

6 **Draft Standard for**
7 **Local and Metropolitan Area Networks—**
8 **Station and Media Access Control**
9 **Connectivity Discovery**

10 Prepared by the
11 **Maintenance Task Group of IEEE 802.1**

12 Sponsor
13 **LAN/MAN Standards Committee of the IEEE Computer Society**

14 **This and the following cover pages are not part of the draft.** They provide revision and other information
15 for IEEE 802.1 Working Group members and participants in the IEEE Standards Association ballot process,
16 and will be updated as convenient. The text proper of this draft begins with the [Title page](#).

Important Notice

This document is an unapproved draft of a proposed IEEE Standard. IEEE hereby grants the named IEEE SA Working Group or Standards Committee Chair permission to distribute this document to participants in the receiving IEEE SA Working Group or Standards Committee, for purposes of review for IEEE standardization activities. No further use, reproduction, or distribution of this document is permitted without the express written permission of IEEE Standards Association (IEEE SA). Prior to any review or use of this draft standard, in part or in whole, by another standards development organization, permission must first be obtained from IEEE SA (stds-copyright@ieee.org). This page is included as the cover of this draft, and shall not be modified or deleted.

IEEE Standards Association
445 Hoes Lane
Piscataway, NJ 08854, USA

1

2 **Current draft**

3 Draft P802.1AB-2016-Rev/D2.1 has been prepared for the first Standards Association (SA) Recirculation
4 Ballot. It includes the resolution to comments received on the SA Ballot of P802.1AB-2016-Rev/D2.0 and
5 determined by the CRG.

6 This draft, P802.1AB-2016-Rev/D2.0, has been prepared for the first Standards Association (SA) Ballot. The
7 comments received on those ballots, and their disposition by the CRG (Comment Resolution Group), are
8 available in myProject.

9 **YANG modules**

10 The YANG modules specified by this standard are attached to the draft pdf as plain text (UTF-8) .yang files.
11 Any changes in the YANG is not represented by change bars since the YANG module is imported from the
12 attached .yang files preventing change bars only on the changed lines.

- ¹ This page intentionally left blank, to allow for the addition of information (e.g. pointers to changed text) during
² the SA Ballot process without the need to renumber the following pages in the draft.

Draft Standard for Local and Metropolitan Area Networks— Station and Media Access Control Connectivity Discovery

Prepared by the

Maintenance Task Group of IEEE 802.1

Sponsor

**LAN/MAN Standards Committee
of the
IEEE Computer Society**

Copyright © 2026 by the IEEE.

Three Park Avenue

New York, New York 10016-5997, USA

All rights reserved.

This document is an unapproved draft of a proposed IEEE Standard. As such, this document is subject to change. USE AT YOUR OWN RISK! IEEE copyright statements SHALL NOT BE REMOVED from draft or approved IEEE standards, or modified in any way. Because this is an unapproved draft, this document must not be utilized for any conformance/compliance purposes. Permission is hereby granted for officers from each IEEE Standards Working Group or Committee to reproduce the draft document developed by that Working Group for purposes of international standardization consideration. IEEE Standards Department must be informed of the submission for consideration prior to any reproduction for international standardization consideration (stds.ipr@ieee.org). Prior to adoption of this document, in whole or in part, by another standards development organization, permission must first be obtained from the IEEE Standards Department (stds.ipr@ieee.org). When requesting permission, IEEE Standards Department will require a copy of the standard development organization's document highlighting the use of IEEE content. Other entities seeking permission to reproduce this document, in whole or in part, must also obtain permission from the IEEE Standards Department.

IEEE Standards Department
445 Hoes Lane
Piscataway, NJ 08854, USA

Abstract: A protocol and a set of managed objects that can be used for discovering the physical topology from adjacent stations in IEEE 802[®] LANs are defined in this document.

Keywords: IEEE 802.1AB[™], link layer discovery protocol, management information base, topology discovery, topology information

The Institute of Electrical and Electronics Engineers, Inc.
3 Park Avenue, New York, NY 10016-5997, USA

Copyright © 2026 by the Institute of Electrical and Electronics Engineers, Inc.
All rights reserved. Unapproved draft.

IEEE and 802 are registered trademarks in the U.S. Patent & Trademark Office, owned by the Institute of Electrical and Electronics Engineers, Incorporated.

PDF: ISBN 978-X-XXX-XXX-X STDXXXXX
Print: ISBN 978-X-XXX-XXX-X STDPDXXXXX

IEEE prohibits discrimination, harassment, and bullying.

For more information, visit <http://www.ieee.org/web/aboutus/whatis/policies/p9-26.html>.

No part of this publication may be reproduced in any form, in an electronic retrieval system or otherwise, without the prior written permission of the publisher.

1 Important Notices and Disclaimers Concerning IEEE Standards Documents

3 IEEE Standards documents are made available for use subject to important notices and legal disclaimers.
4 These notices and disclaimers, or a reference to this page (<https://standards.ieee.org/ipr/disclaimers.html>),
5 appear in all IEEE standards and may be found under the heading “Important Notices and Disclaimers
6 Concerning IEEE Standards Documents.”

7 Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

9 IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards
10 Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus
11 development process, which brings together volunteers representing varied viewpoints and interests to
12 achieve the final product. IEEE Standards are documents developed by volunteers with scientific, academic,
13 and industry-based expertise in technical working groups. Volunteers involved in technical working groups
14 are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While
15 IEEE administers the process and establishes rules to promote fairness in the consensus development
16 process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the
17 soundness of any judgments contained in its standards.

18 IEEE makes no warranties or representations concerning its standards, and expressly disclaims all
19 warranties, express or implied, concerning all standards, including but not limited to the warranties of
20 merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not
21 guarantee safety, security, health, or environmental protection, or compliance with law, or guarantee against
22 interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the
23 use of the material contained in its standards is free from patent infringement. IEEE standards documents are
24 supplied “AS IS” and “WITH ALL FAULTS.”

25 Use of an IEEE standard is wholly voluntary. The existence of an IEEE Standard does not imply that there
26 are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to
27 the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and
28 issued is subject to change brought about through developments in the state of the art and comments
29 received from users of the standard.

30 In publishing and making its standards available, IEEE is not suggesting or rendering professional or other
31 services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any
32 other person or entity to another. Any person utilizing any IEEE Standards document should rely upon their
33 own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate,
34 seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

35 IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
36 EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE
37 NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR
38 BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
39 WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
40 OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON
41 ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND
42 REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

43 Translations

44 The IEEE consensus balloting process involves the review of documents in English only. In the event that an
45 IEEE standard is translated, only the English language version published by IEEE is the approved IEEE
46 standard.

1 Use by artificial intelligence systems

2 In no event shall material in any IEEE Standards documents be used for the purpose of creating, training,
3 enhancing, developing, maintaining, or contributing to any artificial intelligence systems without the
4 express, written consent of the IEEE SA in advance. “Artificial intelligence” refers to any software,
5 application, or other system that uses artificial intelligence, machine learning, or similar technologies, to
6 analyze, train, process, or generate content. Requests for consent can be submitted using the [Contact Us](#)
7 form.¹

8 Official statements

9 A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board
10 Operations Manual is not, and shall not be considered or inferred to be, the official position of IEEE or any
11 of its committees and shall not be considered to be or be relied upon as, a formal position of IEEE or IEEE
12 SA. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE
13 standards shall make it clear that the presenter’s views should be considered the personal views of that
14 individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working
15 Group. Statements made by volunteers may not represent the formal position of their employer(s) or
16 affiliation(s). News releases about IEEE standards issued by entities other than IEEE SA should be
17 considered the view of the entity issuing the release rather than the formal position of IEEE or IEEE SA.

18 Comments on standards

19 Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of
20 membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations,**
21 **consulting information, or advice pertaining to IEEE Standards documents.**

22 Suggestions for changes in documents should be in the form of a proposed change of text, together with
23 appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is
24 important that any responses to comments and questions also receive the concurrence of a balance of interests.
25 For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of
26 Governors are not able to provide an instant response to comments or questions except in those cases where
27 the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation
28 requests. Any person who would like to participate in evaluating comments or revisions to an IEEE standard is
29 welcome to join the relevant IEEE SA working group. You can indicate interest in a working group using the
30 Interests tab in the Manage Profile & Interests area of the [IEEE SA myProject system](#).² An IEEE Account is
31 needed to access the application.

32 Comments on standards should be submitted using the [Contact Us](#) form.¹

33 Laws and regulations

34 Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the
35 provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory
36 requirements. Implementers of the standard are responsible for observing or referring to the applicable
37 regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not
38 in compliance with applicable laws, and these documents may not be construed as doing so.

39 Data privacy

40 Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and
41 data ownership in the context of assessing and using the standards in compliance with applicable laws and
42 regulations.

¹ Available at: <https://standards.ieee.org/content/ieee-standards/en/about/contact/>.

² Available at: <https://development.standards.ieee.org/myproject-web/public/view.html#landing>.

1 Copyrights

2 IEEE draft and approved standards are copyrighted by IEEE under U.S. and international copyright laws.
3 They are made available by IEEE and are adopted for a wide variety of both public and private uses. These
4 include both use, by reference, in laws and regulations, and use in private self-regulation, standardization,
5 and the promotion of engineering practices and methods. By making these documents available for use and
6 adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright
7 to the documents.

8 Photocopies

9 Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license
10 to photocopy portions of any individual standard for company or organizational internal use or individual,
11 non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance
12 Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400;
13 <https://www.copyright.com/>. Permission to photocopy portions of any individual standard for educational
14 classroom use can also be obtained through the Copyright Clearance Center.

15 Updating of IEEE Standards documents

16 Users of IEEE Standards documents should be aware that these documents may be superseded at any time
17 by the issuance of new editions or may be amended from time to time through the issuance of amendments,
18 corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the
19 document together with any amendments, corrigenda, or errata then in effect.

20 Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years
21 old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of
22 some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that
23 they have the latest edition of any IEEE standard.

24 In order to determine whether a given document is the current edition and whether it has been amended
25 through the issuance of amendments, corrigenda, or errata, visit [IEEE Xplore](#) or [contact IEEE](#).³ For more
26 information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

27 Errata

28 Errata, if any, for all IEEE standards can be accessed on the [IEEE SA Website](#).⁴ Search for standard number
29 and year of approval to access the web page of the published standard. Errata links are located under the
30 Additional Resources Details section. Errata are also available in [IEEE Xplore](#). Users are encouraged to
31 periodically check for errata.

32 Patents

33 IEEE Standards are developed in compliance with the [IEEE SA Patent Policy](#).⁵

34 Attention is called to the possibility that implementation of this standard may require use of subject matter
35 covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the
36 existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has
37 filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the
38 IEEE SA Website at <https://standards.ieee.org/about/sasb/patcom/patents.html>. Letters of Assurance may
39 indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without
40 compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of
41 any unfair discrimination to applicants desiring to obtain such licenses.

³ Available at: <https://ieeexplore.ieee.org/browse/standards/collection/ieee>.

⁴ Available at: <https://standards.ieee.org/standard/index.html>.

⁵ Available at: <https://standards.ieee.org/about/sasb/patcom/materials.html>.

1 Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not
2 responsible for identifying Essential Patent Claims for which a license may be required, for conducting
3 inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or
4 conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing
5 agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that
6 determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their
7 own responsibility. Further information may be obtained from the IEEE Standards Association.

8 **IMPORTANT NOTICE**

9 IEEE Standards do not guarantee or ensure safety, security, health, or environmental protection, or ensure
10 against interference with or from other devices or networks. Technologies, application of technologies, and
11 recommended procedures in various industries evolve over time. The IEEE standards development process
12 allows participants to review developments in industries, technologies, and practices, and to determine what,
13 if any, updates should be made to the IEEE standard. During this evolution, the technologies and
14 recommendations in IEEE standards may be implemented in ways not foreseen during the standards
15 development. IEEE Standards development activities consider research and information presented to the
16 standards development group in developing any safety recommendations. Other information about safety
17 practices, changes in technology or technology implementation, or impact by peripheral systems also may
18 be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE
19 Standards documents are responsible for determining and complying with all appropriate safety, security,
20 environmental, health, data privacy, and interference protection practices and all applicable laws and
21 regulations.

1 Participants

2 <<The following lists will be updated in the usual way prior to publication>>

3 At the time this standard was submitted to the IEEE-SA Standards Board for approval, the IEEE 802.1
4 Working Group had the following membership:

5 **Glenn Parsons**, *Chair*
6 **Jessy V. Rouyer**, *Vice Chair*
7 **Mark Hantel**, *Maintenance Task Group Chair*
8 **Paul Bottorff**, *Editor*
9

<<TBA>>

¹ The following members of the individual balloting committee voted on this standard. Balloters may have
² voted for approval, disapproval, or abstention.

<<TBA>>

³ When the IEEE-SA Standards Board approved this standard on XX Month 20xx, it had the following
⁴ membership:

⁵ <<TBA>>

<<TBA>>

⁶
⁷ *Member Emeritus

⁸ **Historical participants**

⁹ Included is a historical list of participants in the IEEE 802.1 Working Group who have dedicated their
¹⁰ valuable time, energy, and knowledge to the creation of this material.

¹¹ The following individuals participated in the 2005 publication of this standard.

¹² **Tony Jeffree, *Chair***
¹³ **Paul Congdon, *Vice Chair***
¹⁴ **Bill Lane, *Technical Editor***
¹⁵ **Mick Seaman, *Interworking Task Group Chair***

1

Les Bell
Paul Bottorff
Jim Burns
Marco Carugi
Dirceu Cavendish
Arjan de Heer
Anush Elangovan
Hesham Elbakoury
David Elie-Dit-Cosaque
Norm Finn
David Frattura
Gerard Goubert
Steve Haddock
Ran Ish-Shalom
Atsushi Iwata

Neil Jarvis
Manu Kaycee
Hal Keen
Roger Lapuh
Loren Larsen
Joe Lawrence
Yannick Le Goff
Marcus Leech
Mahalingam Mani
Dinesh Mohan
Bob Moskowitz
Don O'Connor
Don Pannell
Glenn Parsons

Ken Patton
Frank Reichstein
John Roesé
Allyn Romanow
Dan Romascanu
Jessy V. Rouyer
Ali Sajassi
Dolors Sala
Muneyoshi Suzuki
Jonathan Thatcher
Michel Thorsen
Dennis Volpano
Karl Weber
Ludwig Winkel
Michael D. Wright

1 The following individuals participated in the 2009 publication of this standard.

2
3
4

Tony Jeffree, *Chair and Editor*
Paul Congdon, *Vice Chair*
Stephen Haddock, *Chair, Interworking Task Group*

Osama Aboul-Magd
Zehavit Alon
Caitlin Bestler
Jan Bialkowski
Rob Boatright
Jean-Michel Bonnamy
Paul Bottorff
Rudolf Brandner
Craig W. Carlson
Weiyang Cheng
Rao Cherukuri
Paul Congdon
Diego Crupnicoff
Claudio Desanti
Zhemin Ding
Linda Dunbar
Hesham M. Elbakoury
David Elie-Dit-Cosaque
János Farkas
Donald Fedyk
Norman Finn
Robert Frazier
John Fuller
Geoffrey Garner
Anoop Ghanwani
Franz Goetz
Yannick Le Goff
Eric Gray
Karanvir Grewal
Craig Gunther
Mitch Gusat
Asif Hazarika
Charles Hudson

Romain Insler
Michael Johas Teener
Abhay Karandikar
Prakash Kashyap
Hal Keen
Keti Kilcrease
Yongbum Kim
Philippe Klein
Mike Ko
Vinod Kumar
Bruce Kwan
Kari Laihonen
Michael Lerer
Gael Mace
Ben Mack-Cran
David Martin
Riccardo Martinotti
Alan McGuire
James McIntosh
Menucher Menuchery
John Messenger
Matthew Mora
Eric Multanen
Kevin Nolish
Hiroshi Ohta
David Olsen
Donald Pannell
Glenn Parsons
Joseph Pelissier
David Peterson
Hayim Porat
Max Pritikin

Ananda Rajagopal
Karen T. Randall
Guenter Roeck
Josef Roese
Derek J. Rohde
Dan Romascanu
Moran Roth
Jessy V. Rouyer
Jonathan Sadler
Ali Sajassi
Joseph Salowey
Panagiotis Saltsidis
Satish Sathe
John Sauer
Michael Seaman
Koichiro Seto
Himanshu Shah
Nurit Sprecher
Kevin B. Stanton
Robert A. Sultan
Muneyoshi Suzuki
George Swallow
Attila Takacs
Patricia Thaler
Oliver Thorp
Manoj Wadekar
Yuehua Wei
Brian Weis
Bert Wijnen
Michael D. Wright
Chien-Hsien Wu
Ken Young
Glen Zorn

1 The following individuals participated in the development of Corrigendum 1 to the 2009 publication of this
2 standard.

3 **Tony Jeffree, *Chair and Editor***
4 **Glenn Parsons, *Vice-Chair and Maintenance Task Group Chair***

Ting Ao	Robert Grow	Karen Randall
Kenneth Boehlke	Yingjie Gu	Josef Roesse
Christian Boiger	Craig Gunther	Dan Romascanu
Brad Booth	Stephen Haddock	Jessy Rouyer
Paul Bottorff	Hitoshi Hayakawa	Ali Sajassi
Jeffrey Catlin	Mirko Jakovljevic	Panagiotis Saltsidis
Xin Chang	Markus Jochim	Rick Schell
Weiyang Cheng	Michael Johas Teener	Michael Seaman
Diego Crupnicoff	Girault Jones	Koichiro Seto
Rodney Cummings	Daya Kamath	Daniel Sexton
Donald Eastlake, III	Hal Keen	Rakesh Sharma
János Farkas	Yongbum Kim	Johannes Specht
Donald Fedyk	Philippe Klein	Kevin Stanton
Norman Finn	Oliver Kleineberg	Wilfried Steiner
Andre Fredette	Jeff Lynch	Patricia Thaler
Geoffrey Garner	Ben Mack-Crane	Jeremy Touve
Anoop Ghanwani	John Messenger	Albert Tretter
Franz Goetz	Eric Multanen	Maarten Vissers
Mark Gravel	Henry Muyshondt	Yuehua Wei
Eric Gray	David Olsen	Min Xiao
	Donald Pannell	

5

6

7 The following individuals participated in the development of Corrigendum 2 to the 2009 publication of this
8 standard.

9 **Glenn Parsons, *Working Group Chair***
10 **John Messenger, *Vice-Chair and Maintenance Task Group Chair***
11 **Tony Jeffree, *Editor***

Ting Ao	Hitoshi Hayakawa	Dan Romascanu
Christian Boiger	Jeremy Hitt	Jessy V. Rouyer
Paul Bottorff	Rahil Hussain	Panagiotis Saltsidis
David Chen	Michael Johas Teener	Behcet Sarikaya
Feng Chen	Peter Jones	Michael Seaman
Weiyang Cheng	Hal Keen	Daniel Sexton
Diego Crupnicoff	Marcel Kiessling	Johannes Specht
Rodney Cummings	Yongbum Kim	Kevin B. Stanton
Patrick Diamond	Philippe Klein	Wilfried Steiner
Aboubacar Kader Diarra	Jouni Korhonen	Vahid Tabatabaee
János Farkas	Jeff Lynch	Patricia Thaler
Norman Finn	Ben Mack-Crane	Jeremy Touve
Geoffrey Garner	Christophe Mangin	Karl Weber
Anoop Ghanwani	James McIntosh	Yuehua Wei
Mark Gravel	Eric Multanen	Brian Weis
Eric W. Gray	Donald Pannell	Jordon Woods
Craig Gunther	Karen Randall	Juan-Carlos Zuniga
Stephen Haddock	Maximilian Riegel	

1 The following individuals participated in the 2016 publication of this standard.

2 **Glenn Parsons, *Chair***
3 **John Messenger, *Vice Chair and Maintenance Task Group Chair***
4 **Tony Jeffree, *Editor***

Christian Boiger	Hal Keen	Jessy Rouyer
Paul Bottorff	Stephan Kehrer	Panagiotis Saltsidis
David Chen	Marcel Kiessling	Michael Seaman
Feng Chen	Philippe Klein	Daniel Sexton
Weiyang Cheng	Jouni Korhonen	Johannes Specht
Rodney Cummings	Yizhou Li	Wilfried Steiner
János Farkas	Christophe Mangin	Patricia Thaler
Norman Finn	Tom McBeath	David Thornburg
Geoffrey Garner	James McIntosh	Jeremy Touve
Eric Gray	Hiroki Nakano	Paul Unbehagen
Craig Gunther	Bob Noseworthy	Karl Weber
Stephen Haddock	Donald R. Pannell	Brian Weis
Mark Hantel	Walter Pieniciak	Jordon Woods
Marc Holness	Karen Randall	Helge Zinner
Michael Johas Teener	Maximilian Riegel	Juan Carlos Zuniga
	Dan Romascanu	

5 The following members of the individual balloting committee voted on this standard. Balloters may have
6 voted for approval, disapproval, or abstention.

Thomas Alexander	Raj Jain	Robert Robinson
Butch Anton	Tony Jeffree	Benjamin Rolfe
Lee Armstrong	Michael Johas Teener	Dan Romascanu
Stefan Aust	Adri Jovin	Jessy Rouyer
Christian Boiger	Shinkyo Kaku	John Santhoff
Nancy Bravin	Piotr Karocki	Bartien Sayogo
William Byrd	Stuart Kerry	Michael Seaman
Juan Carreon	Yongbum Kim	Thomas Starai
Rodney Cummings	Paul Lambert	Eugene Stoudenmire
Douglas Dorr	Robert Landman	Rene Struik
János Farkas	David Lewis	Walter Struppler
Yukihiro Fujimoto	Arthur H. Light	Michael Swearingen
David Gregson	William Lumpkins	Patricia Thaler
Randall Groves	Michael Lynch	Mark-Rene Uchida
Craig Gunther	Elvis Maculuba	Lorenzo Vangelista
Stephen Haddock	Jonathon McLendon	Dmitri Varsanofiev
Marek Hajduczenia	Richard Mellitz	George Vlantis
Jerome Henry	John Messenger	Khurram Waheed
Marco Hernandez	Charles Moorwood	Karl Weber
Guido Hiertz	Michael Newman	Hung-Yu Wei
Werner Hoelzl	Nick S. A. Nikjoo	Natalie Wienckowski
Noriyuki Ikeuchi	Satoshi Obara	Andreas Wolf
Sergiu Iordanescu	Alon Regev	Oren Yuen
Atsushi Ito	Maximilian Riegel	Zhen Zhou

7

1 Introduction

This introduction is not part of IEEE Std 802.1AB™-2016, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery.

2 This revision of IEEE Std 802.1AB incorporates the following amendments into the base text of the 2016
3 revision:

- 4 — IEEE Std 802.1ABcu-2021
- 5 — IEEE Std 802.1ABdh-2021

6 This standard contains state-of-the-art material. The area covered by this standard is undergoing evolution.
7 Revisions are anticipated within the next few years to clarify existing material, to correct possible errors, and
8 to incorporate new related material. Information on the current revision state of this and other IEEE 802
9 standards may be obtained from

10 Secretary, IEEE-SA Standards Board
11 445 Hoes Lane
12 Piscataway, NJ 08854-4141
13 USA

Contents

2	1.	Overview	21
3	1.1	Scope	21
4	1.2	Purpose	22
5	2.	Normative references	23
6	3.	Definitions and numerical representation	25
7	3.1	Definitions	25
8	3.2	Numerical representation	27
9	4.	Acronyms and abbreviations	28
10	5.	Conformance	30
11	5.1	Terminology	30
12	5.2	Protocol Implementation Conformance Statement (PICS)	30
13	5.3	Required capabilities	30
14	5.4	Optional capabilities	31
15	6.	Principles of operation	32
16	6.1	Transmission and reception	32
17	6.2	LLDP operational modes	34
18	6.3	LLDP information categories	34
19	6.4	TLV selection	34
20	6.5	Transmission principles	35
21	6.6	Reception principles	35
22	6.7	Systems with multiple LLDP Agents	36
23	6.8	LLDP and Link Aggregation	39
24	6.9	Extended LLDP (XLLDP)	40
25	7.	LLDPDU transmission, reception, and addressing	41
26	7.1	Destination address	41
27	7.2	Source address	44
28	7.3	EtherType use and encoding	44
29	7.4	LLDPDU reception	44
30	8.	LLDPDU and TLV formats	45
31	8.1	LLDPDU bit and octet ordering conventions	45
32	8.2	LLDPDU format	45
33	8.3	TLV categories	46
34	8.4	Basic TLV format	47
35	8.5	Basic management TLV set formats and definitions	49
36	8.6	Extended LLDP set TLV formats and definitions	58
37	8.7	Organizationally Specific TLVs	61
38	9.	LLDP agent operation	63
39	9.1	Overview	63
40	9.2	State machines	68

1	10.	LLDP management	93
2	10.1	Data storage and retrieval	93
3	10.2	The LLDP management entity's responsibilities.....	93
4	10.3	Managed objects	95
5	10.4	Data types	95
6	10.5	LLDP variables	96
7	11.	LLDP MIB definitions.....	99
8	11.1	Internet Standard Management Framework	99
9	11.2	Structure of the LLDP MIB	99
10	11.3	Relationship to other MIBs	104
11	11.4	Security considerations for LLDP base MIB module.....	105
12	11.5	LLDP MIB modules ,	107
13	12.	LLDP YANG definitions.....	159
14	12.1	Internet Standard Management Framework	159
15	12.2	Information model for LLDP management	159
16	12.3	Structure of the LLDP YANG model.....	162
17	12.4	Relationship to other YANG modules	163
18	12.5	Security considerations	164
19	12.6	Definition of the YANG modules,,	165
20	Annex A	(normative) PICS proforma.....	184
21	Annex B	(normative) PTOPO MIB update	191
22	Annex C	(informative) Example LLDP transmission frame formats.....	192
23	Annex D	(informative) Bibliography	193

1 List of figures

2	Figure 6-1, LLDP agent and its relationship to its LLC entity	32
3	Figure 6-2, Relationship between LLDP agents, LLC Entities, MSAPs, and the LLDP management entity	36
4	Figure 6-3, LLDP in a MAC Bridge	37
5	Figure 6-4, LLDP in an end system with port-based network access control	38
6	Figure 6-5, LLDP in a MAC Bridge that uses port-based network access control on both ports	38
7	Figure 6-6, Scope of group MAC addresses	39
8	Figure 6-7, Multiplexing and demultiplexing using shims	39
9	Figure 7-1, MSDU format	41
10	Figure 8-1, LLDPDU formats	46
11	Figure 8-2, Basic TLV format	47
12	Figure 8-3, End Of LLDPDU TLV format	49
13	Figure 8-4, Chassis ID TLV format	49
14	Figure 8-5, Port ID TLV format	51
15	Figure 8-6, Time To Live TLV format	52
16	Figure 8-7, Port Description TLV format	52
17	Figure 8-8, System Name TLV format	53
18	Figure 8-9, System Description TLV format	54
19	Figure 8-10, System Capabilities TLV format	54
20	Figure 8-11, Management Address TLV format	56
21	Figure 8-12, Manifest TLV format	58
22	Figure 8-13, Extension Request TLV format	59
23	Figure 8-14, Extension Identifier TLV format	60
24	Figure 8-15, Format for Organizationally Specific TLVs	61
25	Figure 9-1, Transmit state machine	88
26	Figure 9-2, Receive state machine	90
27	Figure 9-3, Extension receive state machine	91
28	Figure 9-4, Transmit timer state machine	92
29	Figure 11-1, LLDP MIB block diagram	99
30	Figure 12-1, YANG root hierarchy with LLDP YANG modules	160
31	Figure 12-2, LLDP configuration and monitoring objects	160
32	Figure 12-3, LLDP port configuration and monitoring objects	161
33	Figure C.1, IEEE 802.3 LLDP frame format	192
34	Figure C.2, IEEE 802.11 LLDP frame format	192

1 List of tables

2	Table 7-1, Group MAC addresses used by LLDP	42
3	Table 7-2, Support for the Scope MAC Address in different systems	43
4	Table 7-3, LLDP EtherType	44
5	Table 8-1, TLV type values	48
6	Table 8-2, chassis ID subtype enumeration	50
7	Table 8-3, port ID subtype enumeration	51
8	Table 8-4, System capabilities	55
9	Table 9-1, Subclause/operating mode applicability	63
10	Table 9-2, State machine symbols	69
11	Table 11-1, MIB object groups and operating mode applicability	100
12	Table 11-2, LLDP MIB structure and object cross reference	100
13	Table 12-1, Structure of the YANG modules	162
14	Table 12-2, Structure of the ieee802-dot1ab-lldp module	163

1 IEEE Standard for 2 Local and metropolitan area networks— 3 Station and Media Access Control 4 Connectivity Discovery

5 *IMPORTANT NOTICE: IEEE Standards documents are not intended to ensure safety, security, health,*
6 *or environmental protection, or ensure against interference with or from other devices or networks.*
7 *Implementers of IEEE Standards documents are responsible for determining and complying with all*
8 *appropriate safety, security, environmental, health, and interference protection practices and all*
9 *applicable laws and regulations.*

10 *This IEEE document is made available for use subject to important notices and legal disclaimers.*
11 *These notices and disclaimers appear in all publications containing this document and may*
12 *be found under the heading “Important Notice” or “Important Notices and Disclaimers*
13 *Concerning IEEE Documents.” They can also be obtained on request from IEEE or viewed at*
14 <http://standards.ieee.org/IPR/disclaimers.html>.

15 1. Overview

16 The Link Layer Discovery Protocol (LLDP) specified in this standard allows stations attached to an
17 IEEE 802[®] LAN to advertise, to other stations attached to the same IEEE 802 LAN, the major capabilities
18 provided by the system incorporating that station, the management address or addresses of the entity or
19 entities that provide management of those capabilities, and the identification of the station’s point of
20 attachment to the IEEE 802 LAN required by those management entity or entities. Basic LLDP provides a
21 mechanism for advertising information that can fit within a single Protocol Data Unit (PDU), while
22 Extended LLDP (XLLDP) allows advertisement of information spanning multiple PDUs.

23 The information distributed via this protocol is stored in an information repository (RP), which can support
24 both a standard Management Information Base (MIB) for SNMP and standard data models defined in
25 YANG. This enables the information to be accessed by a Network Management System (NMS) through
26 compatible management protocols, such as SNMP or NETCONF.

27 1.1 Scope

28 The scope of this standard is to define a protocol and management elements, suitable for advertising
29 information to stations attached to the same IEEE 802 LAN, for the purpose of populating physical topology
30 and device discovery management information databases. The protocol facilitates the identification of
31 stations connected by IEEE 802 LANs/MANs, their points of interconnection, and access points for
32 management protocols.

1 This standard defines a protocol that

- 2 a) Advertises connectivity and management information about the local station to adjacent stations on
3 the same IEEE 802 LAN.
- 4 b) Receives network management information from adjacent stations on the same IEEE 802 LAN.
- 5 c) Operates with all IEEE 802 access protocols and network media.
- 6 d) Establishes a network management information schema and object definitions that are suitable for
7 storing connection information about adjacent stations.
- 8 e) Can provide compatibility with the IETF PTOPO MIB (IETF RFC 2922 [B8]).¹
- 9 f) Can provide compatibility with the IETF Data Model for Network Topologies (IETF RFC 8345
10 [B14]).²

11 1.2 Purpose

12 This standard specifies the necessary protocol and management elements to

- 13 a) Facilitate multi-vendor inter-operability and the use of standard management tools to discover and
14 make available physical topology information for network management.
- 15 b) Make it possible for network management to discover certain configuration inconsistencies or
16 malfunctions that can result in impaired communication at higher layers.
- 17 c) Provide information to assist network management in making resource changes and/or
18 re-configurations that correct configuration inconsistencies or malfunctions identified in b) above.
- 19 d) Provide IETF MIB (IETF RFC 2922 [B8]) to describe a network's physical topology and associated
20 systems within that topology.
- 21 e) Provide YANG configuration and operational models supporting Station and Media Access Control
22 Connectivity Discovery.

¹ The numbers in brackets correspond to those in the bibliography in Annex D.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in the text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies.

IEEE Std 802[®], IEEE Standard for Local and Metropolitan Area Networks: Overview and Architecture.^{2, 3}

IEEE Std 802.1AC[™], IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

IEEE Std 802.1AE[™], IEEE Standard for Local and Metropolitan Area Networks—Media Access Control (MAC) Security.

IEEE Std 802.1AX[™], IEEE Standard for Local and Metropolitan Area Networks—Link Aggregation.

IEEE Std 802.1Q[™], IEEE Standards for Local and Metropolitan Area Networks: Bridges and Bridged Networks.

IEEE Std 802.1X[™], IEEE Standard for Local and Metropolitan Area Networks—Port-Based Network Access Control.

IEEE Std 802.3[™], IEEE Standard for Ethernet.

IETF RFC 2863, The Interfaces Group MIB, June 2000.⁴

IETF RFC 3046, DHCP Relay Agent Information Option, January 2001.

IETF RFC 3232, Assigned Numbers: RFC 1700 is Replaced by an On-line Database, January 2002.⁵

IETF RFC 3410, Introduction and Applicability Statements for Internet Standard Management Framework, December 2002.

IETF RFC 3417, Transport Mappings for the Simple Network Management Protocol (SNMP), December 2002.

IETF RFC 3418, Management Information Base (MIB) for the Simple Network Management Protocol (SNMP), December 2002.

IETF RFC 3629, UTF-8, a transformation format of ISO 10646, November 2003.

IETF RFC 4502, Remote Network Monitoring Management Information Base Version 2, May 2006.

IETF RFC 4639, Cable Device Management Information Base for Data-Over-Cable Service Interface Specification (DOCSIS) Compliant Cable Modems and Cable Modem Termination Systems, December 2006.

² IEEE and 802 are registered trademarks in the U.S. Patent & Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated.

³ IEEE publications are available from The Institute of Electrical and Electronics Engineers (<http://standards.ieee.org/>).

⁴ IETF documents (i.e., RFCs) are available for download at <http://www.rfc-archive.org/>.

⁵ The IETF RFC 3232 ianaAddressFamilyNumbers on-line database module is accessible at (<http://www.iana.org/>).

¹ IETF RFC 4789, Simple Network Management Protocol (SNMP) over IEEE 802 Networks, November
² 2006.

³ IETF RFC 6241, Network Configuration Protocol (NETCONF), June 2011. ⁶

⁴ IETF RFC 6933, Entity MIB (Version 4), May 2013. IETF RFC 6991, Common YANG Data Types, July
⁵ 2013.

⁶ IETF RFC 6991, Common YANG Data Types, July 2013.

⁷ IETF RFC 7950, The YANG 1.1 Data Modeling Language, August 2016.

⁸ IETF RFC 8343, A YANG Data Model for Interface Management, March 2018.

⁹ IETF RFC 8349, A YANG Data Model for Routing Management (NMDA Version), March 2018.

¹⁰ ISO/IEC 8824-1 [ITU-T Rec. X.680 (2002)], Abstract Syntax Notation One (ASN.1): Specification of Basic
¹¹ Notation. ⁷

⁶ IETF RFCs are available from the Internet Engineering Task Force (<https://www.rfc-archive.org/>).

⁷ ASN.1 standards are available at <http://asn1.elibel.tm.fr/en/standards/index.htm#asn1>.

3. Definitions and numerical representation

3.1 Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause.⁸

alpha-numeric information: Information that is encoded using the 8-bit Universal Character Set (UCS)/Unicode Transformation Format (UTF-8) octet sequence [IETF RFC 3629].

chassis: A physical component incorporating one or more IEEE 802[®] LAN stations and their associated application functionality.

chassis identifier: An administratively assigned name that identifies the particular chassis within the context of an administrative domain that comprises one or more networks.

Extended Link Layer Discovery Protocol (XLLDP): An LLDP extension used to retrieve the frames beyond the Normal LLDPDU of a multi-frame LLDP database.

Extension LLDPDU (XPDU): An LLDPDU containing an Extension Identifier TLV and not containing a Time to Live TLV or Extension Request TLV.

Extension Request LLDPDU (XREQ): An LLDPDU containing an Extension Request TLV and not containing a Time to Live TLV or Extension Identifier TLV.

IEEE 802[®] LAN: Local area network (LAN) technologies that provide a media access control (MAC) Service equivalent to the MAC Service defined in IEEE Std 802.1AC.

IEEE 802[®] LAN station: An IEEE 802-compatible entity that incorporates all the necessary mechanisms to participate in media access control of an IEEE 802 LAN, and that is at least capable of providing the MAC service plus the the protocol identification capabilities required by the station's clients of the MAC service.

NOTE 1—For example, routers and host computers are stations in a bridged network. Bridges, in addition to their relay function, also include station capabilities.⁹

Link Layer Discovery Protocol (LLDP): A media-independent protocol capable of running on all IEEE 802[®] LAN stations and to allow an LLDP agent to learn the connectivity and management information from adjacent stations.

LLDP agent: The protocol entity that implements LLDP for a particular MSAP associated with a Port.

LLDP Information Repository (RP): A management information repository which stores the information exchanged by LLDP in TLVs and can be modeled by SNMP MIBs, YANG, or other data models.

MAC service access point (MSAP): The access point for MAC services provided to the LLC sublayer.

MSAP identifier: The identifier of a MAC service access point.

NOTE 2—In this standard, the concatenation of the chassis identifier and the port identifier is used by LLDP as an MSAP identifier, to identify the port associated with an IEEE 802[®] LAN station.

⁸ *IEEE Standards Dictionary Online* is available at: <http://dictionary.ieee.org>.

⁹ Notes in text, tables, and figures are given for information only and do not contain requirements needed to implement the standard.

1 **management entity:** The protocol entity that implements a particular network management protocol and
2 that provides access support to a information repository associated with the protocol and implemented on a
3 host chassis.

4 **Management Information Base (MIB):** The instantiation of all MIB modules in a managed entity (e.g.,
5 system or device).

6 **Management Information Base module (MIB module):** The specification or schema for a data base that
7 can be populated with the information required to support a network management information system.

8 **Manifest LLDPDU:** A Normal LLDPDU containing a Manifest TLV.

9 **network:** An interconnected group of systems, each comprising one or more IEEE 802[®] LAN stations.

10 **Network Management System (NMS):** A management system that is capable of utilizing the information
11 in a information repository.

12 **Normal LLDPDU:** An LLDPDU containing a Time To Live TLV and not containing an Extension Request
13 TLV or an Extension Identifier TLV.

14 **object identifier (OID):** An identifier used to name an object. Structurally, an OID consists of a node in a
15 hierarchically-assigned namespace, formally defined in ISO/IEC 8824-1, Abstract Syntax Notation 1
16 (ASN.1). OIDs are used in this standard to identify MIB modules and the objects they contain.

17 **physical network topology:** The identification of systems, of IEEE 802[®] LAN stations that compose each
18 system, and of the IEEE 802 LAN stations that attach to the same IEEE 802 LAN.

19 **port:** The entity in a chassis/system to support an MSAP. A port incorporates one and only one MSAP and
20 identifies the collection of manageable entities that provide the MAC Service at the MSAP.

21 **port identifier:** An administratively assigned name that identifies the particular port within the context of a
22 system, where the identification is convenient, local to the system, and persistent for the system's use and
23 management (whereas the MAC address that globally identifies the MSAP can not be).

24 **Scope MAC Address:** The MAC address used as the destination address in Normal LLDPDUs, which
25 determines the limits of propagation in the network.

26 **shutdown LLDPDU:** A Normal LLDPDU with a TTL value of zero.

27 **system:** A managed collection of hardware and software components incorporating one or more chassis,
28 stations and ports.

29 **station only:** A non-forwarding IEEE 802[®] LAN station such as a user workstation, network file server, or
30 print server.

31 **type, length, value (TLV):** A short, variable length encoding of an information element consisting of
32 sequential type, length, and value fields where the type field identifies the type of information, the length
33 field indicates the length of the information field in octets, and the value field contains the information,
34 itself.

35 **YANG:** IETF defined data modeling language, published as IETF RFC 7950.

36 **YANG model:** One or more YANG modules used to configure and monitor the managed element or system.

1 **YANG module:** The description of the data model used to configure and monitor the managed element or
2 system. A YANG module defines a hierarchy of nodes that can be used for NETCONF-based (see IETF
3 RFC 7803 [B12]) and RESTCONF-based (see IETF RFC 8040 [B13]) operations.

4 **3.2 Numerical representation**

5 Decimal, hexadecimal, and binary numbers are used within this document. For clarity, decimal numbers are
6 generally used to represent counts, hexadecimal numbers are used to represent addresses, and binary
7 numbers are used to describe bit patterns within binary fields.

8 Decimal numbers are represented in their usual 0, 1, 2, ... format. Hexadecimal numbers are represented by
9 a string of one or more hexadecimal (0–9, A–F) digits followed by the subscript 16, except in C-code
10 contexts, where they are written as `0x123EF2`, etc. Binary numbers are represented by a string of one or
11 more binary (0,1) digits, followed by the subscript 2. Thus the decimal number “26” may also be represented
12 as “1A₁₆” or “11010₂”.

13 MAC addresses, EtherType values, and OI/OUI/EUI/CID values are represented as strings of 8-bit
14 hexadecimal numbers separated by hyphens and without a subscript, as, for example, “01-80-C2-00-00-15”
15 or “AA-55-11”, using the hexadecimal representation defined in IEEE Std 802.

4. Acronyms and abbreviations

2	AP	Access Point
3	BSSID	basic service set identification
4	DA	Destination Address
5	DS	Distribution System
6	EUI	Extended Unique Identifier
7	FCS	Frame Check Sequence
8	IANA	Internet Assigned Numbers Authority
9	ID	Identifier
10	IETF	Internet Engineering Task Force
11	KaY	Media access control Security Key Agreement Entity
12	LLC	logical link control (sublayer)
13	LLDP	Link Layer Discovery Protocol
14	LLDPDU	Link Layer Discovery Protocol data unit
15	LSAP	link service access point
16	MAC	media access control (sublayer)
17	MIB	Management Information Base (module)
18	MSAP	Media access control service access point
19	MSDU	Media access control service data unit
20	NMS	Network Management System
21	OID	object identifier
22	OI	organizational identifier
23	OUI	Organizationally Unique Identifier
24	PD	Powered Device
25	PHY	physical (sublayer)
26	PSE	Power Sourcing Equipment
27	PTOPO	the name of the Internet Engineering Task Force physical topology Management
28		Information Base

1 RFC	Request for comments
2 RP	LLDP Information Repository
3 SA	Source Address
4 SecY	Media access control Security Entity
5 SNAP	Subnetwork Access Protocol
6 SNMP	Simple Network Management Protocol
7 TPMR	Two-port Media access control relay
8 TLV	type, length, value
9 TTL	time to live (value)
10 UCS	Universal Character Set
11 UML [®]	Unified Modeling Language [™] ¹⁰
12 UTF-8	8-bit Universal Character Set/Unicode Transformation Format
13 XLLDP	Extended Link Layer Discovery Protocol
14 XPDU	Extension LLDPDU
15 XREQ	Extension Request LLDPDU
16	

¹⁰ UML[®] is a registered trademark of the Object Management Group, Inc., and Unified Modeling Language[™] is a trademark of the Object Management Group, Inc.

1 5. Conformance

2 This clause specifies the mandatory and optional capabilities provided by conformant implementations of
3 this standard.

4 5.1 Terminology

5 For consistency with IEEE and existing IEEE 802.1™ standards terminology, requirements placed upon
6 conformant implementations of this standard are expressed using the following terminology:

- 7 a) *Shall* is used for mandatory requirements.
- 8 b) *May* is used to describe implementation or administrative choices (“may” means “is permitted to,”
9 and hence, “may” and “may not” mean precisely the same thing).
- 10 c) *Should* is used for recommended choices (the behaviors described by “should” and “should not” are
11 both permissible but not equally desirable choices).

12 The PICS proforma (see Annex A) reflects the occurrences of the words *shall*, *may*, and *should* within the
13 standard.

14 The standard avoids needless repetition and apparent duplication of its formal requirements by using *is*, *is*
15 *not*, *are*, and *are not* for definitions and the logical consequences of conformant behavior. Behavior that is
16 permitted but is neither always required nor directly controlled by an implementor or administrator, or
17 whose conformance requirement is detailed elsewhere, is described by *can*. Behavior that never occurs in a
18 conformant implementation or system of conformant implementations is described by *can not*. The word
19 *allow* is used as a replacement for the phrase “support the ability for,” and the word *capability* means “is
20 able to, or can be configured to.”

21 5.2 Protocol Implementation Conformance Statement (PICS)

22 The supplier of an implementation that is claimed to conform to this standard shall complete a copy of the
23 PICS proforma provided in Annex A and shall provide the information necessary to identify both the
24 supplier and the implementation.

25 5.3 Required capabilities

26 A system for which conformance to this standard is claimed shall, for all ports for which support is claimed,
27 include the following capabilities:

- 28 a) If port access is controlled by IEEE Std 802.1X,¹¹ LLDP exchanges shall be supported through the
29 controlled port, as specified in Clause 6 of IEEE Std 802.1X-2020.
- 30 b) The destination and source addressing shall conform to 7.1 and 7.2.
- 31 c) EtherType encapsulation shall conform to 7.3.
- 32 d) LLDPDU recognition and reception shall conform to the operation of the receive state machine as
33 defined in 9.2.10.
- 34 e) LLDPDU encapsulation shall conform to the specifications in 8.2.
- 35 f) The basic TLV format capability shall be implemented as defined in 8.4.
- 36 g) The basic management set of TLVs shall be implemented as defined in 8.5.
- 37 h) The Organizationally Specific TLV format capability shall be implemented as defined in 8.7.

¹¹ Information on references can be found in RFC 3629.

- 1 i) The protocol shall conform to the specifications for all Clause 9 subclauses indicated in Table 9-1
2 for the particular operating mode (transmit only, receive only, or transmit and receive) being
3 implemented.
- 4 j) If receipt of LLDPDUs is supported, for every set of TLVs (the basic management set, extended
5 LLDP set, and any organizationally specific sets) supported, support shall be implemented for
6 receipt of every TLV defined in the set.
- 7 k) If transmission of LLDPDUs is supported, for every set of TLVs (the basic management set,
8 extended LLDP set, and any organizationally specific sets) supported, support shall be implemented
9 for transmission of every TLV defined in the set.
- 10 l) If transmission of LLDPDUs is supported, for every set of TLVs (the basic management set,
11 extended LLDP set, and any organizationally specific sets) supported, a capability shall be
12 implemented for users to determine which optional TLVs are transmitted in any particular
13 LLDPDU.
- 14 m) If SNMP is supported, then
 - 15 1) The system shall conform to the LLDP management specifications in Clause 11 and shall
16 implement the sections of version 2 of the basic LLDP MIB indicated in Table 11-1 for the
17 operating mode being implemented.
 - 18 2) The system shall support at least one of the transport mappings defined by IETF RFC 3417 or
19 IETF RFC 4789.
- 20 n) If neither SNMP nor YANG are supported, the system shall provide storage and retrieval capability
21 equivalent to the functionality specified in 10.1 for the operating mode being implemented.
- 22 o) If YANG is supported, then the system shall conform to the LLDP management specifications in
23 Clause 12 and shall implement the sections of the LLDP YANG module for the operating mode
24 being implemented.
- 25 p) If the Extended Link Layer Discovery Protocol is supported, then
 - 26 1) The extended LLDP set of TLVs shall be implemented as defined in 8.6.
 - 27 2) The Extension Request and Extension LLDPDU formats (8.2) and addressing (7.1.2) shall be
28 implemented.
 - 29 3) The extended receive state machine (Figure 9-3) shall be implemented.

30 5.4 Optional capabilities

31 A system for which conformance to this standard is claimed may, for each port for which support is claimed,
32 support the following capabilities:

- 33 a) If port access is controlled by IEEE Std 802.1X, LLDP exchanges may be supported through the
34 uncontrolled port, as specified in Clause 6 of IEEE Std 802.1X-2020.

6. Principles of operation

LLDP is a link layer protocol that allows an IEEE 802 LAN station to advertise the capabilities and current status of the system associated with an MSAP. The MSAP provides the MAC service to an LLC Entity, and that LLC Entity provides an LSAP to an LLDP agent that transmits and receives information to and from the LLDP agents of other stations attached to the same LAN. The information distributed and received in each LLDPDU is stored in one or more Management Information Bases (MIBs) or YANG modules. Figure 6-1 illustrates the LLDP agent and its relationship to its LLC Entity and MSAP, and to additional MIBs or YANG modules designed by the IETF, IEEE 802, and others.

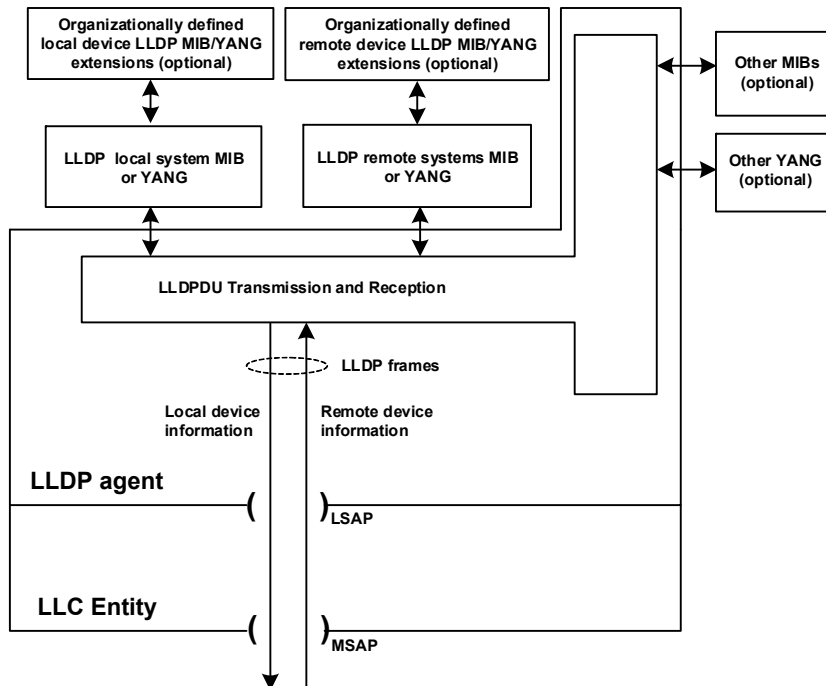


Figure 6-1—LLDP agent and its relationship to its LLC entity

This clause describes general principles of LLDP and Extended LLDP (XLLDP) operation. The following clauses specify transmission, reception, and addressing of LLPDUs (Clause 7); LLDPDU formats (Clause 8); operation of each LLDP agent in detail including state machines (Clause 9); management of LLDP (Clause 10); and the MIB module (Clause 11) or YANG module (Clause 12) used for LLDP management.

NOTE 1—For the purposes of this standard, the terms “LLC” and “LLC entity” include the service provided by the operation of entities that support either Type 3 Protocol Identification Field (PIF) encoding or the use of SNAP encoding of an EtherType, as specified in IEEE Std 802. Hence, “LSAP” in the above description can refer to the use of ISO/IEC 8802-2 [B15] LLC addresses, or an EtherType, as a means of higher layer protocol identification.

NOTE 2—The LLC Entity also provides distinct LSAPs to other higher layer protocol entities (such as those responsible for the Spanning Tree Protocol and IP). In a bridge, the LLC Entity associated with each Bridge Port is modeled as being directly connected to the attached LAN, as discussed in 8.13.9 of IEEE Std 802.1Q-2022, so the operation of LLDP is unaffected by the spanning tree Port State.

6.1 Transmission and reception

The information fields in each LLDP frame are contained in a Link Layer Discovery Protocol Data Unit (LLDPDU) as a sequence of variable length information elements, that each include type, length, and value fields (known as TLVs), where

- 1 — Type identifies what kind of information is being sent.
- 2 — Length indicates the length of the information string in octets.
- 3 — Value is the actual information that needs to be sent (for example, a binary bit map or an
- 4 alpha-numeric string that can contain one or more fields).

5 Each LLDPDU begins with a mandatory sequence of three TLVs, as specified in 8.2, and can contain zero or
6 more additional optional TLVs as selected by network management. The first three TLVs are:

- 7 1) A Chassis ID TLV.
- 8 2) A Port ID TLV.
- 9 3) A Time To Live TLV or, if XLLDP is supported, an Extension Request TLV or an Extension
- 10 Identifier TLV.

11 The first two TLVs contain the chassis ID and the port ID values. These are concatenated to form a logical
12 MSAP identifier. The combination of the MSAP identifier and the Scope MAC Address identify the LLDP
13 agent/port that is the source of the information in the TLVs. Both the chassis ID and port ID values can be
14 defined in a number of convenient forms. Once selected, however, the chassis ID/port ID value combination
15 remains the same as long as the particular port remains operable (8.5.2, 8.5.3, 9.2.4.1).

16 The third TLV differentiates between three types of LLDPDUs:

- 17 a) A Normal LLDPDU, where the third TLV is a Time To Live TLV that tells the receiving LLDP
18 agent how long all information pertaining to this LLDPDU's MSAP identifier is valid so that all of
19 the associated information can later be automatically discarded by the receiving LLDP agent if the
20 sender fails to update it in a timely manner. It is useful to further differentiate two special cases of a
21 Normal LLDPDU:
 - 22 1) A shutdown LLDPDU, where the third TLV is a Time To Live TLV with a TTL value of zero
23 indicating that any information pertaining to this LLDPDU's MSAP identifier is to be
24 discarded immediately. A TTL value of zero can be used, for example, to signal that the
25 sending port has initiated a port shutdown procedure.
 - 26 2) A Manifest LLDPDU, where the third TLV is a Time To Live TLV with a non-zero TTL value
27 and one of the subsequent TLVs is a Manifest TLV.
- 28 b) An Extension Request LLDPDU (XREQ), where the third TLV is an Extension Request TLV that
29 the recipient of a Normal LLDPDU containing a Manifest TLV uses to request one or more of the
30 Extension LLDPDUs described in the Manifest TLV. The Extension Request LLDPDU does not
31 contain a Time to Live TLV.
- 32 c) An Extension LLDPDU (XPDU), where the third TLV is an Extension Identifier TLV. The
33 Extension LLDPDU does not contain a Time to Live TLV.

34 NOTE —The receive state machine (in 9.2.10 and in prior versions of this standard) specifies that an implementation not
35 supporting XLLDP will discard any received LLDPDUs not containing a TTL TLV as the third TLV (i.e., will discard
36 any received Extension Request or Extension LLDPDUs). The state machine further specifies that only implementations
37 supporting XLLDP will process a Manifest TLV and subsequently transmit Extension Request LLDPDUs, and that only
38 implementations supporting XLLDP will process Extension Request TLVs and subsequently transmit Extension
39 LLDPDUs.

40 An optional End Of LLDPDU TLV marks the end of the LLDPDU.

41 The format for the LLDPDU is defined in 8.2. The TLV categories and the basic TLV format are defined in
42 8.3 and 8.4. The specific format and field contents for the Chassis ID TLV are defined in 8.5.2; for the
43 Port ID TLV, in 8.5.3; for the Time To Live TLV in 8.5.4; and for the End Of LLDPDU TLV, in 8.5.1.

1 6.2 LLDP operational modes

2 The information advertised by LLDP is transferred in a single direction. An LLDP agent can periodically
3 advertise information about the capabilities and current status of the system associated with its MSAP
4 identifier. The LLDP agent can also collect information about the capabilities and current status of the
5 system associated with a remote MSAP identifier. LLDP does not itself contain a mechanism for the LLDP
6 agent to solicit specific information from other LLDP agents beyond what is being advertised by those
7 LLDP agents, nor does it provide a specific means of confirming the receipt of information. Extended LLDP
8 allows an LLDP agent collecting information to request detailed information (via an Extension Request
9 LLDPDU) that is only advertised in a summary form (in a Manifest TLV), however this does not change the
10 inherent LLDP property that the choice of information provided is at the discretion of the advertising LLDP
11 agent.

12 NOTE—The LLDP protocol is designed to advertise information useful for discovering pertinent information about a
13 remote port and to populate topology MIBs or YANG modules. It is not intended to act as a configuration protocol for
14 remote systems, nor as a mechanism to signal control information between ports. During the operation of LLDP, it may
15 be possible to discover configuration inconsistencies between systems on the same IEEE 802 LAN. LLDP does not
16 provide a mechanism to resolve those inconsistencies. Rather, it provides a means to report discovered information to
17 higher layer management entities.

18 LLDP allows the advertising function and collecting function of the LLDP agent to be separately enabled,
19 making it possible to configure an implementation so the local LLDP agent can either advertise only or
20 collect only, or so the local LLDP agent can advertise and collect LLDP information.

21 6.3 LLDP information categories

22 The following basic management TLV set (this set is required in all LLDP implementations) is defined by
23 this standard and can be used to describe the system and/or to assist in the detection of configuration
24 inconsistencies associated with the MSAP identifier:

- 25 a) Port Description TLV.
- 26 b) System Name TLV.
- 27 c) System Description TLV.
- 28 d) System Capabilities TLV (indicates both the system's capabilities and its current primary network
29 function, such as end station, bridge, router).
- 30 e) Management Address TLV.

31 Table 8-1 includes a list of the optional TLVs in the basic management set and provides subclause references
32 for their specific definitions.

33 Organizationally Specific TLVs can be defined by either the professional organizations or the individual
34 vendors that are involved with the particular functionality being implemented within a system. The basic
35 format and procedures for defining Organizationally Specific TLVs are provided in 8.7.

36 6.4 TLV selection

37 Information for constructing the various TLVs to be sent is stored in the LLDP local system information
38 repository (RP). The selection of which particular TLVs to send is under control of network management.
39 Information received from remote LLDP agents is stored in the LLDP remote systems information
40 repository.

1 6.5 Transmission principles

2 A transmission cycle can be initiated either by the expiration of a transmit countdown timing counter or by a
3 change in the value of one or more of the information elements (managed objects) associated with the local
4 system. When a transmit cycle is initiated, the LLDP management entity extracts the managed objects from
5 the LLDP local system information repository and formats this information into TLVs. The TLVs are
6 inserted into an LLDPDU that is passed to the LLDP transmit module. When Extended LLDP is supported,
7 some of the TLVs are inserted into one or more Extension LLDPDUs that are passed to the LLDP transmit
8 module when requested by the receipt of a Extension Request LLDPDU. The LLDP transmit module
9 prepends addressing parameters to the LLDPDU as defined in 8.2, 8.3, and 8.4. The LLDPDU and TLV
10 formats are defined in Clause 8. The LLDP transmit state machine is described in 9.2.1 and 9.2.9.

11 NOTE 1—Because a transmission cycle can be initiated whenever a change occurs within the LLDP local system
12 information repository, it is possible that a series of successive changes over a short period of time could trigger a
13 number of LLDP frames to be sent, each reporting only a single change. LLDP utilizes a transmission delay timer that
14 can be set by network management to limit the number of LLDP transmission cycles in a given time period.

15 NOTE 2—Under normal circumstances, the information in the receiving LLDP agent's remote systems information
16 repository is refreshed periodically to avoid being discarded due to ageing. To prevent the receiving LLDP agent's
17 remote systems information repository being aged out because a refresh frame has been lost in transmission, the sending
18 LLDP agent typically sets the TTL value so that several refresh cycles can occur before the received repository
19 information ages out.

20 LLDP can disclose information about a device and network that could be considered sensitive in certain
21 environments. Implementers are encouraged to provide controls that take into account the link protection
22 characteristics when including information in an LLDP message. Different information can be included if an
23 LLDP message is being sent on the uncontrolled port, the controlled port, or a MACsec protected
24 connectivity association.

25 6.6 Reception principles

26 The LLDP receive module uses the services of the LLC entity to recognize that the incoming
27 MA_UNITDATA.indication contains the correct combination of destination address and MSDU header
28 values to identify it as resulting from a received LLDPDU. The LLDPDU recognition procedures are
29 defined in 7.4 and 9.2.8.7.

30 6.6.1 LLDPDU and TLV error handling

31 The LLDPDU is checked to verify that it contains the correct sequence of three mandatory TLVs at the
32 beginning of the frame, as specified in 8.2, and then each optional TLV is validated in succession.
33 LLDPDUs that contain detectable errors in the first three mandatory TLVs are discarded. Optional TLVs that
34 contain detectable errors are discarded [see 9.2.8.7.2 c)]. TLVs that are not recognized, but that also contain
35 no basic format errors, are assumed to be valid and are stored for possible later retrieval by network
36 management (see 9.2.8.7.1 and 9.2.8.5). If the End Of LLDPDU TLV is present, any octets that follow it are
37 discarded.

38 TLVs in which the information string length field contains a value that is greater than the sum of the lengths
39 of the fields within the information string are not discarded.

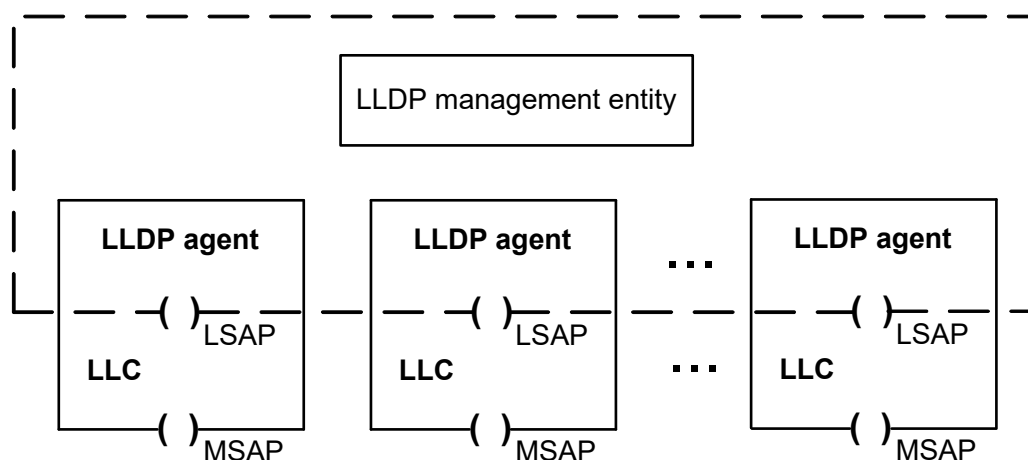
40 NOTE—This approach allows later versions of a TLV to define additional fields at the end of the information string;
41 implementations based on an earlier version of the TLV can therefore continue to process the fields that were defined in
42 that version and ignore any new fields added at the end of the information string in later versions. Any information
43 contained in such new fields is not stored in the information repository and made available to network management
44 protocols via the standard MIB or YANG modules.

6.6.2 LLDP remote systems information repository update

The LLDP remote systems information repository is updated after all TLVs have been validated. LLDP remote system information repository update procedures are defined in 9.2.8.9, 9.2.8.5, and 9.2.8.4. The LLDP receive state machine is described in 9.2.1 and 9.2.10.

6.7 Systems with multiple LLDP Agents

Each LLDP agent advertises a single set of information in the various TLVs it encodes in each transmission cycle, and is associated with the MSAP that supports the LLC entity that the LLDP agent uses to transmit and receive. Each LLDP agent uses its LSAP directly, without the use of any additional multiplexing or addressing above the LSAP to support the use of that LSAP by multiple LLDP agents; and each LLC entity provides service to one and only one protocol entity at each of its LSAPs that it supports, using the service provided by a single MSAP. It follows that each LLDP agent makes use of a unique MSAP, and that the LLDP agent can be uniquely identified by the receiving agent using the MSAP's identifier and the MAC address that determines the LLDP agent's address scope, as specified in 6.1. A single LLDP management entity can support the operation of multiple LLDP agents within the same system. Figure 6-2 illustrates the relationship between the LLDP agents, LLC Entities, MSAPs, and the LLDP management entity.



LLDP - Link Layer Discovery Protocol
LSAP - Link service access point
MSAP - MAC service access point

Figure 6-2—Relationship between LLDP agents, LLC Entities, MSAPs, and the LLDP management entity

A given system can require the use of multiple LLDP agents because it naturally comprises multiple MSAPs, or because it needs to be able to transmit different information (different sets of TLVs) to different sets of peers. These sets of peers can be determined by the way that the MAC service is supported within the system (see IEEE Std 802.1AC for a description of the connectionless connectivity that characterizes an IEEE 802 LAN). A MAC Bridge (see IEEE Std 802.1Q), for example, attaches to multiple LANs, each providing the MAC service at a distinct MSAP. A different example is a station that uses port-based network access control (IEEE Std 802.1X) to provide both a Controlled Port and an Uncontrolled Port, i.e., two distinct MSAPs with potentially different connectivity, for transmission and reception to and from a single LAN. Similarly, different destination addresses can be associated with different sets of potential recipients. Table 7-1 identifies group MAC addresses that can be used for LLDPDU transmission, and 7.1 describes the different transmission scopes associated with each address. LLDP can also be used in conjunction with

individual MAC addresses, and with other group MAC addresses. Distinct LLDP agents, and hence separate and distinct protocol state machines, local and remote information repository tables, and MSAPs are maintained for each LLDPDU destination address, even if the use of two or more MSAPs can result in transmission and reception on the same LAN.

NOTE—For a given MAC address that is used as a destination address in received LLDPDUs, there is the possibility for a station to receive LLDPDUs from multiple senders. All LLDPDUs that carry that destination MAC address are received by a single LLDP agent, via a single MSAP; however, as the LLDPDU carries information that identifies the source of the LLDPDU, information carried in LLDPDUs from different senders is able to be recorded independently in the remote systems information repository.

Figure 6-3 illustrates the use of LLDP in a MAC Bridge, with an LLDP agent for each of the two ports shown. Figure 6-4 shows the use of LLDP in an end system with port-based network access control (IEEE Std 802.1X) supported by MACsec (IEEE Std 802.1AE); an LLDP agent is supported by the Controlled Port and an additional LLDP agent may be supported by the Uncontrolled Port. Figure 6-5 shows LLDP in a MAC Bridge that uses port-based network access control on both ports.

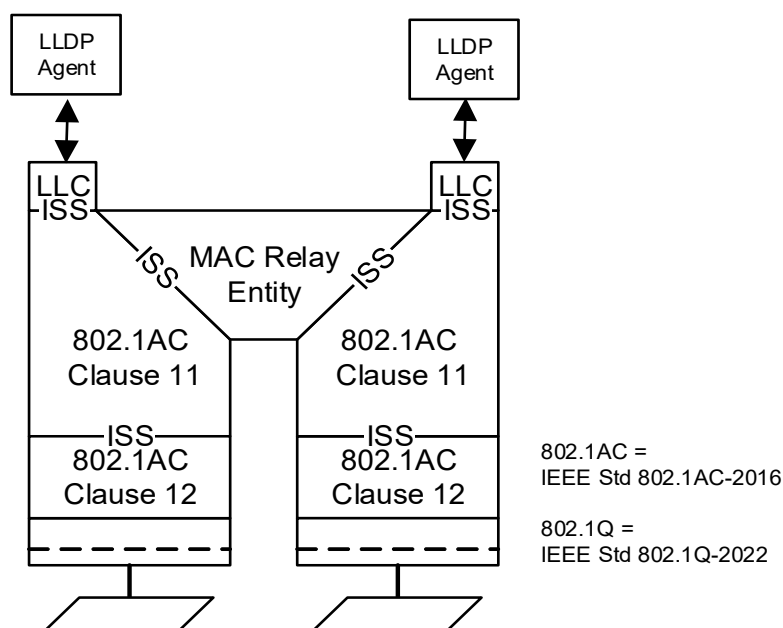


Figure 6-3—LLDP in a MAC Bridge

LLDP frames sent on an unprotected link may be modified by an attacker. If an implementation is going to take action based upon a received LLDP attribute, it is advisable to take into account whether the message was received on a protected link, and allow for the system to be configured such that only information received in protected messages is acted upon.

The group addresses identified in Table 7-1 are taken from the set specified as reserved addresses by bridging standards. Frames (including those conveying LLDPDUs) that use these destination addresses are filtered or not by the various types of bridge components, as specified by Table 8-1 through Table 8-3 of IEEE Std 802.1Q-2022. The choice of a particular address thus allows a given agent to limit the propagation of LLDPDUs to an individual LAN, or to allow them a wider scope. Figure 6-6 illustrates the various scopes achievable with the three destination group MAC addresses identified in Table 7-1, their use is specified further in 7.1.

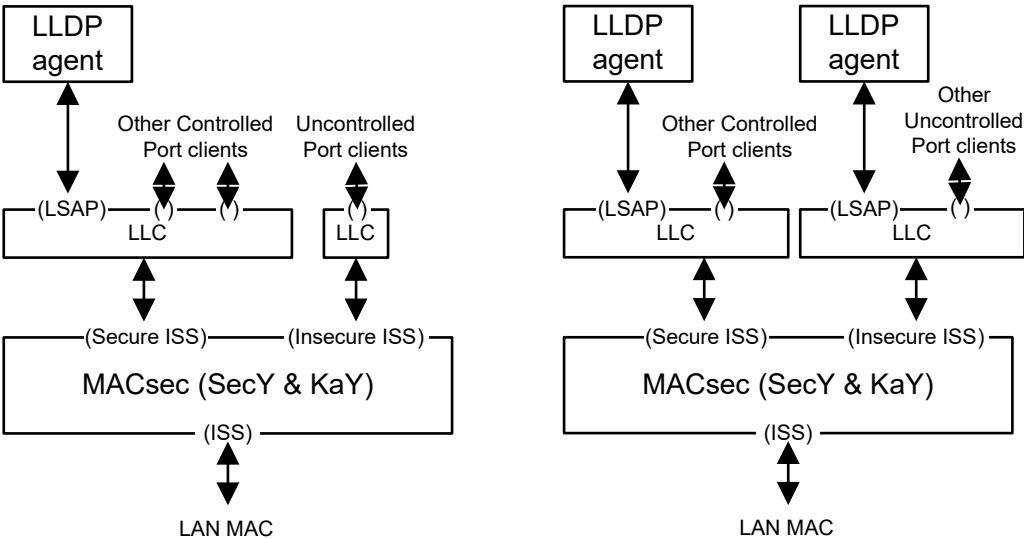


Figure 6-4—LLDP in an end system with port-based network access control

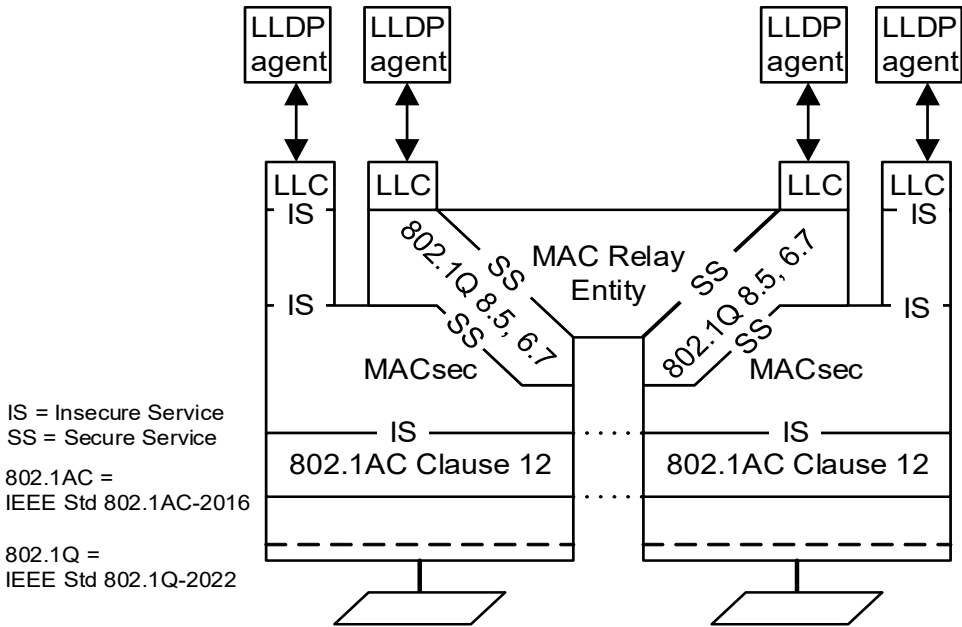


Figure 6-5—LLDP in a MAC Bridge that uses port-based network access control on both ports

1 More than one LLDP agent, each using a different address scope, can be instantiated for a given system port
2 by adding a simple shim that provides the necessary distinct MSAPs by multiplexing and demultiplexing
3 between those MSAPs and a common MSAP for the port, as illustrated in Figure 6-7. Each MSAP shown in

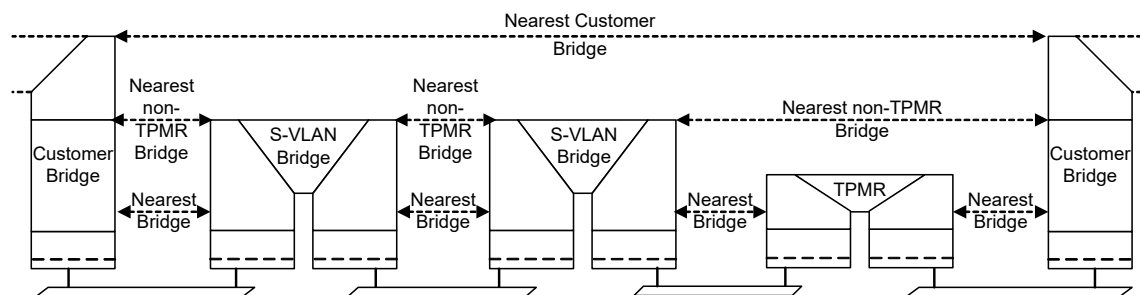


Figure 6-6—Scope of group MAC addresses

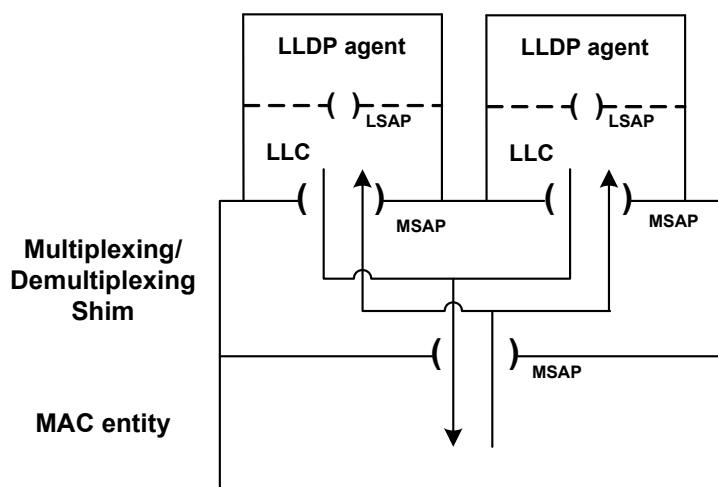


Figure 6-7—Multiplexing and demultiplexing using shims

1 Figure 6-7 is allowed to share the same MSAP Identifier and individual MAC address. The LLDP agents are
2 differentiated by address scope used, i.e., by the MAC address used as a Destination Address in frames
3 containing a Normal LLDPDU.

4 6.8 LLDP and Link Aggregation

5 An LLDP agent can be configured on an IEEE 802.1AX™ Aggregated Port and/or on any number of the
6 physical ports belonging to the Aggregation. Some TLVs (e.g., the Energy-Efficient Ethernet (EEE) TLV
7 designated in subclause 79.3.5 of IEEE Std 802.3-2022) only make sense when transmitted from an LLDP
8 agent configured on a physical port, and can cause incorrect operation if configured on an Aggregation.
9 Other TLVs (e.g., the DCBX TLVs defined in 38.4 of IEEE Std 802.1Q-2022) make sense only when
10 transmitted from an Aggregated Port, because they carry information applicable to the simulated single
11 interface represented by the Aggregation. Standards that define LLDP TLVs should state whether each TLV
12 is applicable to an Aggregation, to a physical port, or to both. Since not all standards that define LLDP TLVs
13 have been careful to state whether each TLV is applicable to an Aggregation, to a physical port, or to both
14 users should be careful to use LLDP TLVs that are appropriate for transmission on a given Port.

1 An LLDP agent configured on an Aggregated Port, or on one of the physical ports of an Aggregation,
2 transmits the Link Aggregation TLV, and processes received LLDPDUs, as specified in F of 802.1AX-2020.
3 An LLDP agent configured on a physical port of an Aggregation uses the LLDP Parser/Multiplexer as
4 specified in 6.1.3 of IEEE Std 802.1AX-2020 for the transmission and reception of LLDPDUs containing
5 TLVs that are specific to that physical port. Since a port not running Link Aggregation is equivalent to an
6 Aggregation with a single physical port, any LLDP agent can transmit the Link Aggregation TLV and
7 process received LLDPDUs as specified in 802.1AX.

8 NOTE—The consideration of aggregated vs. Physical ports is similar to the distinction among destination MAC
9 addresses discussed in 7.1. In both cases, LLDP TLVs that carry information specific to a physical link should be used
10 only in the LLDP agent with the shortest reach.

11 6.9 Extended LLDP (XLLDP)

12 Extended LLDP allows an LLDP agent to advertise more TLVs than would fit within a single LLDPDU by
13 transmitting the additional TLVs in one or more Extension LLDPDUs. The selection of TLVs to be included
14 in LLDPDUs is controlled by management, but the mapping of TLVs to the initial Normal LLDPDU or to an
15 Extension LLDPDU is implementation dependent (10.2.2). When a transmission cycle is initiated, a unique
16 description of each Extension LLDPDU is encoded in a Manifest TLV, which is then included in the Normal
17 LLDPDU transmitted at the beginning of the cycle. A Normal LLDPDU containing a Manifest TLV is
18 referred to as a Manifest LLDPDU. Changes to any of the TLVs in an Extension LLDPDU changes the
19 description included in the Manifest TLV, and thus triggers a new transmission cycle.

20 An implementation not supporting XLLDP that receives a Manifest TLV treats it as an unrecognized TLV
21 and only stores the TLVs that were included in the Normal LLDPDU. An implementation supporting
22 XLLDP that receives a Manifest TLV then requests the transmission of any new or modified Extension
23 LLDPDUs by sending an Extension Request LLDPDU to the LLDP agent sourcing the Manifest LLDPDU.
24 One or more Extension LLDPDUs can be requested in a single Extension Request LLDPDU. A new
25 Extension Request LLDPDU can be transmitted when all Extension LLDPDUs previously requested have
26 been received. This allows the requesting LLDP agent to pace the rate of information transfer. A timer on the
27 request allows an Extension Request LLDPDU to be retried if it appears that either the request or a response
28 has been lost.

29 Extension Request LLDPDUs and Extension LLDPDUs are transmitted with an individual MAC address as
30 the destination address. The destination address in an Extension Request LLDPDU is taken from the
31 contents of the Return MAC Address field of the received Manifest TLV. The requested Extension
32 LLDPDUs are transmitted with a destination address taken from the contents the Return MAC Address field
33 of the Extension Request TLV.

7. LLDPDU transmission, reception, and addressing

This standard is intended to be compatible with all IEEE 802 MACs.

LLDP uses the service provided by the LLDP/LSAP and LLC to transmit and receive LLDPDUs. Each LLDPDU is transmitted as a single MAC service request by an LLC entity that uses a single instance of the MAC Service provided at an MSAP. Each incoming LLDP frame is received at the MSAP by the LLC entity as a MAC service indication.

NOTE 1—For the purposes of this standard, the terms “LLC” and “LLC entity” include the service provided by the operation of entities that support either Type 3 Protocol Identification Field (PIF) encoding or the use of SNAP encoding of an EtherType, as specified in IEEE Std 802. Hence, “LSAP” in the above description can refer to the use of ISO/IEC 8802-2 [B15] LLC addresses, or an EtherType, as a means of higher layer protocol identification.

The parameters of each service request and service indication comprise

- a) Destination address
- b) Source address
- c) EtherType
- d) LLDPDU

The LLDP EtherType, used to identify the LLDP protocol, is prepended to the LLDPDU as shown in Figure 7-1 to form the MSDU of the corresponding MAC service request.



Figure 7-1—MSDU format

The values of the parameters used by LLDP, and their encoding by the LLC entity that supports the LLDP LSAP, are specified in the following subclauses.

7.1 Destination address

7.1.1 Destination address for Normal LLDPDUs

The MAC address used as the destination address in Normal LLDPDUs is referred to as the Scope MAC Address because it determines the limits of propagation of the LLDPDU within a network. A set of standard group MAC addresses that can be used by LLDP as the Scope MAC Address is specified in Table 7-1. These addresses are in the range of IEEE Std 802.1Q C-VLAN and MAC Bridge component Reserved addresses (see Table 8-1 in IEEE Std 802.1Q-2022), the S-VLAN component Reserved addresses (see Table 8-2 in IEEE Std 802.1Q-2022), and the TPMR component Reserved addresses (see Table 8-3 in IEEE Std 802.1Q-2022). The choice of address used determines the scope of propagation of LLDPDUs within a bridged LAN, as follows:

- a) The **nearest bridge** group MAC address is an address that no conformant Two-Port MAC Relay (TPMR) component, S-VLAN component, C-VLAN component, or MAC Bridge component can forward. LLDPDUs transmitted using this destination address can therefore travel no further than those stations that can be reached via a single individual LAN from the originating station.

1 LLDPDUs received on this address therefore represent information about stations that are attached
2 to the same individual LAN segment as the recipient station.

3 NOTE 1—This address was selected in order to make it possible to transmit an LLDP frame containing information
4 specific to a single individual LAN, and for that information not to be propagated further than the extent of that
5 individual LAN, to avoid the meaning of that information being misinterpreted. For example, a TLV containing the
6 auto-negotiation status of an IEEE 802.3™ LAN would be open to misinterpretation if it were to be propagated via a
7 Bridge onto a second IEEE 802.3 LAN, and would be meaningless if propagated onto a LAN based on a different MAC
8 technology. This is the “LLDP_Multicast address” that was used in IEEE Std 802.1AB-2005.

9 b) The **nearest non-TPMR bridge** group MAC address is an address that no conformant C-VLAN
10 component, S-VLAN component, or MAC Bridge can forward. However, this address is relayed by
11 TPMR components. Therefore, LLDPDUs received on this address represent information about
12 stations that are not separated from the recipient station by any intervening C-VLAN component,
13 S-VLAN component, or MAC Bridge. There may, however, be one or more TPMR components in
14 the path between the originating and receiving stations.

15 NOTE 2—This address was selected in order to make it possible to communicate information between adjacent bridges
16 and for that communication to be transparent to the presence or absence of TPMRs in the transmission path. This address
17 is primarily intended for use within provider bridged networks.

18 c) The **nearest Customer Bridge** group MAC address is an address that no conformant C-VLAN
19 component or MAC Bridge forwards. However, this address is relayed by TPMR components and
20 S-VLAN components. Therefore, LLDPDUs received on this address represent information about
21 stations that are not separated from the recipient station by any intervening C-VLAN components
22 (e.g., a C-VLAN Bridge or Provider Edge Bridge). There may, however, be TPMR components
23 and/or S-VLAN components in the path between the originating and receiving stations.

24 NOTE 3—This address was selected in order to make it possible to communicate information between adjacent
25 Customer Bridges and for that communication to be transparent to the presence or absence of TPMRs or S-VLAN
26 components in the communication path. The scope of this address is the same as that of a customer-to-customer
27 MACSec connection.

Table 7-1—Group MAC addresses used by LLDP

Name	Value	Purpose
<i>Nearest bridge</i>	01-80-C2-00-00-0E	Propagation constrained to a single physical link; stopped by all types of bridge
<i>Nearest non-TPMR bridge</i>	01-80-C2-00-00-03	Propagation constrained by all bridges other than TPMRs; intended for use within provider bridged networks
<i>Nearest Customer Bridge</i>	01-80-C2-00-00-00	Propagation constrained by customer bridges; this gives the same coverage as a customer-customer MACSec connection

28 NOTE 4—The LSAP or EtherType field is necessary to distinguish between different protocols conveyed in frames with
29 the same group or individual destination address. Prior to revision of this standard to allow multiple address scopes, the
30 destination address 01-80-C2-00-00-00 was only explicitly specified for Spanning Tree Protocol BPDUs. It is therefore
31 possible that some implementations recognize only a frame’s destination address before applying priority handling
32 resources intended to guard against BPDU loss. Since LLDP rate limits LLDPDU transmission, the additional
33 consumption of these resources by LLDPDUs is unlikely to pose a problem for robust implementations.

34 It is assumed in the foregoing definitions of the scope of these group MAC addresses that an end station
35 attached to an individual LAN is a potential recipient of any of these addresses, and therefore such end
36 stations fall within the scope of these addresses as well as the identified bridge components.

1 In addition to determining the scope of transmission, the support of more than one destination MAC address
2 allows different sets of TLVs to be supported over the different transmission scopes.

3 In addition to the prescribed support for standard group MAC addresses shown in Table 7-1,
4 implementations of LLDP may support the following as the Scope MAC Address:

- 5 d) Any group MAC address.
- 6 e) Any individual MAC address.

7 Support for the use of each of these destination addresses, for both transmission and reception of Normal
8 LLDPDUs, is either mandatory, recommended, permitted, or not permitted, according to the type of system
9 in which LLDP is implemented, as shown in Table 7-2.

Table 7-2—Support for the Scope MAC Address in different systems

Address	C-VLAN Bridge	S-VLAN Bridge	TPMR Bridge	End station
<i>Nearest bridge</i>	Mandatory	Mandatory	Mandatory	Mandatory
<i>Nearest non-TPMR bridge</i>	Mandatory	Mandatory	Not permitted	Recommended
<i>Nearest Customer Bridge</i>	Mandatory	Not permitted	Not permitted	Recommended
<i>Any other group MAC address</i>	Permitted	Permitted	Permitted	Permitted
<i>Any individual MAC address</i>	Permitted	Permitted	Permitted	Permitted

10 NOTE 5 —Where the implementation supports the use of individual MAC addresses, there is a need for some means
11 whereby the sender can ascertain what address to use, and what scope is associated with that address; how this is
12 achieved is outside the scope of this standard.

13 NOTE 6—The option of using an individual MAC addresses supports the use of LLDP in IEEE Std 802.11™ [B1] and
14 other wireless LANs, where it is desirable to target specific TLV content at specific logical Ports. In particular, the use of
15 an individual MAC address allows an access point to direct LLDP frames at an individual station with which it is
16 associated.

17 NOTE 7—If an individual MAC address is specified as the Scope MAC Address, it is used as the destination for
18 transmitted LLDPDUs and also used to validate received LLDPDUs. The primary purpose of using an individual
19 address as the Scope MAC Address is that an LLDP agent configured as “transmit only” sends Normal LLDPDUs to a
20 specific receiving LLDP agent. It is also allowable for an LLDP agent configured as “receive only” to use an individual
21 address (typically the individual MAC address of the LLDP agent’s MSAP) so that only information from an LLDP
22 agent configured as “transmit only” using this same address is collected. Using an individual MAC address as the
23 destination address for an LLDP agent configured to “transmit and receive” is allowed, although it would not typically
24 be useful.

25 NOTE 8—The destination MAC address used by a given LLDP agent defines only the scope of transmission and the
26 intended recipient(s) of the LLDPDUs; it plays no part in protocol identification. In particular, the group MAC addresses
27 identified in Table 7-1 are not used exclusively by LLDP; other protocols that require to use a similar transmission scope
28 are free to use the same addresses.

29 7.1.2 Destination address for Extension Request and Extension LLDPDUs

30 The destination address in an Extension Request LLDPDU or an Extension LLDPDU is an individual
31 address. Extension Request LLDPDUs are sent in response to the receipt of a Manifest LLDPDU, and use
32 the contents of the Return MAC Address field of the Manifest TLV as the destination address of an
33 Extension Request LLDPDU. Likewise, Extension LLDPDUs are sent in response to the receipt of an
34 Extension Request LLDPDU, and use the contents of the Return MAC Address field of the Extension
35 Request TLV as the destination address of the Extension LLDPDU.

7.2 Source address

The source address shall be the individual MAC address of the MSAP supporting the LLDP agent.

7.3 EtherType use and encoding

The EtherType used to identify the LLDP protocol shall be the LLDP EtherType specified in Table 7-3.

Table 7-3—LLDP EtherType

Name	Value
LLDP EtherType	88-CC

The encoding of the LLDP EtherType shall be as specified in IEEE Std 802.1AC.

Where the LLC entity uses an MSAP that is supported by a specific media access control method (for example, IEEE Std 802.3) or a media access control independent entity (for example, IEEE Std 802.1AE) that directly supports encoding of EtherTypes, the LLC entity shall encode the LLDP EtherType as the two octet LLDPDU header in the MSDU of the corresponding MAC service request.

NOTE—Annex C provides example LLDP transmission frame formats for both direct-encoded and SNAP-encoded LLDP EtherType encoding methods.

7.4 LLDPDU reception

The LLDPDU shall be delivered to the LLDP receive module if, and only if

- a) The destination MAC address is equal to the individual MAC address of the MSAP supporting the LLDP agent, or the destination MAC address is equal to the Scope MAC Address associated with the corresponding LLDP agent; and

NOTE—The destination MAC address can be one of the addresses in Table 7-1, or any other valid MAC address—see 7.1.

- b) The EtherType is equal to the value shown in Table 7-3.

1 8. LLDPDU and TLV formats

2 8.1 LLDPDU bit and octet ordering conventions

3 All LLDPDUs contain an integral number of octets. The octets in an LLDPDU are numbered starting from 1
4 and increasing in the order they are put into the LLDPDU. The bits in an octet are numbered from 1 to 8,
5 where bit 1 is the low-order bit.

6 When consecutive bits within an octet are used to represent a binary number, the highest bit number has the
7 most significant value. When consecutive octets are used to represent a binary number, the lower octet
8 number has the most significant value. All TLVs respect these bit and octet ordering conventions.

9 When the encoding of a field or a number of fields is represented using a diagram

- 10 a) Octets are shown with the lowest numbered octet nearest the left of the page, the octet numbering
11 increasing from left to right.
- 12 b) Within an octet, bits are shown with bit 8 to the left and bit 1 to the right.

13 8.2 LLDPDU format

14 The LLDPDU contains the following ordered sequence of three mandatory TLVs followed by zero or more
15 optional TLVs as shown in Figure 8-1. An End of LLDPDU TLV may be present as the last TLV in the
16 LLDPDU.

- 17 a) Three mandatory TLVs are included at the beginning of each LLDPDU and are in the order shown.
 - 18 1) Chassis ID TLV
 - 19 2) Port ID TLV
 - 20 3) Time To Live TLV or, if XLLDP is supported, Extension Request TLV or Extension Identifier
21 TLV
- 22 b) Optional TLVs as selected by network management (may be inserted in any order).

23 NOTE 1—"Optional" in the sense that they are not required for LLDP operation; however, their presence could be
24 required by other system elements that use LLDP.

- 25 c) If the End Of LLDPDU TLV is present, it is the last TLV in the LLDPDU.

1

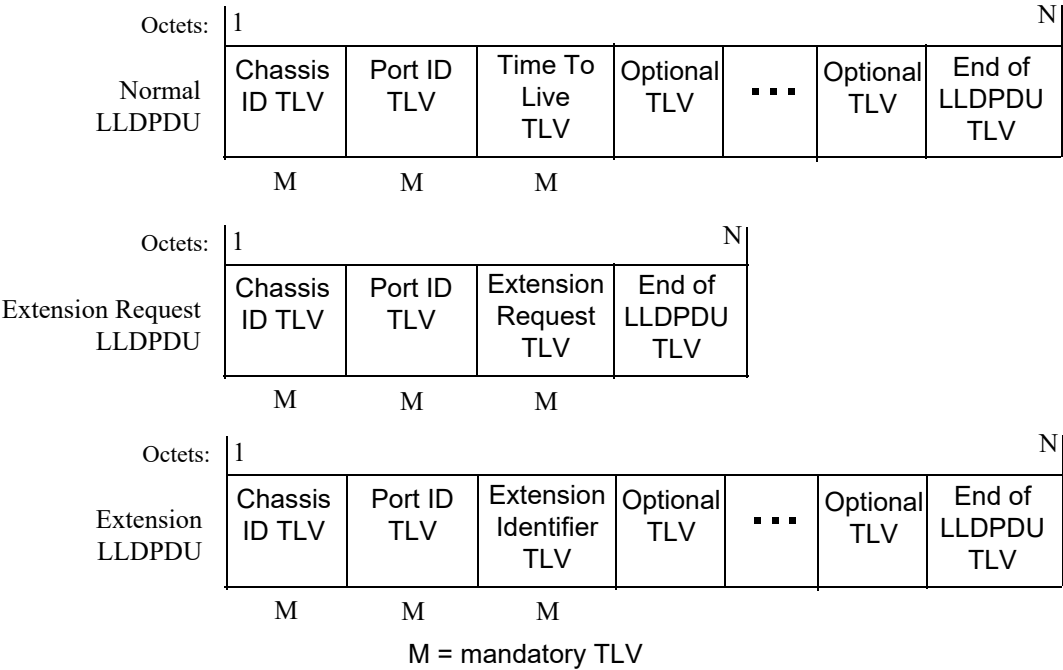


Figure 8-1—LLDPDU formats

2 The maximum length of the LLDPDU is less than or equal to the maximum information field length allowed
3 by the particular transmission rate and protocol. In IEEE 802.3 MACs, for example, the maximum LLDPDU
4 length is the maximum data field length for the basic, untagged MAC frame (1500 octets).

5 NOTE 2—Implementations can restrict the maximum length of the LLDPDU to a value smaller than what would be
6 allowed by the underlying MAC technology. For example, an end station or bridge implementing Time-Sensitive
7 Networking (TSN) features could restrict the maximum length to limit delay variation when transmitting frames in time
8 sensitive streams.

9 NOTE 3—There is no defined minimum length of an LLDPDU, other than that implied by the requirement that
10 conformant implementations support the mandatory TLVs specified in Table 8-1.

11 **8.3 TLV categories**

12 The general format of LLDP TLVs is defined in 8.4. The TLVs are grouped into three categories as follows:

- 13 a) A set of TLVs that are considered to be basic to the management of network stations. Support of the
14 basic management set is required capability of all LLDP implementations. Each TLV in this
15 category is identified by a unique TLV type value that indicates the particular kind of information
16 contained in the TLV. The specific definitions and usage rules for the basic management set TLVs
17 are defined in 8.5.
- 18 b) A set of TLVs that support Extended LLDP (XLLDP) operation. Support of the extended LLDP set
19 is a required capability of all LLDP implementations supporting XLLDP. Each TLV in this category
20 is identified by a unique TLV type value that indicates the particular kind of information contained
21 in the TLV. The specific definitions and usage rules for the extended LLDP set TLVs are defined in
22 8.6.
- 23 c) Organizationally specific extension sets of TLVs that are defined by standards groups such as
24 IEEE 802.1 and IEEE 802.3 and others to enhance management of network stations that are

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- operating with particular media and/or protocols. The format of Organizationally Specific TLVs, along with field definitions and usage rules common to all Organizationally Specific TLVs, are defined in 8.7.
- 1) TLVs in this category are identified by a common TLV type value that indicates the TLV as belonging to the set of Organizationally Specific TLVs.
- 2) Each organization is identified by its organizationally identifier (OI), consisting of either an Organizationally Unique Identifier (OUI) or a Company ID (CID) ¹².
- 3) Organizationally Specific TLV subtype values indicate the kind of information contained in the TLV.

10

8.4 Basic TLV format

11

Figure 8-2 shows the basic TLV format.

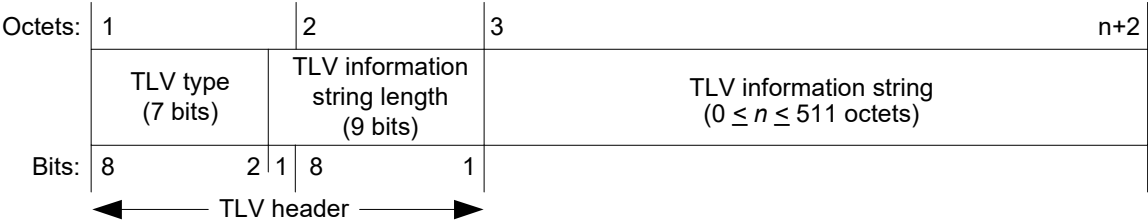


Figure 8-2—Basic TLV format

12

13

14

The TLV type field occupies the seven most significant bits of the first octet of the TLV format. The least significant bit in the first octet of the TLV format is the most significant bit of the TLV information string length field.

15

8.4.1 TLV type

16

The TLV type field is seven bits long and identifies the specific TLV. Two classes of TLVs are defined:

- 17
- 18
- a) Mandatory TLVs that are included in all LLDPDUs.
- b) Optional TLVs that may be included in LLDPDUs.

19

20

Table 8-1 lists the currently defined TLVs, their identifying TLV type values, and whether they are mandatory or optional for inclusion in any particular LLDPDU.

21

8.4.2 TLV information string length

22

The TLV information string length field contain the length of the information string, in octets.

23

8.4.3 TLV information string

24

The information string

- 25
- 26
- 27
- a) May be fixed or variable length.
- b) May include one or more information fields with associated subtype identifiers and field length designators as in, for example, the Management Address TLV (see 8.5.9).

¹² Organizationally Unique Identifier and Company ID assignments are administered by the IEEE Registration Authority, and Organizationally Unique Identifier assignments are included in MAC Address-Large (MA-L) assignments; see <https://standards.ieee.org/products-programs/regauth/>.

Table 8-1—TLV type values

TLV type ^a	TLV name	Usage in LLDPDU	Reference
0	End Of LLDPDU	Optional	8.5.1
1	Chassis ID	Mandatory	8.5.2
2	Port ID	Mandatory	8.5.3
3	Time To Live	Mandatory	8.5.4
4	Port Description	Optional	8.5.5
5	System Name	Optional	8.5.6
6	System Description	Optional	8.5.7
7	System Capabilities	Optional	8.5.8
8	Management Address	Optional	8.5.9
9	Manifest	Optional	8.6.1
10	Extension Request	Optional	8.6.2
11	Extension Identifier	Optional	8.6.3
12–126	Reserved for future standardization	—	—
127	Organizationally Specific TLVs	Optional	8.7

^aTLVs with type values 0–8 are members of the basic management set. TLVs with type values 9–11 are members of the extended LLDP set.

- 1 c) May contain either binary or alpha-numeric information that is instance specific for the particular
- 2 TLV type and/or subtype.
- 3 1) Bit 1 in binary bit maps is the least significant bit in the field.
- 4 2) The first octet of an alpha-numeric field is the most significant octet.
- 5 3) Alpha-numeric information is encoded in UTF-8 [IETF RFC 3629].

1 **8.5 Basic management TLV set formats and definitions**

2 **8.5.1 End Of LLDPDU TLV**

3 The End Of LLDPDU TLV is a 2-octet, all-zero TLV that is used to mark the end of the TLV sequence in
4 LLDPDUs. The format for this TLV is shown in Figure 8-3.

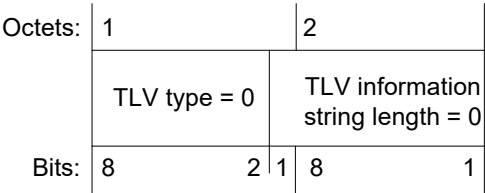


Figure 8-3—End Of LLDPDU TLV format

5 NOTE—Some IEEE 802 MACs require the data field in a frame to contain a minimum number of octets. For example,
6 the IEEE 802.3 MAC adds pad octets to complete a minimum length data field if the user’s data is less than the
7 minimum required length. Since pad octets are unspecified, an End Of LLDPDU TLV can be used to prevent non-zero
8 pad octets from being interpreted by the receiving LLDP agent as another TLV.

9 **8.5.2 Chassis ID TLV**

10 The Chassis ID TLV is a mandatory TLV that identifies the chassis containing the IEEE 802 LAN station
11 associated with the MSAP Identifier associated with the LLDP agent containing the LLDP local system
12 information repository that is the source of the information in TLVs. There are several ways in which a
13 chassis may be identified and a chassis ID subtype is used to indicate the type of component being
14 referenced by the chassis ID field. Each LLDPDU contain one, and only one, Chassis ID TLV and the
15 chassis ID field value remains constant for all LLDPDUs while the MAC status parameter for the Port
16 remains operational.

17 The Chassis ID TLV is the first TLV in the LLDPDU. Its format is shown in Figure 8-4.

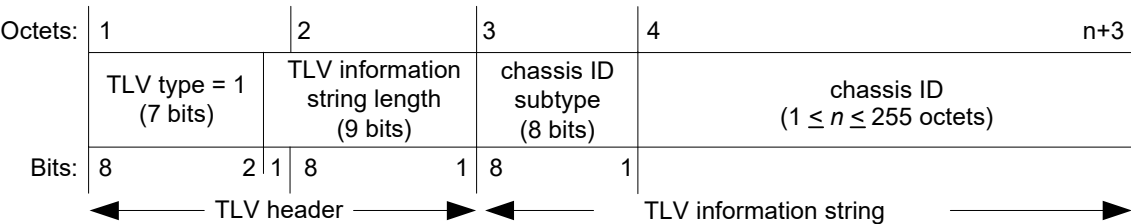


Figure 8-4—Chassis ID TLV format

18 **8.5.2.1 TLV information string length**

19 The TLV information string length field indicates the exact length, in octets, of the (chassis ID subtype +
20 chassis ID) fields.

21 **8.5.2.2 chassis ID subtype**

22 The chassis ID subtype field contains an integer value indicating the basis for the chassis ID entity that is
23 listed in the chassis ID field. The defined chassis ID subtypes and their preferred use order are listed in Table
24 8-2.

Table 8-2—chassis ID subtype enumeration

ID subtype	ID basis	Reference
0	Reserved	—
1	Chassis component	EntPhysicalAlias when entPhysClass has a value of ‘chassis(3)’ (IETF RFC 6933)
2	Interface alias	IfAlias (IETF RFC 2863)
3	Port component	EntPhysicalAlias when entPhysicalClass has a value ‘port(10)’ or ‘backplane(4)’ (IETF RFC 6933)
4	MAC address	MAC address (IEEE Std 802)
5	Network address	networkAddress ^a
6	Interface name	ifName (IETF RFC 2863)
7	Locally assigned	local ^b
8–255	Reserved	—

^anetworkAddress is an octet string that identifies a particular network address family and an associated network address that are encoded in network octet order. An IP address, for example, would be encoded with the first octet containing the IANA Address Family Numbers enumeration value for the specific address type and octets 2 through *n* containing the address value (for example, the encoding for C0-00-02-0A would indicate the IPv4 address 192.0.2.10).

^blocal is an alpha-numeric string and is locally assigned.

1 8.5.2.3 chassis ID

2 The chassis ID field contains an octet string indicating the specific identifier for the particular chassis in this
3 system. Because chassis ID and port ID values are concatenated to form the local MSAP identifier, the value
4 chosen from Table 8-2 for the chassis ID is non-null.

5 8.5.2.4 Chassis ID TLV usage rules

6 An LLDPDU contains exactly one Chassis ID TLV.

7 8.5.3 Port ID TLV

8 The Port ID TLV is a mandatory TLV that identifies the port component of the MSAP identifier associated
9 with the MSAP Identifier associated with the LLDP agent containing the LLDP local system information
10 repository that is the source of the information in TLVs. As with the chassis, there are several ways in which
11 a port is identified. A port ID subtype is used to indicate how the port is being referenced in the port ID field.
12 Each LLDPDU contain one, and only one, Port ID TLV. The port ID value remain constant for all
13 LLDPDUs while the transmitting port remains operational.

14 The chosen port is identified in an unambiguous manner (for example, backplane number, management
15 entity, MAC address).

16 The Port ID TLV is the second TLV in the LLDPDU. Its format is shown in Figure 8-5.

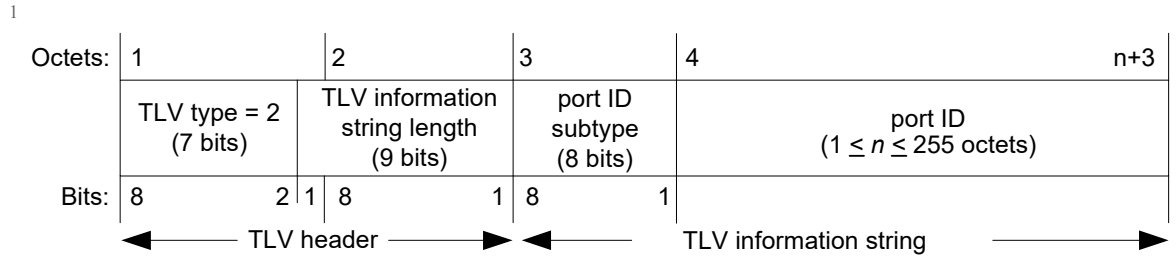


Figure 8-5—Port ID TLV format

2 **8.5.3.1 TLV information string length**

3 The TLV information string length field indicates the length, in octets, of the (port ID subtype + port ID)
4 fields.

5 **8.5.3.2 port ID subtype**

6 The port ID subtype field contains an integer value indicating the basis for the identifier that is listed in the
7 port ID field. The defined port ID subtypes and their preferred use order are listed in Table 8-3.

Table 8-3—port ID subtype enumeration

ID subtype	ID basis	References
0	Reserved	—
1	Interface alias	ifAlias (IETF RFC 2863)
2	Port component	entPhysicalAlias when entPhysicalClass has a value ‘port(10)’ or ‘backplane(4)’ (IETF RFC 6933)
3	MAC address	MAC address (IEEE Std 802)
4	Network address	networkAddress ^a
5	Interface name	ifName (IETF RFC 2863)
6	Agent circuit ID	agent circuit ID (IETF RFC 3046)
7	Locally assigned	local ^b
8–255	Reserved	—

^anetworkAddress is an octet string that identifies a particular network address family and an associated network address that are encoded in network octet order. An IP address, for example, would be encoded with the first octet containing the IANA Address Family Numbers enumeration value for the specific address type and octets 2 through n containing the address value (for example, the encoding for C0-00-02-0A would indicate the IP version 4 address 192.0.2.10).

^blocal is an alpha-numeric string and is locally assigned.

8 **8.5.3.3 port ID**

9 The port ID field is an octet string that contains the specific identifier for the port from which this LLDPDU
10 was transmitted. Because chassis ID and port ID values are concatenated to form the local MSAP identifier,
11 the value chosen from Table 8-3 for the port ID is non-null.

12 NOTE—The port ID can contain alphanumeric data or binary data, depending upon the value of the port ID subtype.

1 8.5.3.4 Port ID TLV usage rules

² An LLDPDU contains exactly one Port ID TLV.

3 8.5.4 Time To Live TLV

4 The Time To Live TLV indicates the number of seconds that the recipient LLDP agent is to regard the
5 information associated with this MSAP identifier to be valid.

- 6 a) When the TTL field is non-zero the receiving LLDP agent is notified to completely replace all
7 information associated with this MSAP identifier with the information in the received LLDPDU.
- 8 b) When the TTL field is set to zero, the receiving LLDP agent is notified to delete all system
9 information associated with the LLDP agent/port. This TLV may be used, for example, to signal that
10 the sending port has initiated a port shutdown procedure.

11 The Time To Live TLV is mandatory for all Normal LLDPDUs, and is the third TLV in the LLDPDU. Its
12 format is shown in Figure 8-6.

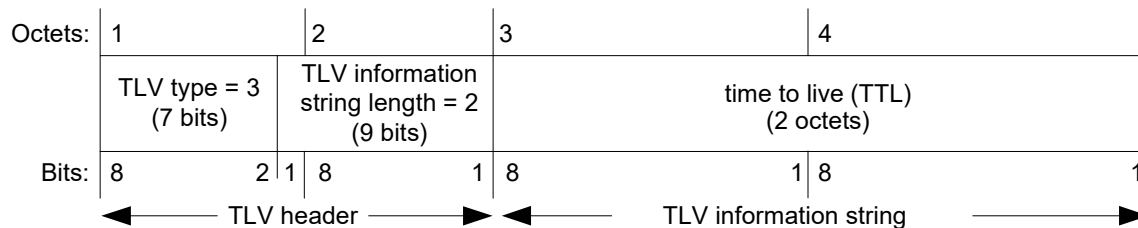


Figure 8-6—Time To Live TLV format

13 8.5.4.1 time to live (TTL)

14 The TTL field contains an integer value in the range $0 \leq t \leq 65535$ seconds and is set to the computed value
15 of txTTL at the time the LLDPDU is constructed (see 9.2.5.22).

16 8.5.4.2 Time To Live TLV usage rules

17 A Normal LLDPDU contains exactly one Time To Live TLV as the third TLV, and does not contain more
18 than one Time To Live TLV. A Time To Live TLV is not contained in an Extension LLDPDU or an
19 Extension Request LLDPDU.

20 8.5.5 Port Description TLV

21 The Port Description TLV allows network management to advertise the IEEE 802 LAN station's port
22 description. The format for this TLV is shown in Figure 8-7.

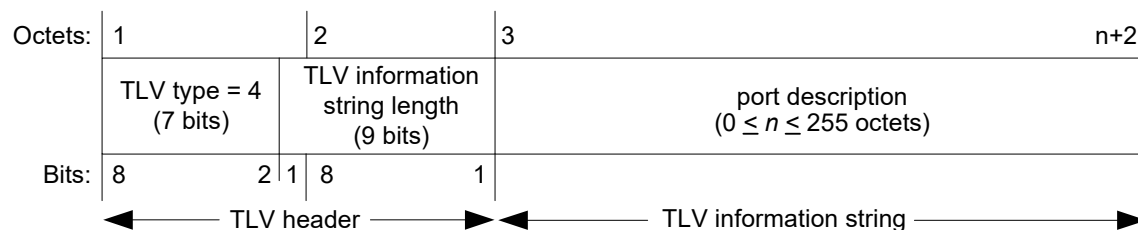


Figure 8-7—Port Description TLV format

1 **8.5.5.1 TLV information string length**

2 The TLV information string length field contains the exact length, in octets, of the port description field.

3 **8.5.5.2 port description**

4 The port description field contains an alpha-numeric string that indicates the port’s description. If
5 IETF RFC 2863 is implemented, the ifDescr object should be used for this field.

6 **8.5.5.3 Port Description TLV usage rules**

7 An LLDPDU should not contain more than one Port Description TLV.

8 **8.5.6 System Name TLV**

9 The System Name TLV allows network management to advertise the system’s assigned name. The format
10 for this TLV is shown in Figure 8-8.

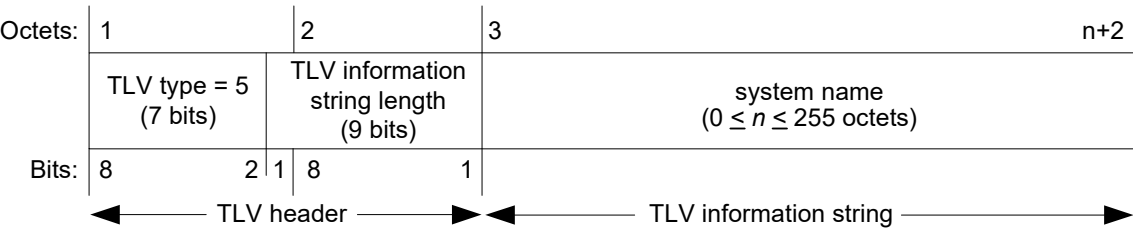


Figure 8-8—System Name TLV format

11 **8.5.6.1 TLV information string length**

12 The TLV information string length field contains the exact length, in octets, of the system name.

13 **8.5.6.2 system name**

14 The system name field contains an alpha-numeric string that indicates the system’s administratively
15 assigned name. The system name should be the system’s fully qualified domain name. If implementations
16 support IETF RFC 3418, the sysName object should be used for this field.

17 **8.5.6.3 System Name TLV usage rules**

18 An LLDPDU does not contain more than one System Name TLV.

1 **8.5.7 System Description TLV**

2 The System Description TLV allows network management to advertise the system’s description. The format
3 for this TLV is shown in Figure 8-9.

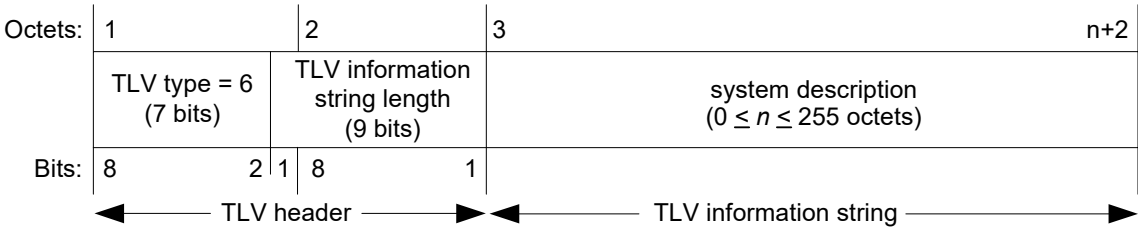


Figure 8-9—System Description TLV format

4 **8.5.7.1 TLV information string length**

5 The TLV information string length field indicates the exact length, in octets, of the system description.

6 **8.5.7.2 system description**

7 The system description field contains an alpha-numeric string that is the textual description of the network
8 entity. The system description should include the full name and version identification of the system’s
9 hardware type, software operating system, and networking software. If implementations support IETF RFC
10 3418, the sysDescr object should be used for this field.

11 **8.5.7.3 System Description TLV usage rules**

12 An LLDPDU does not contain more than one System Description TLV.

13 **8.5.8 System Capabilities TLV**

14 The System Capabilities TLV is an optional TLV that identifies the primary function(s) of the system and
15 whether or not these primary functions are enabled. Figure 8-10 shows the format of this TLV.

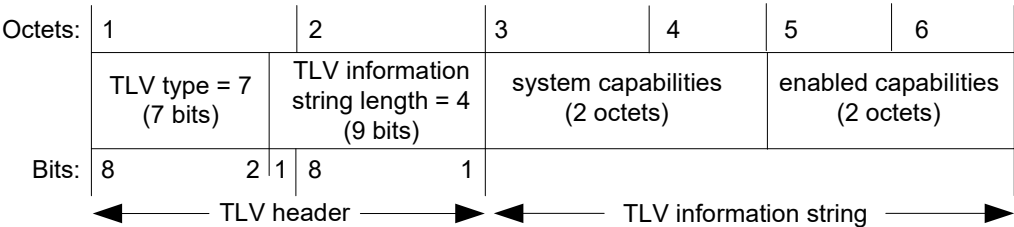


Figure 8-10—System Capabilities TLV format

16 **8.5.8.1 system capabilities**

17 The system capabilities field contains a bit-map of the capabilities that define the primary function(s) of the
18 system. The bit positions for each function and the associated information repository or standard that are
19 likely, but not guaranteed, to be supported are listed in Table 8-4. A binary one in the associated bit indicates
20 the existence of that capability. Individual systems may indicate more than one implemented functional
21 capability (for example, both a bridge and router capability).

Table 8-4—System capabilities

Bit	Capability	Reference
1	Other	—
2	Repeater	IETF RFC 2108 [B6]
3	MAC Bridge component	IEEE Std 802.1Q
4	802.11 Access Point (AP)	IEEE Std 802.11 MIB
5	Router	IETF RFC 1812 [B5]
6	Telephone	IETF RFC 4293 [B9]
7	DOCSIS cable device	IETF RFC 4639 and IETF RFC 4546 [B11]
8	Station Only ^a	IETF RFC 4293 [B9]
9	C-VLAN component	IEEE Std 802.1Q
10	S-VLAN component	IEEE Std 802.1Q
11	Two-port MAC Relay component	IEEE Std 802.1Q
12–16	reserved	—

^aThe Station Only capability is intended for devices that implement only an end station capability, and for which none of the other capabilities in the table apply. Bit 8 should therefore not be set in conjunction with any other bits.

1 8.5.8.2 enabled capabilities

2 The enabled capabilities field contains a bit map of the primary functions listed in Table 8-4. A binary one in
3 a bit position indicates that the function associated with that bit is currently enabled.

4 8.5.8.3 System Capabilities TLV usage rules

5 An LLDPDU does not contain more than one System Capabilities TLV.

6 If the system capabilities field does not indicate the existence of a capability that the enabled capabilities
7 field indicates is enabled, the TLV is interpreted as containing an error and is discarded.

8 8.5.9 Management Address TLV

9 The Management Address TLV identifies an address associated with the local LLDP agent that may be used
10 to reach higher layer entities to assist discovery by network management. The TLV also provides room for
11 the inclusion of both the system interface number and an object identifier (OID) that are associated with this
12 management address, if either or both are known. Figure 8-11 shows the Management Address TLV format.

1

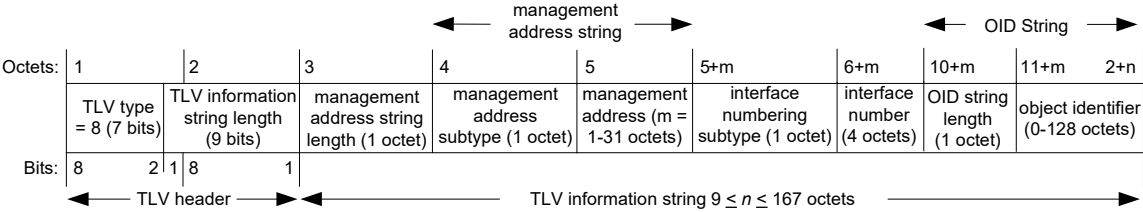


Figure 8-11—Management Address TLV format

2 **8.5.9.1 TLV information string length**

3 The TLV information string length field contains the length, in octets, of all the fields in the TLV
4 information string.

5 **8.5.9.2 management address string length**

6 The management address string length field contains the length, in octets, of the (management address
7 subtype + management address) fields.

8 **8.5.9.3 management address subtype**

9 The management address subtype field contains an integer value indicating the type of address that is listed
10 in the management address field. Enumeration for this field is contained in the `ianaAddressFamilyNumbers`
11 module of the IETF RFC 3232 on-line database that is accessible through a web page (<http://www.iana.org>).
12 The management address subtype is contained in the first octet of the management address string.

13 NOTE—As a consequence of the limitation on the size of the management address field (a maximum of 31 octets), some
14 options for the management address subtype (such as long DNS names) may be impractical.

15 **8.5.9.4 management address**

16 The management address field contains an octet string indicating the particular management address
17 associated with this TLV.

- 18 a) The returned address should be the most appropriate for management use, typically a layer 3 address
19 such as the IPv4 address, 192.0.2.10 (see also Table 8-2, footnote a).
20 b) If no management address is available, the return address should be the MAC address for the station
21 or port. The `ianaAddressFamilyNumber` for a MAC address is all802 (value 6).

22 **8.5.9.5 interface numbering subtype**

23 The interface numbering subtype field contains an integer value indicating the numbering method used for
24 defining the interface number. The following three values are currently defined:

- 25 1) Unknown
26 2) `ifIndex`
27 3) `system port number`

28 **8.5.9.6 interface number**

29 The interface number field contains the assigned number within the system that identifies the specific
30 interface associated with this management address. If the value of the interface subtype is unknown, this
31 field is set to zero.

1 8.5.9.7 object identifier (OID) string length

2 The object identifier string length field contains the length, in octets, of the OID. A value of zero in this field
3 indicates that the OID field is not provided.

4 8.5.9.8 object identifier

5 The object identifier field contains an OID that identifies the type of hardware component or protocol entity
6 associated with the indicated management address. The OID is the value portion of the ASN.1 encoding of
7 the object identifier, encoded according to ASN.1 basic encoding rules [ISO/IEC 8824-1]. If no OID is
8 available, this field is not provided.

9 NOTE—The interface number and OID are included in this TLV to assist NMS discovery by indicating Enterprise
10 Specific or other starting points for the search, such as the Interface or Entity MIB (IETF RFC 6933).

11 8.5.9.9 Management Address TLV usage rules

12 Management Address TLVs are subject to the following:

- 13 a) At least one Management Address TLV should be included in every LLDPDU.
 - 14 b) Since there are typically a number of different addresses associated with a MSAP identifier, an
15 individual LLDPDU may contain more than one Management Address TLV.
 - 16 c) When Management Address TLV(s) are included in an LLDPDU, the included address(es) should
17 be the address(es) offering the best management capability.
 - 18 d) If more than one Management Address TLV is included in an LLDPDU, each management address
19 is different from the management address in any other management address TLV in the LLDPDU.
 - 20 e) If an OID is included in the TLV, it is reachable by the management address.
 - 21 f) In a properly formed Management Address TLV, the TLV information string length is equal to:
22 (management address string length) + (OID string length) + 7.
- 23 If the TLV information string length in a received Management Address TLV is incorrect, then it is
24 ignored and processing of that LLDPDU is terminated.

8.6 Extended LLDP set TLV formats and definitions

8.6.1 Manifest TLV

The Manifest TLV allows an LLDP agent supporting XLLDP to advertise more TLVs than fits in a single LLDPDU. The Manifest TLV contains a list of Extension PDUs that convey the additional TLVs. The format for this TLV is shown in Figure 8-12.

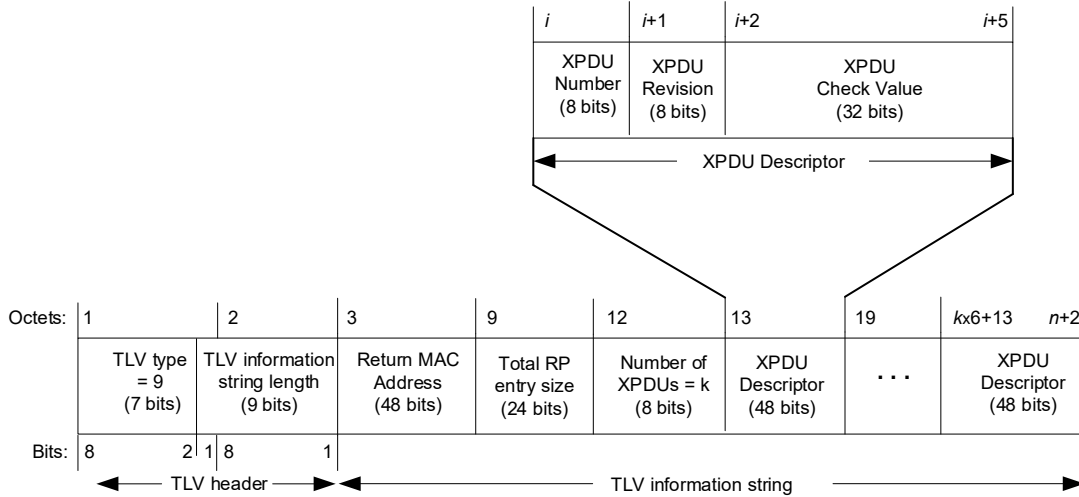


Figure 8-12—Manifest TLV format

8.6.1.1 TLV information string length

The TLV information string length field contains the number of octets in the TLV information string.

8.6.1.2 Return MAC Address

The individual MAC Address of the MSAP supporting the LLDP agent, to be used as the destination address for Extension Request LLDPDUs generated in response to this Manifest TLV.

8.6.1.3 Total information repository (RP) entry size

The total information repository entry size is the number of octets in all TLVs associated with this MSAP Identifier (regardless of the number of XPDUs used to convey those TLVs). It is the sum of the information string length fields in all TLVs, plus two times the number of TLVs, and includes the TLVs in the initial Normal LLDPDU. This allows the LLDP remote systems information repository manager to determine whether it expects to have space to store the information prior to requesting any XPDUs.

8.6.1.4 Number of XPDUs

The number of XPDUs field contains the count of the XPDU Descriptors in this TLV. Since the maximum size of the TLV information string is 511 octets, the minimum number of XPDU Descriptors is 1 and the maximum number is 83. When the number of XPDU Descriptors is k , the TLV information string length is at least $k \times 6 + 10$, but it need not be exactly $k \times 6 + 10$. This allows an implementation to use a fixed length TLV information string with a variable number of XPDU Descriptors.

1 **8.6.1.5 XPDU Descriptor**

2 An XPDU Descriptor comprises an 8-bit XPDU Number, an 8-bit XPDU Revision, and a 32-bit XPDU
3 Check Value. The XPDU Number uniquely identifies each XPDU in a manifest. The XPDU Revision is
4 assigned a random value at initialization, and is incremented (modulo 256) each time there is a change to
5 one or more TLVs in the XPDU. The XPDU Check Value is the low order 32 bits of an MD5 hash calculated
6 across all octets of the XPDU.

7 **8.6.1.6 Manifest TLV usage rules**

8 An LLDPDU does not contain more than one Manifest TLV. Each XPDU Descriptor associated with an
9 MSAP identifier and Scope MAC Address have a unique XPDU Number.

10 **8.6.2 Extension Request TLV**

11 The Extension Request TLV allows an LLDP agent supporting XLLDP to request the neighbor to transmit
12 specific Extension PDUs. The format for this TLV is shown in Figure 8-13.

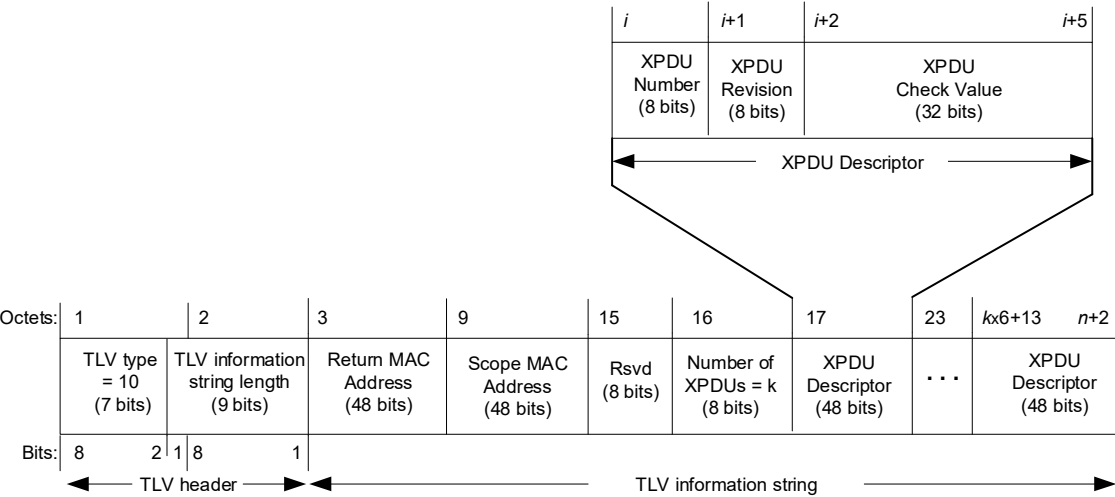


Figure 8-13—Extension Request TLV format

13 **8.6.2.1 TLV information string length**

14 The TLV information string length field contains the number of octets in the TLV information string.

15 **8.6.2.2 Return MAC Address**

16 The individual MAC address of the MSAP supporting the LLDP agent, to be used as the destination address
17 for the requested Extension LLDPDUs.

18 **8.6.2.3 Scope MAC Address**

19 The MAC Address used by this LLDP agent as the destination address for Normal LLDPDUs.

20 **8.6.2.4 Rsvd**

21 The rsvd field is transmitted as zero and ignored on receipt.

1 **8.6.2.5 Number of XPDUs**

2 The number of XPDUs field contains the count of the XPDU Descriptors in this TLV. Since the maximum
3 size of the TLV information string is 511 octets, the minimum number of XPDU Descriptors is 1 and the
4 maximum number is 82. When the number of XPDU Descriptors is k , the TLV information string length is
5 at least $k \times 6 + 14$, but it need not be exactly $k \times 6 + 14$. This allows an implementation to use a fixed length
6 TLV information string with a variable number of XPDU Descriptors.

7 **8.6.2.6 XPDU Descriptor**

8 An XPDU Descriptor comprises an 8-bit XPDU Number, an 8-bit XPDU Revision, and a 32-bit XPDU
9 Check, as specified in 8.6.1.5.

10 **8.6.2.7 Extension Request TLV usage rules**

11 An Extension Request LLDPDU contain an Extension Request TLV as the third TLV, and not contain more
12 than one Extension Request TLV. An Extension Request TLV not be contained in a Normal LLDPDU or an
13 Extension LLDPDU. Each XPDU Descriptor associated with a MSAP identifier and Scope MAC Address
14 have a unique XPDU Number.

15 **8.6.3 Extension Identifier TLV**

16 The Extension Identifier TLV identifies an XPDU containing one or more TLVs. The format for this TLV is
17 shown in Figure 8-14.

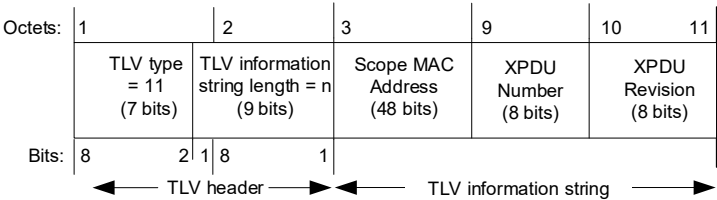


Figure 8-14—Extension Identifier TLV format

18 **8.6.3.1 TLV information string length**

19 The TLV information string length field contain the number of octets in the TLV information string.

20 **8.6.3.2 Scope MAC Address**

21 The MAC Address used by this LLDP agent as the destination address for Normal LLDPDUs.

22 **8.6.3.3 XPDU Number**

23 The XPDU Number portion of the XPDU Descriptor specified in 8.6.1.5.

24 **8.6.3.4 XPDU Revision**

25 The XPDU Revision portion of the XPDU Descriptor specified in 8.6.1.5.

8.6.3.5 Extension Identifier TLV usage rules

An Extension LLDPDU contains an Extension Identifier TLV as the third TLV, and does not contain more than one Extension Identifier TLV. An Extension Identifier TLV is not contained in a Normal LLDPDU or an Extension Request LLDPDU.

8.7 Organizationally Specific TLVs

This TLV category is provided to allow different organizations, such as IEEE 802.1, IEEE 802.3, IETF, as well as individual software and equipment vendors, to define TLVs that advertise information to remote entities attached to the same media, subject to the following restrictions:

- Information transmitted in an Organizationally Specific TLV is intended to be a one way advertisement.
- Information transmitted in an Organizationally Specific TLV is independent from information in a TLV received from a remote port.
- Information transmitted in one Organizationally Specific TLV is not concatenated with information transmitted in another TLV on the same media in order to provide a means for sending messages that are larger than would fit within a single TLV.
- Information received in an Organizationally Specific TLV is not explicitly forwarded to other ports in the system.
- Organizationally Specific TLVs conform to 8.4 and 8.7.1.

Each set of Organizationally Specific TLVs includes associated LLDP MIB or YANG extensions and the associated TLV selection management variables and MIB/YANG to TLV cross reference tables. Systems that implement LLDP and that also support standard protocols for which Organizationally Specific TLV extension sets have been defined support all TLVs and the LLDP MIB or YANG extensions defined for that particular TLV set.

Annex D of IEEE Std 802.1Q-2022 and Clause 79 of IEEE Std 802.3-2022 contain Organizationally Specific TLVs for IEEE 802.1 standards and IEEE Std 802.3-2022, respectively, along with their associated LLDP MIB or YANG extensions and LLDP MIB or YANG to TLV cross reference tables. Organizations wishing to define TLVs for their use should use these annexes as examples.

8.7.1 Organizationally Specific TLV format

The format for Organizationally Specific TLVs is shown in Figure 8-15.

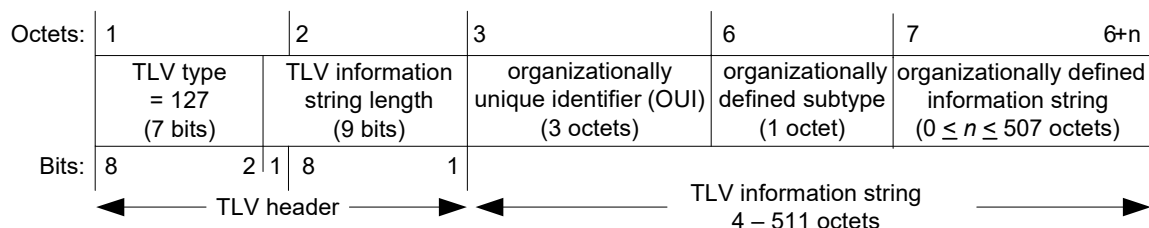


Figure 8-15—Format for Organizationally Specific TLVs

The following subclauses indicate how the individual fields are defined.

8.7.1.1 TLV type

The same Organizationally Specific TLV type value, 127, is used for all organizationally defined TLVs.

1 8.7.1.2 TLV information string length

2 The TLV information string length field contains the length of the information string, in octets.

3 8.7.1.3 organizational identifier (OI)

4 The organizational identifier (OI) field contains the organization's OUI or CID (see Clause 8 of IEEE Std
5 802-2024).

6 8.7.1.4 organizationally defined subtype

7 The organizationally defined subtype field contains a unique subtype value assigned by the defining
8 organization.

9 NOTE—Defining organizations are responsible for maintaining listings of organizationally defined subtypes in order to
10 assure uniqueness.

11 8.7.1.5 organizationally defined information string

12 The actual format of the organizationally defined information string field is organizationally specific and can

- 13 a) Contain either binary or alpha-numeric information that is instance specific for the particular TLV
14 type and/or subtype. Alpha-numeric information should be encoded in UTF-8 (IETF RFC 3629).
- 15 b) Include one or more information fields with their associated field-type identifiers and field length
16 designators similar to those in the Management Address TLV (see 8.5.9).

17 8.7.2 Organizationally Specific TLV usage rules

18 Organizations defining their own Organizationally Specific TLVs should include a subclause that defines
19 any specific usage rules and/or specific conditions that affects how the receiving LLDP agent treats the TLV.

20 The Organizationally Specific TLV usage rules should include the following:

- 21 a) The number of Organizationally Specific TLVs that may be contained in an LLDPDU and any
22 additional information field subtypes that would identify differences between two TLVs with the
23 same OI and organizationally defined subtype.
- 24 b) Any error conditions that are specific to the particular Organizationally Specific TLV and the action
25 that is taken for each defined error condition.

9. LLDP agent operation

This clause describes the operation of the protocol from the point of view of a single LLDP agent (see Clause 6); i.e., it describes the operation of the protocol for the transmission and reception of LLDPDUs on a single Port. Where the use of more than one Scope MAC Address (7.1.1) is supported on a given Port, a separate instance of the LLDP agent is responsible for the operation of the protocol for each Scope MAC Address that is supported.

9.1 Overview

Each LLDP agent is responsible for causing the following tasks to be performed:

- a) Maintaining current information in the LLDP local system information repository (RP).
- b) Extracting and formatting LLDP local system information repository information for transmission to the remote port LLDP agent(s) at regular intervals or whenever there is a change in the system condition or status.
- c) Recognizing and processing received LLDP frames.
- d) Maintaining current values in the LLDP remote system information repository.
- e) Using the procedure somethingChangedLocal() and the procedure somethingChangedRemote() to notify the PTOPO MIB manager and information repository managers of other optional MIBs and YANG modules whenever a value or status change has occurred in one or more objects in the LLDP local system LLDP remote systems information repository.

NOTE 1—A consequence of the support of multiple MAC addresses on a Port is that there are multiple copies of the LLDP local system information repository and remote system information repository, one copy for each address supported.

LLDP allows implementations that support three different operating modes: transmit only, receive only, or both transmit and receive. An LLDP agent shall conform to the specifications of each of the state machines indicated in Table 9-1 for the operating mode that it supports.

Table 9-1—Subclause/operating mode applicability

Subclause number	Subclause title	Transmit only	Receive only	Transmit and receive
9.2.10	Receive state machine	Mandatory if XLLDP is supported	Mandatory	Mandatory
9.2.9	Transmit state machine	Mandatory	—	Mandatory
9.2.11	Transmit timer state machine	Mandatory	—	Mandatory

NOTE 2—When XLLDP is supported and the operating mode is transmit only, the receive state machine is only enabled to recognize and respond to Extension Request LLDPDUs. Information advertised by remote LLDP agents is not stored in the LLDP remote systems information repository.

9.1.1 Frame transmission

An LLDPDU transmission cycle consists of the transmission of a Normal LLDPDU, potentially including a Manifest TLV if the implementation supports XLLDP, and the subsequent transmission of one or more Extension LLDPDUs upon receipt of Extension Request LLDPDUs. For an implementation not supporting XLLDP, the entire transmission cycle consists of the transmission of a single Normal LLDPDU.

1 The LLDP local system information repository contains the information needed for constructing individual
2 TLVs and includes a capability that allows network managers to select the optional TLVs to be included in
3 the Normal LLDPDU and, if XLLDP is supported, in one or more Extension LLDPDUs (see 10.2.2).

4 9.1.1.1 Normal LLDPDUs

5 The transmission of Normal LLDPDUs is performed by the transmit state machine, and the timing of
6 transmission cycles is controlled by the transmit timer state machine. Transmissions occur as a result of a
7 number of different local events, as follows:

- 8 a) A regular background transmission cycle occurs as a result of a transmission timer expiring. This
9 prevents the receiving system from timing out the information received, as a result of the “time to
10 live” associated with the last received LLDPDU expiring.
- 11 b) If a *new neighbor* is recognized (i.e., an LLDPDU is received by the receive state machine from a
12 hitherto unknown remote LLDP agent), then a default of four LLDPDU transmission cycles are
13 performed at shorter time intervals to quickly update the new neighbor with current information
14 about its own neighbors. This rapid transmission behavior is suppressed if the remote systems
15 information repository is not able to accommodate the new neighbor information without removing
16 current information relating to an older neighbor, in order to prevent excessive PDU transmissions
17 resulting from “churn” effects.
- 18 c) If a condition (status or value) change occurs in one or more objects in the LLDP local system
19 information repository, then a transmission cycle is initiated immediately, in order to communicate
20 the local changes as quickly as possible to any neighboring systems. The transmit timer state
21 machine operates a simple credit-based transmission strategy that allows a number of transmission
22 cycles in quick succession (the maximum number being determined by the maximum credit value),
23 thus allowing rapid updating of information in situations where the local state is changing quickly.
24 However, once the LLDP agent’s transmission credit is exhausted, the rate of transmission is cut to
25 an average of one transmission cycle per second.
- 26 d) The overall timing of regular transmission cycles is governed by a system “tick” that operates on a
27 nominal timebase of 1 s. However, in shared media LANs, in order to avoid bunching of
28 transmissions, the intervals between ticks include a random jitter component so that, while the
29 average time interval between ticks is 1 s, the interval between adjacent pairs of ticks can vary (see
30 9.2.2).

31 In order to reduce synchronization effects, it is recommended that transmission cycle intervals for different
32 LLDP agents on the same multi-port implementation are staggered.

33 When the transmit state machine (9.2.9) initiates a transmission cycle, the LLDP information repository
34 manager extracts the selected information from the LLDP local system information repository and
35 constructs a Normal LLDPDU containing the following:

- 36 a) The mandatory TLVs as specified in 8.2:
 - 37 1) Chassis ID TLV (see 8.5.2).
 - 38 2) Port ID TLV (see 8.5.3).
 - 39 3) Time To Live TLV, with the TTL value set equal to txTTL (see 8.5.4).
- 40 b) A Manifest TLV (see 8.6.1), if XLLDP is supported and any of the selected information is to be
41 included in an Extension LLDPDU. The Manifest TLV includes a descriptor for each Extension
42 LLDPDU, even if the information selected to be included in that Extension LLDPDU has not
43 changed.
- 44 c) Additional optional TLVs, from the basic management set or from one or more organizationally
45 specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system
46 information repository by network management (see Table 8-1).
- 47 d) Optionally, an End Of LLDPDU TLV.

Optional TLVs from the basic management set, such as the Management Address TLV, with different TLV type and subtype combinations as well as optional TLVs with different organizationally unique identifier and organizationally defined subtype combinations from one or more organizationally specific sets can be included within the same LLDPDU.

9.1.1.2 Shutdown LLDPDUs

A special procedure exists for the case in which a LLDP agent knows an associated port is about to become non-operational (for example, the adminStatus for the port is transitioning to ‘disabled’). In the event a port, currently configured with LLDP frame transmission enabled, either becomes disabled for LLDP activity, or the interface is administratively disabled, the transmit state machine attempts to send a final LLDP shutdown LLDPDU with:

- a) The Chassis ID TLV.
- b) The Port ID TLV.
- c) The Time To Live TLV with the TTL field set to zero.
- d) Optionally, an End Of LLDPDU TLV.

The shutdown LLDPDU does not include any other optional TLVs and, if possible, should be transmitted before the interface is disabled.

NOTE—There is an inherent race condition between an interface knowing it is going down and its ability to send “one more frame.” If possible, the actual transition of the port to disabled is delayed until after this frame is transmitted. In the event where adminStatus is transitioning to the disabled state and the LLDP agent is shutting down, this shutdown procedure should be executed for all local ports.

9.1.1.3 Extension Request LLDPDUs

Extension Request LLDPDUs are transmitted in response to the reception of a Manifest LLDPDU. The transmission is controlled by the extended receive state machine. When XLLDP is supported, and a Manifest LLDPDU or an Extension LLDPDU is received, and the Manifest TLV includes XPDU Descriptors that do not match the XPDU Descriptors in the LLDP remote systems information repository entry for the MSAP Identifier received in the LLDPDU, the LLDP information repository manager constructs an Extension Request LLDPDU containing the following:

- a) The mandatory TLVs as specified in 8.2:
 - 1) Chassis ID TLV (8.5.2).
 - 2) Port ID TLV (8.5.3).
 - 3) Extension Request TLV with a list of one or more XPDU descriptors (8.6.2).
- b) Optionally, an End Of LLDPDU TLV.

9.1.1.4 Extension LLDPDUs

Extension LLDPDUs are transmitted in response to the reception of an Extension Request LLDPDU. The transmission is controlled by the receive state machine. When XLLDP is supported, and an Extension Request LLDPDU is received, then for each XPDU Number and XPDU Revision pair in the Extension Request TLV that matches an XPDU Number and XPDU Revision in the LLDP local system information repository, the LLDP information repository manager extracts the selected information from the LLDP local system information repository and constructs an Extension LLDPDU containing the following:

- a) The mandatory TLVs as specified in 8.2:
 - 1) Chassis ID TLV (8.5.2).
 - 2) Port ID TLV (8.5.3).
 - 3) Extension Identifier TLV with the requested XPDU Number and XPDU Revision pair (8.6.3).

- 1 b) Additional optional TLVs, from the basic management set or from one or more organizationally
- 2 specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system
- 3 information repository by network management (Table 8-1).
- 4 c) Optionally, an End Of LLDPDU TLV.

5 9.1.2 Frame reception

6 LLDP frame reception consists of four phases: frame recognition, frame validation, TLV collection, and
7 LLDP remote systems information repository updating.

8 9.1.2.1 Frame recognition

9 Frame recognition is performed at the LLDP/LSAP (see Figure 6-2), where the destination address and
10 EtherType values determine whether the frame is an LLDP frame destined for this LLDP agent or not.

11 9.1.2.2 Frame validation

12 Frames that are recognized as LLDP frames are then validated to determine whether they are properly
13 constructed and contain the correct set of mandatory TLVs. Frames that do not pass the validation criteria
14 are discarded.

15 9.1.2.3 TLV collection

16 All received TLVs are validated by checking for format errors, and invalid TLVs are ignored. If the TLVs
17 received in the Normal LLDPDU did not include a Manifest TLV (or the implementation does not support
18 XLLDP and does not recognize the Manifest TLV), then all validated TLVs are passed to the LLDP remote
19 systems information repository immediately. If the implementation supports XLLDP and the Normal
20 LLDPDU includes a Manifest TLV, then this TLV is used to determine which Extension LLDPDUs need to
21 be received before a complete set of TLVs can be passed to the LLDP remote systems information
22 repository.

23 An implementation supporting XLLDP compares the XPDU descriptors in the Manifest TLV to those
24 received in a previous Manifest TLV to determine which XPDU (identified by the XPDU Number) contain
25 new information. If no Manifest TLV has been received previously, then all of the XPDU described in the
26 Manifest TLV contain new information. If there are no XPDU that have new information, then the validated
27 TLVs received in the Normal LLDPDU are passed to the LLDP remote systems information repository.
28 Otherwise these TLVs are retained until the XPDU with new information have been received.

29 If there are XPDU that have new information, then an Extension Request LLDPDU, requesting one or more
30 of the XPDU with new information, is transmitted to the LLDP agent that sourced the Normal LLDPDU. A
31 timer is started when an Extension Request LLDPDU is transmitted, and the request is retried once if the
32 timer expires before all requested XPDU have been received. When all requested XPDU have been
33 received, another Extension Request LLDPDU is transmitted requesting one or more of the remaining
34 XPDU with new information. This process repeats until all XPDU with new information have been
35 received. If a new Manifest LLDPDU is received before all XPDU have been received, any received
36 XPDU that have the same Revision and Checksum in the new Manifest TLV are retained, and any received
37 XPDU that have different Revision and Checksum are discarded. The extended receive state machine then
38 continues to issue Extension Requests for any necessary XPDU.

39 Received frames containing Extension LLDPDU are recognized and validated following the same steps as
40 frames containing Normal LLDPDU. Once validated, the XPDU check value is calculated. This check
41 value, together with the XPDU Number and XPDU Revision from the Extension Identifier TLV, is
42 compared to the XPDU descriptors in the Manifest TLV. If there is a matching descriptor then all valid TLVs

received in this Extension LLDPDU are retained. When all XPDU with new information have been received, all of the retained TLVs are passed to the LLDP remote systems information repository.

9.1.2.4 LLDP remote systems information repository updating

The collected TLVs are used to create, modify, or delete an entry in the LLDP remote systems information repository. The relevant entry in the LLDP remote systems information repository is determined by the originating MSAP identifier formed from the contents of the Chassis ID and Port ID TLVs. The LLDP remote systems information repository is updated as follows:

- a) If the TTL value in the Time To Live TLV is greater than zero and an entry does not exist in the LLDP remote systems information repository for the originating MSAP identifier and there is sufficient space (or sufficient space can be created by the steps in 9.2) for the new information, then a new entry in the information repository is created.
- b) If the TTL value in the Time To Live TLV is greater than zero and an entry already exists in the remote systems information repository for the originating MSAP identifier and there is sufficient space (or sufficient space can be created by the steps in 9.2) for the new information, then that entry is modified as follows:
 - 1) The TLVs received in the Normal LLDPDU are used to replace any information elements in the existing entry in the information repository that were previously received in a Normal LLDPDU. Note that if the Normal LLDPDU did not contain a Manifest TLV (or the implementation does not support XLLDP and does not recognize the Manifest TLV), then the new information contained in the LLDPDU is used to replace the whole of the existing entry in the information repository; i.e., if there are information elements in the existing entry for which there are corresponding elements in the received LLDPDU, then those elements are updated using the information received; any other information elements in the existing entry in the information repository are deleted.
 - 2) The TLVs received in an Extension LLDPDU are used to replace any information elements in the existing entry in the information repository that were previously received in an Extension LLDPDU with the same XPDU Number.
 - 3) Any information elements in the existing entry in the information repository that were previously received in an Extension LLDPDU with an XPDU Number that is not included in the received Manifest TLV are deleted.
- c) If the TTL value in the Time To Live TLV is greater than zero and an entry exists in the LLDP remote systems information repository for the originating MSAP identifier and there is insufficient space for the new information, then the entry is deleted from the information repository.
- d) If the TTL value in the Time To Live TLV is equal to zero and an entry exists in the LLDP remote systems information repository for the originating MSAP identifier, then the entry is deleted from the information repository.

When an entry in the LLDP remote systems information repository is created or modified (even if the modification is to replace all information elements with the same information), the rxInfoTTL timer associated with that entry is set to the TTL value from the Time To Live TLV. This determines how long the information associated with that MSAP identifier and destination MAC address is to be stored in the LLDP remote systems information repository before being aged out and deleted. The ageing out of old data purges the information repository of data that was originated by systems that are no longer neighbors as a result of topology changes, system failure, or system inactivation.

1 9.1.3 Too many neighbors

2 The amount of space needed in the LLDP remote systems information repository to accommodate the
3 creation of several new information repository structures is beyond the scope of this standard. It may not
4 always be possible to accommodate another new neighbor in implementations with limited memory. There
5 are several possibilities for handling this case, including but not limited to the following examples:

- 6 a) Ignore and not process the new neighbor's information.
- 7 b) Delete the information from the oldest neighbor(s) until there is sufficient memory available to store
8 the new neighbor's information.
- 9 c) Arbitrarily delete neighbors until there is sufficient memory available to store the new neighbor's
10 information.

11 The method of handling the case where a new entry in the remote systems information repository can not be
12 created is beyond the scope of this standard, however it is necessary for the implementation to keep track of
13 this case by properly updating the variable tooManyNeighbors and the tooManyNeighborsTimer (9.2.5.16,
14 9.2.2). The variable tooManyNeighbors identifies when there is insufficient space in the LLDP remote
15 systems information repository to store information from all active neighbors. The tooManyNeighborsTimer
16 indicates the minimum time that this condition exists.

17 Further detail on the handling of the too many neighbors condition can be found in 9.2.8.5.1.

18 9.1.4 LLDP remote systems rxInfoTTL timer expiration

19 If the rxInfoTTL timer (9.2.2.1) associated with an MSAP identifier expires, all information associated with
20 that MSAP identifier is deleted and the procedure somethingChangedRemote() notifies the managers of the
21 PTOPO MIB and other optional MIB or YANG modules (see Figure 11-1) that something has changed in the
22 LLDP remote systems information repository associated with that MSAP identifier.

23 9.1.5 LLDP local port/connection failure

24 If the local port or the connection to the remote system fails unexpectedly before a shutdown frame is
25 received, the LLDP management entity does not delete objects in the LLDP remote systems information
26 repository pertaining to information received from any MSAP identifier through that local port until the port
27 is re-initialized or the associated rxInfoTTL timer (9.2.2.1) expires.

28 9.2 State machines

29 The operation of the protocol is represented by the following three state machines and associated timers to
30 indicate when local system managed object values are to be transmitted to refresh the values in remote
31 LLDP agent's LLDP remote systems information repository and when values in the local LLDP agent's
32 remote systems information repository have become invalid:

- 33 a) Transmit state machine (9.2.9).
- 34 b) Receive state machine (9.2.10).
- 35 c) Transmit timer state machine (9.2.11).

36 9.2.1 Notational conventions used in state diagrams

37 State diagrams are used to represent the operation of a function as a group of connected, mutually exclusive
38 states. Only one state of a function can be active at any given time.

1 Each state is represented in the state diagram as a rectangular box, divided into two parts by a horizontal
2 line. The upper part contains the state identifier, written in uppercase letters. The lower part contains any
3 procedures that are executed on entry to the state.

4 All permissible transitions between states are represented by arrows, the arrowhead denoting the direction of
5 the possible transition. Labels attached to arrows denote the condition(s) that shall be met in order for the
6 transition to take place. A transition that is global in nature (i.e., a transition that occurs from any of the
7 possible states if the condition attached to the arrow is met) is denoted by an open arrow; i.e., no specific
8 state is identified as the origin of the transition.

9 On entry to a state, the procedures defined for the state (if any) are executed exactly once, in the order that
10 they appear on the page. Each action is deemed to be atomic; i.e., execution of a procedure completes before
11 the next sequential procedure starts to execute. No procedures execute outside of a state block. On
12 completion of all of the procedures within a state, all exit conditions for the state (including all conditions
13 associated with global transitions) are evaluated continuously until such a time as one of the conditions is
14 met. All exit conditions are regarded as Boolean expressions that evaluate to TRUE or FALSE; if a condition
15 evaluates to TRUE, then the condition is met. When the condition associated with a global transition is met,
16 it supersedes all other exit conditions, including UCT. The label UCT denotes an unconditional transition
17 (i.e., UCT always evaluates to TRUE). The label ELSE denotes a transition that occurs if none of the other
18 conditions for transitions from the state are met (i.e., ELSE evaluates to TRUE if all other possible exit
19 conditions from the state evaluate to FALSE).

20 A variable that is set to a particular value in a state block retains this value until a subsequent state block
21 executes a procedure that modifies the value, or until the value is modified by an external event or timer tick.

22 Where it is necessary to segment a state machine description across more than one diagram, a transition
23 between two states that appear on different diagrams is represented by an exit arrow drawn with dashed
24 lines, plus a reference to the diagram that contains the destination state. Similarly, dashed arrows and a
25 dashed state box are used on the destination diagram to show the transition to the destination state. In a state
26 machine that has been segmented in this way, any global transitions that can cause entry to states defined in
27 one of the diagrams are deemed to be potential exit conditions for all of the states of the state machine,
28 regardless of which diagram the state boxes appear in.

29 Should a conflict exist between the interpretation of a state diagram and either the corresponding global
30 transition tables or the textual description associated with the state machine, the state diagram takes
31 precedence.

32 The interpretation of the special symbols and operators used in the state diagrams is defined in Table 9-2;
33 these symbols and operators are derived from the notation of the “C” programming language.

Table 9-2—State machine symbols

Symbol	Interpretation
()	Used to force the precedence of operators in Boolean expressions and to delimit the argument(s) of actions within state boxes.
;	Used as a terminating delimiter for actions within state boxes. Where a state box contains multiple actions, the order of execution follows the normal English language conventions for reading text.

Table 9-2—State machine symbols (*continued*)

Symbol	Interpretation
=	Assignment action. The value of the expression to the right of the operator is assigned to the variable to the left of the operator. Where this operator is used to define multiple assignments, e.g., a = b = X the action causes the value of the expression following the right-most assignment operator to be assigned to all of the variables that appear to the left of the right-most assignment operator.
!	Logical NOT operator.
&&	Logical AND operator.
	Logical OR operator.
if...then...	Conditional action. If the Boolean expression following the if evaluates to TRUE, then the action following the then is executed.
{statement 1, ... statement N}	Compound statement. Braces are used to group statements that are executed together as if they were a single statement.
!=	Inequality. Evaluates to TRUE if the expression to the left of the operator is not equal in value to the expression to the right.
==	Equality. Evaluates to TRUE if the expression to the left of the operator is equal in value to the expression to the right.
<	Less than. Evaluates to TRUE if the value of the expression to the left of the operator is less than the value of the expression to the right.
<=	Less than or equal to. Evaluates to TRUE if the value of the expression to the left of the operator is either less than or equal to the value of the expression to the right.
>	Greater than. Evaluates to TRUE if the value of the expression to the left of the operator is greater than the value of the expression to the right.
>=	Greater than or equal to. Evaluates to TRUE if the value of the expression to the left of the operator is either greater than or equal to the value of the expression to the right.
+	Arithmetic addition operator.
–	Arithmetic subtraction operator.

9.2.2 Timers

A set of timers is used by the LLDP state machines; these operate as countdown timers (i.e., they expire when their value reaches zero). These timers

- Have a resolution of one tick.
- Have an integer value n , where $0 < n < 65535$ tick intervals.
- Are started by loading an initial integer value.
- Are decremented once per timer tick as long as $n > 0$.
- Represent the remaining time in the period.

For Ports connected to point-to-point LANs, the interval between timer ticks is 1 s. For Ports connected to shared media LANs, the interval between any pair of timer ticks is 0.8 s, plus a “jitter” component that is a random or pseudo-random value between 0.0 s and 0.4 s. Hence, the average interval between timer ticks is 1 s; however, the interval between any pair of ticks varies randomly. When a timer tick occurs, all instances of the variable txTick (9.2.5.22) for the Port are set TRUE.

1 NOTE—The random component of the tick time used in shared media LANs is intended to minimize the possibility of
2 systems connected to the same LAN becoming synchronized in their transmission timings.

3 **9.2.2.1 rxInfoTTL**

4 An instance of this timer exists for each entry in the LLDP remote systems information repository that has
5 been created by this instance of the receive state machine for a given MSAP identifier. The timer value
6 indicates the number of seconds remaining until the information in all the objects in the LLDP remote
7 systems information repository associated with the MSAP identifier is no longer valid and needs to be
8 deleted.

9 **9.2.2.2 tooManyNeighborsTimer**

10 This timer indicates the number of seconds remaining during which it is known that an LLDP neighbor
11 exists that may not be stored in the remote systems information repository.

12 **9.2.2.3 txTTR**

13 An instance of this timer exists for each Agent. The txTTR timer is used to determine the next LLDPDU
14 transmission is due; it is initialized by the transmit timer state machine, with the value msgTxInterval
15 (9.2.5.6) if txFast (9.2.5.19) is equal to zero, or with the value msgFastTx (9.2.5.4) if txFast is greater than
16 zero.

17 **9.2.2.4 txShutdownWhile**

18 An instance of this timer exists for each Agent. The txShutdownWhile timer indicates the number of
19 seconds remaining until LLDP re-initialization can occur.

20 **9.2.2.5 rxTimer**

21 An instance of this timer exists for each extended receive state machine to impose a time limit on each
22 extension request, and on the overall TLV collection process.

23 **9.2.3 Per-System variables**

24 An instance of each of these variables exists per system; they are available for use by all of the state
25 machines of all of the LLDP agents in the system.

26 **9.2.3.1 BEGIN**

27 This Boolean variable is controlled by the system initialization process. A value of TRUE causes all state
28 machines to continuously execute their initial state. A value of FALSE allows all state machines to perform
29 transitions out of their initial state, in accordance with the relevant state machine definitions.

30 **9.2.4 Per-Port variables**

31 An instance of each of these variables exists per Port; they are available to all of the state machines
32 associated with a Port.

33 **9.2.4.1 portEnabled**

34 This variable is externally controlled. Its value reflects the operational state of the MAC service supporting
35 the port. Its value is TRUE if the MAC service supporting the Port is in an operable condition; otherwise, it
36 is FALSE.

1 9.2.5 Per-Agent variables

2 An instance of each of these variables exists per LLDP agent; they are available to all of the state machines
3 associated with an LLDP agent.

4 9.2.5.1 adminStatus

5 This variable indicates whether or not the LLDP agent is enabled. The defined values for this variable are as
6 follows:

- 7 a) enabledRxTx: The LLDP agent is enabled for reception and transmission of LLDPDUs. This is the
8 recommended default value.
- 9 b) enabledTxOnly: The LLDP agent is enabled for transmission of LLDPDUs only.
- 10 c) enabledRxOnly: The LLDP agent is enabled for reception of LLDPDUs only.
- 11 d) disabled: The LLDP agent is disabled for both reception and transmission.

12 9.2.5.2 localChange

13 This variable is set TRUE by the somethingChangedLocal() procedure (see 9.2.8.8) when there is a change
14 in the value of the LLDP local system information repository associated with this LLDP agent. The variable
15 is set FALSE by the operation of the transmit timer state machine (see 9.2.11).

16 9.2.5.3 Scope MAC Address

17 The MAC address associated with this instance of the LLDP agent that defines the scope of propagation of
18 Normal LLDPDUs in the network. It is used as the destination MAC address when the LLDP agent
19 transmits Normal LLDPDUs, and the destination MAC address that, when present in a received LLDPDU,
20 causes the LLDPDU to be passed to this LLDP agent instance for processing.

21 9.2.5.4 msgFastTx

22 This variable defines the time interval in timer ticks between transmissions during fast transmission periods
23 (i.e., txFast is non-zero). The recommended default value of msgFastTx is 1; this value can be changed by
24 management to any value in the range 1 through 3600.

25 9.2.5.5 msgTxHold

26 This variable is used, as a multiplier of msgTxInterval, to determine the value of txTTL that is carried in
27 LLDP frames transmitted by the LLDP agent. The recommended default value of msgTxHold is 4; this
28 value can be changed by management to any value in the range 2 through 10.

29 9.2.5.6 msgTxInterval

30 This variable defines the time interval in timer ticks between transmissions during normal transmission
31 periods (i.e., txFast is zero). The recommended default value for msgTxInterval is 30 s; this value can be
32 changed by management to any value in the range 1 through 3600.

33 9.2.5.7 newNeighbor

34 This variable is set TRUE by the rxProcessFrame() procedure (see 9.2.8.7) when information is received
35 from a new neighbor. It is set FALSE by the Transmit timer state machine (9.2.11).

36 NOTE—The newNeighbor variable is used to force a more rapid regular transmission rate when a new neighbor
37 appears, to ensure that the new neighbor also receives new information from the receiving system.

1 9.2.5.8 rcvFrame

2 This variable indicates that an LLDP frame has been recognized by the LLDP LSAP function and is ready to
3 be processed by the LLDP receive state machine. If both of the following conditions are TRUE, the variable
4 rcvFrame is set to TRUE and the frame is made available to the receive state machine for validation and
5 processing:

- 6 a) The destination address value is the Scope MAC Address associated with this specific LLDP agent
7 or the individual MAC address of the MSAP supporting the LLDP agent.
- 8 b) The EtherType value carried by the frame is the LLDP EtherType defined in Table 7-3.

9 NOTE—The model that is assumed for reception is that incoming LLDPDUs are presented to all LLDP agents
10 associated with the reception Port, and if the Scope MAC Address is not one that is associated with a given LLDP agent,
11 that LLDPDU is ignored by that LLDP agent. In practice, at most one LLDP agent processes any received LLDPDU, as
12 there is only one LLDP agent associated with any given Scope MAC Address on a given reception Port.

13 9.2.5.9 reinitDelay

14 This parameter indicates the amount of delay from when adminStatus becomes ‘disabled’ until
15 re-initialization is attempted. The reinitDelay can take a value in the range from 1 s. through 10 s. The
16 recommended default value for reinitDelay is 2 s.

17 9.2.5.10 remoteChanges

18 A Boolean indication of whether the status/value of one or more selected objects in the LLDP remote
19 systems information repository has changed or that all objects associated with a particular MSAP identifier
20 have been deleted. This variable takes the value (*rxInfoAge OR (rxChanges && (rxTTL !=0))*)

21 9.2.5.11 rxChanges

22 This variable indicates that the incoming LLDPDU has been received with different TLV values from those
23 currently in the LLDP remote systems information repository associated with the LLDPDU’s MSAP
24 identifier.

25 9.2.5.12 rxInfoAge

26 This variable is set TRUE when a timer expiry event occurs for the rxInfoTTL timing counter associated
27 with a particular MSAP identifier in the LLDP remote systems information repository (i.e., the counter
28 transitions from non-zero to zero). It is set FALSE by the operation of the Receive state machine.

29 9.2.5.13 rxTTL

30 This variable indicates the time to live value associated with all TLVs received in the current frame.

31 9.2.5.14 rxType

32 This variable indicates the type of the most recently received LLDP frame. It is set by the rxProcessFrame()
33 procedure to one of the following values:

- 34 a) xreq: The LLDP agent supports XLLDP and identifies the frame as containing a request for
35 extended PDUs.
- 36 b) xpdu: The LLDP agent supports XLLDP and identifies the frame as containing an extended PDU.
- 37 c) shutdown: The frame contains a shutdown LLDPDU (which has an rxTTL value equal to zero).
- 38 d) manifest: The LLDP agent supports XLLDP and the frame is a Normal LLDPDU that has a rxTTL
39 value greater than zero and contains a Manifest TLV.

- 1 e) normal: The frame contains a Normal LLDPDU that has an rxTTL value greater than zero, and
- 2 either the LLDP agent does not support XLLDP or the frame does not contain a Manifest TLV.
- 3 f) invalid: The frame is improperly formatted for a valid LLDPDU, or the frame contains an extended
- 4 PDU or a request for extended PDUs but the LLDP agent does not support XLLDP.

5 **9.2.5.15 sentManifest**

6 This Boolean variable is set to TRUE when the transmit state machine generates a Manifest LLDPDU, and
7 is set to FALSE when the transmit state machine generates a Normal LLDPDU that does not contain a
8 Manifest TLV. It is used to enable the receive state machine to recognize Extension Request LLDPDUs and
9 respond with Extension LLDPDUs.

10 **9.2.5.16 tooManyNeighbors**

11 This Boolean variable indicates that there is insufficient space in the LLDP remote systems information
12 repository to store information from all connected active remote ports (see 9.2.8.5.1).

13 **9.2.5.17 txCredit**

14 The number of consecutive LLDPDUs that can be transmitted at any time. This variable is incremented by
15 the txAddCredit() procedure, up to the maximum value specified by the txCreditMax variable (see 9.2.5.18),
16 and is decremented whenever an LLDPDU is transmitted by the transmit state machine.

17 **9.2.5.18 txCreditMax**

18 The maximum value of txCredit. The recommended default value is 5; this value can be changed by
19 management to any value in the range 1 through 10.

20 **9.2.5.19 txFast**

21 This variable is used as a down counter to count the number of transmissions to be made during a fast
22 transmission period. Fast transmission periods are initiated when a new neighbor is detected, and cause
23 LLDPDU transmissions to take place at shorter time intervals than during normal (steady state) operation of
24 the protocol. The fast transmission period ensures that more than one LLDPDU is transmitted when a new
25 neighbor is detected. The first transmission is immediate; the subsequent transmissions occur at intervals
26 determined by msgFastTx. The number of transmissions is determined by the value of txFastInit (see
27 9.2.5.20).

28 **9.2.5.20 txFastInit**

29 This variable is used as the initial value for the txFast variable (see 9.2.5.19). This value determines the
30 number of LLDPDUs that are transmitted during a fast transmission period. The recommended default value
31 of txFastInit is 4; this value can be changed by management to any value in the range 1 through 8.

32 **9.2.5.21 txNow**

33 This variable is used to signal to the transmit state machine that a transmission is required. It is set TRUE by
34 the transmit timer state machine, and set FALSE by the transmit state machine when the transmission has
35 been initiated. It remains TRUE until cleared by the transmit state machine or reset by initialization.

36 **9.2.5.22 txTick**

37 This variable is set TRUE by the system timer tick at an average interval of one second (see 9.2.2). It is set
38 FALSE by the operation of the transmit timer state machine.

1 9.2.5.23 txTTL

2 This variable indicates the time remaining before information in the outgoing LLDPDU is no longer valid.
3 The TTL field in the Time To Live TLV is set to txTTL during LLDPDU construction (see 9.1.2). The value
4 of txTTL depends on the following:

- 5 a) During normal operation, txTTL is set to whichever is the smaller of the values represented by
6 Equation (1) and Equation (2):

$$7 \quad (\text{msgTxInterval} \times \text{msgTxHold}) + 1 \quad (1)$$

$$8 \quad 65535 \quad (2)$$

9 where msgTxInterval is defined in 9.2.5.6 and msgTxHold is defined in 9.2.5.5.

- 10 b) If adminStatus is transitioning to ‘disabled’ or portEnabled is transitioning to FALSE, txTTL shall
11 be set to zero.

12 9.2.6 Per-Neighbor variables

13 An instance of each of these variables exists per neighbor (identified by an MSAP) for each LLDP agent;
14 they are available to all of the state machines associated with an LLDP agent.

15 9.2.6.1 createExtRx

16 This Boolean is used to initialize a per-neighbor extended receive state machine. It is set to TRUE when an
17 extended receive state machine is created in response to the receipt of a Manifest LLDPDU from a neighbor
18 for which an extended receive state machine does not already exist. It is set to FALSE by the extended
19 receive state machine following initialization.

20 9.2.6.2 rxExtended

21 This Boolean is used to pass control between the per-agent receive state machine and the per-neighbor
22 extended receive state machine. It is set to TRUE by the receive state machine when an LLDPDU containing
23 a Manifest TLV or an Extension Identifier TLV is received, and is set to FALSE by the extended receive
24 state machine.

25 9.2.6.3 manifestComplete

26 This Boolean indicates to the receive state machine whether the extended receive state machine has received
27 all TLVs necessary to update the information for this MSAP in the LLDP remote systems information
28 repository.

29 9.2.6.4 xreqComplete

30 This Boolean is used within the extended receive state machine to indicate that all XPDUs requested for this
31 MSAP have been received.

32 9.2.6.5 rxTick

33 This Boolean is set TRUE by the system timer tick at an average interval of one second (see 9.2.2). It is set
34 FALSE by the operation of the extended receive state machine.

1 **9.2.6.6 rxTTLExpired**

2 This Boolean is used to indicate that the extended receive state machine has been unable to collect all the
3 TLVs within a time interval greater than the received rxTTL value. It is set to TRUE when the rxTimer is
4 decremented to zero and either all Extension LLDPDUs for the most recently transmitted Extension Request
5 LLDPDU have been received, or that Extension Request LLDPDU was a retry of a previous request.

6 **9.2.7 Per-Agent statistical counters**

7 An instance of each of these statistical counters exists per LLDP agent; they are used to count significant
8 events in the transmit and receive state machines and are made available to the management functions
9 described in Clause 10 and Clause 11. All counters are maintained as unsigned integer values, four octets in
10 length, and are initialized to zero only on system initialization (i.e., when the BEGIN system variable is set
11 TRUE).

12 **9.2.7.1 statsAgeoutsTotal**

13 This counter provides a count of the times that a neighbor's information has been deleted from the LLDP
14 remote systems information repository because the rxInfoTTL timer associated with the neighbor's MSAP
15 has expired.

16 **9.2.7.2 statsFramesDiscardedTotal**

17 This counter provides a count of all LLDPDUs received and then discarded for any of the following reasons:

- 18 a) One or more of the three mandatory TLVs at the beginning of the LLDPDU is missing, out of order,
19 or contains an out of range information string length.
- 20 b) There is insufficient space in the remote systems information repository to store the LLDPDU.

21 **9.2.7.3 statsFramesInErrorsTotal**

22 This counter provides a count of all LLDPDUs received with one or more detectable errors.

23 **9.2.7.4 statsFramesInTotal**

24 This counter provides a count of all LLDP frames received.

25 **9.2.7.5 statsFramesOutTotal**

26 This counter provides a count of all LLDP frames transmitted by this instance of the transmit state machine.
27 The counter shall be maintained as an unsigned integer value, four octets in length.

28 **9.2.7.6 statsTLVsDiscardedTotal**

29 This counter provides a count of all TLVs received and then discarded for any reason.

30 **9.2.7.7 statsTLVsUnrecognizedTotal**

31 This counter provides a count of all TLVs received on the port that are not recognized by the LLDP local
32 agent.

1 9.2.7.8 lldpduLengthErrors

2 A count of the number of LLDPDU length restriction errors detected by the rpConstrInfoLLDPDU()
3 procedure (9.2.8.2).

4 9.2.8 State machine procedures

5 9.2.8.1 dec (variable-name)

6 If the state machine variable *variable-name* has a non-zero value, this procedure decrements the value of the
7 variable by 1.

8 9.2.8.2 rpConstrInfoLLDPDU()

9 The rpConstrInfoLLDPDU() procedure constructs an information LLDPDU, structured according to the
10 specification in 8.2, the associated basic TLV formats defined in 8.4, plus any optional Organizationally
11 Specific TLVs as specified and their associated individual organizationally defined formats (see 8.7). The
12 information LLDPDU contains the following:

- 13 a) The set of mandatory TLVs as specified in 8.2, with the value of txTTL set in accordance with the
14 definition in 9.2.5.23.
- 15 b) Additional optional TLVs, from the basic management set or from one or more organizationally
16 specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system
17 information repository by network management (see Table 8-1).
- 18 c) Optionally, an End Of LLDPDU TLV.

19 If the set of TLVs that is selected in the LLDP local system information repository by network management
20 would result in an LLDPDU that violates LLDPDU length restrictions and XLLDP is supported, each TLV
21 selected for transmission is mapped to the Normal LLDPDU or to an Extension LLDPDU, as defined in
22 10.2.2. If the set of TLVs selected for transmission would result in more than 83 Extension LLDPDUs, the
23 variable lldpduLengthErrors is incremented by 1. The mandatory TLVs plus as many optional TLVs in the
24 set that will fit are mapped to the Normal LLDPDU and the 83 Extension LLDPDU.

25 If the set of TLVs that is selected in the LLDP local system information repository by network management
26 would result in an LLDPDU that violates LLDPDU length restrictions and XLLDP is not supported, then
27 the variable lldpduLengthErrors (9.2.8.2) is incremented by 1, and an LLDPDU is sent containing the
28 mandatory TLVs plus as many of the optional TLVs in the set as will fit in the remaining LLDPDU length.

29 9.2.8.3 rpConstrShutdownLLDPDU()

30 The rpConstrShutdownLLDPDU() procedure constructs a shutdown LLDPDU according to the LLDPDU
31 and the associated TLV formats specified in 8.2 and 8.5. The shutdown LLDPDU contains only the
32 following items:

- 33 a) The Chassis ID and Port ID TLVs.
- 34 b) The Time To Live TLV with the TTL field set to zero.
- 35 c) Optionally, an End Of LLDPDU TLV.

36 9.2.8.4 rpDeleteObjects()

37 The rpDeleteObjects() procedure deletes all information in the LLDP remote systems information repository
38 associated with a given MSAP identifier (and destroys the associated extended receive state machine if one
39 exists) if an LLDPDU is received with an rxTTL value of zero (see 9.2.8.7.1), the extended receive state
40 machine sets rxTTLExpired to TRUE, or the timing counter rxInfoTTL expires (see 9.2.2.1).

9.2.8.5 rpUpdateObjects()

The rpUpdateObjects() procedure updates the information repository objects corresponding to the TLVs contained in the received Normal LLDPDU, and all TLVs contained in any received Extension LLDPDUs, for the LLDP remote system if information repository space is available, as follows:

- a) Compare the MSAP identifier in the current LLDPDU with the MSAP identifiers in the LLDP remote systems information repository:
 - 1) If a match is found, but no difference exists between the information in the TLVs just received and the information in the LLDP remote systems information repository, then set the control variable rxChanges to FALSE, and set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.
 - 2) If a match is found and sufficient space is not available in the LLDP remote systems information repository for the updated information in the TLVs just received, then follow the tooManyNeighbors procedure (9.2.8.5.1) to determine whether to discard the TLVs from a new neighbor or to delete information from an existing neighbor that is already in the LLDP remote systems information repository.
 - 3) If a match is found, and sufficient space is available (or has been made available) in the LLDP remote systems information repository for the updated information in the TLVs just received, then:
 - i) Replace all information associated with the MSAP identifier in the LLDP remote systems information repository that was previously received in a Normal LLDPDU with the information in the just received Normal LLDPDU.
 - ii) For each received Extension LLDPDU associated with the MSAP identifier, if any, replace all information in the LLDP remote systems information repository that was previously received in an Extension LLDPDU of the same XPDU Number with the information in the just received Extension LLDPDU. If a received Extension LLDPDU has an XPDU Number that has not been previously received, add the new information to the LLDP remote system information repository entry.
 - iii) Set the control variable rxChanges to TRUE.
 - iv) Set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.
 - 4) If no match is found and sufficient space is not available in the LLDP remote systems information repository, then follow the tooManyNeighbors procedure (9.2.8.5.1) to determine whether to discard the TLVs from a new neighbor or to delete information from an existing neighbor that is already in the LLDP remote systems information repository.
 - 5) If no match is found and sufficient space is available (or has been made available), create a new information repository structure to receive information associated with the new MSAP identifier, set these information repository objects to the values indicated in their respective TLVs, set the control variable rxChanges to TRUE, and set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.

If an incoming TLV is not recognized by the receiving LLDP agent, the TLV is stored in the LLDP remote systems information repository as follows:

- b) If the TLV type value is in the range of the reserved TLV types in Table 8-1, the TLV can be from a later version of the basic management set and is stored according to the basic TLV format shown in Figure 8-2. These TLVs are indexed by their TLV type.
- c) If the TLV type value is 127, the TLV is an Organizationally Specific TLV and is stored according to the basic format for Organizationally Specific TLVs shown in Figure 8-15. These TLVs are indexed by their organizationally unique identifier and organizationally defined TLV subtype.

9.2.8.5.1 Too many neighbors

When there is insufficient space in the LLDP remote systems information repository to store information from a new neighbor something needs to be discarded. Either the received LLDPDU is discarded or existing information within the current remote systems information repository is discarded in order to make space for the new information received in the LLDPDU. If the space required to store the information from the new neighbor is larger than the storage space available to the remote systems information repository then the received LLDPDU shall be discarded rather than discarding existing information in the remote systems information repository. The procedure is as follows:

- a) Set the flag variable `tooManyNeighbors` to TRUE.
- b) If the information selected to be discarded is the information in the current LLDPDU:
 - 1) Set the `tooManyNeighborsTimer` as specified in Equation (3).

$$\text{tooManyNeighborsTimer} = \max (\text{tooManyNeighborsTimer}, \text{rxTTL}) \quad (3)$$

where `rxTTL` is the TTL value in the current LLDPDU

- 2) Discard the current LLDPDU and increment the `statsFramesDiscardedTotal` counter.
- 3) Set the control variable `rxChanges` to FALSE. Wait for the next LLDPDU.
- c) If the information selected to be discarded is currently in the LLDP remote systems information repository:
 - 1) From the data in the remote information repository, select a neighbor.
 - 2) Delete all information associated with the selected neighbor's MSAP identifier from the LLDP remote systems information repository, and destroy the associated extended receive state machine if one exists.
 - 3) Set the `tooManyNeighborsTimer` as specified in Equation (4).

$$\text{tooManyNeighborsTimer} = \max (\text{tooManyNeighborsTimer}, \text{rxInfoTTL}) \quad (4)$$

where `rxInfoTTL` = the selected neighbor's TTL value

- 4) If sufficient space exists to store the information received in the current LLDPDU in the LLDP remote systems information repository, set control variable `rxChanges` to TRUE.
- 5) If sufficient space still does not exist to store the information received in the current LLDPDU in the LLDP remote systems information repository, perform steps 1–4 again.

The variable `tooManyNeighbors` is automatically set to FALSE whenever the `tooManyNeighborsTimer` expires.

NOTE—The `tooManyNeighborsTimer` is not precise, as it is only cleared (and the `tooManyNeighbors` variable set FALSE) by timer expiration, and not by the removal of the too many neighbors condition.

9.2.8.6 rxInitializeLLDP()

The `rxInitializeLLDP()` procedure initializes the LLDP receive module. The `rxInitializeLLDP()` procedure waits till `portEnabled` is equal to TRUE and after the variable `portEnabled` is equal to TRUE, the LLDP receive module is initialized or re-initialized for frame reception. During this process, the local LLDP agent performs the following tasks:

- a) The variable `tooManyNeighbors` is set to FALSE.
- b) All information in the remote systems information repository associated with this LLDP agent is deleted.

1 9.2.8.7 rxProcessFrame()

2 The rxProcessFrame() procedure:

- 3 a) Validates the LLDPDU, sets the value of rxType, and validate the TLVs contained in the LLDPDU
4 as specified in 9.2.8.7.1.
5

6 9.2.8.7.1 LLDPDU validation

7 The receive module processes each incoming LLDPDU as it is received. The statsFramesInTotal counter for
8 the port is incremented and the LLDPDU is checked to verify the presence of the three mandatory TLVs at
9 the beginning of the LLDPDU as defined in 8.2.

- 10 a) The first TLV is extracted:
- 11 1) If the extracted TLV type value does not match the Chassis ID TLV type:
- 12 i) The LLDPDU is discarded.
- 13 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
14 incremented.
- 15 iii) The rxType value is set equal to invalid.
- 16 iv) The procedure rxProcessFrame() is terminated.
- 17 2) If the extracted TLV type value matches the Chassis ID TLV type, and the chassis ID TLV
18 information string length is not within the range $2 \leq n \leq 256$:
- 19 i) The LLDPDU is discarded.
- 20 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
21 incremented.
- 22 iii) The rxType value is set equal to invalid.
- 23 iv) The procedure rxProcessFrame() is terminated.
- 24 3) If the extracted TLV type value matches the Chassis ID TLV type, and the chassis ID TLV
25 information string length is within the range $2 \leq n \leq 256$, the chassis ID value is extracted to
26 become the first part of the MSAP identifier.
- 27 b) The second TLV is extracted:
- 28 1) If the extracted TLV type value does not match the Port ID TLV type:
- 29 i) The LLDPDU is discarded.
- 30 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
31 incremented.
- 32 iii) The rxType value is set equal to invalid.
- 33 iv) The procedure rxProcessFrame() is terminated.
- 34 2) If the extracted TLV type value matches the Port ID TLV type, and the port ID TLV
35 information string length is not within the range $2 \leq n \leq 256$:
- 36 i) The LLDPDU is discarded.
- 37 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
38 incremented.
- 39 iii) The rxType value is set equal to invalid.
- 40 iv) The procedure rxProcessFrame() is terminated.
- 41 3) If the extracted TLV type value matches the Port ID TLV type, and the port ID TLV
42 information string length is within the range $2 \leq n \leq 256$, the port ID value is extracted and
43 appended to the chassis ID value to complete construction of the MSAP identifier.
- 44 c) The third TLV is extracted:

- 1) If the extracted TLV type value does not match the Time To Live, Extension Request, or Extension TLV type, the LLDPDU is invalid:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
- 2) If the extracted TLV type value matches the Time To Live or Extension TLV type, and the adminStatus variable is enableTxOnly, the LLDPDU is discarded because the receive state machine is only enabled to receive Extension Request LLDPDUs.
 - i) The LLDPDU is discarded.
 - ii) The rxType value is set equal to invalid.
 - iii) The procedure rxProcessFrame() is terminated.
- 3) If the extracted TLV type value matches the Time To Live TLV type, and the Time To Live TLV information string length is less than 2:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
- 4) If the extracted TLV type value matches the Time To Live TLV type, and the Time To Live TLV information string length is greater than or equal to 2:
 - i) The first two octets of the TLV information string is extracted and rxTTL is set to this value.
 - ii) If rxTTL equals zero, the rxType value is set equal to shutdown and the procedure rxProcessFrame() is terminated.
 - iii) If rxTTL is non-zero, the rxType value is set to normal.
- 5) If the extracted TLV type value matches the Extension Request TLV type, XLLDP is supported, and the received MSAP identifier and Scope MAC Address match the MSAP and Scope MAC Address of this LLDP agent:
 - i) The rxType value is set equal to xreq.
 - ii) The procedure rxProcessFrame() is terminated.
- 6) If the extracted TLV type value matches the Extension Request TLV type, and either XLLDP is not supported or the received MSAP identifier and Scope MAC Address combination does not match the MSAP and Scope MAC Address combination of this LLDP agent:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
- 7) If the extracted TLV type value matches the Extension TLV type, XLLDP is supported, and an extended receive state machine exists for the neighbor identified by the received MSAP and Scope MAC Address:
 - i) The rxType value is set equal to xpdu.
- 8) If the extracted TLV type value matches the Extension TLV type, and either XLLDP is not supported or an extended receive state machine does not exist for the neighbor identified by the received MSAP and Scope MAC Address:

- 1 i) The LLDPDU is discarded.
- 2 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
- 3 incremented.
- 4 iii) The rxType value is set equal to invalid.
- 5 iv) The procedure rxProcessFrame() is terminated.

6 Each of the remaining TLV information elements are decoded in succession as required by their particular
7 TLV format definitions:

- 8 d) If the TLV type value equals 0, the TLV is the End Of LLDPDU TLV, and the procedure
9 rxProcessFrame() is terminated.
- 10 e) If the TLV_type_value is less than 127 and is recognized by the LLDP implementation, it is
11 validated according to the general rules for all TLVs defined in 9.2.8.7.2 as well as any specific rules
12 defined for the particular TLVs defined in 8.5.
- 13 f) If the TLV_type_value is less than 127 and is not recognized by the LLDP implementation, it may
14 be a basic TLV from a later LLDP version. The statsTLVsUnrecognizedTotal counter is
15 incremented, and the TLV is assumed to be validated.
- 16 g) If the implementation supports XLLDP and the TLV type value matches the Manifest TLV type:
 - 17 1) Calculate the additional required space in the LLDP remote system information repository to
18 store new or updated neighbor information:
 - 19 i) If the MSAP identifier in the current LLDPDU does not match an MSAP identifier in the
20 LLDP remote systems information repository, the total information repository size is
21 extracted from the Manifest TLV and used as the additional required space.
 - 22 ii) If the MSAP identifier in the current LLDPDU matches an MSAP identifier in the LLDP
23 remote system, the additional required space is the difference between the total
24 information repository entry size as extracted from the Manifest TLV and the current size
25 of the data associated with the MSAP identifier in the LLDP remote system information
26 repository.
 - 27 2) If there is insufficient space in the LLDP remote systems information repository to store the
28 neighbor information (based on the additional required space calculated in 1 above), and the
29 policy is to discard the current LLDPDU rather than delete information for other MSAPs or if
30 there would be insufficient space after deleting all possible information for other MSAPs from
31 the remote system information repository then:
 - 32 i) The flag variable tooManyNeighbors is set to TRUE.
 - 33 ii) If the rxTTL value in the LLDPDU is greater than the value of the
34 tooManyNeighborsTimer, the tooManyNeighborsTimer is set to the rxTTL value.
 - 35 iii) The statsFrameDiscardedTotal counter is incremented.
 - 36 iv) The rxType value is changed to invalid.
 - 37 3) If there is sufficient space in the LLDP remote systems information repository or the policy
38 allows deleting information from other MSAPs rather than discarding the current LLDPDU
39 and sufficient space can be created (base on the space calculated in 1 above), then:
 - 40 i) The rxType value is changed to manifest.
 - 41 ii) If an extended receive state machine does not already exist for the MSAP identifier in the
42 current LLDPDU, an extended receive state machine for that MSAP identifier is created
43 and its createExtRx variable is set to TRUE.
- 44 h) If the TLV type value is 127, the TLV is an Organizationally Specific TLV:
 - 45 1) If the TLV's organizationally unique identifier and organizationally defined subtype are
46 recognized, the TLV is validated according to the general rules for all TLVs defined in 9.2.8.7.2
47 as well as the general rules for Organizationally Specific TLVs defined in 9.2.8.7.3 plus any
48 specific rules defined for the particular TLV.

- 1 2) If the TLV's organizationally unique identifier and/or organizationally defined subtype are not
2 recognized, the statsTLVsUnrecognizedTotal counter is incremented, and the TLV is assumed
3 to be validated.
- 4 i) If the end of the LLDPDU has been reached the procedure rxProcessFrame() is terminated.

5 **9.2.8.7.2 General validation rules for all TLVs**

6 The value in the TLV information string length field is the value used to validate the TLV and to indicate the
7 location of the next TLV in the LLDPDU.

8 All TLVs are subject to the general validation rules listed in this subclause as well as any specific usage rules
9 defined for the particular TLV (for example, see systems capabilities TLV usage rules in 8.5.8.3):

- 10 a) If the LLDPDU contains more than one Chassis ID TLV, Port ID TLV, or Time To Live TLV:
 - 11 1) The LLDPDU is discarded.
 - 12 2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 13 3) The rxType value is set equal to invalid.
 - 14 4) The procedure rxProcessFrame() is terminated.
- 15 b) If the implementation supports XLLDP and the LLDPDU contains more than one Manifest TLV,
16 Extension Request TLV, or Extension Identifier TLV:
 - 17 1) The LLDPDU is discarded.
 - 18 2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 19 3) The rxType value is set equal to invalid.
 - 20 4) The procedure rxProcessFrame() is terminated.
- 21 c) If the LLDPDU contains more than one out of the 3 TLV types Time to Live TLV, Extension
22 Request TLV and Extension TLV:
 - 23 1) The LLDPDU is discarded.
 - 24 2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 25 3) The rxType value is set equal to invalid.
 - 26 4) The procedure rxProcessFrame() is terminated.
- 27 d) If the LLDPDU contains both a Manifest TLV and an Extension TLV:
 - 28 1) The LLDPDU is discarded.
 - 29 2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 30 3) The rxType value is set equal to invalid.
 - 31 4) The procedure rxProcessFrame() is terminated.
- 32 e) If the TLV information string length value is not greater than or equal to the sum of the lengths of all
33 fields contained in the TLV information string:
 - 34 1) The LLDPDU is discarded.
 - 35 2) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 36 3) The rxType value is set equal to invalid.
 - 37 4) The procedure rxProcessFrame() is terminated.
- 38 f) If any TLV contains an error condition specified for that particular TLV:
 - 39 1) The TLV is discarded.
 - 40 2) The statsTLVsDiscardedTotal counter is incremented and if the statsFramesInErrors has not
41 been previously updated for this LLDPDU then the statsFramesInErrorsTotal counter is
42 incremented.
 - 43 3) The location of the next TLV is based on the TLV information string length value of the current
44 TLV.
- 45 g) With the exception of the TTL TLV for which specific rules apply (see 9.2.8.7.1), if the length of
46 any field of a TLV is outside the length range specified for that particular TLV (for example, the

- 1 TLV information string length is greater than the maximum permitted length for the TLV
2 information string as specified for that TLV):
- 3 1) The TLV is discarded.
 - 4 2) The statsTLVsDiscardedTotal counter is incremented and if the statsFramesInErrors has not
5 been previously updated for this LLDPDU then the statsFramesInErrorsTotal counter is
6 incremented.
 - 7 3) The location of the next TLV is based on the TLV information string length value of the current
8 TLV.
- 9 h) If any TLV extends past the physical end of the frame:
- 10 1) The TLV is discarded.
 - 11 2) The statsTLVsDiscardedTotal counter is incremented and if the statsFramesInErrors has not
12 been previously updated for this LLDPDU then the statsFramesInErrorsTotal counter is
13 incremented.
- 14 i) Any information following an End Of LLDPDU TLV is ignored.
- 15 NOTE—Usage rules for individual TLVs allow some TLVs to appear more than once in an LLDPDU. Duplicate TLVs
16 result in any one of the values being placed in the information repository, can cause the discard stats to increment, and
17 can cause the change marker for the information repository entry to change if any of the TLV copies change the value
18 even if the value finally recorded is unchanged. The only thing guaranteed is that the information repository value is set
19 to one (unspecified) of the TLV values, and if that value is different to what was previously in the information repository
20 then the change marker is set.

21 9.2.8.7.3 General validation rules for all Organizationally Specific TLVs

22 If an Organizationally Specific TLV is not recognized by the receiving LLDP agent, the content of the TLV
23 can be ignored but the TLV is stored in the LLDP remote systems information repository for possible
24 retrieval by network management using the basic Organizationally Specific TLV format (see 8.7) and the
25 StatsTLVsUnrecognizedTotal counter is incremented. If more than one unrecognized Organizationally
26 Specific TLV is received with the same OI and organizationally defined subtype, but with identifiable
27 differences in the organizationally defined information strings, all copies are assigned a temporary
28 identification index and stored.

29 9.2.8.8 somethingChangedLocal()

30 This procedure is called to indicate that the status/value of one or more of the selected objects in the LLDP
31 local system information repository has changed, and that there is therefore a need to transmit new
32 information. It is used to notify the managers of the PTOPO and other optional MIBs or YANG modules that
33 something has changed in the LLDP local systems information repository.

34 This procedure also sets the value of the variable localChange (see 9.2.5.2) to signal the need for the LLDP
35 agent(s) to transmit the new information, and copies the value of the variable txFastInit (see 9.2.5.20) to the
36 variable txFast (see 9.2.5.19) to ensure that the new values will be seen by the neighbors on the port.

37 It is assumed that the system provides the ability for any processes that need to receive such indications to
38 register with this procedure so that appropriate signals can be received by those processes when the
39 procedure is called.

1 **9.2.8.9 somethingChangedRemote()**

2 This procedure is called after all the information associated with the particular MSAP identifier has been
3 updated in the LLDP remote systems information repository. The procedure call serves as an indication to
4 the managers of the PTOPO and other optional MIBs or YANG modules that one or more of the following
5 conditions is true:

- 6 a) That the status/value of one or more objects in the LLDP remote systems information repository
7 associated with the particular MSAP identifier has changed.
- 8 b) That an incoming LLDPDU contained an MSAP identifier requiring creation of a new information
9 repository structure in the LLDP remote systems information repository to receive the information
10 in that LLDPDU.
- 11 c) That information in the LLDP remote systems information repository associated with the MSAP
12 identifier has been deleted.

13 It is assumed that the system provides the ability for any processes that need to receive such indications to
14 register with this procedure so that appropriate signals can be received by those processes when the
15 procedure is called.

16 **9.2.8.10 txAddCredit()**

17 This procedure increments the value of the txCredit variable (9.2.5.17) by one if the current value of the
18 txCredit variable is less than the value of txCreditMax (9.2.5.18).

19 **9.2.8.11 txFrame()**

20 The txFrame() procedure makes use of the services of LLC to transmit the LLDPDU constructed by either
21 the rpConstrInfoLLDPDU(), rpConstrShutdownLLDPDU(), constructXreqPDU(), or rxProcessXreq()
22 procedures (9.2.8.2, 9.2.8.3, 9.2.8.16, 9.2.8.18). For Normal LLDPDUs, the destination MAC address used
23 is the MAC address associated with the instance of the LLDP agent (see 7.1.1 and 9.2.5.3). For Extension
24 Request and Extension LLDPDUs, the destination MAC address used is the Return MAC Address in the
25 received Manifest TLV or Extension Request TLV respectively (see 8.6.1.2 and 8.6.2.2). The source MAC
26 address used is the individual MAC address of the transmitting port. The EtherType used is the LLDP
27 EtherType identified in Table 7-3.

28 If the frame conveys a Manifest LLDPDU, the sentManifest variable is set to TRUE; otherwise the
29 sentManifest variable is set to FALSE.

30 After the frame has been transmitted, the statsFramesOutTotal counter (9.2.7.5) is incremented.

31 **9.2.8.12 txInitializeLLDP()**

32 The txInitializeLLDP() procedure initializes the LLDP transmit module. It performs the following tasks:

- 33 a) If applicable, either the non-volatile configuration for the LLDP local system information repository
34 are retrieved or the appropriate default values are assigned to all LLDP configuration variables.
- 35 b) The internal (implementation specific) data structures are initialized with appropriate local physical
36 topology information.
- 37 c) The setManifest variable is set to FALSE.
- 38 d) System default values are set for the following timing parameters:
 - 39 1) reinitDelay (9.2.5.9)
 - 40 2) msgTxHold (9.2.5.5)
 - 41 3) msgTxInterval (9.2.5.6)

1 4) msgFastTx (9.2.5.4)

2 NOTE—It is recommended to introduce a degree of stagger into starting the LLDP local agent initialization in
3 multi-port implementations so that the timing of LLDP frame transmission cycles can be distributed among system
4 ports, and distributed among supported MAC addresses on each port. The method of accomplishing the stagger is an
5 implementation issue and is beyond the scope of this standard.

6 **9.2.8.13 rxProcessManifest()**

7 The rxProcessManifest() procedure performs the following tasks:

- 8 a) Form the MSAP identifier from the chassis ID and Port ID of the received LLDPDU.
- 9 b) All TLVs received in the Manifest LLDPDU and passed from the receive state machine are retained
10 in temporary storage associated with the MSAP identifier calculated in step a), replacing any TLVs
11 retained from a previous Manifest LLDPDU.
- 12 c) If there is no current entry in the LLDP remote systems information repository for the MSAP
13 identifier calculated in step a), or if the current entry does not contain a Manifest TLV, all XPDU
14 Descriptors in the new Manifest TLV are assigned a status of “update”.
- 15 d) If there is a current manifest in the LLDP remote systems information repository for the MSAP
16 identifier calculated in step a), each XPDU Descriptor in the new manifest is compared to the XPDU
17 Descriptor in the current manifest. Any XPDU Descriptors in the new manifest that match an XPDU
18 Descriptor in the current manifest are assigned a status of “current”. Any XPDU Descriptors in the
19 new manifest that do not have a matching XPDU Descriptor in the current manifest are assigned a
20 status of “update”.
- 21 e) If there are any XPDU in temporary storage associated with this MSAP identifier that match an
22 XPDU Descriptor with a status of “update” in the new manifest, those XPDU are retained and the
23 status of the corresponding XPDU Descriptors is changed to “new”. Any XPDU in temporary
24 storage associated with the MSAP identifier calculated in step a) that do not match an XPDU
25 descriptor received in the Manifest message with a status of update are discarded.

26 NOTE—Usage rules for the Manifest TLV specify that each XPDU Descriptor has a unique XPDU Number. There is no
27 requirement for the LLDP agent receiving a Manifest TLV to check for duplicate XPDU Numbers, and the response of
28 the receiving LLDP agent to duplicated XPDU Numbers is unspecified and implementation dependent.

29 **9.2.8.14 rxProcessXPDU()**

30 The rxProcessXPDU() procedure performs the following tasks:

- 31 a) The statsFramesInTotal counter for the port is incremented.
- 32 b) The 32-bit XPDU Check value of the XPDU is calculated. If the XPDU Number, Sequence, and
33 Check do not match an XPDU Descriptor in the new manifest, the statsFramesDiscardedTotal and
34 statsFramesInErrorsTotal counters are both incremented and the procedure rxProcessXPDU() is
35 terminated.
- 36 c) If the XPDU Number, Sequence, and Check match an XPDU Descriptor in the new manifest, the
37 status of that XPDU Descriptor is changed to “new”.
- 38 d) Validates the TLVs contained in the LLDPDU as defined in 9.2.8.7.
- 39 e) Retains all validated TLVs in temporary local storage associated with this XPDU Descriptor.
- 40 f) If there are any XPDU Descriptors in the new manifest with a status of “requested” or “retried”, the
41 variable xreqComplete is set to FALSE. Otherwise the variable xreqComplete is set to TRUE.

42 **9.2.8.15 checkManifest()**

43 The checkManifest() procedure reviews the status of all XPDU Descriptors in the new manifest. If the status
44 of one or more XPDU Descriptors is “update”, “requested”, or “retried”, the manifestComplete variable is
45 set to FALSE. Otherwise the manifestComplete variable is set to TRUE and rxTimer is set to rxTTL.

1 **9.2.8.16 constructXreqPDU()**

2 The constructXreqPDU() procedure performs the following tasks:

- 3 a) If there are any XPDU Descriptors with a status of “requested” in the new manifest, these XPDU
4 Descriptors are selected to be included in the Extension Request TLV, the status of these XPDU
5 Descriptors is changed to “retried”, and rxTimer is set to rxTTL.
- 6 b) If there are no XPDU Descriptors with a status of “requested” or “retried” in the new manifest, one
7 or more XPDU Descriptors with a status of “update” are selected to be included in the request. The
8 status of these XPDU Descriptors is changed to “requested”, and rxTimer is set to the value of
9 rxTTL divided by 32 and rounded up to the nearest integer.
- 10 c) If any XPDU have been selected in steps a) or b) then construct an Extension Request LLDPDU
11 (8.2) including an Extension Request TLV, structured according to the specification in 8.6.2,
12 containing the selected XPDU Descriptors..

13 The LLDP agent generating the Extension Request LLDPDU can pace the rate at which Extension
14 LLDPDUs arrive by adjusting the number of XPDU Descriptors in a single Extension Request TLV and the
15 timing of the Extension Request LLDPDU transmission. This pacing is implementation dependent and
16 beyond the scope of this standard. An LLDP agent only transmits an Extension Request LLDPDU when it is
17 prepared (e.g., has sufficient buffer resources) to receive all requested Extension LLDPDUs.

18 **9.2.8.17 checkRxTimer()**

19 The checkRxTimer() procedure reviews the status of all XPDU Descriptors in the new manifest. If the value
20 of rxTimer is zero and there are any XPDU Descriptors with a status of “requested”, then this is a timeout for
21 an Extension Request LLDPDU and the rxTTLExpired variable is set to FALSE. If the value of rxTimer is
22 zero and there are no XPDU Descriptors with a status “requested”, then this is a timeout for the TLV
23 collection process and rxTTLExpired is set to TRUE.

24 **9.2.8.18 rxProcessXREQ()**

25 If the MSAP identifier and Scope MAC Address in the received Extension Request LLDPDU match this
26 LLDP agent, then for each XPDU Descriptor in the Extension Request TLV that matches one of the XPDU
27 Descriptors in the most recent Manifest TLV created by the LLDP manager and transmitted in a Normal
28 LLDPDU, an Extension LLDPDU (8.2) is constructed, including an Extension Identifier TLV structured
29 according to the specification in 8.6.3, and the other TLVs selected by the LLDP manager, and transmits the
30 frame using the txFrame() procedure.

31 NOTE—When the Extension Request TLV includes more than one XPDU Descriptor, the rate at which Extension
32 LLDPDUs are transmitted is at the discretion of the transmitting agent and is implementation dependent.

9.2.9 Transmit state machine

The transmit state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-1 and the attendant definitions contained in 9.2.3 through 9.2.8.

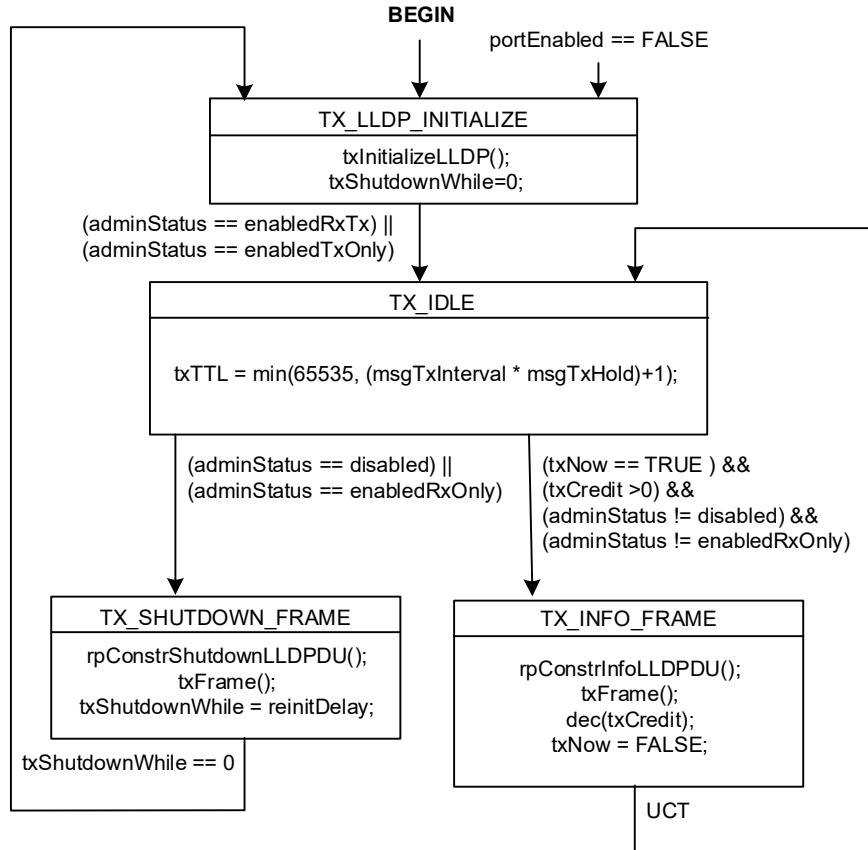


Figure 9-1—Transmit state machine

9.2.10 Receive state machine

The receive state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-2 and the attendant definitions contained in 9.2.2 through 9.2.8. Additionally, when an implementation supports Extended LLDP and a Manifest TLV is received in an LLDPDU, an extended receive state machine for the received chassis ID and port ID shall implement the function specified by the state diagram contained in Figure 9-3 and the attendant definitions contained in 9.2.2 through 9.2.8.

Because there can be multiple extended receive state machines, the receive state machine uses a notational convention to reference per-neighbor variables (9.2.6) associated with an extended receive state machine. The variable is prefaced by “MSAP.” to indicate a variable associated with the specific MSAP Identifier contained in the frame currently being processed by the receive state machine. The variable is prefaced with “any.” to indicate the logical OR of the variable from all extended state machines.

An extended receive state machine associated with a neighbor is dynamically instantiated, if it does not already exist, when a Normal LLDPDU is received that contains a Manifest TLV. It is destroyed, and any

¹ TLVs held in temporary storage associated with this state machine are discarded, when a Normal LLDPDU
² is received that does not contain a Manifest TLV, or by the assertion of BEGIN.

1

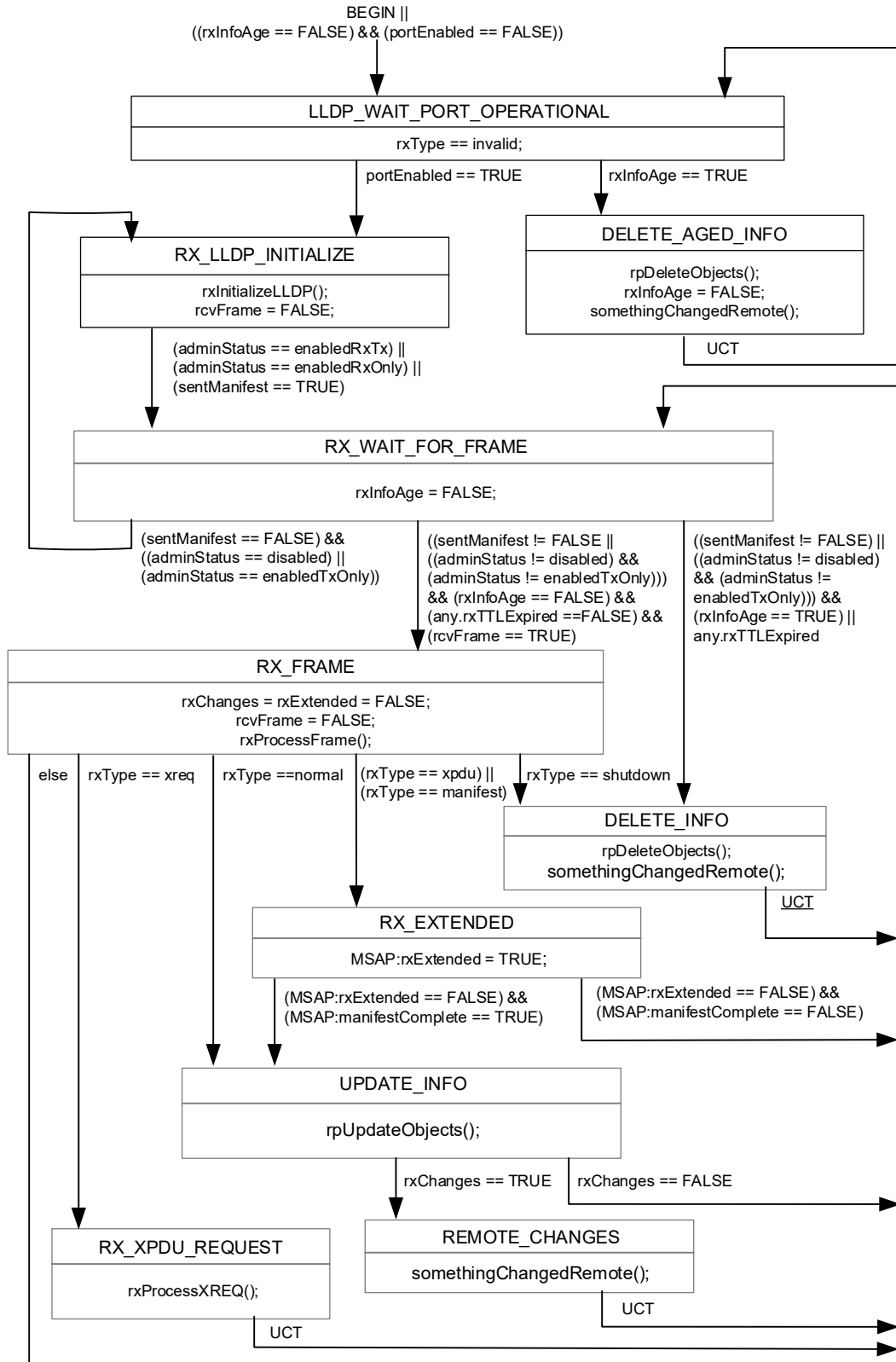


Figure 9-2— Receive state machine

1

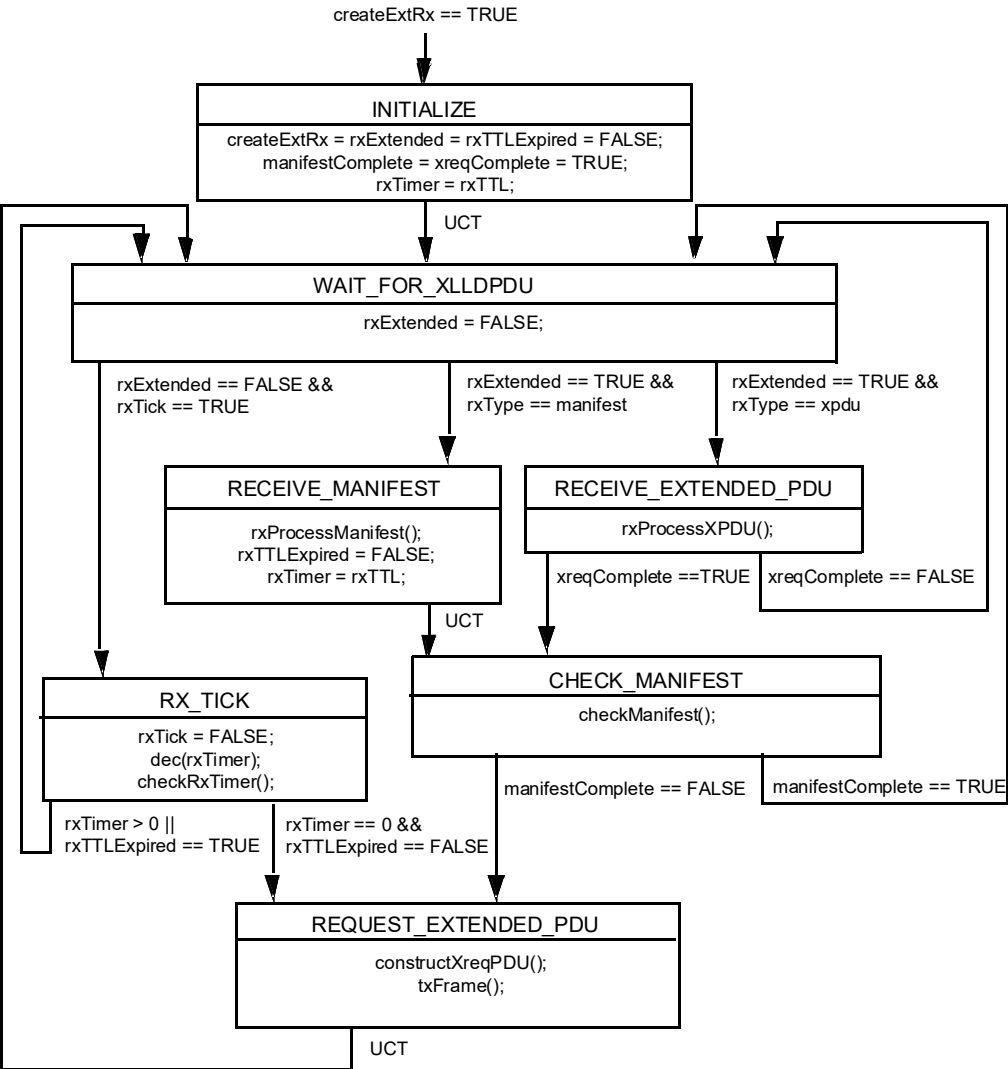


Figure 9-3—Extension receive state machine

2

3

4

1 **9.2.11 Transmit timer state machine**

2 The transmit timer state machine for an individual port and destination MAC address (see 7.1) shall
3 implement the function specified by the state diagram contained in Figure 9-4 and the attendant definitions
4 contained in 9.2.3 through 9.2.8.

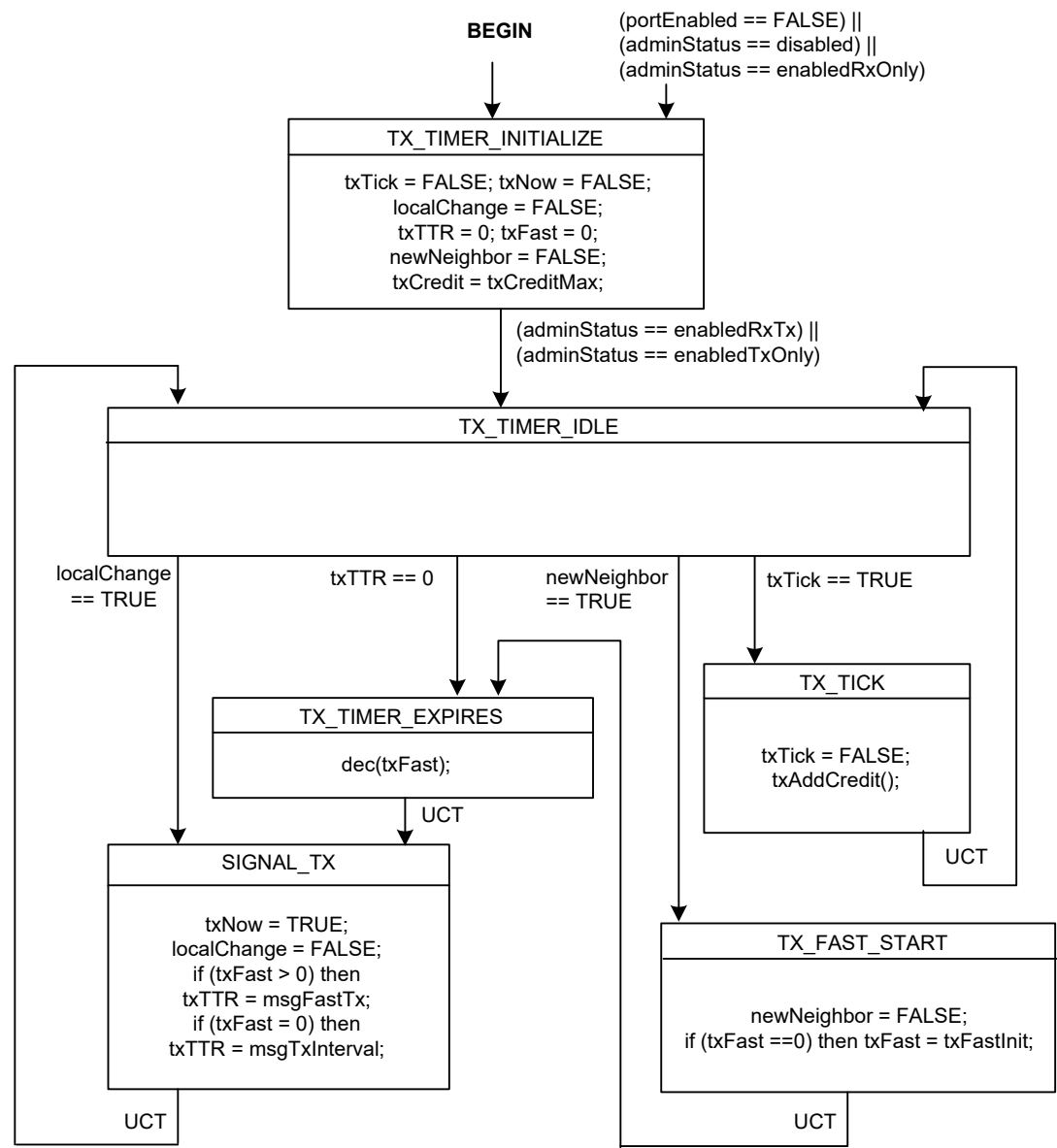


Figure 9-4—Transmit timer state machine

1 10. LLDP management

2 This clause defines the set of managed objects and their functionality that allow administrative configuration
3 and monitoring of LLDP operation.

4 10.1 Data storage and retrieval

5 LLDP agents need to have a place to store both information about the local system and information they
6 have received about remote systems. Data storage and retrieval capability to support the functionality
7 defined in 7, 8, 9, and 10 shall be provided. No particular implementation is implied.

8 10.2 The LLDP management entity's responsibilities

9 The LLDP management entity has the following responsibilities:

- 10 a) Starting the transmit and receive state machines.
- 11 b) Providing a means for the network manager to select TLVs to be included in outgoing LLDPDUs.
- 12 c) Extracting the necessary information from the LLDP local system information repository and
13 constructing the individual TLVs selected for insertion into an outgoing LLDPDU.
- 14 d) Prepending appropriate addressing and LLDP EtherType to the LLDPDU before submitting the
15 MA_UNITDATA.request for frame transmission.
- 16 e) Updating the LLDP remote systems information repository and monitoring for information
17 repository object adds, deletions and value changes.
- 18 f) Maintaining operational statistics.

19 10.2.1 Protocol initialization management

20 Protocol initialization consists of the following:

- 21 a) Retrieving non-volatile configuration values for the LLDP local system information repository.
- 22 b) Assigning default or management assigned values to LLDP configuration variables.
- 23 c) Loading physical topology information into their associated LLDP local system information
24 repository objects.

25 10.2.2 TLV selection management

26 TLV selection management consists of providing the network manager with the means to select which
27 specific TLVs are enabled for inclusion in an LLDPDU. The following LLDP variables cross reference to
28 LLDP local systems configuration information repository tables indicating which specific TLVs are enabled
29 for the particular port(s) on the system. The specific port(s) through which each TLV is enabled for
30 transmission may be set (or reset) by the network manager.

- 31 a) **lldpV2PortConfigTLVsTxEnableV2:** This variable lists the single-instance-use basic management
32 TLVs, each with a bit map indicating the system ports through which the referenced TLV is enabled
33 for transmission.
- 34 b) **lldpV2ManAddrConfigTxEnable:** This variable lists the different management addresses (by
35 subtype) that are defined for the system, each with a bit map indicating the system ports through
36 which the particular management address/subtype TLV is enabled for transmission.

37 NOTE—Implementers of new TLVs should be aware that provision needs to be made to allow similar network
38 management selection of new TLVs and that doing so requires linkage to the LLDP information repository, regardless of
39 whether or not the TLV is part of the basic management set or part of an Organizationally Specific TLV set.

1 When XLLDP is not supported, the LLDP agent can only transmit the TLVs that fit within a single Normal
2 LLDPU. If more TLVs are selected for inclusion in the LLDPU than fit, some TLVs are not transmitted.
3 Which TLVs are not transmitted is implementation dependent and not determined by this standard.

4 When XLLDP is supported, each TLV selected for transmission is mapped to the Normal LLDPU or to an
5 Extension LLDPU. Each Extension LLDPU is identified by an XPDU Number. The mapping of TLVs to
6 XPDU is implementation dependent. The LLDP management entity maintains a list of which TLVs are
7 mapped to which XPDU Number, and uses this list to create a Manifest TLV for inclusion in a Manifest
8 LLDPU at the start of a transmission cycle. The addition, deletion, or modification of any TLV mapped to
9 Extension LLDPU since the last Manifest LLDPU was transmitted cause the XPDU Revision for that
10 Extension LLDPU to be incremented prior to creating a new Manifest TLV. Maintaining a consistent
11 mapping of TLVs to Extension LLDPU whenever possible reduces the number of Extension LLDPU
12 that need to be requested and transmitted during a transmission cycle.

13 10.2.3 Transmission management

14 Transmission management consists of the following:

- 15 a) Determining when a new transmission cycle is required, as signaled by the
16 somethingChangedLocal() procedure (see 9.2.8.8), or when an Extension LLDPU has been
17 requested by the rxProcessXREQ() procedure (see 9.2.8.18).
- 18 b) Extracting the appropriate LLDP local system information repository information for the three
19 mandatory TLVs and inserting them at the beginning of the LLDPU in proper format order.
- 20 c) Creating a Manifest TLV for inclusion in the Normal LLDPU that starts a transmission cycle if
21 XLLDP is supported and there are TLVs that are mapped to one or more Extension LLDPU.
- 22 d) Extracting the appropriate LLDP local system information repository information for the selected
23 optional TLVs and inserting them into the LLDPU.
- 24 e) Optionally, appending an End Of LLDPU TLV after the last optional TLV in the LLDPU.
- 25 f) Maintaining length control during LLDPU construction so that the TLVs do not exceed the
26 maximum length allowed for the LLDPU.
- 27 g) Submitting the frame for transmission.

28 10.2.4 Reception management

29 Reception management consists of the following:

- 30 a) Receiving and parsing the incoming LLDPU.
- 31 b) Validating, and checking the types of the first three TLVs to ensure that they are the required type
32 and in LLDPU format order.
- 33 c) Validating optional TLVs and extracting information values.
- 34 d) Checking whether or not the current LLDPU represents a new MSAP identifier.
- 35 e) Monitoring whether or not there is sufficient space available in the LLDP remote systems
36 information repository for all active neighbors.
- 37 f) Arbitrating which information is to be discarded in cases where insufficient space is available in the
38 LLDP remote systems information repository for all active neighbors.
- 39 g) Checking whether or not the received information represents a status or value change to the existing
40 information repository object.
- 41 h) Updating remote systems information repository objects as necessary.

42 When XLLDP is supported, the LLDP management entity associates each TLV that is stored in the remote
43 systems information repository with the XPDU Number of the Extension LLDPU in which it was
44 received. (For convenience an XPDU Number of zero can be used for TLVs received in a Normal LLDPU,

1 since no valid Extension LLDPDU has an XPDU Number equal to zero.) This association is necessary so
2 that the appropriate TLVs are updated or deleted when a new Extension LLDPDU with that XPDU Number
3 is received.

4 **10.2.5 Performance management**

5 Performance management consists of the following:

- 6 a) Monitoring the LLDP local system information repository update activities for status or value
7 changes to selected information repository objects and:
 - 8 1) Calling the somethingChangedLocal() procedure (see 9.2.8.8) whenever a status/value change
9 occurs.
 - 10 2) Initiating a new transmit cycle if txCredit > 0.
- 11 b) Monitoring the txTTR timer for countdown expiration and initiating a new transmit cycle if txCredit
12 > 0.
- 13 c) Monitoring the rxInfoTTL timer for countdown expiration and:
 - 14 1) Deleting the associated objects from the LLDP remote systems information repository
15 whenever the timing counter expires.
 - 16 2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.
- 17 d) Monitoring rxTTL in the incoming LLDPDUs for shutdown indication and:
 - 18 1) Deleting the associated objects from the LLDP remote systems information repository
19 whenever rxTTL = 0.
 - 20 2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.
- 21 e) Monitoring the “tooManyNeighbors” variable to determine whether an existing neighbor’s
22 information needs to be deleted and:
 - 23 1) Deleting the objects associated with a selected MSAP identifier from the LLDP remote systems
24 information repository whenever existing information is selected to be deleted.
 - 25 2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.
- 26 f) Monitoring the tooManyNeighborsTimer to determine when there is not sufficient space available to
27 accommodate information from all active neighbors and setting the variable tooManyNeighbors
28 associated with the port to FALSE when the tooManyNeighborsTimer expires.
- 29 g) Notifying the managers of the PTOPO MIB and other optional MIB and YANG modules (see Figure
30 11-1) whenever the procedures somethingChangedLocal() or somethingChangedRemote() are
31 performed.
- 32 h) Maintaining operational statistics regarding the LLDP frames sent and received.

33 **10.3 Managed objects**

34 Managed objects model the semantics of management operations. Operations upon a managed object supply
35 information concerning, or facilitate control over, the process or entity associated with that managed object.

36 Management of LLDP is described in terms of the managed resources that are associated with individual
37 TLVs and that support frame transmission and reception.

38 **10.4 Data types**

39 This subclause specifies the semantics of operations independent of their encoding in management protocol.
40 The data types of the parameters of operations are defined for that specification.

1 The following data types are used:

- 2 a) Boolean.
- 3 b) Enumerated, for a collection of named values.
- 4 c) Unsigned, for all parameters specified as “number of” some quantity.
- 5 d) MAC address.
- 6 e) Time interval, an Unsigned value representing a positive integral number of seconds for all time out
- 7 parameters.
- 8 f) Counter, for all parameters specified as a “count” of some quantity (all counters increment and wrap
- 9 with a modulus of 2 to the power 32).

10 10.5 LLDP variables

11 LLDP managed objects fall into the following categories:

- 12 a) Status/control variables necessary for operation of the protocol.
- 13 b) Variables that accumulate operational statistics.
- 14 c) Variables that are required by the particular TLVs.

15 10.5.1 LLDP operational status and control

- 16 a) Global parameters and variables:
 - 17 1) **adminStatus**: The authority that controls whether or not a local LLDP agent is enabled for
 - 18 transmit and receive, transmit only, or receive only; or is disabled (9.2.5.1).
- 19 b) Transmit state machine parameters and variables:
 - 20 1) **msgTxHold**: A multiplier on msgTxInterval used to compute the TTL value of txTTL (9.2.5.5,
 - 21 9.2.5.23).
 - 22 2) **msgTxInterval**: The interval between successive transmit cycles in normal transmission mode
 - 23 (9.2.5.6).
 - 24 3) **reinitDelay**: The delay after adminStatus becomes ‘disabled’ before re-initialization is
 - 25 attempted (9.2.5.9).
 - 26 4) **txCreditMax**: The maximum value of transmission credit (9.2.5.18).
 - 27 5) **txFastInit**: The interval between successive LLDPDU transmissions in fast transmission mode
 - 28 (9.2.5.20).
- 29 c) Receive state machine parameters and variables:
 - 30 1) **remoteChanges**: An indication that the status/value of one or more selected objects in the
 - 31 LLDP remote systems information repository has changed or that all objects associated with a
 - 32 particular MSAP identifier have been deleted (9.2.5.10).
 - 33 2) **tooManyNeighbors**: An indication that there is insufficient space in the LLDP remote systems
 - 34 information repository to store information from all active neighbors (9.2.5.16).

35 10.5.2 LLDP operational statistics counters

36 The following counters provide operational statistics on a per port basis:

- 37 a) Transmission counters:
 - 38 1) **statsFramesOutTotal**: A count of all LLDP frames transmitted through the port (9.2.7.5).
 - 39 2) **statsLengthErrorsTotal**: A count of all LLDP length errors detected when constructing
 - 40 LLDPDU frames for transmission through the port (9.2.8.2).
- 41 b) Reception counters:
 - 42 1) **statsAgeoutsTotal**: A count of the times that a neighbor’s information is deleted from the
 - 43 LLDP remote systems information repository because of rxInfoTTL timer expiration (9.2.7.1).

- 1 2) **statsFramesDiscardedTotal:** A count of all LLDPDUs received and then discarded (9.2.7.2).
- 2 3) **statsFramesInErrorsTotal:** A count of all LLDPDUs received at the port with one or more
- 3 detectable errors (9.2.7.3).
- 4 4) **statsFramesInTotal:** A count of all LLDP frames received at the port (9.2.7.4).
- 5 5) **statsTlvsDiscardedTotal:** A count of all TLVs received at the port and discarded for any
- 6 reason. (9.2.7.6)
- 7 6) **statsTLVsUnrecognizedTotal:** This counter provides a count of all TLVs not recognized by
- 8 the receiving LLDP local agent (9.2.7.7).

9 **10.5.3 TLV required variables**

10 Variables in this category are defined by the requirements of the particular TLVs. They are maintained with
11 reference to the local system in the LLDP local system information repository; and with reference to the
12 remote system, in the LLDP remote systems information repository. TLV variables are outputs from the
13 local LLDP agent and inputs to the remote LLDP agent.

14 Variables that pertain to basic set TLVs are listed in the following subclauses.

15 **10.5.3.1 Chassis ID TLV objects**

- 16 a) **chassis ID subtype:** The type of identifier used for the chassis (see 8.5.2.2).
- 17 b) **chassis ID:** The identification assigned to the chassis containing the port (see 8.5.2.3).

18 **10.5.3.2 Port ID TLV objects**

- 19 a) **port ID subtype:** The type of identifier used for the port (see 8.5.3.2).
- 20 b) **port ID:** The identification assigned to the port (see 8.5.3.3).

21 **10.5.3.3 Port description TLV object**

- 22 a) **port description:** The port's description (see 8.5.5.2).

23 **10.5.3.4 System name TLV object**

- 24 a) **system name:** The system's assigned name (see 8.5.6.2).

25 **10.5.3.5 System description TLV object**

- 26 a) **system description:** The system's description (see 8.5.7.2).

27 **10.5.3.6 System capabilities TLV objects**

- 28 a) **system capabilities:** The primary capabilities of the system (see 8.5.8.1).
- 29 b) **enabled capabilities:** The system's enabled capabilities (see 8.5.8.2).

30 **10.5.3.7 Management address TLV objects**

- 31 a) **management address length:** The length of the management address (see 8.5.9.2).
- 32 b) **management address subtype:** The management address type (see 8.5.9.3).
- 33 c) **management address:** The management address (see 8.5.9.4).
- 34 d) **interface numbering subtype:** The interface type (see 8.5.9.5).
- 35 e) **interface number:** The interface number (see 8.5.9.6).

- 1 f) **OID length:** The object identifier length (see 8.5.9.7).
- 2 g) **OID:** The object identifier (see 8.5.9.8).

11. LLDP MIB definitions

This clause defines the LLDP basic MIB for use with SNMP in TCP/IP based internets. In particular, it defines objects for managing the operation of LLDP based on the specifications of 7, 8, 9, and 10.

11.1 Internet Standard Management Framework

LLDP MIBs are designed to operate in a manner consistent with the principles of the Internet Standard Management Framework, which describes the separation of a data modeling language (for example, SMIV2) from content-specific data models (for example, the LLDP remote systems MIB), and from messages and protocol operations used to manipulate the data (for example, SNMPv3).

For a detailed overview of the documents that describe the current Internet Standard Management Framework, please refer to section 7 of IETF RFC 3410.

Managed objects are accessed via a virtual information store, termed the Management Information Base or MIB. MIB objects are generally accessed through the Simple Network Management Protocol (SNMP). Objects in the MIB are defined using the mechanisms defined in the Structure of Management Information (SMI). This clause specifies a MIB module that is compliant to the SMIV2, which is described in IETF RFC 2578 (STD 58) [B2], IETF RFC 2579 (STD 58) [B3], and IETF RFC 2580 (STD 58) [B4].

11.2 Structure of the LLDP MIB

The LLDP MIB consists of two types of MIB modules, the mandatory basic MIB defined in this clause and from zero to n optional organizationally specific MIB extensions such as the IEEE 802.1 MIB in Annex D of IEEE Std 802.1Q-2022 and the IEEE 802.3 MIB in Clause 79 of IEEE Std 802.3-2022.

Each MIB module is divided into two major sections as shown in Figure 11-1 to allow selective MIB support for the particular operating mode (transmit only, receive only, or both transmit and receive) being implemented.

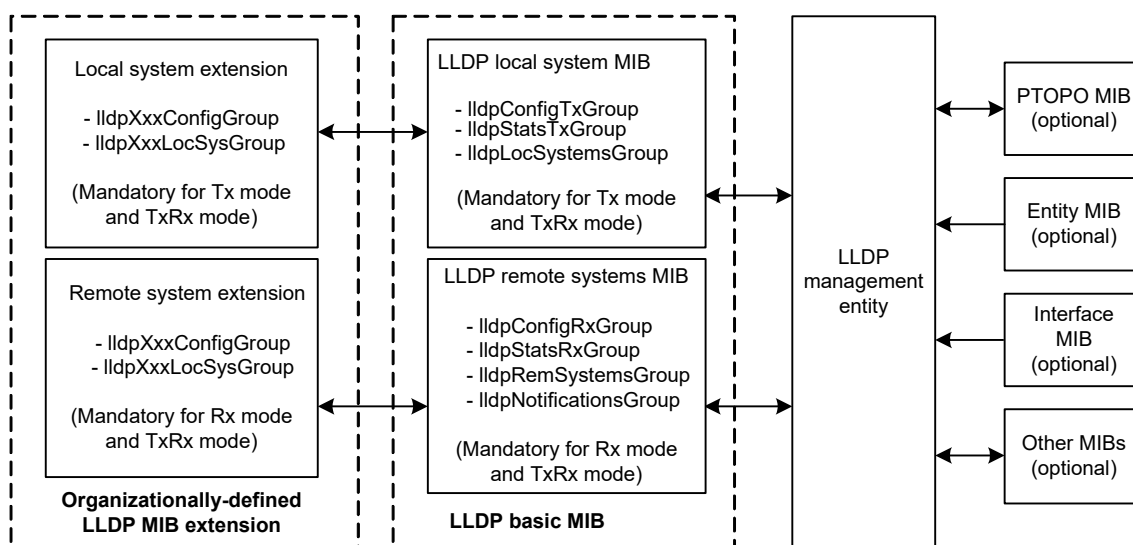


Figure 11-1—LLDP MIB block diagram

1 Table 11-1 summarizes the particular object groups that are required for each operating mode. The basic MIB
2 shall comply with the MIB conformance section for the particular operating mode (Rx, Tx, or TxRx mode)
3 being supported.

Table 11-1—MIB object groups and operating mode applicability

MIB group	Rx mode	Tx mode	TxRx mode
lldpV2ConfigGroup			
lldpV2ConfigRxGroup	M ^a	—	M
lldpV2ConfigTxGroup	—	M	M
lldpV2StatsRxGroup	M	—	M
lldpV2StatsTxGroup	—	M	M
lldpV2LocSysGroup	—	M	M
lldpV2RemSysGroup	M	—	M
lldpV2NotificationsGroup	M	—	M
ifGeneralInformationGroup	M	M	M

^aM=Mandatory

4 Table 11-2 shows the structure of the MIB and the relationship of the MIB objects to the LLDP operational
5 status/control variables, LLDP statistics variables, and TLV variables.

Table 11-2—LLDP MIB structure and object cross reference

MIB table	MIB object	LLDP reference
<i>LLDP Configuration group</i>		
	lldpV2MessageTxInterval	msgTxInterval, 9.2.5.6
	lldpV2MessageTxHoldMultiplier	msgTxHold, 9.2.5.5
	lldpV2ReinitDelay	reinitDelay, 9.2.5.9
	lldpV2NotificationInterval	msgTxInterval, 9.2.5.6
	lldpV2TxCreditMax	txCreditMax, 9.2.5.18
	lldpV2MessageFastTx	msgFastTx, 9.2.5.4
	lldpV2TxFastInit	txFastInit, 9.2.5.20

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
lldpV2PortConfigTableV2		
	lldpV2PortConfigIfIndexV2	(Table index)
	lldpV2PortConfigDestAddressIndexV2	(Table index)
	lldpV2PortConfigAdminStatusV2	adminStatus, 9.2.5.1
	lldpV2PortMessageTxInterval	msgTxInterval, 9.2.5.6
	lldpV2PortMessageTxHoldMultiplier	msgTxHold, 9.2.5.5
	lldpV2PortReinitDelay	reinitDelay, 9.2.5.9
	lldpV2PortNotificationInterval	msgTxInterval, 9.2.5.6
	lldpV2PortTxCreditMax	txCreditMax, 9.2.5.18
	lldpV2PortMessageFastTx	msgFastTx, 9.2.5.4
	lldpV2PortTxFastInit	txFastInit, 9.2.5.20
	lldpV2PortConfigNotificationEnableV2	—
	lldpV2PortConfigTLVsTxEnableV2	9.1.1.1
lldpV2DestAddressTable		
	lldpV2AddressTableIndex	(Table index)
	lldpV2DestMacAddress	(Table index)
lldpV2ManAddrConfigTxPortsTable		
	lldpV2ManAddrConfigIfIndex	(Table index)
	lldpV2ManAddrConfigDestAddressIndex	(Table index)
	lldpV2ManAddrConfigLocManAddrSubtype	8.5.9.3 (Table index)
	lldpV2ManAddrConfigLocManAddr	8.5.9.4 (Table index)
	lldpV2ManAddrConfigTxEnable	9.1.1.1
	lldpV2ManAddrConfigRowStatus	—
<i>LLDP Statistics group</i>		
	lldpV2StatsRemTablesLastChangeTime	—
	lldpV2StatsRemTablesInserts	—
	lldpV2StatsRemTablesDeletes	—
	lldpV2StatsRemTablesDrops	—
	lldpV2StatsRemTablesAgeouts	—

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
lldpV2StatsTxPortTable		
	lldpV2StatsTxIfIndex	(Table index)
	lldpV2StatsTxDestMACAddress	(Table index)
	lldpV2StatsTxPortFramesTotal	statsFramesOutTotal, 9.2.7.5
	lldpV2StatsTxLLDPDULengthErrors	lldpduLengthErrors, 9.2.7.8
lldpV2StatsRxPortTable		
	lldpV2StatsRxDestIfIndex	(Table index)
	lldpV2StatsRxDestMACAddress	(Table index)
	lldpV2StatsRxPortFramesDiscardedTotal	statsFramesDiscardedTotal, 9.2.7.2
	lldpV2StatsRxPortFramesErrors	statsFramesInErrorsTotal, 9.2.7.3
	lldpV2StatsRxPortFramesTotal	statsFramesInTotal, 9.2.7.4
	lldpV2StatsRxPortTLVsDiscardedTotal	statsTLVsDiscardedTotal, 9.2.7.6
	lldpV2StatsRxPortTLVsUnrecognizedTotal	statsTLVsUnrecognizedTotal, 9.2.7.7
	lldpV2StatsRxPortAgeoutsTotal	statsAgeoutsTotal, 9.2.7.1
<i>Local System Data group</i>		
	lldpV2LocChassisIdSubtype	chassis ID subtype, 8.5.2.2
	lldpV2LocChassisId	chassis ID, 8.5.2.3
	lldpV2LocSysName	system name, 8.5.6.2
	lldpV2LocSysDesc	system description, 8.5.7.2
	lldpV2LocSysCapSupported	system capabilities, 8.5.8.1
	lldpV2LocSysCapEnabled	enabled capabilities, 8.5.8.2
lldpV2LocPortTable		
	lldpV2LocPortIfIndex	(Table index)
	lldpV2LocPortIdSubtype	port ID subtype, 8.5.3.2
	lldpV2LocPortId	port ID, 8.5.3.3
	lldpV2LocPortDesc	port description, 8.5.5.2
lldpV2LocManAddrTable		
	lldpV2LocManAddrSubtype	management address subtype, 8.5.9.3 (Table index)
	lldpV2LocManAddr	management address, 8.5.9.4 (Table index)
	lldpV2LocManAddrLen	management address string length, 8.5.9.2
	lldpV2LocManAddrIfSubtype	interface numbering subtype, 8.5.9.5

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
	lldpV2LocManAddrIfId	interface number, 8.5.9.6
	lldpV2LocManAddrOID	object identifier, 8.5.9.8
<i>Remote Systems Data group</i>		
lldpV2RemTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemChassisIdSubtype	chassis ID subtype, 8.5.2.2
	lldpV2RemChassisId	chassis ID, 8.5.2.3
	lldpV2RemPortIdSubtype	port ID sybtype, 8.5.3.2
	lldpV2RemPortId	port ID, 8.5.3.3
	lldpV2RemPortDesc	port description, 8.5.5.2
	lldpV2RemSysName	system name, 8.5.6.2
	lldpV2RemSysDesc	system description, 8.5.7.2
	lldpV2RemSysCapSupported	system capabilities, 8.5.8.1
	lldpV2RemSysCapEnabled	enabled capabilities, 8.5.8.2
	lldpV2RemRemoteChanges	remoteChanges, 9.2.5.10
	lldpV2RemTooManyNeighbors	tooManyNeighbors, 9.2.5.16
lldpV2RemManAddrTable		(Table index)
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemManAddrSubtype	management address subtype, 8.5.9.3 (Table index)
	lldpV2RemManAddr	management address, 8.5.9.4 (Table index)
	lldpV2RemManAddrIfSubtype	interface numbering subtype, 8.5.9.5
	lldpV2RemManAddrIfId	interface number, 8.5.9.6
	lldpV2RemManAddrOID	object identifier, 8.5.9.8

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
lldpV2RemUnknownTLVTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemUnknownTLVType	LLDPDU validation, 9.2.8.7.1 (Table index)
	lldpV2RemUnknownTLVInfo	LLDPDU validation, 9.2.8.7.1
lldpV2RemOrgDefInfoTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemOrgDefInfoOUI	organizationally identifier, 8.7.1.3 (Table index)
	lldpV2RemOrgDefInfoSubtype	organizationally defined subtype, 8.7.1.4 (Table index)
	lldpV2RemOrgDefInfoIndex	(Table index)
	lldpV2RemOrgDefInfo	organizationally defined information, 8.7.1.5
LLDP MIB Notifications		
	lldpV2RemTablesChange	

11.3 Relationship to other MIBs

This clause includes specifications for an LLDP Textual Conventions MIB module, an LLDP MIB module, and for IEEE 802.1 and IEEE 802.3 extension MIB modules that are compliant with the SMIV2 as defined in IETF RFC 2578 (STD 58) [B2], IETF RFC 2579 (STD 58) [B3], and IETF RFC 2580 (STD 58) [B4].

LLDP is designed to operate in conjunction with MIB modules defined by the IETF, IEEE 802, and others. The following subclauses discuss the relationship between LLDP and IETF MIB modules. LLDP agents automatically notify the managers of these MIB modules whenever there is a value or status change in an LLDP MIB object.

LLDP managed objects for the local system are stored in the LLDP local system MIB. Information received from a remote LLDP agent is stored in the local LLDP agent's LLDP remote system MIB.

NOTE—In this standard, managed objects for an LLDP MIB module are defined as they would be for an SNMP MIB. However, it is not required for LLDP implementations to support SNMP to store and retrieve system data. LLDP agents need to have a place to store both information about the local system and information they have received about remote systems. No particular implementation is implied.

1 11.3.1 Relationship to LLDP Version 1 MIB

2 The version 1 MIB module that was published in the initial 2005 publication of this standard has been
3 superseded by the version 2 MIB module specified in 11.5.1, and support of the version 2 module is a
4 requirement for conformance to the required or optional capabilities (Clause 5) in this revision of the
5 standard. The version 2 MIB module reflects changes in indexation of the MIB objects that support the use
6 of LLDP with multiple destination MAC addresses, as discussed in Clause 6.

7 In addition to the changes in indexation, the version 2 MIB module also supports changes to the
8 management of the LLDP protocol that have resulted from changes to the LLDP protocol state machines. As
9 these protocol changes affect only the timing and frequency of transmission of LLDPDUs, and not the
10 content of the LLDPDUs, version 1 and version 2 implementations can successfully co-exist and
11 interoperate.

12 From the perspective of a version 2 implementation, a version 1 neighbor behaves as a version 2 neighbor
13 that is only using a single destination MAC address (the 01-80-C2-00-00-0E address specified for use in the
14 IEEE 802.1AB-2005 version of the standard). From the perspective of a version 1 implementation, a version
15 2 neighbor that uses the 01-80-C2-00-00-0E destination MAC address behaves as a version 1 neighbor.

16 11.3.2 IETF Physical Topology MIB

17 The IETF Physical Topology (PTOPO) MIB (IETF RFC 2922 [B8]) allows a LLDP agent to expose learned
18 physical topology information, using an IETF MIB. LLDP is intended to support the PTOPO MIB.

19 NOTE—The LLDP MIB module is a logical superset of the IETF PTOPO MIB; therefore, from a functional point of
20 view, if the LLDP MIB module is implemented, it is likely not to be necessary to implement the PTOPO MIB defined in
21 IETF RFC 2922 [B8]. However, for backward compatibility, this standard also supports the use of the PTOPO MIB.

22 11.3.3 IETF Entity MIB

23 The Entity MIB (IETF RFC 6933) allows the physical component inventory and hierarchy to be identified.
24 Chassis IDs passed in the LLDPDU can identify entPhysicalTable entries. SNMP agents that implement the
25 LLDP MIB should implement the entPhysicalAlias object from the Entity MIB version 2 or higher.

26 11.3.4 IETF Interfaces MIB

27 The Interfaces MIB (IETF RFC 2863) provides a standard mechanism for managing network ports. Port IDs
28 passed in the LLDPDU can identify ifTable (or entPhysicalTable) entries. SNMP agents that implement the
29 LLDP MIB shall also implement the ifTable and ifXTable for the ports that are represented in the Interfaces
30 MIB.

31 11.4 Security considerations for LLDP base MIB module

32 There are a number of management objects defined in this MIB module with a MAX-ACCESS clause of
33 read-write.¹³ Such objects may be considered sensitive or vulnerable in some network environments. The
34 support for SET operations in a non-secure environment without proper protection can have a negative
35 effect on network operations.

36 a) Setting the following objects to incorrect values can result in an excessive number of LLDP packets
37 being sent by the LLDP agent:

- 38 1) lldpV2MessageTxInterval, lldpV2PortMessageTxInterval
- 39 2) lldpV2TxCreditMax, lldpV2PortTxCreditMax

¹³ In IETF MIB definitions, the MAX-ACCESS clause defines the type of access that is allowed for particular data elements in the MIB. An explanation of the MAX-ACCESS mappings is given in section 7.3 of IETF RFC 2578 [B2].

- 1 3) lldpV2MessageFastTx, lldpV2PortMessageFastTx
- 2 4) lldpV2TxFastInit, lldpV2PortTxFastInit
- 3 b) Setting the object, lldpV2MessageTxHoldMultiplier or lldpV2PortMessageTxHoldMultiplier, to
- 4 incorrect values can cause the LLDP agent to transmit LLDPDUs with too-high TTL values, which
- 5 affect the expiration time of objects grouped under lldpV2RemoteSystemsData identifier.
- 6 c) Setting the object, lldpV2ReinitDelay or lldpV2PortReinitDelay, to too low a value can cause the
- 7 transmit state machine to attempt excessive re-initializations.
- 8 d) Setting incorrect bits in the object, lldpV2PortConfigTLVsTxEnableV2, can cause the LLDP agent
- 9 to transmit LLDPDUs with an undesired optional TLV sequence.
- 10 e) Setting incorrect bits in the object, lldpV2ConfigManAddrPortsTxEnable, can cause the LLDP
- 11 agent to advertise management addresses that were not meant to be disclosed and/or to omit
- 12 addresses that were desired.
- 13 f) Setting the following objects to incorrect values can result in improper operation of the MIB
- 14 notification process:
- 15 1) lldpV2NotificationInterval
- 16 2) lldpV2PortNotificationInterval
- 17 3) lldpV2PortConfigNotificationEnableV2
- 18 4) lldpV2ManAddrConfigTxEnable
- 19 5) lldpV2ManAddrConfigRowStatus
- 20 g) Setting the object, lldpV2PortConfigAdminStatusV2, to the incorrect value can result in enabling a
- 21 non-desired operational mode.

22 The following readable objects in this MIB module may be considered to be sensitive or vulnerable in some
23 network environments:

- 24 h) Objects that are associated with the transmit mode
- 25 1) lldpV2LocChassisIdSubtype
- 26 2) lldpV2LocChassisId
- 27 3) lldpV2LocPortIdSubtype
- 28 4) lldpV2LocPortId
- 29 5) lldpV2LocPortDesc
- 30 6) lldpV2LocSysName
- 31 7) lldpV2LocSysDesc
- 32 8) lldpV2LocSysCapSupported
- 33 9) lldpV2LocSysCapEnabled
- 34 10) lldpV2LocManAddrLen
- 35 11) lldpV2LocManAddrIfSubtype
- 36 12) lldpV2LocManAddrIfId
- 37 13) lldpV2LocManAddrOID
- 38 i) Objects that are associated with the receive mode
- 39 1) lldpV2NotificationInterval
- 40 2) lldpV2PortConfigNotificationEnable
- 41 3) lldpV2RemChassisIdSubtype
- 42 4) lldpV2RemChassisId
- 43 5) lldpV2RemPortIdSubtyp
- 44 6) lldpV2RemPortId
- 45 7) lldpV2RemPortDesc
- 46 8) lldpV2RemSysName
- 47 9) lldpV2RemSysDesc
- 48 10) lldpV2RemSysCapSupported
- 49 11) lldpV2RemSysCapEnabled

- 1 12) lldpV2RemManAddrIfSubtype
- 2 13) lldpV2RemManAddrIfId
- 3 14) lldpV2RemManAddrOID
- 4 15) lldpV2RemUnknownTLVInfo
- 5 16) lldpV2RemOrgDefInfo

6 This concern applies both to objects that describe the configuration of the local host, as well as for objects
7 that describe information from the remote hosts, acquired via LLDP and displayed by the objects in this
8 MIB module. It is thus important to control even GET and/or NOTIFY access to these objects and possibly
9 to even encrypt the values of these objects when sending them over the network via SNMP.

10 SNMP versions prior to SNMPv3 did not include adequate security. Even if the network itself is secure (for
11 example, by using IPSec), even then, there is no control as to who on the secure network is allowed to
12 access and GET/SET (read/change/create/delete) the objects in this MIB module.

13 Implementers should consider the security features as provided by the SNMPv3 framework (see
14 IETF RFC 3410, section 8), including full support for the SNMPv3 cryptographic mechanisms (for
15 authentication and privacy).

16 Further, implementers should not deploy SNMP versions prior to SNMPv3. Instead, implementers should
17 deploy SNMPv3 to enable cryptographic security. It is then a customer/operator responsibility to ensure that
18 the SNMP entity giving access to an instance of this MIB module is properly configured to give access to the
19 objects only to those principals (users) that have legitimate rights to indeed GET or SET
20 (change/create/delete) them.

21 **11.5 LLDP MIB modules** ^{14, 15}

22 Two MIB modules are defined—a textual conventions MIB module (11.5.1) that contains the textual
23 conventions used by the LLDP version 2 MIB module and by the version 2 extension MIB module in
24 Annex D of IEEE Std 802.1Q-2022, and the LLDP version 2 MIB module itself (11.5.2). The textual
25 conventions defined in the Textual-Convention MIB module are also available by extension MIB modules
26 defined in other standards, such as the extension MIB modules defined in IEEE Std 802.1Q.

27

¹⁴ *Copyright release for MIBs:* Users of this standard may freely reproduce the MIB contained in this subclause so that it can be used for its intended purpose.

¹⁵ An ASCII version of this MIB module can be obtained by Web browser from the IEEE 802.1 Website at <http://www.ieee802.org/1/pages/MIBS.html>.

11.5.1 Definitions for the LLDP-V2-TC MIB module

In the following MIB module, should any discrepancy between the DESCRIPTION text and the corresponding definition in Clause 9 and Clause 10 occur, the definition in Clause 9 and Clause 10 shall take precedence.

```
LLDP-V2-TC-MIB DEFINITIONS ::= BEGIN
IMPORTS
    MODULE-IDENTITY,
    Unsigned32,
    org
        FROM SNMPv2-SMI
    TEXTUAL-CONVENTION
        FROM SNMPv2-TC;

lldpV2TcMIB MODULE-IDENTITY
    LAST-UPDATED "202602120000Z" -- February 12, 2026
    ORGANIZATION "IEEE 802.1 Working Group"
    CONTACT-INFO
        " WG-URL: http://www.ieee802.org/1/
          WG-EMail: stds-802-1-L@ieee.org

          Contact: IEEE 802.1 Working Group Chair
          Postal: C/O IEEE 802.1 Working Group
                  IEEE Standards Association
                  445 Hoes Lane
                  Piscataway
                  NJ 08854
                  USA
          E-mail: stds-802-1-L@ieee.org"
    DESCRIPTION
        "Textual conventions used throughout the IEEE Std 802.1AB
        version 2 and later MIB modules.

        Unless otherwise indicated, the references in this
        MIB module are to IEEE 802.1AB-2026.

        The TCs in this MIB are taken from the original LLDP-MIB,
        LLDP-EXT-DOT1-MIB, and LLDP-EXT-DOT3-MIB published in
        IEEE Std 802-1D-2004, with the addition of TCs to support
        the management address table. They have been made available
        as a separate TC MIB module to facilitate referencing from
        other MIB modules.

        Copyright (C) IEEE (2026). This version of this MIB module
        is published as 11.5.1 of IEEE Std 802.1AB-2026;
        see the standard itself for full legal notices."

    REVISION "202602120000Z" -- February 12, 2026

    DESCRIPTION
        "Published as part of IEEE Std 802.1AB-2026.
        This revision updated references and resolved comments on the
        first Standards Association Ballot."

    REVISION "201603110000Z" -- March 11, 2016

    DESCRIPTION
        "Published as part of IEEE Std 802.1AB-2016.
```

```
1           This revision updated references."
2
3     REVISION "200906080000Z" -- June 08, 2009
4
5
6     DESCRIPTION
7         "Published as part of IEEE Std 802.1AB-2009 revision."
8
9     ::= { org ieee(111) standards-association-numbers-series-standards(2)
10         lan-man-stds(802) ieee802dot1(1) 1 12 }
11
12 --
13 -- Definition of the root OID arc for IEEE 802.1 MIBs
14 --
15
16 ieee802dot1mibs OBJECT IDENTIFIER
17     ::= { org ieee(111) standards-association-numbers-series-standards(2)
18         lan-man-stds(802) ieee802dot1(1) 1 }
19
20 --
21 -- *****
22 --
23 -- Textual Conventions
24 --
25 -- *****
26
27 LldpV2ChassisIdSubtype ::= TEXTUAL-CONVENTION
28     STATUS         current
29     DESCRIPTION
30         "This TC describes the source of a chassis identifier.
31
32         The enumeration 'chassisComponent(1)' represents a chassis
33         identifier based on the value of entPhysicalAlias object
34         (defined in IETF RFC 6933) for a chassis component (i.e.,
35         an entPhysicalClass value of 'chassis(3)').
36
37         The enumeration 'interfaceAlias(2)' represents a chassis
38         identifier based on the value of ifAlias object (defined in
39         IETF RFC 2863) for an interface on the containing chassis.
40
41         The enumeration 'portComponent(3)' represents a chassis
42         identifier based on the value of entPhysicalAlias object
43         (defined in IETF RFC 6933) for a port or backplane
44         component (i.e., entPhysicalClass value of 'port(10)' or
45         'backplane(4)'), within the containing chassis.
46
47         The enumeration 'macAddress(4)' represents a chassis
48         identifier based on the value of a unicast source address
49         (encoded in network byte order and IEEE 802.3 canonical bit
50         order), of a port on the containing chassis as defined in
51         IEEE Std 802.
52
53         The enumeration 'networkAddress(5)' represents a chassis
54         identifier based on a network address, associated with
55         a particular chassis. The encoded address is actually
56         composed of two fields. The first field is a single octet,
57         representing the IANA AddressFamilyNumbers value for the
58         specific address type, and the second field is the network
59         address value.
```

```

1
2     The enumeration 'interfaceName(6)' represents a chassis
3     identifier based on the value of ifName object (defined in
4     IETF RFC 2863) for an interface on the containing chassis.
5
6     The enumeration 'local(7)' represents a chassis identifier
7     based on a locally defined value."
8 SYNTAX  INTEGER {
9     chassisComponent(1),
10    interfaceAlias(2),
11    portComponent(3),
12    macAddress(4),
13    networkAddress(5),
14    interfaceName(6),
15    local(7)
16 }
17
18 LldpV2ChassisId ::= TEXTUAL-CONVENTION
19     DISPLAY-HINT "1x:"
20     STATUS      current
21     DESCRIPTION
22         "This TC describes the format of a chassis identifier string.
23         Objects of this type are always used with an associated
24         LldpChassisIdSubtype object, which identifies the format of
25         the particular LldpChassisId object instance.
26
27         If the associated LldpChassisIdSubtype object has a value of
28         'chassisComponent(1)', then the octet string identifies
29         a particular instance of the entPhysicalAlias object
30         (defined in IETF RFC 6933) for a chassis component (i.e.,
31         an entPhysicalClass value of 'chassis(3)').
32
33         If the associated LldpChassisIdSubtype object has a value
34         of 'interfaceAlias(2)', then the octet string identifies
35         a particular instance of the ifAlias object (defined in
36         IETF RFC 2863) for an interface on the containing chassis.
37         If the particular ifAlias object does not contain any values,
38         another chassis identifier type should be used.
39
40         If the associated LldpChassisIdSubtype object has a value
41         of 'portComponent(3)', then the octet string identifies a
42         particular instance of the entPhysicalAlias object (defined
43         in IETF RFC 6933) for a port or backplane component within
44         the containing chassis.
45
46         If the associated LldpChassisIdSubtype object has a value of
47         'macAddress(4)', then this string identifies a particular
48         unicast source address (encoded in network byte order and
49         IEEE 802.3 canonical bit order), of a port on the containing
50         chassis as defined in IEEE Std 802.
51
52         If the associated LldpChassisIdSubtype object has a value of
53         'networkAddress(5)', then this string identifies a particular
54         network address, encoded in network byte order, associated
55         with one or more ports on the containing chassis. The first
56         octet contains the IANA Address Family Numbers enumeration
57         value for the specific address type, and octets 2 through
58         N contain the network address value in network byte order.
59

```

1 If the associated LldpChassisIdSubtype object has a value
2 of 'interfaceName(6)', then the octet string identifies
3 a particular instance of the ifName object (defined in
4 IETF RFC 2863) for an interface on the containing chassis.
5 If the particular ifName object does not contain any values,
6 another chassis identifier type should be used.
7
8 If the associated LldpChassisIdSubtype object has a value of
9 'local(7)', then this string identifies a locally assigned
10 Chassis ID."
11 SYNTAX OCTET STRING (SIZE (1..255))
12
13 LldpV2PortIdSubtype ::= TEXTUAL-CONVENTION
14 STATUS current
15 DESCRIPTION
16 "This TC describes the source of a particular type of port
17 identifier used in the LLDP MIB.
18
19 The enumeration 'interfaceAlias(1)' represents a port
20 identifier based on the ifAlias MIB object, defined in IETF
21 RFC 2863.
22
23 The enumeration 'portComponent(2)' represents a port
24 identifier based on the value of entPhysicalAlias (defined in
25 IETF RFC 6933) for a port component (i.e., entPhysicalClass
26 value of 'port(10)'), within the containing chassis.
27
28 The enumeration 'macAddress(3)' represents a port identifier
29 based on a unicast source address (encoded in network
30 byte order and IEEE 802.3 canonical bit order), which has
31 been detected by the agent and associated with a particular
32 port (IEEE Std 802).
33
34 The enumeration 'networkAddress(4)' represents a port
35 identifier based on a network address, detected by the agent
36 and associated with a particular port.
37
38 The enumeration 'interfaceName(5)' represents a port
39 identifier based on the ifName MIB object, defined in IETF
40 RFC 2863.
41
42 The enumeration 'agentCircuitId(6)' represents a port
43 identifier based on the agent-local identifier of the circuit
44 (defined in IETF RFC 3046), detected by the agent and associated
45 with a particular port.
46
47 The enumeration 'local(7)' represents a port identifier
48 based on a value locally assigned."
49
50 SYNTAX INTEGER {
51 interfaceAlias(1),
52 portComponent(2),
53 macAddress(3),
54 networkAddress(4),
55 interfaceName(5),
56 agentCircuitId(6),
57 local(7)
58 }
59

```
1 LldpV2PortId ::= TEXTUAL-CONVENTION
2   DISPLAY-HINT "1x:"
3   STATUS      current
4   DESCRIPTION
5       "This TC describes the format of a port identifier string.
6       Objects of this type are always used with an associated
7       LldpPortIdSubtype object, which identifies the format of the
8       particular LldpPortId object instance.
9
10      If the associated LldpPortIdSubtype object has a value of
11      'interfaceAlias(1)', then the octet string identifies a
12      particular instance of the ifAlias object (defined in IETF
13      RFC 2863). If the particular ifAlias object does not contain
14      any values, another port identifier type should be used.
15
16      If the associated LldpPortIdSubtype object has a value of
17      'portComponent(2)', then the octet string identifies a
18      particular instance of the entPhysicalAlias object (defined
19      in IETF RFC 6933) for a port or backplane component.
20
21      If the associated LldpPortIdSubtype object has a value of
22      'macAddress(3)', then this string identifies a particular
23      unicast source address (encoded in network byte order
24      and IEEE 802.3 canonical bit order) associated with the port
25      (IEEE Std 802).
26
27      If the associated LldpPortIdSubtype object has a value of
28      'networkAddress(4)', then this string identifies a network
29      address associated with the port. The first octet contains
30      the IANA AddressFamilyNumbers enumeration value for the
31      specific address type, and octets 2 through N contain the
32      networkAddress address value in network byte order.
33
34      If the associated LldpPortIdSubtype object has a value of
35      'interfaceName(5)', then the octet string identifies a
36      particular instance of the ifName object (defined in IETF
37      RFC 2863). If the particular ifName object does not contain
38      any values, another port identifier type should be used.
39
40      If the associated LldpPortIdSubtype object has a value of
41      'agentCircuitId(6)', then this string identifies a agent-local
42      identifier of the circuit (defined in IETF RFC 3046).
43
44      If the associated LldpPortIdSubtype object has a value of
45      'local(7)', then this string identifies a locally
46      assigned port ID."
47   SYNTAX      OCTET STRING (SIZE (1..255))
48
49 LldpV2ManAddrIfSubtype ::= TEXTUAL-CONVENTION
50   STATUS      current
51   DESCRIPTION
52       "This TC defines an enumeration value that identifies
53       the interface numbering method used for defining the
54       interface number associated with a management address.
55       An object with this syntax defines the format of an
56       interface number object.
57
58       The enumeration 'unknown(1)' represents the case where the
59       interface is not known. In this case, the corresponding
```


1 interface number is of zero length.
2
3 The enumeration 'ifIndex(2)' represents interface identifier
4 based on the ifIndex MIB object.
5
6 The enumeration 'systemPortNumber(3)' represents interface
7 identifier based on the system port numbering convention."
8 REFERENCE
9 "8.5.9.5"
10
11 SYNTAX INTEGER {
12 unknown(1),
13 ifIndex(2),
14 systemPortNumber(3)
15 }
16
17 LldpV2ManAddress ::= TEXTUAL-CONVENTION
18 DISPLAY-HINT "1x:"
19 STATUS current
20 DESCRIPTION
21 "The value of a management address associated with the LLDP
22 agent that may be used to reach higher layer entities to
23 assist discovery by network management.
24
25 It should be noted that appropriate security credentials,
26 such as SNMP engineId, may be required to access the LLDP
27 agent using a management address. These necessary credentials
28 should be known by the network management and the objects
29 associated with the credentials are not included in the
30 LLDP agent."
31 SYNTAX OCTET STRING (SIZE (1..31))
32
33 LldpV2SystemCapabilitiesMap ::= TEXTUAL-CONVENTION
34 STATUS current
35 DESCRIPTION
36 "This TC describes the system capabilities.
37
38 The bit 'other(0)' indicates that the system has capabilities
39 other than those listed below.
40
41 The bit 'repeater(1)' indicates that the system has repeater
42 capability.
43
44 The bit 'bridge(2)' indicates that the system has bridge
45 capability.
46
47 The bit 'wlanAccessPoint(3)' indicates that the system has
48 WLAN access point capability.
49
50 The bit 'router(4)' indicates that the system has router
51 capability.
52
53 The bit 'telephone(5)' indicates that the system has telephone
54 capability.
55
56 The bit 'docsisCableDevice(6)' indicates that the system has
57 DOCSIS Cable Device capability (IETF RFC 4639 & 2670).
58
59 The bit 'stationOnly(7)' indicates that the system has only

```

1         station capability and nothing else.
2
3         The bit 'cVLANComponent(8)' indicates that the system has
4         C-VLAN component functionality.
5
6         The bit 'sVLANComponent(8)' indicates that the system has
7         S-VLAN component functionality.
8
9         The bit 'twoPortMACRelay(10)' indicates that the system has
10        Two-port MAC Relay (TPMR) functionality."
11    SYNTAX BITS {
12        other(0),
13        repeater(1),
14        bridge(2),
15        wlanAccessPoint(3),
16        router(4),
17        telephone(5),
18        docsisCableDevice(6),
19        stationOnly(7),
20        cVLANComponent(8),
21        sVLANComponent(9),
22        twoPortMACRelay(10)
23    }
24
25
26
27 LldpV2DestAddressTableIndex ::= TEXTUAL-CONVENTION
28     DISPLAY-HINT "d"
29     STATUS      current
30     DESCRIPTION
31         "An index value, used as the index to the table of destination
32         MAC addresses used both as the destination addresses on
33         transmitted LLDPDUs and on received LLDPDUs. This index value
34         is also used as a secondary index value in tables indexed
35         by fields of type ifIndex, in order to associate
36         a destination address with each row of the table."
37     SYNTAX      Unsigned32(1..4096)
38
39 LldpV2PowerPortClass ::= TEXTUAL-CONVENTION
40     STATUS      current
41     DESCRIPTION
42         "This TC describes the Power over Ethernet (PoE) port class."
43     SYNTAX      INTEGER {
44         pClassPSE(1),
45         pClassPD(2)
46     }
47
48 END
49

```

```
1
2 LLDP-V2-MIB DEFINITIONS ::= BEGIN
3 IMPORTS
4     MODULE-IDENTITY,
5     OBJECT-TYPE,
6     Unsigned32,
7     Counter32,
8     NOTIFICATION-TYPE
9         FROM SNMPv2-SMI
10    TimeStamp,
11    TruthValue,
12    MacAddress,
13    RowStatus
14        FROM SNMPv2-TC
15    SnmpAdminString
16        FROM SNMP-FRAMEWORK-MIB
17    MODULE-COMPLIANCE,
18    OBJECT-GROUP,
19    NOTIFICATION-GROUP
20        FROM SNMPv2-CONF
21    TimeFilter,
22    ZeroBasedCounter32
23        FROM RMON2-MIB
24    AddressFamilyNumbers
25        FROM IANA-ADDRESS-FAMILY-NUMBERS-MIB
26    ifGeneralInformationGroup,
27    InterfaceIndex
28        FROM IF-MIB
29    LldpV2ChassisIdSubtype,
30    LldpV2ChassisId,
31    LldpV2PortIdSubtype,
32    LldpV2PortId,
33    LldpV2ManAddrIfSubtype,
34    LldpV2ManAddress,
35    LldpV2SystemCapabilitiesMap,
36    LldpV2DestAddressTableIndex,
37    ieee802dot1mibs
38        FROM LLDP-V2-TC-MIB;
39
40 lldpV2MIB MODULE-IDENTITY
41     LAST-UPDATED "202602120000Z" -- February 12, 2026
42     ORGANIZATION "IEEE 802.1 Working Group"
43     CONTACT-INFO
44         "WG-URL: http://www.ieee802.org/1/
45         WG-EMail: stds-802-1-L@ieee.org
46
47         Contact: IEEE 802.1 Working Group Chair
48         Postal:  IEEE Standards Board
49                 445 Hoes Lane
50                 Piscataway, NJ 08854
51                 USA
52         E-mail:  stds-802-1-L@ieee.org"
53     DESCRIPTION
54         "Management Information Base module for LLDP configuration,
55         statistics, local system data and remote systems data
56         components.
57
58         This MIB module supports the architecture described in
```

1 Clause 6, where multiple LLDP agents can be associated with
2 a single Port, each supporting transmission by means of a
3 different MAC address.

4
5 Unless otherwise indicated, the references in this
6 MIB module are to IEEE Std 802.1AB-2026.

7
8 Copyright (C) IEEE (2026). This version of this MIB module
9 is published as 11.5.2 of IEEE Std 802.1AB-2026;
10 see the standard itself for full legal notices."

11
12 REVISION "202602120000Z" -- February 12, 2026

13 DESCRIPTION

14 "Published as part of the 2026 revision of
15 IEEE Std 802.1AB.
16 This revision updates references to 802.1AB,
17 copyright dates, and resolution to comments from the
18 first Standards Association ballot."

19
20 REVISION "201603110000Z" -- March 11, 2016

21
22 DESCRIPTION

23 "Published as part of the 2016 revision of
24 IEEE Std 802.1AB.
25 This revision incorporated changes to the MIB to
26 address issues identified in maintenance item
27 0121 - see <http://www.ieee802.org/1/maint.html>.
28 The changes involved correcting the way that
29 lldpV2MessageTxHoldMultiplier is calculated, in
30 accordance with 9.2.5.23."

31
32
33 REVISION "201502160000Z" -- February 16, 2015

34
35 DESCRIPTION

36 "Published as part of IEEE Std 802.1AB-2009 Cor-2.
37 This corrigendum incorporated changes to the MIB to
38 address issues identified in maintenance item
39 0121 - see <http://www.ieee802.org/1/maint.html>."

40
41 REVISION "200906080000Z" -- June 08, 2009

42
43 DESCRIPTION

44 "Published as part of IEEE Std 802.1AB-2009 revision.
45 This revision incorporated changes to the MIB to
46 support the use of LLDP with multiple destination MAC
47 addresses."

48
49 ::= { ieee802dot1mibs 13 }

50
51 lldpV2Notifications OBJECT IDENTIFIER ::= { lldpV2MIB 0 }
52 lldpV2Objects OBJECT IDENTIFIER ::= { lldpV2MIB 1 }
53 lldpV2Conformance OBJECT IDENTIFIER ::= { lldpV2MIB 2 }

54
55 --

56 -- LLDP MIB Objects

57 --

58

59 lldpV2Configuration OBJECT IDENTIFIER ::= { lldpV2Objects 1 }

```

1 lldpV2Statistics          OBJECT IDENTIFIER ::= { lldpV2Objects 2 }
2 lldpV2LocalSystemData     OBJECT IDENTIFIER ::= { lldpV2Objects 3 }
3 lldpV2RemoteSystemsData   OBJECT IDENTIFIER ::= { lldpV2Objects 4 }
4 lldpV2Extensions          OBJECT IDENTIFIER ::= { lldpV2Objects 5 }
5
6 --
7 -- *****
8 --
9 --             L L D P       C O N F I G
10 --
11 -- *****
12 --
13
14 lldpV2MessageTxInterval OBJECT-TYPE
15     SYNTAX      Unsigned32(1..32768)
16     UNITS       "seconds"
17     MAX-ACCESS   read-write
18     STATUS      current
19     DESCRIPTION
20         "The interval at which LLDP frames are transmitted on
21         behalf of this LLDP agent.
22
23         The default value for lldpV2MessageTxInterval object is
24         30 seconds.The range for this managed object is from 1 to 3600.
25
26         The value of this object is used as the initial value of
27         the lldpV2PortMessageTxInterval object on row creation in
28         the lldpV2PortConfigTableV2.
29
30         The value of this object is restored from non-volatile
31         storage after a re-initialization of the management system."
32     REFERENCE
33         "9.2.5.6"
34     DEFVAL      { 30 }
35     ::= { lldpV2Configuration 1 }
36
37 lldpV2MessageTxHoldMultiplier OBJECT-TYPE
38     SYNTAX      Unsigned32(2..10)
39     MAX-ACCESS   read-write
40     STATUS      current
41     DESCRIPTION
42         "The time to live value expressed as a multiple of the
43         lldpV2MessageTxInterval object. The actual time to live
44         value used in LLDP frames, transmitted on behalf of this
45         LLDP agent, can be expressed by the following formula:
46         TTL = min(65535,
47         (lldpV2MessageTxInterval*lldpV2MessageTxHoldMultiplier)+1)
48         For example, if the value of lldpV2MessageTxInterval is
49         '30', and the value of lldpV2MessageTxHoldMultiplier
50         is '4', then the value '121' is encoded in the
51         TTL field in the LLDP header.
52
53         The default value for lldpV2MessageTxHoldMultiplier
54         object is 4.
55
56         The value of this object is used as the initial value of
57         the lldpV2PortMessageTxHoldMultiplier object on row creation in
58         the lldpV2PortConfigTableV2.
59

```

```

1           The value of this object is restored from non-volatile
2           storage after a re-initialization of the management system."
3   REFERENCE
4       "9.2.5.5"
5   DEFVAL      { 4 }
6   ::= { lldpV2Configuration 2 }
7
8 lldpV2ReinitDelay OBJECT-TYPE
9   SYNTAX      Unsigned32(1..10)
10  UNITS        "seconds"
11  MAX-ACCESS   read-write
12  STATUS       current
13  DESCRIPTION
14      "The lldpV2ReinitDelay indicates the delay (in units of
15      seconds) from when lldpPortConfigAdminStatus object of a
16      particular port becomes 'disabled' until re-initialization
17      is attempted.
18
19      The default value for lldpV2ReinitDelay is 2 s.
20
21      The value of this object is used as the initial value of
22      the lldpV2PortReinitDelay object on row creation in
23      the lldpV2PortConfigTableV2.
24
25      The value of this object is restored from non-volatile
26      storage after a re-initialization of the management system."
27  REFERENCE
28      "9.2.5.9"
29  DEFVAL      { 2 }
30  ::= { lldpV2Configuration 3 }
31
32 lldpV2NotificationInterval OBJECT-TYPE
33  SYNTAX      Unsigned32(5..3600)
34  UNITS        "seconds"
35  MAX-ACCESS   read-write
36  STATUS       current
37  DESCRIPTION
38      "This object controls the interval between transmission of
39      LLDP notifications during normal transmission periods.
40
41      The value of this object is used as the initial value of
42      the lldpV2PortNotificationInterval object on row creation in
43      the lldpV2PortConfigTableV2.
44
45      The value of this object is restored from non-volatile
46      storage after a re-initialization of the management system."
47  DEFVAL { 30 }
48  ::= { lldpV2Configuration 4 }
49
50 lldpV2TxCreditMax OBJECT-TYPE
51  SYNTAX      Unsigned32(1..100)
52  UNITS        "PDUs"
53  MAX-ACCESS   read-write
54  STATUS       current
55  DESCRIPTION
56      "The maximum number of consecutive LLDPDUs that can be
57      transmitted at any time.
58
59      The default value for lldpV2TxCreditMax object is 5 PDUs. The

```

```

1         range for this managed object is from 1 to 10.
2
3         The value of this object is used as the initial value of
4         the lldpV2PortTxCreditMax object on row creation in
5         the lldpV2PortConfigTableV2.
6
7         The value of this object is restored from non-volatile
8         storage after a re-initialization of the management system."
9     REFERENCE
10         "9.2.5.18"
11     DEFVAL { 5 }
12     ::= { lldpV2Configuration 5 }
13
14 lldpV2MessageFastTx OBJECT-TYPE
15     SYNTAX Unsigned32(1..3600)
16     UNITS "seconds"
17     MAX-ACCESS read-write
18     STATUS current
19     DESCRIPTION
20         "The interval at which LLDP frames are transmitted on
21         behalf of this LLDP agent during fast transmission period
22         (e.g., when a new neighbor is detected).
23
24         The default value for lldpV2MessageFastTx object is
25         1 second.
26
27         The value of this object is used as the initial value of
28         the lldpV2PortMessageFastTx object on row creation in
29         the lldpV2PortConfigTableV2.
30
31         The value of this object is restored from non-volatile
32         storage after a re-initialization of the management system."
33     REFERENCE
34         "9.2.5.4"
35     DEFVAL { 1 }
36     ::= { lldpV2Configuration 6 }
37
38 lldpV2TxFastInit OBJECT-TYPE
39     SYNTAX Unsigned32(1..8)
40     MAX-ACCESS read-write
41     STATUS current
42     DESCRIPTION
43         "The initial value used to initialize the txFast variable
44         which determines the number of transmissions that are
45         made in fast transmission mode.
46
47         The default value for lldpV2TxFastInit object is 4.
48
49         The value of this object is used as the initial value of
50         the lldpV2PortTxFastInit object on row creation in
51         the lldpV2PortConfigTableV2.
52
53         The value of this object is restored from non-volatile
54         storage after a re-initialization of the management system."
55     REFERENCE
56         "9.2.5.20"
57     DEFVAL { 4 }
58     ::= { lldpV2Configuration 7 }
59 --

```

```

1-- lldpV2PortConfigTable: LLDP configuration indexed on a per port,
2-- per destination address basis. The ifIndex, coupled with an
3-- index into the lldpDestAddressTable, is used to index per port
4-- per destination MAC address.
5-- ***This table and its associated objects are now deprecated
6-- and replaced by lldpV2PortConfigTableV2.***
7--
8
9 lldpV2PortConfigTable    OBJECT-TYPE
10  SYNTAX      SEQUENCE OF LldpV2PortConfigEntry
11  MAX-ACCESS  not-accessible
12  STATUS      deprecated
13  DESCRIPTION
14      "The table that controls LLDP frame transmission on individual
15      ports that uses particular destination MAC addresses."
16  ::= { lldpV2Configuration 8 }
17
18 lldpV2PortConfigEntry    OBJECT-TYPE
19  SYNTAX      LldpV2PortConfigEntry
20  MAX-ACCESS  not-accessible
21  STATUS      deprecated
22  DESCRIPTION
23      "LLDP configuration information for a particular port and
24      destination MAC address.
25
26      This configuration parameter controls the transmission and
27      the reception of LLDP frames on those interface/address
28      combinations whose rows are created in this table.
29
30      Rows in this table can only be created for MAC addresses
31      that can validly be used in association with the type of
32      interface concerned, as defined by Table 7-2.
33
34      The contents of this table is persistent across
35      re-initializations or re-boots."
36  INDEX  { lldpV2PortConfigIfIndex,
37          lldpV2PortConfigDestAddressIndex  }
38  ::= { lldpV2PortConfigTable 1 }
39
40 LldpV2PortConfigEntry ::= SEQUENCE {
41     lldpV2PortConfigIfIndex      InterfaceIndex,
42     lldpV2PortConfigDestAddressIndex  LldpV2DestAddressTableIndex,
43     lldpV2PortConfigAdminStatus   INTEGER,
44     lldpV2PortConfigNotificationEnable  TruthValue,
45     lldpV2PortConfigTLVsTxEnable   BITS }
46
47 lldpV2PortConfigIfIndex    OBJECT-TYPE
48  SYNTAX      InterfaceIndex
49  MAX-ACCESS  not-accessible
50  STATUS      deprecated
51  DESCRIPTION
52      "The interface index value used to identify the port
53      associated with this entry. Its value is an index into
54      the interfaces MIB.
55
56      The value of this object is used as an index to the
57      lldpV2PortConfigTable."
58  ::= { lldpV2PortConfigEntry 1 }
59

```



```

1 lldpV2PortConfigDestAddressIndex OBJECT-TYPE
2     SYNTAX      LldpV2DestAddressTableIndex
3     MAX-ACCESS  not-accessible
4     STATUS      deprecated
5     DESCRIPTION
6         "The index value used to identify the destination
7         MAC address associated with this entry. Its value identifies
8         the row in the lldpV2DestAddressTable where the MAC address
9         can be found.
10
11         The value of this object is used as an index to the
12         lldpV2PortConfigTable."
13     ::= { lldpV2PortConfigEntry 2 }
14
15 lldpV2PortConfigAdminStatus OBJECT-TYPE
16     SYNTAX INTEGER {
17         txOnly(1),
18         rxOnly(2),
19         txAndRx(3),
20         disabled(4)
21     }
22     MAX-ACCESS  read-write
23     STATUS      deprecated
24     DESCRIPTION
25         "The administratively desired status of the local LLDP agent.
26
27         If the associated lldpV2PortConfigAdminStatus object is
28         set to a value of 'txOnly(1)', then LLDP agent transmits
29         LLDPframes on this port and it does not store any
30         information about the remote systems connected.
31
32         If the associated lldpV2PortConfigAdminStatus object is
33         set to a value of 'rxOnly(2)', then the LLDP agent
34         receives, but it does not transmit LLDP frames on this port.
35
36         If the associated lldpV2PortConfigAdminStatus object is set
37         to a value of 'txAndRx(3)', then the LLDP agent transmits
38         and receives LLDP frames on this port.
39
40         If the associated lldpV2PortConfigAdminStatus object is set
41         to a value of 'disabled(4)', then LLDP agent does not
42         transmit or receive LLDP frames on this port. If there is
43         remote systems information that is received on this port
44         and stored in other tables, before the port's
45         lldpV2PortConfigAdminStatus becomes disabled, then that
46         information is deleted."
47     REFERENCE
48         "9.2.5.1"
49     DEFVAL { txAndRx }
50     ::= { lldpV2PortConfigEntry 3 }
51
52 lldpV2PortConfigNotificationEnable OBJECT-TYPE
53     SYNTAX      TruthValue
54     MAX-ACCESS  read-write
55     STATUS      deprecated
56     DESCRIPTION
57         "The lldpV2PortConfigNotificationEnable controls, on a per
58         agent basis, whether or not notifications from the agent
59         are enabled. The value true(1) means that notifications are

```

```
1         enabled; the value false(2) means that they are not."
2     DEFVAL { false }
3     ::= { lldpV2PortConfigEntry 4 }
4
5 lldpV2PortConfigTLVsTxEnable OBJECT-TYPE
6     SYNTAX      BITS {
7         portDesc(0),
8         sysName(1),
9         sysDesc(2),
10        sysCap(3)
11    }
12    MAX-ACCESS   read-write
13    STATUS        deprecated
14    DESCRIPTION
15        "The lldpV2PortConfigTLVsTxEnable, defined as a bitmap,
16        includes the basic set of LLDP TLVs whose transmission is
17        allowed on the local LLDP agent by the network management.
18        Each bit in the bitmap corresponds to a TLV type associated
19        with a specific optional TLV.
20
21        It should be noted that the organizationally-specific TLVs
22        are excluded from the lldpV2PortConfigTLVsTxEnable bitmap.
23
24        LLDP Organization Specific Information Extension MIBs should
25        have similar configuration objects to control transmission
26        of their organizationally defined TLVs.
27
28        The bit 'portDesc(0)' indicates that LLDP agent should
29        transmit 'Port Description TLV'.
30
31        The bit 'sysName(1)' indicates that LLDP agent should transmit
32        'System Name TLV'.
33
34        The bit 'sysDesc(2)' indicates that LLDP agent should transmit
35        'System Description TLV'.
36
37        The bit 'sysCap(3)' indicates that LLDP agent should transmit
38        'System Capabilities TLV'.
39
40        There is no bit reserved for the management address TLV type
41        since transmission of management address TLVs are controlled
42        by another object, lldpV2ConfigManAddrTable.
43
44        The default value for lldpV2PortConfigTLVsTxEnable object is
45        empty set, which means no enumerated values are set.
46
47        The value of this object is restored from non-volatile
48        storage after a re-initialization of the management system."
49    REFERENCE
50        "9.1.2.1"
51    DEFVAL { { } }
52    ::= { lldpV2PortConfigEntry 5 }
53
54 --
55 -- lldpV2PortConfigTableV2: LLDP configuration indexed on a per port,
56 -- per destination address basis. The ifIndex, coupled with an
57 -- index into the lldpDestAddressTable, is used to index per port
58 -- per destination MAC address.
59 --
```

```

1 -- V2 extends the original table definition to include per-port
2 -- per-MAC address parameters msgTxInterval, msgTxHold, reinitDelay,
3 -- notificationInterval, txCreditMax, msgFastTx, and txFastInit.
4 --
5
6 lldpV2PortConfigTableV2    OBJECT-TYPE
7     SYNTAX      SEQUENCE OF LldpV2PortConfigEntryV2
8     MAX-ACCESS  not-accessible
9     STATUS      current
10    DESCRIPTION
11        "The table that controls LLDP frame transmission on individual
12        ports and using particular destination MAC addresses."
13    ::= { lldpV2Configuration 11 }
14
15 lldpV2PortConfigEntryV2    OBJECT-TYPE
16     SYNTAX      LldpV2PortConfigEntryV2
17     MAX-ACCESS  not-accessible
18     STATUS      current
19     DESCRIPTION
20        "LLDP configuration information for a particular port and
21        destination MAC address.
22
23        This configuration parameter controls the transmission and
24        the reception of LLDP frames on those interface/address
25        combinations whose rows are created in this table.
26
27        Rows in this table can only be created for MAC addresses
28        that can validly be used in association with the type of
29        interface concerned, as defined by Table 7-2.
30
31        The contents of this table is persistent across
32        re-initializations or re-boots."
33     INDEX  { lldpV2PortConfigIfIndexV2,
34             lldpV2PortConfigDestAddressIndexV2  }
35     ::= { lldpV2PortConfigTableV2 1 }
36
37 LldpV2PortConfigEntryV2 ::= SEQUENCE {
38     lldpV2PortConfigIfIndexV2      InterfaceIndex,
39     lldpV2PortConfigDestAddressIndexV2  LldpV2DestAddressTableIndex,
40     lldpV2PortConfigAdminStatusV2    INTEGER,
41     lldpV2PortMessageTxInterval      Unsigned32,
42     lldpV2PortMessageTxHoldMultiplier Unsigned32,
43     lldpV2PortReinitDelay            Unsigned32,
44     lldpV2PortNotificationInterval   Unsigned32,
45     lldpV2PortTxCreditMax            Unsigned32,
46     lldpV2PortMessageFastTx          Unsigned32,
47     lldpV2PortTxFastInit             Unsigned32,
48     lldpV2PortConfigNotificationEnableV2 TruthValue,
49     lldpV2PortConfigTLVsTxEnableV2   BITS }
50
51 lldpV2PortConfigIfIndexV2    OBJECT-TYPE
52     SYNTAX      InterfaceIndex
53     MAX-ACCESS  not-accessible
54     STATUS      current
55     DESCRIPTION
56        "The interface index value used to identify the port
57        associated with this entry. Its value is an index into
58        the interfaces MIB.
59

```

```

1           The value of this object is used as an index to the
2           lldpV2PortConfigTable."
3 ::= { lldpV2PortConfigEntryV2 1 }
4
5 lldpV2PortConfigDestAddressIndexV2 OBJECT-TYPE
6     SYNTAX      LldpV2DestAddressTableIndex
7     MAX-ACCESS  not-accessible
8     STATUS      current
9     DESCRIPTION
10        "The index value used to identify the destination
11        MAC address associated with this entry. Its value identifies
12        the row in the lldpV2DestAddressTable where the MAC address
13        can be found.
14
15        The value of this object is used as an index to the
16        lldpV2PortConfigTable."
17 ::= { lldpV2PortConfigEntryV2 2 }
18
19 lldpV2PortConfigAdminStatusV2 OBJECT-TYPE
20     SYNTAX INTEGER {
21         txOnly(1),
22         rxOnly(2),
23         txAndRx(3),
24         disabled(4)
25     }
26     MAX-ACCESS  read-write
27     STATUS      current
28     DESCRIPTION
29        "The administratively desired status of the local LLDP agent.
30
31        If the associated lldpV2PortConfigAdminStatus object is
32        set to a value of 'txOnly(1)', then LLDP agent transmits
33        LLDPframes on this port and it does not store any
34        information about the remote systems connected.
35
36        If the associated lldpV2PortConfigAdminStatus object is
37        set to a value of 'rxOnly(2)', then the LLDP agent
38        receives, but it does not transmit, LLDP frames on this port.
39
40        If the associated lldpV2PortConfigAdminStatus object is set
41        to a value of 'txAndRx(3)', then the LLDP agent transmits
42        and receives LLDP frames on this port.
43
44        If the associated lldpV2PortConfigAdminStatus object is set
45        to a value of 'disabled(4)', then LLDP agent does not
46        transmit or receive LLDP frames on this port. If there is
47        remote systems information that is received on this port
48        and stored in other tables, before the port's
49        lldpV2PortConfigAdminStatus becomes disabled, then that
50        information is deleted."
51     REFERENCE
52         "9.2.5.1"
53     DEFVAL { txAndRx }
54 ::= { lldpV2PortConfigEntryV2 3 }
55
56 lldpV2PortMessageTxInterval OBJECT-TYPE
57     SYNTAX      Unsigned32(1..32768)
58     UNITS        "seconds"
59     MAX-ACCESS  read-write

```

```

1     STATUS      current
2     DESCRIPTION
3         "The interval at which LLDP frames are transmitted on
4         behalf of this LLDP agent.
5
6         This object takes its initial value from the
7         lldpV2MessageTxInterval object on table row creation. The
8         range for this managed object is from 1 to 3600.
9
10        The value of this object is restored from non-volatile
11        storage after a re-initialization of the management system."
12     REFERENCE
13         "9.2.5.6"
14     DEFVAL      { 30 }
15     ::= { lldpV2PortConfigEntryV2 4 }
16
17 lldpV2PortMessageTxHoldMultiplier OBJECT-TYPE
18     SYNTAX      Unsigned32(2..10)
19     MAX-ACCESS  read-write
20     STATUS      current
21     DESCRIPTION
22         "The time to live value expressed as a multiple of the
23         lldpV2MessageTxInterval object. The actual time to live
24         value used in LLDP frames, transmitted on behalf of this
25         LLDP agent, can be expressed by the following formula:
26         TTL = min(65535,
27         (lldpV2MessageTxInterval*lldpV2MessageTxHoldMultiplier)+1)
28         For example, if the value of lldpV2MessageTxInterval is
29         '30', and the value of lldpV2MessageTxHoldMultiplier
30         is '4', then the value '121' is encoded in the
31         TTL field in the LLDP header.
32
33         This object takes its initial value from the
34         lldpV2PortMessageTxHoldMultiplier object on table row creation.
35
36         The value of this object is restored from non-volatile
37         storage after a re-initialization of the management system."
38     REFERENCE
39         "9.2.5.5"
40     DEFVAL      { 4 }
41     ::= { lldpV2PortConfigEntryV2 5 }
42
43 lldpV2PortReinitDelay OBJECT-TYPE
44     SYNTAX      Unsigned32(1..10)
45     UNITS       "seconds"
46     MAX-ACCESS  read-write
47     STATUS      current
48     DESCRIPTION
49         "The lldpV2ReinitDelay indicates the delay (in units of
50         seconds) from when lldpPortConfigAdminStatus object of a
51         particular port becomes 'disabled' until re-initialization
52         is attempted.
53
54         This object takes its initial value from the
55         lldpV2PortReinitDelay object on table row creation.
56
57         The value of this object is restored from non-volatile
58         storage after a re-initialization of the management system."
59     REFERENCE

```

```

1         "9.2.5.9"
2     DEFVAL      { 2 }
3     ::= { lldpV2PortConfigEntryV2 6 }
4
5 lldpV2PortNotificationInterval OBJECT-TYPE
6     SYNTAX      Unsigned32(5..3600)
7     UNITS       "seconds"
8     MAX-ACCESS  read-write
9     STATUS      current
10    DESCRIPTION
11        "This object controls the interval between transmission of
12        LLDP notifications during normal transmission periods.
13
14        This object takes its initial value from the
15        lldpV2PortNotificationInterval object on table row creation.
16
17        The value of this object is restored from non-volatile
18        storage after a re-initialization of the management system."
19    DEFVAL { 30 }
20    ::= { lldpV2PortConfigEntryV2 7 }
21
22 lldpV2PortTxCreditMax OBJECT-TYPE
23     SYNTAX      Unsigned32(1..100)
24     UNITS       "PDUs"
25     MAX-ACCESS  read-write
26     STATUS      current
27     DESCRIPTION
28        "The maximum number of consecutive LLDPDUs that can be
29        transmitted at any time.
30
31        This object takes its initial value from the
32        lldpV2PortTxCreditMax object on table row creation. The
33        range for this managed object is from 1 to 10.
34
35        The value of this object is restored from non-volatile
36        storage after a re-initialization of the management system."
37     REFERENCE
38        "9.2.5.18"
39     DEFVAL { 5 }
40     ::= { lldpV2PortConfigEntryV2 8 }
41
42 lldpV2PortMessageFastTx OBJECT-TYPE
43     SYNTAX      Unsigned32(1..3600)
44     UNITS       "seconds"
45     MAX-ACCESS  read-write
46     STATUS      current
47     DESCRIPTION
48        "The interval at which LLDP frames are transmitted on
49        behalf of this LLDP agent during fast transmission period
50        (e.g., when a new neighbor is detected).
51
52        This object takes its initial value from the
53        lldpV2PortMessageFastTx object on table row creation.
54
55        The value of this object is restored from non-volatile
56        storage after a re-initialization of the management system."
57     REFERENCE
58        "9.2.5.4"
59     DEFVAL { 1 }

```

```

1      ::= { lldpV2PortConfigEntryV2 9 }
2
3 lldpV2PortTxFastInit OBJECT-TYPE
4     SYNTAX Unsigned32(1..8)
5     MAX-ACCESS read-write
6     STATUS current
7     DESCRIPTION
8         "The initial value used to initialize the txFast variable
9         which determines the number of transmissions that are
10        made in fast transmission mode.
11
12        This object takes its initial value from the
13        lldpV2PortTxFastInit object on table row creation.
14
15        The value of this object is restored from non-volatile
16        storage after a re-initialization of the management system."
17     REFERENCE
18         "9.2.5.20"
19     DEFVAL { 4 }
20     ::= { lldpV2PortConfigEntryV2 10 }
21
22
23 lldpV2PortConfigNotificationEnableV2 OBJECT-TYPE
24     SYNTAX      TruthValue
25     MAX-ACCESS  read-write
26     STATUS      current
27     DESCRIPTION
28         "The lldpV2PortConfigNotificationEnableV2 controls, on a per
29         agent basis, whether or not notifications from the agent
30         are enabled. The value true(1) means that notifications are
31         enabled; the value false(2) means that they are not."
32     DEFVAL { false }
33     ::= { lldpV2PortConfigEntryV2 11 }
34
35 lldpV2PortConfigTLVsTxEnableV2 OBJECT-TYPE
36     SYNTAX      BITS {
37         portDesc(0),
38         sysName(1),
39         sysDesc(2),
40         sysCap(3)
41     }
42     MAX-ACCESS  read-write
43     STATUS      current
44     DESCRIPTION
45         "The lldpV2PortConfigTLVsTxEnableV2, defined as a bitmap,
46         includes the basic set of LLDP TLVs whose transmission is
47         allowed on the local LLDP agent by the network management.
48         Each bit in the bitmap corresponds to a TLV type associated
49         with a specific optional TLV.
50
51         It should be noted that the organizationally-specific TLVs
52         are excluded from the lldpV2PortConfigTLVsTxEnable bitmap.
53
54         LLDP Organization Specific Information Extension MIBs should
55         have similar configuration objects to control transmission
56         of their organizationally defined TLVs.
57
58         The bit 'portDesc(0)' indicates that LLDP agent should
59         transmit 'Port Description TLV'.
```

1
2 The bit 'sysName(1)' indicates that LLDP agent should transmit
3 'System Name TLV'.
4
5 The bit 'sysDesc(2)' indicates that LLDP agent should transmit
6 'System Description TLV'.
7
8 The bit 'sysCap(3)' indicates that LLDP agent should transmit
9 'System Capabilities TLV'.
10
11 There is no bit reserved for the management address TLV type
12 since transmission of management address TLVs are controlled
13 by another object, lldpV2ConfigManAddrTable.
14
15 The default value for lldpV2PortConfigTLVsTxEnable object is
16 empty set, which means no enumerated values are set.
17
18 The value of this object is restored from non-volatile
19 storage after a re-initialization of the management system."
20 REFERENCE
21 "9.1.2.1"
22 DEFVAL { { } }
23 ::= { lldpV2PortConfigEntryV2 12 }
24
25
26 --
27 -- lldpV2DestAddressTable: Destination MAC addresses used by LLDP
28 --
29
30 lldpV2DestAddressTable OBJECT-TYPE
31 SYNTAX SEQUENCE OF LldpV2DestAddressTableEntry
32 MAX-ACCESS not-accessible
33 STATUS current
34 DESCRIPTION
35 "The table that contains the set of MAC addresses used
36 by LLDP for transmission and reception of LLDPDUs. The
37 agent pre-populates this table with all Destination MAC
38 addresses listed in Table 7-1 and any other supported
39 addresses."
40 ::= { lldpV2Configuration 9 }
41
42 lldpV2DestAddressTableEntry OBJECT-TYPE
43 SYNTAX LldpV2DestAddressTableEntry
44 MAX-ACCESS not-accessible
45 STATUS current
46 DESCRIPTION
47 "Destination MAC address information for LLDP."
48
49 This configuration parameter identifies a MAC address
50 corresponding to a LldpV2DestAddressTableIndex value.
51
52 Rows in this table are created as necessary, to support
53 MAC addresses needed by other tables in the MIB that
54 are indexed by MAC address.
55
56 A given row in this table cannot be deleted if the MAC
57 address table index value is in use in any other table
58 in the MIB.
59


```

1           The contents of this table are persistent across
2           re-initializations or re-boots."
3     INDEX { lldpV2AddressTableIndex }
4     ::= { lldpV2DestAddressTable 1 }
5
6 lldpV2DestAddressTableEntry ::= SEQUENCE {
7     lldpV2AddressTableIndex      LldpV2DestAddressTableIndex,
8     lldpV2DestMacAddress         MacAddress }
9
10 lldpV2AddressTableIndex OBJECT-TYPE
11     SYNTAX      LldpV2DestAddressTableIndex
12     MAX-ACCESS  not-accessible
13     STATUS      current
14     DESCRIPTION
15         "The index value used to identify the destination
16         MAC address associated with this entry."
17
18         The value of this object is used as an index to the
19         lldpV2DestAddressTable."
20     ::= { lldpV2DestAddressTableEntry 1 }
21
22 lldpV2DestMacAddress OBJECT-TYPE
23     SYNTAX      MacAddress
24     MAX-ACCESS  read-only
25     STATUS      current
26     DESCRIPTION
27         "The MAC address associated with this entry."
28
29         The octet string identifies an individual or a group
30         MAC address that is in use by LLDP as a destination
31         MAC address."
32
33         The MAC address is encoded in the octet string in
34         canonical format (see IEEE Std 802)."
35     ::= { lldpV2DestAddressTableEntry 2 }
36
37 --
38 -- lldpV2ManAddrConfigTxPortsTable : selection of management addresses
39 -- to be transmitted on a specified set of port/destination
40 -- address pairs.
41 --
42
43 lldpV2ManAddrConfigTxPortsTable OBJECT-TYPE
44     SYNTAX      SEQUENCE OF LldpV2ManAddrConfigTxPortsEntry
45     MAX-ACCESS  not-accessible
46     STATUS      current
47     DESCRIPTION
48         "The table that controls selection of LLDP management address
49         TLV instances to be transmitted on individual port/
50         destination address pairs."
51     ::= { lldpV2Configuration 10 }
52
53 lldpV2ManAddrConfigTxPortsEntry OBJECT-TYPE
54     SYNTAX      LldpV2ManAddrConfigTxPortsEntry
55     MAX-ACCESS  not-accessible
56     STATUS      current
57     DESCRIPTION
58         "LLDP configuration information that specifies the set
59         of port/destination address pairs on which the local

```

```

1         system management address instance is transmitted.
2
3         Each active lldpManAddrConfigTxPortsTableV2Entry is
4         restored from non-volatile storage and re-created
5         after a re-initialization of the management system."
6     INDEX {
7         lldpV2ManAddrConfigIfIndex,
8         lldpV2ManAddrConfigDestAddressIndex,
9         lldpV2ManAddrConfigLocManAddrSubtype,
10        lldpV2ManAddrConfigLocManAddr }
11    ::= { lldpV2ManAddrConfigTxPortsTable 1 }
12
13 lldpV2ManAddrConfigTxPortsEntry ::= SEQUENCE {
14     lldpV2ManAddrConfigIfIndex      InterfaceIndex,
15     lldpV2ManAddrConfigDestAddressIndex LldpV2DestAddressTableIndex,
16     lldpV2ManAddrConfigLocManAddrSubtype AddressFamilyNumbers,
17     lldpV2ManAddrConfigLocManAddr     LldpV2ManAddress,
18     lldpV2ManAddrConfigTxEnable       TruthValue,
19     lldpV2ManAddrConfigRowStatus       RowStatus
20 }
21
22 lldpV2ManAddrConfigIfIndex OBJECT-TYPE
23     SYNTAX      InterfaceIndex
24     MAX-ACCESS  not-accessible
25     STATUS      current
26     DESCRIPTION
27         "The interface index value used to identify the port
28         associated with this entry. Its value is an index into
29         the interfaces MIB.
30
31         The value of this object is used as an index to the
32         lldpV2PortConfigTable.
33
34         The value in this column of the table MUST match
35         the IfIndex value specified in the BridgePort table."
36     ::= { lldpV2ManAddrConfigTxPortsEntry 1 }
37
38 lldpV2ManAddrConfigDestAddressIndex OBJECT-TYPE
39     SYNTAX      LldpV2DestAddressTableIndex
40     MAX-ACCESS  not-accessible
41     STATUS      current
42     DESCRIPTION
43         "The index value used to identify the destination
44         MAC address associated with this entry. Its value identifies
45         the row in the lldpV2DestAddressTable where the MAC address
46         can be found.
47
48         The value of this object is used as an index to the
49         lldpV2PortConfigTable."
50     ::= { lldpV2ManAddrConfigTxPortsEntry 2 }
51
52
53 lldpV2ManAddrConfigLocManAddrSubtype OBJECT-TYPE
54     SYNTAX      AddressFamilyNumbers
55     MAX-ACCESS  not-accessible
56     STATUS      current
57     DESCRIPTION
58         "The type of management address identifier encoding used in
59         the associated 'lldpLocManagmentAddr' object.

```

```

1
2         It should be noted that only a subset of the possible
3         address encodings enumerated in AddressFamilyNumbers
4         are appropriate for use as a LLDP management
5         address, either because some are just not applicable or
6         because the maximum size of a LldpV2ManAddress octet string
7         would prevent the use of some address identifier encodings."
8     REFERENCE
9         "8.5.9.3"
10    ::= { lldpV2ManAddrConfigTxPortsEntry 3 }
11
12 lldpV2ManAddrConfigLocManAddr OBJECT-TYPE
13     SYNTAX      LldpV2ManAddress
14     MAX-ACCESS  not-accessible
15     STATUS      current
16     DESCRIPTION
17         "The string value used to identify the management address
18         component associated with the local system. The purpose of
19         this address is to contact the management entity."
20     REFERENCE
21         "8.5.9.4"
22    ::= { lldpV2ManAddrConfigTxPortsEntry 4 }
23
24
25
26 lldpV2ManAddrConfigTxEnable OBJECT-TYPE
27     SYNTAX      TruthValue
28     MAX-ACCESS  read-create
29     STATUS      current
30     DESCRIPTION
31         "A Boolean controlling the transmission of system
32         management address instance for the specified port,
33         destination, subtype, and MAN address used to index
34         this table. If set to the default value of false,
35         no transmission occurs. If set to true, the
36         appropriate information is transmitted out of the
37         port specified in the row's index."
38     REFERENCE
39         "9.1.2.1"
40     DEFVAL { false } -- not transmitted
41    ::= { lldpV2ManAddrConfigTxPortsEntry 5 }
42
43 lldpV2ManAddrConfigRowStatus OBJECT-TYPE
44     SYNTAX RowStatus
45     MAX-ACCESS read-create
46     STATUS current
47     DESCRIPTION
48         "Indicates the status of an entry in this table, and is used
49         to create/delete entries.
50         The corresponding instances of the following objects
51         must be set before this object can be made active(1):
52             lldpV2ManAddrConfigDestAddressIndex
53             lldpV2ManAddrConfigLocManAddrSubtype
54             lldpV2ManAddrConfigLocManAddr
55             lldpV2ManAddrConfigTxEnable
56
57         The corresponding instances of the following objects
58         may not be changed while this object is active(1):
59             lldpV2ManAddrConfigDestAddressIndex

```

```

1          lldpV2ManAddrConfigLocManAddrSubtype
2          lldpV2ManAddrConfigLocManAddr "
3 ::= { lldpV2ManAddrConfigTxPortsEntry 6 }
4
5 --
6 -- *****
7 --
8 --          L L D P      S T A T S
9 --
10 -- *****
11 --
12 -- LLDP Stats Group
13
14 lldpV2StatsRemTablesLastChangeTime OBJECT-TYPE
15     SYNTAX      TimeStamp
16     MAX-ACCESS  read-only
17     STATUS      current
18     DESCRIPTION
19         "The value of sysUpTime object (defined in IETF RFC 3418)
20         at the time an entry is created, modified, or deleted in the
21         in tables associated with the lldpV2RemoteSystemsData objects
22         and all LLDP extension objects associated with remote systems.
23
24         An NMS can use this object to reduce polling of the
25         lldpV2RemoteSystemsData objects."
26     ::= { lldpV2Statistics 1 }
27
28 lldpV2StatsRemTablesInserts OBJECT-TYPE
29     SYNTAX      ZeroBasedCounter32
30     UNITS       "table entries"
31     MAX-ACCESS  read-only
32     STATUS      current
33     DESCRIPTION
34         "The number of times the complete set of information
35         advertised by a particular MSAP has been inserted into tables
36         contained in lldpV2RemoteSystemsData and lldpV2Extensions objects.
37
38         The complete set of information received from a particular
39         MSAP should be inserted into related tables. If partial
40         information cannot be inserted for a reason such as lack
41         of resources, all of the complete set of information should
42         be removed.
43
44         This counter should be incremented only once after the
45         complete set of information is successfully recorded
46         in all related tables. Any failures during inserting
47         information set that result in deletion of previously
48         inserted information should not trigger any changes in
49         lldpV2StatsRemTablesInserts since the insert is not completed
50         yet or in lldpStatsRemTablesDeletes since the deletion
51         would only be a partial deletion. If the failure was the
52         result of lack of resources, the lldpStatsRemTablesDrops
53         counter should be incremented once."
54     ::= { lldpV2Statistics 2 }
55
56 lldpV2StatsRemTablesDeletes OBJECT-TYPE
57     SYNTAX      ZeroBasedCounter32
58     UNITS       "table entries"
59     MAX-ACCESS  read-only

```

```

1     STATUS      current
2
3     DESCRIPTION
4         "The number of times the complete set of information
5         advertised by a particular MSAP has been deleted from
6         tables contained in lldpV2RemoteSystemsData and lldpV2Extensions
7         objects.
8
9         This counter should be incremented only once when the
10        complete set of information is completely deleted from all
11        related tables. Partial deletions, such as deletion of
12        rows associated with a particular MSAP from some tables,
13        but not from all tables are not allowed, thus should not
14        change the value of this counter."
15    ::= { lldpV2Statistics 3 }
16
17 lldpV2StatsRemTablesDrops OBJECT-TYPE
18     SYNTAX      ZeroBasedCounter32
19     UNITS        "table entries"
20     MAX-ACCESS   read-only
21
22     STATUS      current
23     DESCRIPTION
24         "The number of times the complete set of information
25         advertised by a particular MSAP could not be entered into
26         tables contained in lldpV2RemoteSystemsData and lldpV2Extensions
27         objects because of insufficient resources."
28     ::= { lldpV2Statistics 4 }
29
30 lldpV2StatsRemTablesAgeouts OBJECT-TYPE
31     SYNTAX      ZeroBasedCounter32
32     UNITS        "table entries"
33     MAX-ACCESS   read-only
34     STATUS      current
35     DESCRIPTION
36         "The number of times the complete set of information
37         advertised by a particular MSAP has been deleted from tables
38         contained in lldpV2RemoteSystemsData and lldpV2Extensions objects
39         because the information timeliness interval has expired.
40
41         This counter should be incremented only once when the complete
42         set of information is completely invalidated (aged out)
43         from all related tables. Partial ageing, similar to deletion
44         case, is not allowed, and thus, should not change the value
45         of this counter."
46     ::= { lldpV2Statistics 5 }
47
48 --
49 -- TX statistics
50 -- Indexed by port (via ifIndex) and
51 -- destination MAC address.
52 --
53
54 lldpV2StatsTxPortTable OBJECT-TYPE
55     SYNTAX      SEQUENCE OF LldpV2StatsTxPortEntry
56     MAX-ACCESS   not-accessible
57     STATUS      current
58     DESCRIPTION
59         "A table containing LLDP transmission statistics for

```

```

1         individual port/destination address combinations.
2         Entries are not required to exist in
3         this table while the lldpPortConfigEntry object is equal to
4         'disabled(4)'.
5         ::= { lldpV2Statistics 6 }
6
7 lldpV2StatsTxPortEntry    OBJECT-TYPE
8     SYNTAX                LldpV2StatsTxPortEntry
9     MAX-ACCESS             not-accessible
10    STATUS                 current
11    DESCRIPTION
12        "LLDP frame transmission statistics for a particular port
13        and destination MAC address.
14        The port is contained in the same chassis as the
15        LLDP agent.
16
17        All counter values in a particular entry shall be
18        maintained on a continuing basis and shall not be deleted
19        upon expiration of rxInfoTTL timing counters in the LLDP
20        remote systems MIB of the receipt of a shutdown frame from
21        a remote LLDP agent.
22
23        All statistical counters associated with a particular
24        port on the local LLDP agent become frozen whenever the
25        adminStatus is disabled for the same port.
26
27        Rows in this table can only be created for MAC addresses
28        that can validly be used in association with the type of
29        interface concerned, as defined by Table 7-2."
30    INDEX { lldpV2StatsTxIfIndex,
31            lldpV2StatsTxDestMACAddress }
32    ::= { lldpV2StatsTxPortTable 1 }
33
34 lldpV2StatsTxPortEntry ::= SEQUENCE {
35     lldpV2StatsTxIfIndex          InterfaceIndex,
36     lldpV2StatsTxDestMACAddress   LldpV2DestAddressTableIndex,
37     lldpV2StatsTxPortFramesTotal  Counter32,
38     lldpV2StatsTxLLDPDULengthErrors Counter32 }
39
40 lldpV2StatsTxIfIndex    OBJECT-TYPE
41     SYNTAX                InterfaceIndex
42     MAX-ACCESS             not-accessible
43     STATUS                 current
44     DESCRIPTION
45         "The interface index value used to identify the port
46         associated with this entry. Its value is an index
47         into the interfaces MIB.
48
49         The value of this object is used as an index to the
50         lldpV2StatsTxPortTable."
51     ::= { lldpV2StatsTxPortEntry 1 }
52
53 lldpV2StatsTxDestMACAddress OBJECT-TYPE
54     SYNTAX                LldpV2DestAddressTableIndex
55     MAX-ACCESS             not-accessible
56     STATUS                 current
57     DESCRIPTION
58         "The index value used to identify the destination
59         MAC address associated with this entry. Its value identifies

```

```

1         the row in the lldpV2DestAddressTable where the MAC address
2         can be found.
3
4         The value of this object is used as an index to the
5         lldpV2StatsTxPortTable."
6 ::= { lldpV2StatsTxPortEntry 2 }
7
8 lldpV2StatsTxPortFramesTotal OBJECT-TYPE
9     SYNTAX      Counter32
10    UNITS        "LLDP frames"
11    MAX-ACCESS   read-only
12    STATUS       current
13    DESCRIPTION
14        "The number of LLDP frames transmitted by this LLDP agent
15        on the indicated port to the destination MAC address
16        associated with this row of the table."
17    REFERENCE
18        "9.2.7.5"
19 ::= { lldpV2StatsTxPortEntry 3 }
20
21 lldpV2StatsTxLLDPDULengthErrors OBJECT-TYPE
22     SYNTAX      Counter32
23     UNITS        "LLDP frames"
24     MAX-ACCESS   read-only
25     STATUS       current
26     DESCRIPTION
27         "The number of LLDPDU Length Errors recorded for the Port."
28     REFERENCE
29         "9.2.7.8"
30 ::= { lldpV2StatsTxPortEntry 4 }
31
32 --
33 -- lldpV2StatsRxPortTable - RX statistics
34 -- This table is indexed by ifIndex and destination MAC address.
35 --
36
37 lldpV2StatsRxPortTable OBJECT-TYPE
38     SYNTAX      SEQUENCE OF LldpV2StatsRxPortEntry
39     MAX-ACCESS   not-accessible
40     STATUS       current
41     DESCRIPTION
42         "A table containing LLDP reception statistics for individual
43         ports and destination MAC addresses.
44         Entries are not required to exist in this table while
45         the lldpPortConfigEntry object is equal to 'disabled(4)'."
46 ::= { lldpV2Statistics 7 }
47
48 lldpV2StatsRxPortEntry OBJECT-TYPE
49     SYNTAX      LldpV2StatsRxPortEntry
50     MAX-ACCESS   not-accessible
51     STATUS       current
52     DESCRIPTION
53         "LLDP frame reception statistics for a particular port.
54         The port is contained in the same chassis as the
55         LLDP agent.
56
57         All counter values in a particular entry shall be
58         maintained on a continuing basis and shall not be deleted
59         upon expiration of rxInfoTTL timing counters in the LLDP

```

```

1         remote systems MIB of the receipt of a shutdown frame from
2         a remote LLDP agent.
3
4         All statistical counters associated with a particular
5         port on the local LLDP agent become frozen whenever the
6         adminStatus is disabled for the same port.
7
8         Rows in this table can only be created for MAC addresses
9         that can validly be used in association with the type of
10        interface concerned, as defined by Table 7-2.
11
12        The contents of this table is persistent across
13        re-initializations or re-boots."
14    INDEX { lldpV2StatsRxDestIfIndex,
15            lldpV2StatsRxDestMACAddress }
16    ::= { lldpV2StatsRxPortTable 1 }
17
18    lldpV2StatsRxPortEntry ::= SEQUENCE {
19        lldpV2StatsRxDestIfIndex          InterfaceIndex,
20        lldpV2StatsRxDestMACAddress        LldpV2DestAddressTableIndex,
21        lldpV2StatsRxPortFramesDiscardedTotal Counter32,
22        lldpV2StatsRxPortFramesErrors      Counter32,
23        lldpV2StatsRxPortFramesTotal       Counter32,
24        lldpV2StatsRxPortTLVsDiscardedTotal Counter32,
25        lldpV2StatsRxPortTLVsUnrecognizedTotal Counter32,
26        lldpV2StatsRxPortAgeoutsTotal      ZeroBasedCounter32
27    }
28
29    lldpV2StatsRxDestIfIndex OBJECT-TYPE
30        SYNTAX      InterfaceIndex
31        MAX-ACCESS   not-accessible
32        STATUS       current
33        DESCRIPTION
34            "The interface index value used to identify the port
35            associated with this entry. Its value is an index
36            into the interfaces MIB.
37
38            The value of this object is used as an index to the
39            lldpV2StatsRxPortTable."
40    ::= { lldpV2StatsRxPortEntry 1 }
41
42    lldpV2StatsRxDestMACAddress OBJECT-TYPE
43        SYNTAX      LldpV2DestAddressTableIndex
44        MAX-ACCESS   not-accessible
45        STATUS       current
46        DESCRIPTION
47            "The index value used to identify the destination
48            MAC address associated with this entry. Its value identifies
49            the row in the lldpV2DestAddressTable where the MAC address
50            can be found.
51
52            The value of this object is used as an index to the
53            lldpV2StatsRxPortTable."
54    ::= { lldpV2StatsRxPortEntry 2 }
55
56
57    lldpV2StatsRxPortFramesDiscardedTotal OBJECT-TYPE
58        SYNTAX      Counter32
59        UNITS        "LLDP frames"

```



```

1     MAX-ACCESS read-only
2     STATUS      current
3     DESCRIPTION
4         "The number of LLDP frames received by this LLDP agent on
5         the indicated port, and then discarded for any reason.
6         This counter can provide an indication that LLDP header
7         formatting problems may exist with the local LLDP agent in
8         the sending system or that LLDPDU validation problems may
9         exist with the local LLDP agent in the receiving system."
10    REFERENCE
11        "9.2.7.2"
12    ::= { lldpV2StatsRxPortEntry 3 }
13
14    lldpV2StatsRxPortFramesErrors OBJECT-TYPE
15        SYNTAX      Counter32
16        UNITS        "LLDP frames"
17        MAX-ACCESS  read-only
18        STATUS      current
19        DESCRIPTION
20            "The number of invalid LLDP frames received by this LLDP
21            agent on the indicated port, while this LLDP agent is enabled."
22        REFERENCE
23            "9.2.7.3"
24        ::= { lldpV2StatsRxPortEntry 4 }
25
26    lldpV2StatsRxPortFramesTotal OBJECT-TYPE
27        SYNTAX      Counter32
28        UNITS        "LLDP frames"
29        MAX-ACCESS  read-only
30        STATUS      current
31        DESCRIPTION
32            "The number of valid LLDP frames received by this LLDP agent
33            on the indicated port, while this LLDP agent is enabled."
34        REFERENCE
35            "9.2.7.4"
36        ::= { lldpV2StatsRxPortEntry 5 }
37
38    lldpV2StatsRxPortTLVsDiscardedTotal OBJECT-TYPE
39        SYNTAX      Counter32
40        UNITS        "TLVs"
41        MAX-ACCESS  read-only
42        STATUS      current
43        DESCRIPTION
44            "The number of LLDP TLVs discarded for any reason by this LLDP
45            agent on the indicated port."
46        REFERENCE
47            "9.2.7.6"
48        ::= { lldpV2StatsRxPortEntry 6 }
49
50    lldpV2StatsRxPortTLVsUnrecognizedTotal OBJECT-TYPE
51        SYNTAX      Counter32
52        UNITS        "TLVs"
53        MAX-ACCESS  read-only
54        STATUS      current
55        DESCRIPTION
56            "The number of LLDP TLVs received on the given port that
57            are not recognized by this LLDP agent on the indicated port.
58
59            An unrecognized TLV is referred to as the TLV whose type value

```

```

1         is in the range of reserved TLV types (000 1100 - 111 1110)
2         in Table 8-1 of IEEE Std 802.1AB. An unrecognized
3         TLV may be a basic management TLV from a later LLDP version."
4     REFERENCE
5         "9.2.7.7"
6     ::= { lldpV2StatsRxPortEntry 7 }
7
8 lldpV2StatsRxPortAgeoutsTotal    OBJECT-TYPE
9     SYNTAX      ZeroBasedCounter32
10    MAX-ACCESS   read-only
11    STATUS       current
12    DESCRIPTION
13        "The counter that represents the number of age-outs that
14        occurred on a given port. An age-out is the number of
15        times the complete set of information advertised by a
16        particular MSAP has been deleted from tables contained in
17        lldpV2RemoteSystemsData and lldpV2Extensions objects because
18        the information timeliness interval has expired.
19
20        This counter is similar to lldpV2StatsRemTablesAgeouts, except
21        that the counter is on a per port basis. This enables NMS to
22        poll tables associated with the lldpV2RemoteSystemsData objects
23        and all LLDP extension objects associated with remote systems
24        on the indicated port only.
25
26        This counter is set to zero during agent initialization
27        and its value should not be saved in non-volatile storage.
28
29        This counter is incremented only once when the
30        complete set of information is invalidated (aged out) from
31        all related tables on a particular port. Partial ageing
32        is not allowed."
33    REFERENCE
34        "9.2.7.1"
35    ::= { lldpV2StatsRxPortEntry 8 }
36
37
38 -- *****
39 --
40 --             L O C A L       S Y S T E M       D A T A
41 --
42 -- *****
43
44 lldpV2LocChassisIdSubtype    OBJECT-TYPE
45     SYNTAX      LldpV2ChassisIdSubtype
46     MAX-ACCESS   read-only
47     STATUS       current
48     DESCRIPTION
49         "The type of encoding used to identify the chassis
50         associated with the local system."
51     REFERENCE
52         "8.5.2.2"
53     ::= { lldpV2LocalSystemData 1 }
54
55 lldpV2LocChassisId    OBJECT-TYPE
56     SYNTAX      LldpV2ChassisId
57     MAX-ACCESS   read-only
58     STATUS       current
59     DESCRIPTION

```

```
1         "The string value used to identify the chassis component
2         associated with the local system."
3     REFERENCE
4         "8.5.2.3"
5     ::= { lldpV2LocalSystemData 2 }
6
7 lldpV2LocSysName OBJECT-TYPE
8     SYNTAX      SnmpAdminString (SIZE(0..255))
9     MAX-ACCESS  read-only
10    STATUS      current
11    DESCRIPTION
12        "The string value used to identify the system name of the
13        local system. If the local agent supports IETF RFC 3418,
14        lldpLocSysName object should have the same value as sysName
15        object."
16    REFERENCE
17        "8.5.6.2"
18    ::= { lldpV2LocalSystemData 3 }
19
20 lldpV2LocSysDesc OBJECT-TYPE
21     SYNTAX      SnmpAdminString (SIZE(0..255))
22     MAX-ACCESS  read-only
23     STATUS      current
24     DESCRIPTION
25         "The string value used to identify the system description
26         of the local system. If the local agent supports IETF RFC 3418,
27         lldpLocSysDesc object should have the same value as sysDesc
28         object."
29     REFERENCE
30         "8.5.7.2"
31     ::= { lldpV2LocalSystemData 4 }
32
33 lldpV2LocSysCapSupported OBJECT-TYPE
34     SYNTAX      LldpV2SystemCapabilitiesMap
35     MAX-ACCESS  read-only
36     STATUS      current
37     DESCRIPTION
38         "The bitmap value used to identify which system capabilities
39         are supported on the local system."
40     REFERENCE
41         "8.5.8.1"
42     ::= { lldpV2LocalSystemData 5 }
43
44 lldpV2LocSysCapEnabled OBJECT-TYPE
45     SYNTAX      LldpV2SystemCapabilitiesMap
46     MAX-ACCESS  read-only
47     STATUS      current
48     DESCRIPTION
49         "The bitmap value used to identify which system capabilities
50         are enabled on the local system."
51     REFERENCE
52         "8.5.8.2"
53     ::= { lldpV2LocalSystemData 6 }
54
55
56 --
57 -- lldpV2LocPortTable : Port specific Local system data
58 -- Indexed by ifIndex.
59 --
```

```

1
2 lldpV2LocPortTable OBJECT-TYPE
3     SYNTAX      SEQUENCE OF LldpV2LocPortEntry
4     MAX-ACCESS  not-accessible
5     STATUS      current
6     DESCRIPTION
7         "This table contains one row per port of information
8         associated with the local system known to this agent."
9     ::= { lldpV2LocalSystemData 7 }
10
11 lldpV2LocPortEntry OBJECT-TYPE
12     SYNTAX      LldpV2LocPortEntry
13     MAX-ACCESS  not-accessible
14     STATUS      current
15     DESCRIPTION
16         "Information about a particular port component.
17
18         Entries may be created and deleted in this table by the
19         agent.
20
21         Rows in this table can only be created for MAC addresses
22         that can validly be used in association with the type of
23         interface concerned, as defined by Table 7-2.
24
25         The contents of this table is persistent across
26         re-initializations or re-boots."
27     INDEX      { lldpV2LocPortIfIndex }
28     ::= { lldpV2LocPortTable 1 }
29
30 LldpV2LocPortEntry ::= SEQUENCE {
31     lldpV2LocPortIfIndex      InterfaceIndex,
32     lldpV2LocPortIdSubtype    LldpV2PortIdSubtype,
33     lldpV2LocPortId           LldpV2PortId,
34     lldpV2LocPortDesc         SnmpAdminString
35 }
36
37 lldpV2LocPortIfIndex OBJECT-TYPE
38     SYNTAX      InterfaceIndex
39     MAX-ACCESS  not-accessible
40     STATUS      current
41     DESCRIPTION
42         "The interface index value used to identify the port
43         associated with this entry. Its value is an index
44         into the interfaces MIB.
45
46         The value of this object is used as an index to the
47         lldpV2LocPortTable."
48     ::= { lldpV2LocPortEntry 1 }
49
50 lldpV2LocPortIdSubtype OBJECT-TYPE
51     SYNTAX      LldpV2PortIdSubtype
52     MAX-ACCESS  read-only
53     STATUS      current
54     DESCRIPTION
55         "The type of port identifier encoding used in the associated
56         'lldpLocPortId' object."
57     REFERENCE
58         "8.5.3.2"
59     ::= { lldpV2LocPortEntry 2 }

```

```

1
2 lldpV2LocPortId    OBJECT-TYPE
3     SYNTAX          LldpV2PortId
4     MAX-ACCESS      read-only
5     STATUS          current
6     DESCRIPTION
7         "The string value used to identify the port component
8         associated with a given port in the local system."
9     REFERENCE
10        "8.5.3.3"
11    ::= { lldpV2LocPortEntry 3 }
12
13 lldpV2LocPortDesc  OBJECT-TYPE
14     SYNTAX          SnmpAdminString (SIZE(0..255))
15     MAX-ACCESS      read-only
16     STATUS          current
17     DESCRIPTION
18         "The string value used to identify the IEEE 802 LAN station's port
19         description associated with the local system. If the local
20         agent supports IETF RFC 2863, lldpLocPortDesc object should
21         have the same value of ifDescr object."
22     REFERENCE
23        "8.5.5.2"
24    ::= { lldpV2LocPortEntry 4 }
25
26 --
27 -- lldpV2LocManAddrTable : Management addresses of the local system
28 --
29
30 lldpV2LocManAddrTable OBJECT-TYPE
31     SYNTAX          SEQUENCE OF LldpV2LocManAddrEntry
32     MAX-ACCESS      not-accessible
33     STATUS          current
34     DESCRIPTION
35         "This table contains management address information on the
36         local system known to this agent."
37    ::= { lldpV2LocalSystemData 8 }
38
39 lldpV2LocManAddrEntry OBJECT-TYPE
40     SYNTAX          LldpV2LocManAddrEntry
41     MAX-ACCESS      not-accessible
42     STATUS          current
43     DESCRIPTION
44         "Management address information about a particular chassis
45         component. There may be multiple management addresses
46         configured on the system identified by a particular
47         lldpLocChassisId. Each management address should have
48         distinct 'management address type' (lldpV2LocManAddrSubtype) and
49         'management address' (lldpLocManAddr.)
50
51         Entries may be created and deleted in this table by the
52         agent.
53
54         Since a variable length octetstring is used as an index
55         in a table, the address length is encoded as part of the OID
56         (as per IETF RFC 2578)."
```

```

57
58     INDEX          { lldpV2LocManAddrSubtype,
59                     lldpV2LocManAddr }
```

```

1      ::= { lldpV2LocManAddrTable 1 }
2
3 LldpV2LocManAddrEntry ::= SEQUENCE {
4     lldpV2LocManAddrSubtype    AddressFamilyNumbers,
5     lldpV2LocManAddr          LldpV2ManAddress,
6     lldpV2LocManAddrLen       Unsigned32,
7     lldpV2LocManAddrIfSubtype LldpV2ManAddrIfSubtype,
8     lldpV2LocManAddrIfId      Unsigned32,
9     lldpV2LocManAddrOID       OBJECT IDENTIFIER
10 }
11
12 lldpV2LocManAddrSubtype OBJECT-TYPE
13     SYNTAX      AddressFamilyNumbers
14     MAX-ACCESS  not-accessible
15     STATUS      current
16     DESCRIPTION
17         "The type of management address identifier encoding used in
18         the associated 'lldpLocManagmentAddr' object.
19
20         It should be noted that only a subset of the possible
21         address encodings enumerated in AddressFamilyNumbers
22         are appropriate for use as a LLDP management
23         address, either because some are just not applicable or
24         because the maximum size of a LldpV2ManAddress octet string
25         would prevent the use of some address identifier encodings."
26     REFERENCE
27         "8.5.9.3"
28     ::= { lldpV2LocManAddrEntry 1 }
29
30 lldpV2LocManAddr OBJECT-TYPE
31     SYNTAX      LldpV2ManAddress
32     MAX-ACCESS  not-accessible
33     STATUS      current
34     DESCRIPTION
35         "The string value used to identify the management address
36         component associated with the local system. The purpose of
37         this address is to contact the management entity."
38     REFERENCE
39         "8.5.9.4"
40     ::= { lldpV2LocManAddrEntry 2 }
41
42 lldpV2LocManAddrLen OBJECT-TYPE
43     SYNTAX      Unsigned32
44     MAX-ACCESS  read-only
45     STATUS      current
46     DESCRIPTION
47         "The total length of the management address subtype and the
48         management address fields in LLDPDUs transmitted by the
49         local LLDP agent.
50
51         The management address length field is needed so that the
52         receiving systems that do not implement SNMP are not
53         required to implement an Internet Assigned Numbers
54         Authority (IANA) family numbers/address length equivalency
55         table in order to decode the management address."
56     REFERENCE
57         "8.5.9.2"
58     ::= { lldpV2LocManAddrEntry 3 }
59

```

```

1 lldpV2LocManAddrIfSubtype OBJECT-TYPE
2     SYNTAX      LldpV2ManAddrIfSubtype
3     MAX-ACCESS  read-only
4     STATUS      current
5     DESCRIPTION
6         "The enumeration value that identifies the interface numbering
7         method used for defining the interface number
8         (lldpV2LocManAddrIfId), associated with the local system."
9     REFERENCE
10        "8.5.9.5"
11    ::= { lldpV2LocManAddrEntry 4 }
12
13 lldpV2LocManAddrIfId OBJECT-TYPE
14     SYNTAX      Unsigned32
15     MAX-ACCESS  read-only
16     STATUS      current
17     DESCRIPTION
18         "The integer value used to identify the interface number
19         regarding the management address component associated with
20         the local system."
21     REFERENCE
22        "8.5.9.6"
23    ::= { lldpV2LocManAddrEntry 5 }
24
25 lldpV2LocManAddrOID OBJECT-TYPE
26     SYNTAX      OBJECT IDENTIFIER
27     MAX-ACCESS  read-only
28     STATUS      current
29     DESCRIPTION
30         "The OID value used to identify the type of hardware component
31         or protocol entity associated with the management address
32         advertised by the local system agent."
33     REFERENCE
34        "8.5.9.8"
35    ::= { lldpV2LocManAddrEntry 6 }
36
37
38 -- *****
39 --
40 --             R E M O T E       S Y S T E M S       D A T A
41 --
42 -- *****
43
44
45 --
46 -- lldpV2RemTable
47 -- Indexed by ifIndex and destination MAC address.
48 --
49
50 lldpV2RemTable OBJECT-TYPE
51     SYNTAX      SEQUENCE OF LldpV2RemEntry
52     MAX-ACCESS  not-accessible
53     STATUS      current
54     DESCRIPTION
55         "This table contains one or more rows per physical network
56         connection known to this agent. The agent may wish to ensure
57         that only one lldpRemEntry is present for each local port
58         and destination MAC address,
59         or it may choose to maintain multiple lldpRemEntries for

```

1 the same local port and destination MAC address.

2

3 The following procedure may be used to retrieve remote

4 systems information updates from an LLDP agent:

5

6 1. NMS polls all tables associated with remote systems

7 and keeps a local copy of the information retrieved.

8 NMS polls periodically the values of the following

9 objects:

10 a. lldpV2StatsRemTablesInserts

11 b. lldpV2StatsRemTablesDeletes

12 c. lldpV2StatsRemTablesDrops

13 d. lldpV2StatsRemTablesAgeouts

14 e. lldpV2StatsRxPortAgeoutsTotal for all ports.

15

16 2. LLDP agent updates remote systems MIB objects, and

17 sends out notifications to a list of notification

18 destinations.

19

20 3. NMS receives the notifications and compares the new

21 values of objects listed in step 1.

22

23 Periodically, NMS should poll the object

24 lldpV2StatsRemTablesLastChangeTime to find out if anything

25 has changed since the last poll. If something has

26 changed, NMS polls the objects listed in step 1 to

27 figure out what kind of changes occurred in the tables.

28

29 If value of lldpV2StatsRemTablesInserts has changed,

30 then NMS walks all tables by employing TimeFilter

31 with the last-poll time value. This request

32 returns new objects or objects whose values have been

33 updated since the last poll.

34

35 If value of lldpV2StatsRemTablesAgeouts has changed,

36 then NMS walks the lldpStatsRxPortAgeoutsTotal and

37 compares the new values with previously recorded ones.

38 For ports whose lldpStatsRxPortAgeoutsTotal value is

39 greater than the recorded value, NMS can

40 retrieve objects associated with those ports from

41 table(s) without employing a TimeFilter (which is

42 performed by specifying 0 for the TimeFilter).

43

44 lldpV2StatsRemTablesDeletes and lldpV2StatsRemTablesDrops

45 objects are provided for informational purposes."

46 ::= { lldpV2RemoteSystemsData 1 }

47

48 lldpV2RemEntry OBJECT-TYPE

49 SYNTAX LldpV2RemEntry

50 MAX-ACCESS not-accessible

51 STATUS current

52 DESCRIPTION

53 "Information about a particular physical network connection.

54 Entries may be created and deleted in this table by the agent,

55 if a physical topology discovery process is active.

56

57 Rows in this table can only be created for MAC addresses

58 that can validly be used in association with the type of

59 interface concerned, as defined by Table 7-2.


```

1
2         The contents of this table is persistent across
3         re-initializations or re-boots."
4     INDEX {
5         lldpV2RemTimeMark,
6         lldpV2RemLocalIfIndex,
7         lldpV2RemLocalDestMACAddress,
8         lldpV2RemIndex
9     }
10    ::= { lldpV2RemTable 1 }
11
12    lldpV2RemEntry ::= SEQUENCE {
13        lldpV2RemTimeMark          TimeFilter,
14        lldpV2RemLocalIfIndex       InterfaceIndex,
15        lldpV2RemLocalDestMACAddress LldpV2DestAddressTableIndex,
16        lldpV2RemIndex              Unsigned32,
17        lldpV2RemChassisIdSubtype    LldpV2ChassisIdSubtype,
18        lldpV2RemChassisId           LldpV2ChassisId,
19        lldpV2RemPortIdSubtype       LldpV2PortIdSubtype,
20        lldpV2RemPortId              LldpV2PortId,
21        lldpV2RemPortDesc             SnmpAdminString,
22        lldpV2RemSysName              SnmpAdminString,
23        lldpV2RemSysDesc              SnmpAdminString,
24        lldpV2RemSysCapSupported      LldpV2SystemCapabilitiesMap,
25        lldpV2RemSysCapEnabled        LldpV2SystemCapabilitiesMap,
26        lldpV2RemRemoteChanges        TruthValue,
27        lldpV2RemTooManyNeighbors     TruthValue
28    }
29
30    lldpV2RemTimeMark OBJECT-TYPE
31        SYNTAX          TimeFilter
32        MAX-ACCESS       not-accessible
33        STATUS           current
34        DESCRIPTION
35            "A TimeFilter for this entry. See the TimeFilter textual
36            convention in IETF RFC 4502 and
37            http://www.ietf.org/IESG/Implementations/RFC2021-Implementation.txt
38            to see how TimeFilter works."
39        REFERENCE
40            "IETF RFC 4502 section 6"
41        ::= { lldpV2RemEntry 1 }
42
43
44    lldpV2RemLocalIfIndex OBJECT-TYPE
45        SYNTAX          InterfaceIndex
46        MAX-ACCESS       not-accessible
47        STATUS           current
48        DESCRIPTION
49            "The interface index value used to identify the port
50            associated with this entry. Its value is an index
51            into the interfaces MIB
52
53            The value of this object is used as an index to the
54            lldpV2RemTable."
55        ::= { lldpV2RemEntry 2 }
56
57    lldpV2RemLocalDestMACAddress OBJECT-TYPE
58        SYNTAX          LldpV2DestAddressTableIndex
59        MAX-ACCESS       not-accessible

```

```

1      STATUS      current
2      DESCRIPTION
3          "The index value used to identify the destination
4          MAC address associated with this entry. Its value identifies
5          the row in the lldpV2DestAddressTable where the MAC address
6          can be found.
7
8          The value of this object is used as an index to the
9          lldpV2RemTable."
10     ::= { lldpV2RemEntry 3 }
11
12
13 lldpV2RemIndex    OBJECT-TYPE
14     SYNTAX      Unsigned32(1..2147483647)
15     MAX-ACCESS   not-accessible
16     STATUS      current
17     DESCRIPTION
18         "This object represents an arbitrary local integer value used
19         by this agent to identify a particular connection instance,
20         unique only for the indicated remote system.
21
22         An agent is encouraged to assign monotonically increasing
23         index values to new entries, starting with one, after each
24         reboot. It is considered unlikely that the lldpRemIndex
25         can wrap between reboots."
26     ::= { lldpV2RemEntry 4 }
27
28 lldpV2RemChassisIdSubtype    OBJECT-TYPE
29     SYNTAX      LldpV2ChassisIdSubtype
30     MAX-ACCESS   read-only
31     STATUS      current
32     DESCRIPTION
33         "The type of encoding used to identify the chassis associated
34         with the remote system."
35     REFERENCE
36         "8.5.2.2"
37     ::= { lldpV2RemEntry 5 }
38
39 lldpV2RemChassisId    OBJECT-TYPE
40     SYNTAX      LldpV2ChassisId
41     MAX-ACCESS   read-only
42     STATUS      current
43     DESCRIPTION
44         "The string value used to identify the chassis component
45         associated with the remote system."
46     REFERENCE
47         "8.5.2.3"
48     ::= { lldpV2RemEntry 6 }
49
50 lldpV2RemPortIdSubtype    OBJECT-TYPE
51     SYNTAX      LldpV2PortIdSubtype
52     MAX-ACCESS   read-only
53     STATUS      current
54     DESCRIPTION
55         "The type of port identifier encoding used in the associated
56         'lldpRemPortId' object."
57     REFERENCE
58         "8.5.3.2"
59     ::= { lldpV2RemEntry 7 }

```

```
1
2 lldpV2RemPortId    OBJECT-TYPE
3     SYNTAX          LldpV2PortId
4     MAX-ACCESS      read-only
5     STATUS          current
6     DESCRIPTION
7         "The string value used to identify the port component
8         associated with the remote system."
9     REFERENCE
10        "8.5.3.3"
11    ::= { lldpV2RemEntry 8 }
12
13 lldpV2RemPortDesc   OBJECT-TYPE
14     SYNTAX          SnmpAdminString (SIZE(0..255))
15     MAX-ACCESS      read-only
16     STATUS          current
17     DESCRIPTION
18         "The string value used to identify the description of
19         the given port associated with the remote system."
20     REFERENCE
21        "8.5.5.2"
22    ::= { lldpV2RemEntry 9 }
23
24 lldpV2RemSysName     OBJECT-TYPE
25     SYNTAX          SnmpAdminString (SIZE(0..255))
26     MAX-ACCESS      read-only
27     STATUS          current
28     DESCRIPTION
29         "The string value used to identify the system name of the
30         remote system."
31     REFERENCE
32        "8.5.6.2"
33    ::= { lldpV2RemEntry 10 }
34
35 lldpV2RemSysDesc     OBJECT-TYPE
36     SYNTAX          SnmpAdminString (SIZE(0..255))
37     MAX-ACCESS      read-only
38     STATUS          current
39     DESCRIPTION
40         "The string value used to identify the system description
41         of the remote system."
42     REFERENCE
43        "8.5.7.2"
44    ::= { lldpV2RemEntry 11 }
45
46 lldpV2RemSysCapSupported OBJECT-TYPE
47     SYNTAX          LldpV2SystemCapabilitiesMap
48     MAX-ACCESS      read-only
49     STATUS          current
50     DESCRIPTION
51         "The bitmap value used to identify which system capabilities
52         are supported on the remote system."
53     REFERENCE
54        "8.5.8.1"
55    ::= { lldpV2RemEntry 12 }
56
57 lldpV2RemSysCapEnabled OBJECT-TYPE
58     SYNTAX          LldpV2SystemCapabilitiesMap
59     MAX-ACCESS      read-only
```

```

1     STATUS      current
2     DESCRIPTION
3         "The bitmap value used to identify which system capabilities
4         are enabled on the remote system."
5     REFERENCE
6         "8.5.8.2"
7     ::= { lldpV2RemEntry 13 }
8
9 lldpV2RemRemoteChanges OBJECT-TYPE
10    SYNTAX      TruthValue
11    MAX-ACCESS  read-only
12    STATUS      current
13    DESCRIPTION
14        "Indicates that there are changes in the remote systems
15        MIB, as determined by the variable remoteChanges."
16    REFERENCE
17        "9.2.5.10"
18    ::= { lldpV2RemEntry 14 }
19
20 lldpV2RemTooManyNeighbors OBJECT-TYPE
21    SYNTAX      TruthValue
22    MAX-ACCESS  read-only
23    STATUS      current
24    DESCRIPTION
25        "Indicates that there are too many neighbors
26        as determined by the variable tooManyNeighbors."
27    REFERENCE
28        "9.2.5.16"
29    ::= { lldpV2RemEntry 15 }
30
31 --
32 -- lldpV2RemManAddrTable : Management addresses of the remote system
33 -- Version 2 includes additional index values for ifIndex and
34 -- destination MAC address.
35 --
36
37 lldpV2RemManAddrTable OBJECT-TYPE
38    SYNTAX      SEQUENCE OF LldpV2RemManAddrEntry
39    MAX-ACCESS  not-accessible
40    STATUS      current
41    DESCRIPTION
42        "This table contains one or more rows per management address
43        information on the remote system learned on a particular port
44        contained in the local chassis known to this agent."
45    ::= { lldpV2RemoteSystemsData 2 }
46
47 lldpV2RemManAddrEntry OBJECT-TYPE
48    SYNTAX      LldpV2RemManAddrEntry
49    MAX-ACCESS  not-accessible
50    STATUS      current
51    DESCRIPTION
52        "Management address information about a particular chassis
53        component. There may be multiple management addresses
54        configured on the remote system identified by a particular
55        lldpRemIndex whose information is received on
56        an interface of the local system and a given destination
57        MAC address. Each management
58        address should have distinct 'management address
59        type' (lldpRemManAddrSubtype) and 'management address'

```

```

1          (lldpRemManAddr).
2
3          Entries may be created and deleted in this table by the
4          agent.
5          Since a variable length octetstring is used as an index
6          in a table, the address length is encoded as part of the OID
7          (as per IETF RFC 2578)."
```

INDEX	{	<pre> lldpV2RemTimeMark, lldpV2RemLocalIfIndex, lldpV2RemLocalDestMACAddress, lldpV2RemIndex, lldpV2RemManAddrSubtype, lldpV2RemManAddr </pre>
	}	

```

15 ::= { lldpV2RemManAddrTable 1 }
16
17
18 LldpV2RemManAddrEntry ::= SEQUENCE {
19     lldpV2RemManAddrSubtype    AddressFamilyNumbers,
20     lldpV2RemManAddr           LldpV2ManAddress,
21     lldpV2RemManAddrIfSubtype  LldpV2ManAddrIfSubtype,
22     lldpV2RemManAddrIfId       Unsigned32,
23     lldpV2RemManAddrOID        OBJECT IDENTIFIER
24 }
25
26 lldpV2RemManAddrSubtype OBJECT-TYPE
27     SYNTAX      AddressFamilyNumbers
28     MAX-ACCESS  not-accessible
29     STATUS      current
30     DESCRIPTION
31         "The type of management address identifier encoding used in
32         the associated 'lldpRemManagmentAddr' object.
33
34         It should be noted that only a subset of the possible
35         address encodings enumerated in AddressFamilyNumbers
36         are appropriate for use as a LLDP management
37         address, either because some are just not applicable or
38         because the maximum size of a LldpV2ManAddress octet string
39         would prevent the use of some address identifier encodings."
40     REFERENCE
41         "8.5.9.3"
42 ::= { lldpV2RemManAddrEntry 1 }
43
44 lldpV2RemManAddr OBJECT-TYPE
45     SYNTAX      LldpV2ManAddress
46     MAX-ACCESS  not-accessible
47     STATUS      current
48     DESCRIPTION
49         "The string value used to identify the management address
50         component associated with the remote system. The purpose
51         of this address is to contact the management entity."
52     REFERENCE
53         "8.5.9.4"
54 ::= { lldpV2RemManAddrEntry 2 }
55
56 lldpV2RemManAddrIfSubtype OBJECT-TYPE
57     SYNTAX      LldpV2ManAddrIfSubtype
58     MAX-ACCESS  read-only
59     STATUS      current

```

```

1  DESCRIPTION
2      "The enumeration value that identifies the interface numbering
3      method used for defining the interface number, associated
4      with the remote system."
5  REFERENCE
6      "8.5.9.5"
7      ::= { lldpV2RemManAddrEntry 3 }
8
9  lldpV2RemManAddrIfId  OBJECT-TYPE
10     SYNTAX      Unsigned32
11     MAX-ACCESS  read-only
12     STATUS      current
13     DESCRIPTION
14         "The integer value used to identify the interface number
15         regarding the management address component associated with
16         the remote system. The value depends upon the value of the
17         lldpV2RemManAddrIfSubtype for the table row."
18     REFERENCE
19         "8.5.9.6"
20     ::= { lldpV2RemManAddrEntry 4 }
21
22 lldpV2RemManAddrOID  OBJECT-TYPE
23     SYNTAX      OBJECT IDENTIFIER
24     MAX-ACCESS  read-only
25     STATUS      current
26     DESCRIPTION
27         "The OID value used to identify the type of hardware component
28         or protocol entity associated with the management address
29         advertised by the remote system agent."
30     REFERENCE
31         "8.5.9.8"
32     ::= { lldpV2RemManAddrEntry 5 }
33
34
35 --
36 -- lldpV2RemUnknownTLVTable : Unrecognized TLV information
37 -- This version has additional indexes for
38 -- ifIndex and destination MAC address
39 --
40
41 lldpV2RemUnknownTLVTable  OBJECT-TYPE
42     SYNTAX      SEQUENCE OF LldpV2RemUnknownTLVEntry
43     MAX-ACCESS  not-accessible
44     STATUS      current
45     DESCRIPTION
46         "This table contains information about an incoming TLV that
47         is not recognized by the receiving LLDP agent. The TLV may
48         be from a later version of the basic management set.
49
50         This table should only contain TLVs that are found in
51         a single LLDP frame. Entries in this table, associated
52         with an MAC service access point (MSAP, the access point
53         for MAC services provided to the LCC sublayer, defined
54         in IEEE Standards Dictionary Online, which is also
55         identified with a particular lldpRemLocalPortNum,
56         lldpRemIndex pair) are overwritten with most recently
57         received unrecognized TLV from the same MSAP, or they
58         naturally age out when the rxInfoTTL timer
59         (associated with the MSAP) expires."

```

```

1  REFERENCE
2      "9.2.8.7.1"
3  ::= { lldpV2RemoteSystemsData 3 }
4
5  lldpV2RemUnknownTLVEntry OBJECT-TYPE
6      SYNTAX      LldpV2RemUnknownTLVEntry
7      MAX-ACCESS  not-accessible
8      STATUS      current
9      DESCRIPTION
10         "Information about an unrecognized TLV received from a
11         physical network connection. Entries may be created and
12         deleted in this table by the agent, if a physical topology
13         discovery process is active."
14  INDEX  {
15         lldpV2RemTimeMark,
16         lldpV2RemLocalIfIndex,
17         lldpV2RemLocalDestMACAddress,
18         lldpV2RemIndex,
19         lldpV2RemUnknownTLVType
20     }
21  ::= { lldpV2RemUnknownTLVTable 1 }
22
23  LldpV2RemUnknownTLVEntry ::= SEQUENCE {
24      lldpV2RemUnknownTLVType      Unsigned32,
25      lldpV2RemUnknownTLVInfo      OCTET STRING
26  }
27
28  lldpV2RemUnknownTLVType OBJECT-TYPE
29      SYNTAX      Unsigned32(9..126)
30      MAX-ACCESS  not-accessible
31      STATUS      current
32      DESCRIPTION
33         "This object represents the value extracted from the type
34         field of the TLV."
35      REFERENCE
36         "9.2.8.7.1"
37  ::= { lldpV2RemUnknownTLVEntry 1 }
38
39  lldpV2RemUnknownTLVInfo OBJECT-TYPE
40      SYNTAX      OCTET STRING (SIZE(0..511))
41      MAX-ACCESS  read-only
42      STATUS      current
43      DESCRIPTION
44         "This object represents the value extracted from the value
45         field of the TLV."
46      REFERENCE
47         "9.2.8.7.1"
48  ::= { lldpV2RemUnknownTLVEntry 2 }
49
50 -----
51 -- Remote Systems Extension Table - Organizationally Defined Information
52 -----
53 --
54 -- lldpV2RemOrgDefInfoTable - indexed by ifIndex and destination
55 -- MAC address.
56 --
57
58  lldpV2RemOrgDefInfoTable OBJECT-TYPE
59      SYNTAX      SEQUENCE OF LldpV2RemOrgDefInfoEntry

```

```

1  MAX-ACCESS    not-accessible
2  STATUS        current
3  DESCRIPTION
4      "This table contains one or more rows per physical network
5      connection which advertises the organizationally defined
6      information.
7
8      Note that this table contains one or more rows of
9      organizationally defined information that is not recognized
10     by the local agent.
11
12     If the local system is capable of recognizing any
13     organizationally defined information, appropriate extension
14     MIBs from the organization should be used for information
15     retrieval."
16 ::= { lldpV2RemoteSystemsData 4 }
17
18 lldpV2RemOrgDefInfoEntry OBJECT-TYPE
19 SYNTAX          LldpV2RemOrgDefInfoEntry
20 MAX-ACCESS      not-accessible
21 STATUS          current
22 DESCRIPTION
23     "Information about the unrecognized organizationally
24     defined information advertised by the remote system.
25     The lldpRemTimeMark, lldpRemLocalPortNum, lldpRemIndex,
26     lldpRemOrgDefInfoOUI, lldpRemOrgDefInfoSubtype, and
27     lldpRemOrgDefInfoIndex are indexes to this table. If there is
28     an lldpRemOrgDefInfoEntry associated with a particular remote
29     system identified by the lldpRemLocalPortNum and lldpRemIndex,
30     then there is an lldpRemEntry associated with the same
31     instance (i.e., using same indexes.) When the lldpRemEntry
32     for the same index is removed from the lldpRemTable, the
33     associated lldpRemOrgDefInfoEntry is removed from
34     the lldpRemOrgDefInfoTable. Note, the lldpV2RemOrgDefInfoOUI
35     contains an OI, rather than an OUI.
36
37     Entries may be created and deleted in this table by the
38     agent."
39 INDEX          { lldpV2RemTimeMark,
40                 lldpV2RemLocalIfIndex,
41                 lldpV2RemLocalDestMACAddress,
42                 lldpV2RemIndex,
43                 lldpV2RemOrgDefInfoOUI,
44                 lldpV2RemOrgDefInfoSubtype,
45                 lldpV2RemOrgDefInfoIndex }
46 ::= { lldpV2RemOrgDefInfoTable 1 }
47
48 lldpV2RemOrgDefInfoEntry ::= SEQUENCE {
49     lldpV2RemOrgDefInfoOUI      OCTET STRING,
50     lldpV2RemOrgDefInfoSubtype  Unsigned32,
51     lldpV2RemOrgDefInfoIndex    Unsigned32,
52     lldpV2RemOrgDefInfo         OCTET STRING
53 }
54
55 lldpV2RemOrgDefInfoOUI OBJECT-TYPE
56 SYNTAX          OCTET STRING (SIZE(3))
57 MAX-ACCESS      not-accessible
58 STATUS          current
59 DESCRIPTION

```



```

1         "The Organizationally Unique Identifier (OUI), as defined
2         in IEEE Std 802, is a 24 bit (three octets) globally
3         unique assigned number referenced by various standards,
4         of the information received from the remote system."
5     REFERENCE
6         "8.7.1.3"
7     ::= { lldpV2RemOrgDefInfoEntry 1 }
8
9 lldpV2RemOrgDefInfoSubtype OBJECT-TYPE
10    SYNTAX      Unsigned32(1..255)
11    MAX-ACCESS  not-accessible
12    STATUS      current
13    DESCRIPTION
14        "The integer value used to identify the subtype of the
15        organizationally defined information received from the
16        remote system.
17
18        The subtype value is required to identify different instances
19        of organizationally defined information that could not be
20        retrieved without a unique identifier that indicates the
21        particular type of information contained in the information
22        string."
23    REFERENCE
24        "8.7.1.4"
25    ::= { lldpV2RemOrgDefInfoEntry 2 }
26
27 lldpV2RemOrgDefInfoIndex OBJECT-TYPE
28    SYNTAX      Unsigned32(1..2147483647)
29    MAX-ACCESS  not-accessible
30    STATUS      current
31    DESCRIPTION
32        "This object represents an arbitrary local integer value
33        used by this agent to identify a particular unrecognized
34        organizationally defined information instance, unique only
35        for the lldpRemOrgDefInfoOUI and lldpRemOrgDefInfoSubtype
36        from the same remote system.
37
38        An agent is encouraged to assign monotonically increasing
39        index values to new entries, starting with one, after each
40        reboot. It is considered unlikely that the
41        lldpRemOrgDefInfoIndex can wrap between reboots."
42    ::= { lldpV2RemOrgDefInfoEntry 3 }
43
44 lldpV2RemOrgDefInfo OBJECT-TYPE
45    SYNTAX      OCTET STRING(SIZE(0..507))
46    MAX-ACCESS  read-only
47    STATUS      current
48    DESCRIPTION
49        "The string value used to identify the organizationally
50        defined information of the remote system. The encoding for
51        this object should be as defined for SnmpAdminString TC."
52    REFERENCE
53        "8.7.1.5"
54    ::= { lldpV2RemOrgDefInfoEntry 4 }
55
56
57 --
58 -- *****
59 --

```

```

1 --          L L D P   M I B   N O T I F I C A T I O N S
2 --
3 -- *****
4 --
5
6 lldpV2NotificationPrefix OBJECT IDENTIFIER ::= { lldpV2Notifications 0 }
7
8 lldpV2RemTablesChange NOTIFICATION-TYPE
9     OBJECTS {
10         lldpV2StatsRemTablesInserts,
11         lldpV2StatsRemTablesDeletes,
12         lldpV2StatsRemTablesDrops,
13         lldpV2StatsRemTablesAgeouts
14     }
15     STATUS          current
16     DESCRIPTION
17         "A lldpV2RemTablesChange notification is sent when the value
18         of lldpV2StatsRemTablesLastChangeTime changes. It can be
19         utilized by an NMS to trigger LLDP remote systems table
20         maintenance polls.
21
22         Note that transmission of lldpV2RemTablesChange
23         notifications are throttled by the agent, as specified by the
24         'lldpV2NotificationInterval' object."
25     ::= { lldpV2NotificationPrefix 1 }
26
27
28 --
29 -- *****
30 --
31 --          L L D P   M I B   C O N F O R M A N C E
32 --
33 -- *****
34 --
35
36 lldpV2Compliances OBJECT IDENTIFIER ::= { lldpV2Conformance 1 }
37 lldpV2Groups      OBJECT IDENTIFIER ::= { lldpV2Conformance 2 }
38
39 -- compliance statements
40
41 lldpV2TxRxCompliance MODULE-COMPLIANCE
42     --V2 to add ifGeneralInformationGroup
43     --and support re-indexed tables
44     STATUS current
45     DESCRIPTION
46         "A compliance statement for all SNMP entities that
47         implement the LLDP MIB as either a transmitter or
48         a receiver of LLDPDUs.
49
50         This version defines compliance requirements for
51         V2 of the LLDP MIB module."
52     MODULE -- this module
53         MANDATORY-GROUPS { lldpV2ConfigGroup,
54                             ifGeneralInformationGroup
55         }
56
57     ::= { lldpV2Compliances 1 }
58
59 lldpV2TxCompliance MODULE-COMPLIANCE

```

```

1          --V2 requirements for transmitters of LLDPDUs
2          --and support re-indexed tables
3  STATUS   current
4  DESCRIPTION
5          "A compliance statement for SNMP entities that implement
6          the LLDP MIB and have the capability of transmitting
7          LLDP frames.
8
9          This version defines compliance requirements for
10         V2 of the LLDP MIB module."
11  MODULE   -- this module
12          MANDATORY-GROUPS { lldpV2ConfigTxGroup,
13                              lldpV2StatsTxGroup,
14                              lldpV2LocSysGroup
15          }
16
17          ::= { lldpV2Compliances 2 }
18
19 lldpV2RxCompliance MODULE-COMPLIANCE
20         --V2 requirements for receivers of LLDPDUs
21         --and support re-indexed tables
22  STATUS   current
23  DESCRIPTION
24         "The compliance statement for SNMP entities that implement
25         the LLDP MIB and have the capability of receiving
26         LLDP frames.
27
28         This version defines compliance requirements for
29         V2 of the LLDP MIB module."
30  MODULE   -- this module
31          MANDATORY-GROUPS { lldpV2ConfigRxGroup,
32                              lldpV2StatsRxGroup,
33                              lldpV2RemSysGroup,
34                              lldpV2NotificationsGroup
35          }
36
37          ::= { lldpV2Compliances 3 }
38
39 -- MIB groupings
40
41
42 lldpV2ConfigGroup   OBJECT-GROUP
43  OBJECTS {
44      lldpV2PortConfigAdminStatusV2
45  }
46  STATUS   current
47  DESCRIPTION
48      "The collection of objects that are used to configure the
49      LLDP implementation behavior."
50  ::= { lldpV2Groups 1 }
51
52
53 lldpV2ConfigRxGroup OBJECT-GROUP
54  OBJECTS {
55      lldpV2NotificationInterval,
56      lldpV2PortConfigNotificationEnableV2
57  }
58  STATUS   current
59  DESCRIPTION

```

```

1         "The collection of objects that are used to configure the
2         LLDP reception implementation behavior."
3 ::= { lldpV2Groups 2 }
4
5
6 lldpV2ConfigTxGroup    OBJECT-GROUP
7   OBJECTS {
8     lldpV2MessageTxInterval,
9     lldpV2MessageTxHoldMultiplier,
10    lldpV2ReinitDelay,
11    lldpV2PortConfigTLVsTxEnableV2,
12    lldpV2ManAddrConfigTxEnable,
13    lldpV2ManAddrConfigRowStatus,
14    lldpV2TxCreditMax,
15    lldpV2MessageFastTx,
16    lldpV2TxFastInit,
17    lldpV2DestMacAddress,
18    lldpV2PortMessageTxInterval,
19    lldpV2PortMessageTxHoldMultiplier,
20    lldpV2PortReinitDelay,
21    lldpV2PortNotificationInterval,
22    lldpV2PortTxCreditMax,
23    lldpV2PortMessageFastTx,
24    lldpV2PortTxFastInit
25  }
26  STATUS current
27  DESCRIPTION
28    "The collection of objects that are used to configure the
29    LLDP transmission implementation behavior."
30 ::= { lldpV2Groups 3 }
31
32
33 lldpV2StatsRxGroup    OBJECT-GROUP
34   OBJECTS {
35     lldpV2StatsRemTablesLastChangeTime,
36     lldpV2StatsRemTablesInserts,
37     lldpV2StatsRemTablesDeletes,
38     lldpV2StatsRemTablesDrops,
39     lldpV2StatsRemTablesAgeouts,
40     lldpV2StatsRxPortFramesDiscardedTotal,
41     lldpV2StatsRxPortFramesErrors,
42     lldpV2StatsRxPortFramesTotal,
43     lldpV2StatsRxPortTLVsDiscardedTotal,
44     lldpV2StatsRxPortTLVsUnrecognizedTotal,
45     lldpV2StatsRxPortAgeoutsTotal
46   }
47  STATUS current
48  DESCRIPTION
49    "The collection of objects that are used to represent LLDP
50    reception statistics."
51 ::= { lldpV2Groups 4 }
52
53
54 lldpV2StatsTxGroup    OBJECT-GROUP
55   OBJECTS {
56     lldpV2StatsTxPortFramesTotal,
57     lldpV2StatsTxLLDPDULengthErrors
58   }
59  STATUS current

```

```

1      DESCRIPTION
2          "The collection of objects that are used to represent LLDP
3          transmission statistics."
4      ::= { lldpV2Groups 5 }
5
6
7 lldpV2LocSysGroup  OBJECT-GROUP
8     OBJECTS {
9         lldpV2LocChassisIdSubtype,
10        lldpV2LocChassisId,
11        lldpV2LocPortIdSubtype,
12        lldpV2LocPortId,
13        lldpV2LocPortDesc,
14        lldpV2LocSysDesc,
15        lldpV2LocSysName,
16        lldpV2LocSysCapSupported,
17        lldpV2LocSysCapEnabled,
18        lldpV2LocManAddrLen,
19        lldpV2LocManAddrIfSubtype,
20        lldpV2LocManAddrIfId,
21        lldpV2LocManAddrOID
22    }
23     STATUS  current
24     DESCRIPTION
25         "The collection of objects that are used to represent LLDP
26         Local System Information."
27     ::= { lldpV2Groups 6 }
28
29 lldpV2RemSysGroup  OBJECT-GROUP
30     OBJECTS {
31         lldpV2RemChassisIdSubtype,
32         lldpV2RemChassisId,
33         lldpV2RemPortIdSubtype,
34         lldpV2RemPortId,
35         lldpV2RemPortDesc,
36         lldpV2RemSysName,
37         lldpV2RemSysDesc,
38         lldpV2RemSysCapSupported,
39         lldpV2RemSysCapEnabled,
40         lldpV2RemRemoteChanges,
41         lldpV2RemTooManyNeighbors,
42         lldpV2RemManAddrIfSubtype,
43         lldpV2RemManAddrIfId,
44         lldpV2RemManAddrOID,
45         lldpV2RemUnknownTLVInfo,
46         lldpV2RemOrgDefInfo
47     }
48     STATUS  current
49     DESCRIPTION
50         "The collection of objects that are used to represent
51         LLDP Remote Systems Information. The objects represent the
52         information associated with the basic TLV set. Please note
53         that even if the agent does not implement some of the optional
54         TLVs, it shall recognize all the optional TLV information
55         that the remote system may advertise."
56     ::= { lldpV2Groups 7 }
57
58 lldpV2NotificationsGroup  NOTIFICATION-GROUP
59     NOTIFICATIONS {

```

```
1      lldpV2RemTablesChange
2  }
3  STATUS current
4  DESCRIPTION
5      "The collection of notifications used to indicate LLDP MIB
6      data consistency and general status information."
7  ::= { lldpV2Groups 8 }
8
9 END
```

12. LLDP YANG definitions

This clause specifies YANG modules that provide control and status monitoring of systems and system components that implement functionality specified in this standard.

This clause:

- a) Introduces the YANG framework that governs the naming and hierarchy of configuration and operational data structures in the data models, and the modeling of network interfaces (12.1).
- b) Describes the information data model and its relationship to the operational processes and managed objects specified in the other clauses of this standard, and provides a Unified Modeling Language (UML) representation of each data model (12.2).
- c) Describes the structure of the data models, each of which comprises or makes use of one or more YANG modules (12.3).
- d) Includes a relationship description of other modules imported in YANG modules (12.4).
- e) Reviews security considerations applicable to each of the modules, with specific reference to data nodes in the YANG modules that compose the model (12.5).
- f) Includes each of the YANG modules and its data schema (12.6).

12.1 Internet Standard Management Framework

This YANG module uses the YANG 1.1 Data Modeling Language as specified in IETF RFC 7950.

The YANG framework applies hierarchy in the following areas:

- a) The uniform resource name (URN), as specified in IEEE Std 802.
- b) The YANG objects form a hierarchy of configuration and operational data structures that define the YANG model.

12.2 Information model for LLDP management

The YANG objects are based on the TLVs detailed in 8.5 and 8.7, and the agent operation variables detailed in 9.1 and 9.2. A UML-like representation of the management model is provided in the following subclauses.

The purpose of a UML-like¹⁶ diagram is to express the model design on a single piece of paper. The structure of the UML-like representation shows the name of the object followed by a list of properties for the object. The properties indicate its type and accessibility. It should be noted that the UML-like representation is meant to express simplified semantics for the properties. It is not meant to provide the specific datatype as used to encode the object in either MIB or YANG. In the UML-like representation, a box with a white background represents information that comes from sources outside of the IEEE. A box with a gray background represents objects that are defined by this IEEE Standard.

The YANG hierarchical structure that incorporates the LLDP YANG modules supported by this standard is represented by Figure 12-1. In the figures in this clause, items that are shaded gray are described in this document, items with no background shading are defined elsewhere. The YANG data model is realized in two YANG modules. One module *ieee802-dot1ab-types* provides data types that are needed by the LLDP configuration and monitoring objects. The *ieee802-dot1ab-lldp* module provides the LLDP configuration

¹⁶ A description of the UML-like diagrams used in this clause is provided at <https://1.ieee802.org/uml-like-diagrams>

1 and monitoring objects. The ability to augment the port container to support extension TLVs is also shown.

2 The LLDP capabilities are not only applicable to IEEE Std 802.1Q bridges, but also (for example) end

3 stations, and routers.

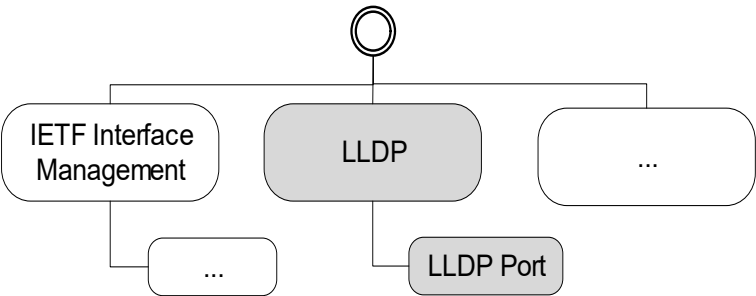


Figure 12-1—YANG root hierarchy with LLDP YANG modules

4 12.2.1 LLDP UML

5 The LLDP configuration and monitoring objects in Figure 12-2 show the objects that are applicable on a

6 per-agent or chassis associated with the LLDP agent.

7

lldp			
uint32	message-fast-tx;	// (9.2.5.5)	r-w
uint32	message-tx-hold-multiplier;	// (9.2.5.6)	r-w
uint32	message-tx-interval;	// (9.2.5.7)	r-w
uint32	reinit-delay;	// (9.2.5.10)	r-w
uint32	tx-credit-max;	// (9.2.5.17)	r-w
uint32	tx-fast-init;	// (9.2.5.19)	r-w
uint32	notification-interval;	// (9.2.5.7)	r-w
remote-statistics			
timestamp	last-change-time;	//	r
zero-based-counter32	remote-inserts;	//	r
zero-based-counter32	remote-deletes;	//	r
zero-based-counter32	remote-drops;	//	r
zero-based-counter32	remote-ageouts;	//	r
local-system-data			
enum	chassis-id-subtype;	// (8.5.2.2)	r
string	chassis-id;	// (8.5.2.3)	r
string	system-name;	// (8.5.6.2)	r
string	system-description;	// (8.5.7.2)	r
bits	system-capabilities-supported	// (8.5.8.1)	r
bits	system-capabilities-enabled	// (8.5.8.2)	r

Figure 12-2—LLDP configuration and monitoring objects

1 12.2.2 LLDP Port UML

2 The LLDP port configuration and monitoring objects in Figure 12-3 show the objects that are applicable to
3 an LLDP port.

4

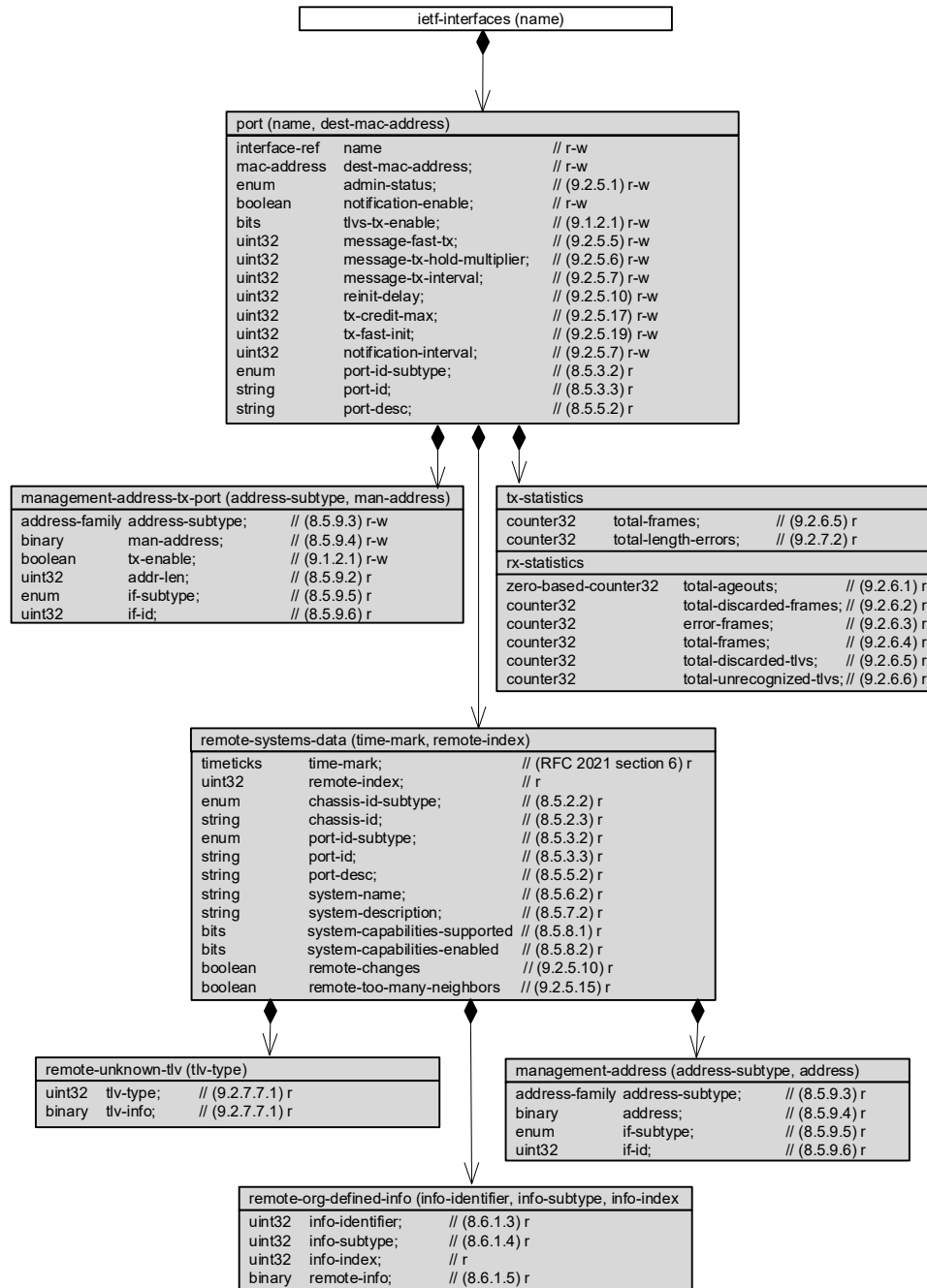


Figure 12-3—LLDP port configuration and monitoring objects

1 **12.3 Structure of the LLDP YANG model**

2 The IEEE YANG model specified in this standard is divided into two YANG modules. A summary of the
3 modules contained in this clause is represented in Table 12-1.

Table 12-1—Structure of the YANG modules

Module	Subclause	Notes
ieee802-dot1ab-types	Clause 12.6.2.1	Type definitions used for LLDP YANG
ieee802-dot1ab-lldp	Clause 12.6.2.2	LLDP Generic Management

4 In the YANG module definitions below, if any discrepancy between the DESCRIPTION text and the
5 corresponding definition in any other part of this standard occurs, the definitions outside this subclause take
6 precedence.

1 12.3.1 Structure of the *ieee802-dot1ab-lldp* module

2 The *ieee802-dot1ab-lldp* YANG module is divided into several YANG branches (e.g., subtrees). A summary
3 of the YANG subtrees associated with this module is presented in Table 12-2.

Table 12-2—Structure of the *ieee802-dot1ab-lldp* module

Branches	References	Notes
lldp		Link Layer Discovery Protocol (LLDP) configuration and operational information
message-fast-tx	9.2.5.4	
message-tx-hold-multiplier	9.2.5.5	
message-tx-interval	9.2.5.7	
reinit-delay	9.2.5.9	
tx-credit-max	9.2.5.18	
tx-fast-init	9.2.5.20	
notification-interval	9.2.5.6	
remote-statistics	10.5.2	LLDP remote operational statistics data
local-system-data	10.5.1	LLDP local system operational data
port	9.2.4	LLDP configuration information for a particular port
management-address-tx-port	8.5.9	Set of ports on which the local system management address instance will be transmitted
tx-statistics	9.2.7	LLDP frame transmission statistics for a particular port
rx-statistics	9.2.7	LLDP frame reception statistics for a particular port
remote-system-data	9.2.6	Information about a particular physical network connection

4 12.4 Relationship to other YANG modules

5 This clause describes how the *ieee802-dot1ab-lldp* YANG module is related to the YANG modules that are
6 imported.

7 12.4.1 IEEE LLDP Types Module

8 The *ieee802-dot1ab-types* module provides reusable types that are used by the *ieee802-dot1ab-lldp* module.

9 12.4.2 IETF Routing Module

10 The *ietf-routing* YANG module (IETF RFC 8349) is a module defined by the IETF for management of a
11 routing subsystem. This document only uses the address-family identity, which is used as the base identity
12 for address families.

1 12.4.3 IETF YANG Types Module

2 The *ietf-yang-types* YANG module (IETF RFC 6991) contains a set of derived YANG types. This document
3 leverages timestamp, timeticks, zero-based-counter32, and counter32.

4 12.4.4 IETF Interfaces YANG Module

5 The *ietf-interfaces* YANG module (IETF RFC 8343) contains a set of YANG definitions for managing
6 network interfaces. This document models a port as an interface.

7 12.4.5 IEEE 802 Types Module

8 The *ieee802-types* module provides reusable types that are used in IEEE 802 standards.

9 The type for mac-addresses defined in *ieee802-types* has a pattern that allows upper and lower case letters.
10 To avoid issues with string comparison, it is suggested to only use upper case for the letters in the
11 hexadecimal numbers. Implementers using code comparing MAC addresses should note that there is still an
12 issue with a difference between the IETF mac-address definition and the IEEE mac-address definition.

13 12.5 Security considerations

14 The YANG modules defined in this clause are designed to be accessed via a network configuration protocol
15 (e.g., NETCONF protocol). In the case of NETCONF, the lowest NETCONF layer is the secure transport
16 layer and the mandatory to implement secure transport is SSH. The NETCONF access control model
17 provides the means to restrict access for particular NETCONF users to a pre-configured subset of all
18 available NETCONF protocol operations and content.

19 It is the responsibility of a system's implementor and administrator to ensure that the protocol entities in the
20 system that support NETCONF, and any other remote configuration protocols that make use of these YANG
21 modules, are properly configured to allow access only to those users who have legitimate rights to read or
22 write data nodes. This standard does not specify how the credentials of those users are to be stored or
23 validated.

24 12.5.1 Security considerations of the *ieee802-dot1ab-lldp* YANG module

25 There are several management objects defined in the *ieee802-dot1ab-lldp* YANG module that are
26 configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive
27 or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF
28 RFC 6241), can support protocol operations that can edit or delete YANG module configuration data
29 (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper
30 protection, then negative effects on the network operation are possible.

31 The following objects in the *ieee802-dot1ab-lldp* YANG module can be manipulated to interfere with the
32 operation of LLDP. This could, for example, be used to return incorrect information about neighbor nodes,
33 or cause a denial-of-service attack.

34 lldp/message-fast-tx
35 lldp/message-tx-hold-multiplier
36 lldp/message-tx-interval
37 lldp/reinit-delay
38 lldp/tx-credit-max
39 lldp/tx-fast-init

1 lldp/notification-interval
2 lldp/port/name
3 lldp/port/dest-mac-address
4 lldp/port/admin-status
5 lldp/port/notification-enable
6 lldp/port/tlvs-tx-enable
7 lldp/port/message-fast-tx
8 lldp/port/message-tx-hold-multiplier
9 lldp/port/message-tx-interval
10 lldp/port/reinit-delay
11 lldp/port/tx-credit-max
12 lldp/port/tx-fast-init
13 lldp/port/notification-interval
14 lldp/port/management-address-tx-port/address-subtype
15 lldp/port/management-address-tx-port/man-address
16 lldp/port/management-address-tx-port/tx-enable

17 Some of the readable data in this YANG module may be considered sensitive or vulnerable in some network
18 environments. It is important to control all types of access (e.g., including NETCONF get and get-config
19 operations) to these objects and possibly to even encrypt the values of these objects when sending them over
20 the network. For example the system name and other information about the remote systems could provide
21 information about the configuration and topology of the network and could be considered a privacy threat.

22 12.6 Definition of the YANG modules^{17, 18, 19}

23 12.6.1 YANG schema definitions

24 A simplified graphical representation of the data model is used in this document. The meaning of the
25 symbols in these diagrams is as follows:

- 26 — Brackets “[“ and “]” enclose list keys.
- 27 — Abbreviations before data node names: “rw” means configuration (read-write), and “ro” means state
28 data (read-only).
- 29 — Symbols after data node names: “?” means an optional node, “!” means a presence container, and
30 “*” denotes a list and leaf-list.
- 31 — Parentheses enclose choice and case nodes, and case nodes are also marked with a colon (“:”).
- 32 — Ellipsis (“...”) stand for contents of subtrees that are not shown.

33 12.6.1.1 YANG schema definition for *ieee802-dot1ab-lldp* YANG module

```
34 module: ieee802-dot1ab-lldp
35   +---rw lldp
36     +---rw message-fast-tx?          uint32
37     +---rw message-tx-hold-multiplier?  uint32
```

¹⁷ *Copyright release for YANG modules:* Users of this standard may freely reproduce the YANG modules contained in this subclause so that they can be used for their intended purpose.

¹⁸ The ieee802-types YANG module is common across various IEEE 802.1 standards and can be updated by any IEEE 802.1 standard that imports the ieee802-types YANG module.

¹⁹ Consult the IEEE 802.1 website at <https://1.ieee802.org/yang-modules/> to determine and obtain the current revision of any IEEE 802.1 YANG module.

```

1      +---rw message-tx-interval?          uint32
2      +---rw reinit-delay?                 uint32
3      +---rw tx-credit-max?                uint32
4      +---rw tx-fast-init?                 uint32
5      +---rw notification-interval?         uint32
6      +---ro remote-statistics
7      |   +---ro last-change-time?         yang:timestamp
8      |   +---ro remote-inserts?           yang:zero-based-counter32
9      |   +---ro remote-deletes?           yang:zero-based-counter32
10     |   +---ro remote-drops?              yang:zero-based-counter32
11     |   +---ro remote-ageouts?           yang:zero-based-counter32
12     +---ro local-system-data
13     |   +---ro chassis-id-subtype?        ieee:chassis-id-subtype-type
14     |   +---ro chassis-id?                ieee:chassis-id-type
15     |   +---ro system-name?               string
16     |   +---ro system-description?        string
17     |   +---ro system-capabilities-supported? lldp-types:system-capabilities-map
18     |   +---ro system-capabilities-enabled? lldp-types:system-capabilities-map
19     +---rw port* [name dest-mac-address]
20         +---rw name                       if:interface-ref
21         +---rw dest-mac-address            ieee:mac-address
22         +---rw admin-status?               enumeration
23         +---rw notification-enable?         boolean
24         +---rw tlvs-tx-enable?             bits
25         +---rw message-fast-tx?            uint32
26         +---rw message-tx-hold-multiplier? uint32
27         +---rw message-tx-interval?         uint32
28         +---rw reinit-delay?                uint32
29         +---rw tx-credit-max?                uint32
30         +---rw tx-fast-init?                uint32
31         +---rw notification-interval?        uint32
32         +---rw management-address-tx-port* [address-subtype man-address]
33         |   +---rw address-subtype         identityref
34         |   +---rw man-address              lldp-types:man-addr-type
35         |   +---rw tx-enable?               boolean
36         |   +---ro addr-len?                uint32
37         |   +---ro if-subtype?              lldp-types:man-addr-if-subtype
38         |   +---ro if-id?                   uint32
39         +---ro port-id-subtype?              ieee:port-id-subtype-type
40         +---ro port-id?                      ieee:port-id-type
41         +---ro port-desc?                    string
42         +---ro tx-statistics
43         |   +---ro total-frames?            yang:counter32
44         |   +---ro total-length-errors?      yang:counter32
45         +---ro rx-statistics
46         |   +---ro total-ageouts?            yang:zero-based-counter32
47         |   +---ro total-discarded-frames?   yang:counter32
48         |   +---ro error-frames?             yang:counter32
49         |   +---ro total-frames?             yang:counter32
50         |   +---ro total-discarded-tlvs?     yang:counter32
51         |   +---ro total-unrecognized-tlvs?  yang:counter32
52         +---ro remote-systems-data* [time-mark remote-index]
53         |   +---ro time-mark                 yang:timeticks
54         |   +---ro remote-index              uint32
55         |   +---ro remote-too-many-neighbors? boolean
56         |   +---ro remote-changes?           boolean
57         |   +---ro chassis-id-subtype?        ieee:chassis-id-subtype-type
58         |   +---ro chassis-id?                ieee:chassis-id-type
59         |   +---ro port-id-subtype?           ieee:port-id-subtype-type

```

```

1      +---ro port-id?                                ieee:port-id-type
2      +---ro port-desc?                              string
3      +---ro system-name?                            string
4      +---ro system-description?                      string
5          +---ro system-capabilities-supported?      lldp-types:system-capabili-
6 ties-map
7          +---ro system-capabilities-enabled?        lldp-types:system-capabili-
8 ties-map
9      +---ro management-address* [address-subtype address]
10     | +---ro address-subtype      identityref
11     | +---ro address              lldp-types:man-addr-type
12     | +---ro if-subtype?          lldp-types:man-addr-if-subtype
13     | +---ro if-id?              uint32
14     +---ro remote-unknown-tlv* [tlv-type]
15     | +---ro tlv-type            uint32
16     | +---ro tlv-info?          binary
17         +---ro remote-org-defined-info* [info-identifier info-subtype
18 info-index]
19         +---ro info-identifier    uint32
20         +---ro info-subtype       uint32
21         +---ro info-index         uint32
22         +---ro remote-info?      binary
23
24 notifications:
25     +---n remote-table-change
26         +---ro remote-insert?    -> /lldp/remote-statistics/remote-inserts
27         +---ro remote-delete?    -> /lldp/remote-statistics/remote-deletes
28         +---ro remote-drops?     -> /lldp/remote-statistics/remote-drops
29         +---ro remote-ageouts?   -> /lldp/remote-statistics/remote-ageouts
30

```

31 12.6.2 YANG data model definitions

32 12.6.2.1 Definition for the *ieee802-dot1ab-types* module

```

33 module ieee802-dot1ab-types {
34   yang-version "1.1";
35   namespace urn:ieee:std:802.1Q:yang:ieee802-dot1ab-types;
36   prefix lldp-types;
37   organization
38     "IEEE 802.1 Working Group";
39   contact
40     "WG-URL: http://ieee802.org/1/
41     WG-EMail: stds-802-1-1@ieee.org
42     Contact: IEEE 802.1 Working Group Chair
43     Postal: C/O IEEE 802.1 Working Group
44     IEEE Standards Association
45     445 Hoes Lane
46     Piscataway, NJ 08854
47     USA
48
49     E-mail: stds-802-1-chairs@ieee.org";
50   description
51     "Common types used within ieee802-dot1ab-lldp modules. Copyright(C) IEEE
52     (2025). All rights reserved. This version of this YANG module is part of
53     IEEE Std 802.1AB-2026; see the standard itself for full legal notices.";
54   revision 2025-09-29 {
55     description

```

```
1      "Published as part of IEEE Std 802.1AB-2026.
2
3      The following reference statement identifies each referenced IEEE
4      Standard as updated by applicable amendments.";
5  reference
6      "IEEE Std 802.1AB Station and Media Access Control Connectivity Discovery:
7      IEEE Std 802.1AB-2026.";
8  }
9  revision 2022-03-15 {
10     description
11         "Published as part of IEEE Std 802.1ABcu.";
12     reference
13         "IEEE Std 802.1AB-2016";
14 }
15 typedef man-addr-if-subtype {
16     type enumeration {
17         enum unknown {
18             value 1;
19             description
20                 "Interface is not known.";
21         }
22         enum port-ref {
23             value 2;
24             description
25                 "Interface based on the port-ref MIB object.";
26         }
27         enum system-port-number {
28             value 3;
29             description
30                 "Interface based on the system port number.";
31         }
32     }
33     description
34         "Management address interface subtype.";
35     reference
36         "8.5.9.3 of IEEE Std 802.1AB";
37 }
38 typedef man-addr-type {
39     type string {
40         pattern "[0-9A-F]{2}([0-9A-F]{2}){0,30}";
41     }
42     description
43         "Management address associated with the LLDP agent.";
44     reference
45         "8.5.9.4 of IEEE Std 802.1AB";
46 }
47 typedef system-capabilities-map {
48     type bits {
49         bit other {
50             position 0;
51             description
52                 "System has capabilities other than those listed below.";
53         }
54         bit repeater {
55             position 1;
56             description
57                 "System has repeater capability.";
58         }
59         bit bridge {
```



```
1      position 2;
2      description
3      "System has bridge capability.";
4  }
5  bit wlan-access-point {
6      position 3;
7      description
8      "System has WLAN access point capability.";
9  }
10 bit router {
11     position 4;
12     description
13     "System has router capability.";
14 }
15 bit telephone {
16     position 5;
17     description
18     "System has telephone capability.";
19 }
20 bit docsis-cable-device {
21     position 6;
22     description
23     "System has DOCSIS Cable Device capability.";
24     reference
25     "IETF RFC 4639";
26 }
27 bit station-only {
28     position 7;
29     description
30     "System has only station capability.";
31 }
32 bit cvlan-component {
33     position 8;
34     description
35     "System has C-VLAN component functionality.";
36 }
37 bit svlan-component {
38     position 9;
39     description
40     "System has S-VLAN component functionality.";
41 }
42 bit two-port-mac-relay {
43     position 10;
44     description
45     "System has Two-port MAC Relay (TPMR) functionality.";
46 }
47 }
48 description
49 "This describes system capabilities.";
50 reference
51 "8.5.8.1 of IEEE Std 802.1AB";
52 }
53 typedef port-list {
54     type binary {
55         length "0..512";
56     }
57     description
58     "Each octet within this value specifies a set of eight ports, with the
59     first octet specifying ports 1 through 8, the second octet specifying
```

```

1      ports 9 through 16, etc. Within each octet, the most significant bit
2      represents the lowest numbered port, and the least significant bit
3      represents the highest numbered port. Thus, each port of the system is
4      represented by a single bit within the value of this object. If that
5      bit has a value of '1' then that port is included in the set of ports;
6      the port is not included if its bit has a value of '0'.
7      reference
8      "IETF RFC 2674 section 5";
9  }
10 }
11

```

12 12.6.2.2 Definition for *ieee802-dot1ab-lldp* YANG module

```

13 module ieee802-dot1ab-lldp {
14   yang-version 1.1;
15   namespace "urn:ieee:std:802.1AB:yang:ieee802-dot1ab-lldp";
16   prefix lldp;
17
18   import ieee802-dot1ab-types {
19     prefix lldp-types;
20   }
21   import ietf-routing {
22     prefix rt;
23   }
24   import ietf-yang-types {
25     prefix yang;
26   }
27   import ietf-interfaces {
28     prefix if;
29   }
30   import ieee802-types {
31     prefix ieee;
32   }
33
34   organization
35     "Institute of Electrical and Electronics Engineers";
36   contact
37     "WG-URL: http://ieee802.org/1/
38     WG-EMail: stds-802-1-1@ieee.org
39     Contact: IEEE 802.1 Working Group Chair
40     Postal: C/O IEEE 802.1 Working Group
41     IEEE Standards Association
42       445 Hoes Lane
43       Piscataway, NJ 08854
44       USA
45
46     E-mail: stds-802-1-chairs@ieee.org";
47   description
48     "Management Information Base module for LLDP configuration, statistics,
49     local system data, and remote systems data components. Copyright(C) IEEE
50     (2026). All rights reserved. This version of this YANG module is part of
51     IEEE Std 802.1AB-2026; see the standard itself for full legal notices.";
52
53   revision 2026-02-12 {
54     description
55       "Published as part of IEEE Std 802.1AB-2026.
56

```

```
1      The following reference statement identifies each referenced IEEE
2      Standard as updated by applicable amendments.";
3      reference
4      "IEEE Std 802.1AB Station and Media Access Control Connectivity Discovery:
5      IEEE Std 802.1AB-2026.";
6  }
7  revision 2022-03-15 {
8      description
9      "Published as part of IEEE Std 802.1ABcu.";
10     reference
11     "IEEE Std 802.1AB-2016";
12 }
13
14 grouping lldp-cfg {
15     description
16     "LLDP basic configuration group.";
17     leaf message-fast-tx {
18         type uint32 {
19             range "1..3600";
20         }
21         units "ticks";
22         default "1";
23         description
24         "Time interval in timer ticks between transmissions during fast
25         transmission periods (i.e., txFast is non-zero).";
26         reference
27         "9.1.1, 9.2.5.4 of IEEE Std 802.1AB";
28     }
29     leaf message-tx-hold-multiplier {
30         type uint32 {
31             range "2..10";
32         }
33         default "4";
34         description
35         "Multiplier of msg-tx-interval.";
36         reference
37         "9.2.5.5 of IEEE Std 802.1AB";
38     }
39     leaf message-tx-interval {
40         type uint32 {
41             range "1..3600";
42         }
43         units "ticks";
44         default "30";
45         description
46         "Time interval in timer ticks between transmissions during normal
47         transmission periods (i.e., txFast is zero).";
48         reference
49         "9.1.1, 9.2.5.6 of IEEE Std 802.1AB";
50     }
51     leaf reinit-delay {
52         type uint32 {
53             range "1..10";
54         }
55         units "second";
56         default "2";
57         description
58         "Amount of delay (in units of seconds) from when admin-status becomes
59         'disabled' until re-initialization is attempted.";
```

```

1      reference
2      "9.2.5.9 of IEEE Std 802.1AB";
3  }
4  leaf tx-credit-max {
5      type uint32 {
6          range "1..10";
7      }
8      default "5";
9      description
10     "The maximum number of consecutive LLDPDUs that can be transmitted at
11     any time.";
12     reference
13     "9.2.5.18 of IEEE Std 802.1AB";
14 }
15 leaf tx-fast-init {
16     type uint32 {
17         range "1..8";
18     }
19     default "4";
20     description
21     "Initial value for the fast transmitting LLDPDU.";
22     reference
23     "9.2.5.20 of IEEE Std 802.1AB";
24 }
25 leaf notification-interval {
26     type uint32 {
27         range "1..3600";
28     }
29     units "second";
30     default "30";
31     description
32     "Controls the transmission of LLDP notifications.";
33     reference
34     "9.2.5.6 of IEEE Std 802.1AB";
35 }
36 }
37
38 container lldp {
39     description
40     "Link Layer Discovery Protocol configuration and operational
41     information.";
42     uses lldp-cfg;
43     container remote-statistics {
44         config false;
45         description
46         "LLDP remote operational statistics data.";
47         leaf last-change-time {
48             type yang:timestamp;
49             description
50             "The value of sysUpTime object.";
51             reference
52             "11.5.1 of IEEE Std 802.1AB:
53             lldpV2StatsRemTablesLastChangeTime";
54         }
55         leaf remote-inserts {
56             type yang:zero-based-counter32;
57             units "table entries";
58             description
59             "The number of times the complete set of information advertised by

```

```
1         a particular MSAP has been inserted into tables contained in
2         remote-systems-data and lldpExtensions objects.";
3     reference
4         "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesInserts";
5 }
6 leaf remote-deletes {
7     type yang:zero-based-counter32;
8     units "table entries";
9     description
10        "The number of times the complete set of information advertised by
11        a particular MSAP has been deleted from tables contained in
12        remote-systems-data and lldpExtensions objects.";
13    reference
14        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDeletes";
15 }
16 leaf remote-drops {
17     type yang:zero-based-counter32;
18     units "table entries";
19     description
20        "The number of times the complete set of information advertised by
21        a particular MSAP could not be entered into tables contained in
22        remote-systems-data and lldpExtensions objects because of
23        insufficient resources.";
24    reference
25        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDrops";
26 }
27 leaf remote-ageouts {
28     type yang:zero-based-counter32;
29     description
30        "The number of times the complete set of information advertised by
31        a particular MSAP has been deleted from tables contained in
32        remote-systems-data and lldpExtensions objects because the
33        information timeliness interval has expired.";
34    reference
35        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesAgeouts";
36 }
37 }
38 container local-system-data {
39     config false;
40     description
41        "LLDP local system operational data.";
42     leaf chassis-id-subtype {
43         type ieee:chassis-id-subtype-type;
44         description
45            "The type of encoding used to identify the chassis associated with
46            the local system.";
47         reference
48            "8.5.2.2 of IEEE Std 802.1AB";
49     }
50     leaf chassis-id {
51         type ieee:chassis-id-type;
52         description
53            "Chassis component associated with the local system.";
54         reference
55            "8.5.2.3 of IEEE Std 802.1AB";
56     }
57     leaf system-name {
58         type string {
59             length "0..255";
```

```
1      }
2      description
3          "System name of the local system.";
4      reference
5          "8.5.6.2 of IEEE Std 802.1AB";
6  }
7  leaf system-description {
8      type string {
9          length "0..255";
10     }
11     description
12         "System description of the local system.";
13     reference
14         "8.5.7.2 of IEEE Std 802.1AB";
15 }
16 leaf system-capabilities-supported {
17     type lldp-types:system-capabilities-map;
18     description
19         "System capabilities are supported on the local system.";
20     reference
21         "8.5.8.1 of IEEE Std 802.1AB";
22 }
23 leaf system-capabilities-enabled {
24     type lldp-types:system-capabilities-map;
25     description
26         "System capabilities that are enabled on the local system.";
27     reference
28         "8.5.8.2 of IEEE Std 802.1AB";
29 }
30 }
31 list port {
32     key "name dest-mac-address";
33     description
34         "LLDP configuration information for a particular port.";
35     leaf name {
36         type if:interface-ref;
37         description
38             "The port name used to identify the port component (contained in
39              the local chassis with the LLDP agent) associated with this entry.";
40     }
41     leaf dest-mac-address {
42         type ieee:mac-address;
43         description
44             "Destination MAC address. The ieee:mac-address type has a pattern
45              that allows upper and lower case letters. To avoid issues with
46              string comparison, it is suggested to only use upper case for the
47              letters in the hexadecimal numbers. Implementers using code
48              comparing MAC addresses should note that there is still an issue
49              with a difference between the IETF mac-address definition and the
50              IEEE mac-address definition.";
51     }
52     leaf admin-status {
53         type enumeration {
54             enum tx-only {
55                 value 1;
56                 description
57                     "Transmit LLDP frames only.";
58             }
59             enum rx-only {
```

```

1         value 2;
2         description
3             "Receive LLDP frames only.";
4     }
5     enum tx-and-rx {
6         value 3;
7         description
8             "Transmit and Receive LLDP frames.";
9     }
10    enum disabled {
11        value 4;
12        description
13            "Do Not Transmit or Receive LLDP frames.";
14    }
15 }
16 default "tx-and-rx";
17 description
18     "Administrative status of the local LLDP agent.";
19 reference
20     "9.2.5.1 of IEEE Std 802.1AB";
21 }
22 leaf notification-enable {
23     type boolean;
24     default "false";
25     description
26         "Notification status.";
27 }
28 leaf tlvs-tx-enable {
29     type bits {
30         bit port-desc {
31             position 0;
32             description
33                 "Transmit 'Port Description TLV'.";
34         }
35         bit sys-name {
36             position 1;
37             description
38                 "Transmit 'System Name TLV'.";
39         }
40         bit sys-desc {
41             position 2;
42             description
43                 "Transmit 'System Description TLV'.";
44         }
45         bit sys-cap {
46             position 3;
47             description
48                 "Transmit 'System Capabilities TLV'.";
49         }
50     }
51     description
52         "LLDP TLVs whose transmission is allowed on the local LLDP agent by
53         the network management.";
54     reference
55         "9.1.1.1 of IEEE Std 802.1AB";
56 }
57 uses lldp-cfg;
58 list management-address-tx-port {
59     key "address-subtype man-address";

```

```
1      description
2          "Set of ports (represented as a PortList) on which the local system
3          management address instance will be transmitted.";
4      leaf address-subtype {
5          type identityref {
6              base rt:address-family;
7          }
8          description
9              "Type of address.";
10         reference
11             "8.5.9.3 of IEEE Std 802.1AB";
12     }
13     leaf man-address {
14         type lldp-types:man-addr-type;
15         description
16             "Management address associated with this TLV.";
17         reference
18             "8.5.9.4 of IEEE Std 802.1AB";
19     }
20     leaf tx-enable {
21         type boolean;
22         default "false";
23         description
24             "Transmission enabled status.";
25         reference
26             "9.1.1.1 of IEEE Std 802.1AB";
27     }
28     leaf addr-len {
29         type uint32;
30         config false;
31         description
32             "Length of the management address subtype and the management
33             address fields in LLDPDUs transmitted by the local LLDP agent.";
34         reference
35             "8.5.9.2 of IEEE Std 802.1AB";
36     }
37     leaf if-subtype {
38         type lldp-types:man-addr-if-subtype;
39         config false;
40         description
41             "Interface numbering method used for defining the interface
42             number, associated with the local system.";
43         reference
44             "8.5.9.5 of IEEE Std 802.1AB";
45     }
46     leaf if-id {
47         type uint32;
48         config false;
49         description
50             "Interface number for the management address component associated
51             with the local system.";
52         reference
53             "8.5.9.6 of IEEE Std 802.1AB";
54     }
55 }
56 leaf port-id-subtype {
57     type ieee:port-id-subtype-type;
58     config false;
59     description
```



```
1         "Port identifier encoding used in the associated 'port-id' object.";
2     reference
3         "8.5.3.2 of IEEE Std 802.1AB";
4 }
5 leaf port-id {
6     type ieee:port-id-type;
7     config false;
8     description
9         "Port component associated with a given port in the local system.";
10    reference
11        "8.5.3.3 of IEEE Std 802.1AB";
12 }
13 leaf port-desc {
14     type string {
15         length "0..255";
16     }
17     config false;
18     description
19         "802 LAN station's port description associated with the local
20         system.";
21     reference
22         "8.5.5.2 of IEEE Std 802.1AB";
23 }
24 container tx-statistics {
25     config false;
26     description
27         "LLDP frame transmission statistics for a particular port.";
28     leaf total-frames {
29         type yang:counter32;
30         description
31             "A count of all LLDP frames transmitted through the port.";
32         reference
33             "9.2.7.5 of IEEE Std 802.1AB";
34     }
35     leaf total-length-errors {
36         type yang:counter32;
37         description
38             "A count of all LLDP length errors detected when constructing
39             LLDPDU frames for transmission through the port.";
40         reference
41             "9.2.7.8 of IEEE Std 802.1AB";
42     }
43 }
44 container rx-statistics {
45     config false;
46     description
47         "LLDP frame reception statistics for a particular port.";
48     leaf total-ageouts {
49         type yang:zero-based-counter32;
50         description
51             "A count of the times that a neighbor's information is deleted
52             because of rxInfoTTL timer expiration.";
53         reference
54             "9.2.7.1 of IEEE Std 802.1AB";
55     }
56     leaf total-discarded-frames {
57         type yang:counter32;
58         description
59             "A count of all LLDPDUs received and then discarded.";
```

```

1      reference
2      "9.2.7.2 of IEEE Std 802.1AB";
3  }
4  leaf error-frames {
5      type yang:counter32;
6      description
7          "A count of all LLDPDUs received at the port with one or more
8              detectable errors.";
9      reference
10         "9.2.7.3 of IEEE Std 802.1AB";
11  }
12  leaf total-frames {
13      type yang:counter32;
14      description
15          "A count of all LLDP frames received at the port.";
16      reference
17         "9.2.7.4 of IEEE Std 802.1AB";
18  }
19  leaf total-discarded-tlvs {
20      type yang:counter32;
21      description
22          "A count of all TLVs received at the port and discarded for any
23              reason.";
24      reference
25         "9.2.7.6 of IEEE Std 802.1AB";
26  }
27  leaf total-unrecognized-tlvs {
28      type yang:counter32;
29      description
30          "A count of all TLVs not recognized by the receiving LLDP local
31              agent.";
32      reference
33         "9.2.7.7 of IEEE Std 802.1AB";
34  }
35  }
36  list remote-systems-data {
37      key "time-mark remote-index";
38      config false;
39      description
40          "Information about a particular physical network connection.";
41      leaf time-mark {
42          type yang:timeticks;
43          description
44              "A TimeFilter for this entry.";
45          reference
46              "IETF RFC 2021 section 6";
47      }
48      leaf remote-index {
49          type uint32 {
50              range "1..2147483647";
51          }
52          description
53              "Represents an arbitrary local integer value used to identify a
54                  remote system.";
55          reference
56              "11.5.1 of IEEE Std 802.1AB: lldpV2RemIndex";
57      }
58      leaf remote-too-many-neighbors {
59          type boolean;

```

```
1      description
2      "Indicates that there are too many neighbors as determined by the
3      variable tooManyNeighbors.";
4      reference
5      "9.2.5.16 of IEEE Std 802.1AB";
6  }
7  leaf remote-changes {
8      type boolean;
9      description
10     "Indicates that there are changes in the remote system's data, as
11     determined by the variable remoteChanges.";
12     reference
13     "9.2.5.10 of IEEE Std 802.1AB";
14 }
15 leaf chassis-id-subtype {
16     type ieee:chassis-id-subtype-type;
17     description
18     "Identify the chassis associated with the remote system.";
19     reference
20     "8.5.2.2 of IEEE Std 802.1AB";
21 }
22 leaf chassis-id {
23     type ieee:chassis-id-type;
24     description
25     "Identify the chassis component associated with the remote
26     system.";
27     reference
28     "8.5.2.3 of IEEE Std 802.1AB";
29 }
30 leaf port-id-subtype {
31     type ieee:port-id-subtype-type;
32     description
33     "The type of port identifier encoding used in the associated
34     'port-id' object.";
35     reference
36     "8.5.3.2 of IEEE Std 802.1AB";
37 }
38 leaf port-id {
39     type ieee:port-id-type;
40     description
41     "Port component associated with the remote system.";
42     reference
43     "8.5.3.3 of IEEE Std 802.1AB";
44 }
45 leaf port-desc {
46     type string {
47         length "0..255";
48     }
49     description
50     "Description of the given port associated with the remote system.";
51     reference
52     "8.5.5.2 of IEEE Std 802.1AB";
53 }
54 leaf system-name {
55     type string {
56         length "0..255";
57     }
58     description
59     "System name of the remote system.";
```

```
1         reference
2         "8.5.6.2 of IEEE Std 802.1AB";
3     }
4     leaf system-description {
5         type string {
6             length "0..255";
7         }
8         description
9         "System description of the remote system.";
10        reference
11        "8.5.7.2 of IEEE Std 802.1AB";
12    }
13    leaf system-capabilities-supported {
14        type lldp-types:system-capabilities-map;
15        description
16        "Capabilities that are supported on the remote system.";
17        reference
18        "8.5.8.1 of IEEE Std 802.1AB";
19    }
20    leaf system-capabilities-enabled {
21        type lldp-types:system-capabilities-map;
22        description
23        "System capabilities that are enabled on the remote system.";
24        reference
25        "8.5.8.2 of IEEE Std 802.1AB";
26    }
27    list management-address {
28        key "address-subtype address";
29        description
30        "Management address information about a particular chassis
31        component.";
32        leaf address-subtype {
33            type identityref {
34                base rt:address-family;
35            }
36            description
37            "Management address identifier encoding.";
38            reference
39            "8.5.9.3 of IEEE Std 802.1AB";
40        }
41        leaf address {
42            type lldp-types:man-addr-type;
43            description
44            "Management address component associated with the remote
45            system.";
46            reference
47            "8.5.9.4 of IEEE Std 802.1AB";
48        }
49        leaf if-subtype {
50            type lldp-types:man-addr-if-subtype;
51            description
52            "Interface numbering method used for defining the interface
53            number, associated with the remote system.";
54            reference
55            "8.5.9.5 of IEEE Std 802.1AB";
56        }
57        leaf if-id {
58            type uint32;
59            description
```

```
1         "Interface number regarding the management address component
2         associated with the remote system.";
3     reference
4         "8.5.9.6 of IEEE Std 802.1AB";
5 }
6 }
7 list remote-unknown-tlv {
8     key "tlv-type";
9     description
10        "Information about an unrecognized TLV received from a physical
11        network connection. Entries may be created and deleted in this
12        table by the agent, if a physical topology discovery process is
13        active.";
14     leaf tlv-type {
15         type uint32 {
16             range "9..126";
17         }
18         description
19             "Type of TLV.";
20         reference
21             "9.2.8.7.1 of IEEE Std 802.1AB";
22     }
23     leaf tlv-info {
24         type binary {
25             length "0..511";
26         }
27         description
28             "Value extracted from TLV.";
29         reference
30             "9.2.8.7.1 of IEEE Std 802.1AB";
31     }
32 }
33 list remote-org-defined-info {
34     key "info-identifier info-subtype info-index";
35     description
36        "Information about the unrecognized organizationally defined
37        information advertised by the remote system.";
38     leaf info-identifier {
39         type uint32 {
40             range "0..16777215";
41         }
42         description
43             "The Organizationally Unique Identifier (OUI) or Company ID
44             (CID).";
45         reference
46             "8.7.1.3 of IEEE Std 802.1AB";
47     }
48     leaf info-subtype {
49         type uint32 {
50             range "1..255";
51         }
52         description
53             "The subtype of the organizationally defined information
54             received from the remote system.";
55         reference
56             "8.7.1.4 of IEEE Std 802.1AB";
57     }
58     leaf info-index {
59         type uint32 {
```

```
1         range "1..2147483647";
2     }
3     description
4         "Arbitrary local integer value.";
5     }
6     leaf remote-info {
7         type binary {
8             length "0..507";
9         }
10        description
11            "The organizationally defined information of the remote system.";
12        reference
13            "8.7.1.5 of IEEE Std 802.1AB";
14    }
15 }
16 }
17 }
18 }
19
20 notification remote-table-change {
21     description
22         "A rem-table-change notification is sent when the value of
23         remote-table-last-change-time changes. It can be utilized by an NMS to
24         trigger LLDP remote systems table maintenance polls.";
25     leaf remote-insert {
26         type leafref {
27             path "/lldp:lldp/lldp:remote-statistics/lldp:remote-inserts";
28         }
29         description
30             "The number of times the complete set of information advertised by a
31             particular MSAP has been inserted into tables contained in
32             remote-systems-data and lldpExtensions objects.";
33         reference
34             "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesInserts";
35     }
36     leaf remote-delete {
37         type leafref {
38             path "/lldp:lldp/lldp:remote-statistics/lldp:remote-deletes";
39         }
40         description
41             "The number of times the complete set of information advertised by a
42             particular MSAP has been deleted from tables contained in
43             remote-systems-data and lldpExtensions objects.";
44         reference
45             "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDeletes";
46     }
47     leaf remote-drops {
48         type leafref {
49             path "/lldp:lldp/lldp:remote-statistics/lldp:remote-drops";
50         }
51         description
52             "The number of times the complete set of information advertised by a
53             particular MSAP could not be entered into tables contained in
54             remote-systems-data and lldpExtensions objects because of
55             insufficient resources.";
56         reference
57             "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesDrops";
58     }
59     leaf remote-ageouts {
```

```
1     type leafref {
2         path "/lldp:lldp/lldp:remote-statistics/lldp:remote-ageouts";
3     }
4     description
5         "The number of times the complete set of information advertised by a
6         particular MSAP has been deleted from tables contained in
7         remote-systems-data and lldpExtensions objects because the
8         information timeliness interval has expired.";
9     reference
10        "11.5.1 of IEEE Std 802.1AB: lldpV2StatsRemTablesAgeouts";
11 }
12 }
13 }
14
15
```

1 Annex A

2 (normative)

3 PICS proforma²⁰

4 A.1 Introduction

5 The supplier of a protocol implementation that is claimed to conform to this standard shall complete the
6 following Protocol Implementation Conformance Statement (PICS) proforma.

7 A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of
8 which capabilities and options of the protocol have been implemented. The PICS can have a number of uses,
9 including use

- 10 a) By the protocol implementers, as a checklist to reduce the risk of failure to conform to the standard
11 through oversight.
- 12 b) By the supplier and acquirer—or potential acquirer—of the implementation, as a detailed indication
13 of the capabilities of the implementation, stated relative to the common basis for understanding
14 provided by the standard PICS proforma.
- 15 c) By the user—or potential user—of the implementation, as a basis for initially checking the
16 possibility of interworking with another implementation (note that although interworking can never
17 be guaranteed, failure to interwork can often be predicted from incompatible PICS).
- 18 d) By a protocol tester, as the basis for selecting appropriate tests against which to assess the claim for
19 conformance of the implementation.

20 A.2 Abbreviations and special symbols

21 A.2.1 Status symbols

- 22 M Mandatory
- 23 O Optional
- 24 *O.n* Optional, but support of at least one of the group of options labeled by the same numeral *n* is
25 required
- 26 X Prohibited
- 27 pred: Conditional-item symbol, including predicate identification: See B.3.4
- 28 ¬ Logical negation, applied to a conditional item's predicate

29 A.2.2 General abbreviations

- 30 N/A Not applicable
- 31 PICS Protocol Implementation Conformance Statement

²⁰ *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

1 **A.3 Instructions for completing the PICS proforma**

2 **A.3.1 General structure of the PICS proforma**

3 The first part of the PICS proforma, implementation identification and protocol summary, is to be completed
4 as indicated with the information necessary to identify fully both the supplier and the implementation.

5 The main part of the PICS proforma is a fixed-format questionnaire, divided into several subclauses, each
6 containing a number of individual items. Answers to the questionnaire items are to be provided in the
7 right-most column, either by simply marking an answer to indicate a restricted choice (usually Yes or No), or
8 by entering a value or a set or range of values. (Note that there are some items in which two or more choices
9 from a set of possible answers can apply; all relevant choices are to be marked.)

10 Each item is identified by an item reference in the first column. The second column contains the question to
11 be answered; the third column records the status of the item—whether support is mandatory, optional, or
12 conditional; see also B.3.4. The fourth column contains the reference or references to the material that
13 specifies the item in the main body of this standard, and the fifth column provides the space for the answers.

14 A supplier may also provide (or be required to provide) further information, categorized as either Additional
15 Information or Exception Information. When present, each kind of further information is to be provided in a
16 further subclause of items labeled A_i or X_i , respectively, for cross-referencing purposes, where i is any
17 unambiguous identification for the item (e.g., simply a numeral). There are no other restrictions on its format
18 and presentation.

19 A completed PICS proforma, including any Additional Information and Exception Information, is the
20 Protocol Implementation Conformation Statement for the implementation in question.

21 NOTE—Where an implementation is capable of being configured in more than one way, a single PICS may be able to
22 describe all such configurations. However, the supplier has the choice of providing more than one PICS, each covering
23 some subset of the implementation's configuration capabilities, in case that makes for easier and clearer presentation of
24 the information.

25 **A.3.2 Additional information**

26 Items of Additional Information allow a supplier to provide further information intended to assist the
27 interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and a PICS
28 can be considered complete without any such information. Examples might be an outline of the ways in
29 which a (single) implementation can be set up to operate in a variety of environments and configurations, or
30 information about aspects of the implementation that are outside the scope of this standard but that have a
31 bearing on the answers to some items.

32 References to items of Additional Information may be entered next to any answer in the questionnaire and
33 may be included in items of Exception Information.

34 **A.3.3 Exception information**

35 It may occasionally happen that a supplier will wish to answer an item with mandatory status (after any
36 conditions have been applied) in a way that conflicts with the indicated requirement. No preprinted answer
37 will be found in the Support column for this: instead, the supplier shall write the missing answer into the
38 Support column, together with an X_i reference to an item of Exception Information, and shall provide the
39 appropriate rationale in the Exception item.

- 1 An implementation for which an Exception item is required in this way does not conform to this standard.
2 NOTE—A possible reason for the situation described above is that a defect in this standard has been reported, a
3 correction for which is expected to change the requirement not met by the implementation.

4 **A.3.4 Conditional status**

5 **A.3.4.1 Conditional items**

6 The PICS proforma contains a number of conditional items. These are items for which both the applicability
7 of the item itself, and its status if it does apply—mandatory or optional—are dependent upon whether or not
8 certain other items are supported.

9 Where a group of items is subject to the same condition for applicability, a separate preliminary question
10 about the condition appears at the head of the group, with an instruction to skip to a later point in the
11 questionnaire if the “Not Applicable” answer is selected. Otherwise, individual conditional items are
12 indicated by a conditional symbol in the Status column.

13 A conditional symbol is of the form “pred: S,” where pred is a predicate as described in A.3.4.2, and S is a
14 status symbol, M or O.

15 If the value of the predicate is TRUE (see A.3.4.2), the conditional item is applicable, and its status is
16 indicated by the status symbol following the predicate: the answer column is to be marked in the usual way.
17 If the value of the predicate is FALSE, the “Not Applicable” (N/A) answer is to be marked.

18 **A.3.4.2 Predicates**

19 A predicate is one of the following:

- 20 a) An item-reference for an item in the PICS proforma: The value of the predicate is TRUE if the item
21 is marked as supported, and is FALSE otherwise.
- 22 b) A predicate-name, for a predicate defined as a Boolean expression constructed by combining
23 item-references using the Boolean operator OR: The value of the predicate is TRUE if one or more
24 of the items is marked as supported.
- 25 c) A predicate-name, for a predicate defined as a Boolean expression constructed by combining
26 item-references using the Boolean operator AND: The value of the predicate is TRUE if all of the
27 items are marked as supported.
- 28 d) The logical negation symbol “¬” prefixed to an item-reference or predicate-name: The value of the
29 predicate is TRUE if the value of the predicate formed by omitting the “¬” symbol is FALSE, and
30 vice versa.

31 Each item whose reference is used in a predicate or predicate definition, or in a preliminary question for
32 grouped conditional items, is indicated by an asterisk²¹ in the Item column.

33

34

35

36 **PICS proforma for IEEE Std 802.1AB**

²¹ Asterisks are currently not used in this PICS.

1

A.3.5 Implementation identification

Supplier	
Contact point for queries about the PICS	
Implementation Name(s) and Version(s)	
Other information necessary for full identification—e.g., name(s) and version(s) of machines and/or operating system names	
NOTE 1—Only the first three items are required for all implementations; other information may be completed as appropriate in meeting the requirement for full identification. NOTE 2—The terms Name and Version should be interpreted appropriately to correspond with a supplier’s terminology (e.g., Type, Series, Model).	

A.3.6 Protocol summary, IEEE Std 802.1AB

Identification of protocol specification	IEEE Std 802.1AB, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery			
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd.	:	Corr.	:
	Amd.	:	Corr.	:
Have any Exception items been required? (See B.3.3: The answer Yes means that the implementation does not conform to IEEE Std 802.1AB-2026)	No []		Yes []	

Date of Statement	
-------------------	--

2

A.4 Major capabilities and options

Item	Feature	Status	References	Support
cntrlport	Are LLDP exchanges supported through a controlled port if port access is controlled by IEEE Std 802.1X?	M	5.3, 6	Yes [] N/A []
uncntrlport	Are LLDP exchanges supported through the uncontrolled port if port access is controlled by IEEE Std 802.1X?	O	5.4, 6	Yes [] No [] N/A []
addr	Are LLDP addressing and LLDP EtherType encoding for Normal LLDPDUs in conformance with the defined requirements?			
	<i>All systems:</i>			
	DA = Any group MAC address permitted by Table 7-2	O	7.1	Yes [] No []
	DA = Any individual MAC address	O	7.1	Yes [] No []
	SA = station MAC address	M	7.2	Yes []
	LLDP EtherType encoding	M	7.3	Yes []
	<i>C-VLAN Bridge:</i>			
	DA = Nearest bridge address	M	7.1	Yes [] N/A []
	DA = Nearest non-TPMR bridge address	M	7.1	Yes [] N/A []
	DA = Nearest customer bridge address	M	7.1	Yes [] N/A []
	<i>S-VLAN Bridge:</i>			
	DA = Nearest bridge address	M	7.1	Yes [] N/A []
	DA = Nearest non-TPMR bridge address	M	7.1	Yes [] N/A []
	DA = Nearest customer bridge address	X	7.1	No [] N/A []
	<i>TPMR Bridge:</i>			
	DA = Nearest bridge address	M	7.1	Yes [] N/A []
	DA = Nearest non-TPMR bridge address	X	7.1	No [] N/A []
	DA = Nearest customer bridge address	X	7.1	No [] N/A []
	<i>End station:</i>			
	DA = Nearest bridge address	M	7.1	Yes [] N/A []
	DA = Nearest non-TPMR bridge address	O	7.1	Yes [] No [] N/A []
	DA = Nearest customer bridge address	O	7.1	Yes [] No [] N/A []
lldpdu	Is the Normal LLDPDU encapsulation in conformance with the TLV order specified by the Normal LLDPDU format?	M	8.2	Yes []
tlvfmt	Is the basic TLV capability implemented?	M	8.4	Yes []
basictlv	Is each TLV in the basic management set implemented?			
	End Of LLDPDU TLV	M	8.5.1	Yes []
	Chassis ID TLV	M	8.5.2	Yes []
	Port ID TLV	M	8.5.3	Yes []
	Time To Live TLV	M	8.5.4	Yes []
	Port Description TLV	M	8.5.5	Yes []
	System Name TLV	M	8.5.6	Yes []
	System Description TLV	M	8.5.2	Yes []
	System Capabilities TLV	M	8.5.8	Yes []
	Management Address TLV	M	8.5.9	Yes []
xtlvfmt	Is the Organizationally Specific TLV capability implemented?	M	8.7	Yes []

A.4 Major capabilities and options

Item	Feature	Status	References	Support
tlvsel	User configuration of transmission of implemented optional TLVs	M	5.3 l)	Yes []
	Which of the following operational modes are implemented?			
optxrx	Transmit and receive	O.1	6.1	Yes [] No []
optx	Transmit only	O.1	6.1	Yes [] No []
opr	Receive only	O.1	6.1	Yes [] No []
txmode	Is the transmit mode in conformance with all operational specifications indicated for the Tx mode in Table 9-1?	optxrx OR optx: M	Clause 9	Yes [] N/A []
rxmode	Is the receive module in conformance with all operational specifications indicated for the Rx mode in Table 9-1?	optxrx OR opr: M	Clause 9	Yes [] N/A []
	Which type of data store/retrieval is implemented?			
mib	SNMP MIB is supported	O.2	11.5, 5.3	Yes [] No []
nomib	SNMP MIB is not supported	O.2	10.1, 5.3	Yes [] No []
snmpmib	Is the MIB module in conformance with the MIB sections indicated in Table 11-1 for the operating mode being implemented?	mib:M	11.5, 5.3	Yes [] N/A []
snmpsupport	Which of the transport mappings defined by IETF RFC 3417 or IETF RFC 4789 is used to support SNMP?			
	IETF RFC 3417	mib:O.3	5.3, 5.4	Yes [] No [] N/A []
	IETF RFC 4789	mib:O.3	5.3, 5.4	Yes [] No [] N/A []
equivstor	If the SNMP is not supported, is functionally equivalent storage and retrieval capability specified in Clause 8, Clause 9, Clause10 provided for the operating mode being implemented?	nomib:M	10.1	Yes [] N/A []
xlldp	Is the Extended Link Layer Discovery Protocol supported?	O		Yes [] No []

A.4 Major capabilities and options

Item	Feature	Status	References	Support
exttlv	Extended LLDP set of TLVs implemented	xlldp:M	5.3	Yes [] N/A []
xlldpdu	Are the Extension and Extension Request LLDPDU encapsulation in conformance with the TLV order specified?	xlldp:M	8.2	Yes [] N/A []
xaddr	Are LLDP addressing and LLDP EtherType encoding for Extension and Extension Request LLDPDUs in conformance with the defined requirements? <i>Extension Request LLDPDUs:</i> DA = Return MAC address from Manifest TLV SA = station MAC address LLDP EtherType encoding <i>Extension LLDPDU:</i> DA = Return MAC address from Extension Request TLV SA = station MAC address LLDP EtherType encoding	xlldp:M xlldp:M xlldp:M xlldp:M xlldp:M xlldp:M	7.1.2 7.2 7.4 7.1.2 7.2 7.4	Yes [] N/A [] Yes [] N/A [] Yes [] N/A [] Yes [] N/A [] Yes [] N/A [] Yes [] N/A []
yang	Does the implementation support management operations using YANG modules?	O	12.6	Yes[] No[]
yang modules	Is the ieee802-dot1ab-lldp module supported? Is the ieee802-dot1ab-types module supported?	yang:M yang:M	12.6.2.2 12.6.2.1	Yes[] N/A[] Yes[] N/A[]

1 Annex B

2 (normative)

3 PTOPO MIB update

4 The PTOPO MIB should be updated according to the following rules:

- 5 a) If any objects in the LLDP remote systems MIB age out, the equivalent objects in the PTOPO MIB
6 should be deleted.
- 7 b) If the TTL value in the Time To Live TLV is zero, then the port is being shutdown and the PTOPO
8 MIB objects associated with the MSAP identifier should be deleted.
- 9 c) If the TTL field is non-zero, then the appropriate ptopoRemEntry is found or created, based on the
10 data elements included in the LLDP frame. If the indicated entry is dynamic (i.e., ptopoConnIsStatic
11 is FALSE), then the current sysUpTime value is stored in the ptopoConnLastVerifyTime field for the
12 entry.
- 13 d) If a ptopoRemEntry was added then the ptopoConnTabInserts counter is incremented.
- 14 e) If any ptopoRemEntry was added or deleted, or if information other than the
15 ptopoRemLastVerifyTime changed for any entry due to the processing of this LLDP frame, the
16 ptopoLastChangeTime object is set with the current sysUpTime, and a ptopoConfigChange trap
17 event is generated. (See the PTOPO MIB for information on ptopoConfigChange trap generation.)

1 **Annex C**

2 (informative)

3 **Example LLDP transmission frame formats**

4 The LLDP MAC frame format is based on the particular transmission protocol. The following example
5 formats illustrate the indicated LLDP EtherType encoding method.

6 **C.1 Direct-encoded LLDP frame format**

7 The IEEE 802.3 LLDP frame format is illustrated in Figure C.1.

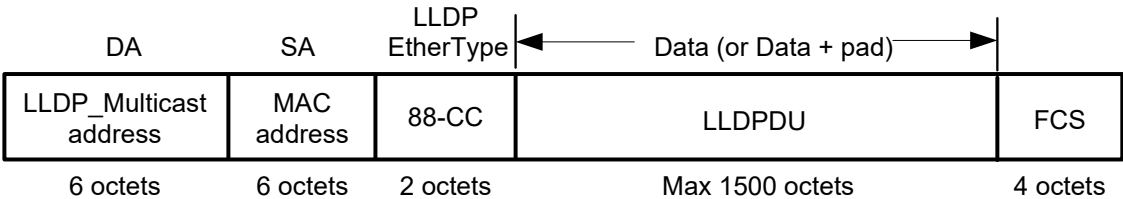


Figure C.1—IEEE 802.3 LLDP frame format

8 NOTE—The illustration shows the simplest form of an LLDP frame on an IEEE 802.3 medium; i.e., where the frame
9 has had no IEEE 802.1Q tag header, or IEEE 802.1AE™ security tag, or any other form of encapsulation applied to it.

10 **C.2 SNAP-encoded LLDP frame format**

11 The IEEE 802.11™ frame format is illustrated in Figure C.2, for the case where the value of To DS and
12 From DS in the Frame Control field are both zero.

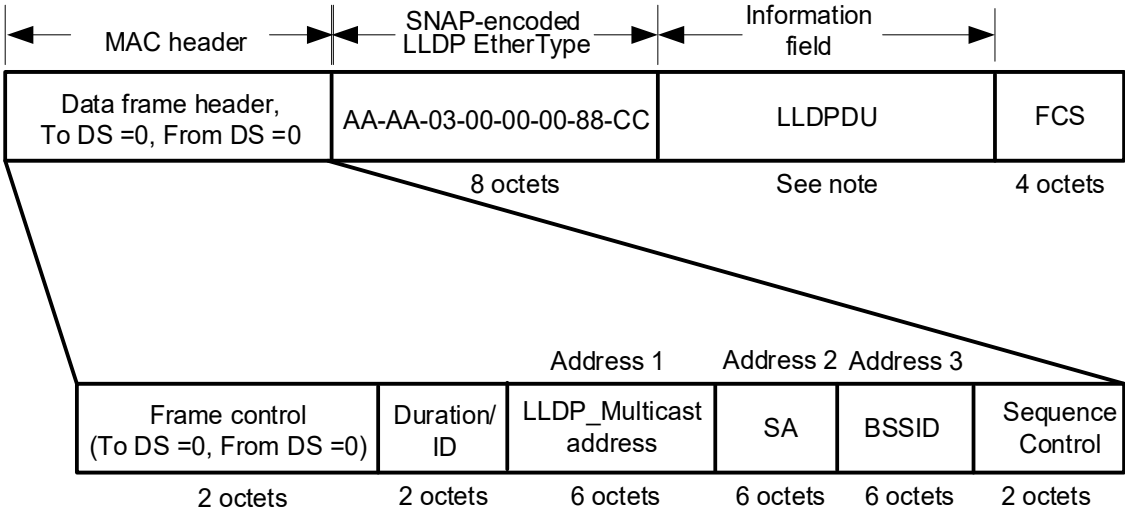


Figure C.2—IEEE 802.11 LLDP frame format

13 NOTE—The illustration shows the simplest form of an LLDP frame on an IEEE 802.11 medium; i.e., where the frame
14 has had no IEEE 802.1Q tag header, or IEEE 802.1AE security tag, or any other form of encapsulation applied to it.

Annex D

(informative)

Bibliography

- [B1] IEEE Std 802.11, Standard for Information technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) specifications.²²
- [B2] IETF RFC 2578 (STD 58), Structure of Management Information for Version 2 of the Simple Network Management Protocol (SNMPv2), McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999.²³
- [B3] IETF RFC 2579 (STD 58), Textual Conventions for Version 2 of the Simple Network Management Protocol (SNMPv2), McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999.
- [B4] IETF RFC 2580 (STD 58), Conformance Statements for SMIV2, McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999.
- [B5] IETF RFC 1812, Requirements for IP Version 4 Routers, June 1995.
- [B6] IETF RFC 2108, Definitions of Managed Objects for IEEE 802.3 Repeater Devices using SMIV2, February 1997.
- [B7] IETF RFC 2670, Radio Frequency (RF) Interface Management Information Base for MCNS/DOCSIS compliant RF interfaces, August 1999.
- [B8] IETF RFC 2922, Physical Topology MIB, Bierman, A., and Jones, K., November 1998.
- [B9] IETF RFC 4293, Management Information Base for the Internet Protocol (IP), April 2006.
- [B10] IETF RFC 4363, Definitions of Managed Objects for Bridges with Traffic Classes, Multicast Filtering, and Virtual LAN Extensions, January 2006.
- [B11] IETF RFC 4546, Radio Frequency (RF) Interface Management Information Base for Data over Cable Service Interface Specifications (DOCSIS) 2.0 Compliant RF Interfaces, June 2006.
- [B12] IETF RFC 7803, Changing the Registration Policy for the NETCONF Capability URNs Registry, February 2016.
- [B13] IETF RFC 8040, RESTCONF Protocol, January 2017.
- [B14] IETF RFC 8345, A YANG Data Model for Network Topologies, March 2018.
- [B15] ISO/IEC 8802-2:1998, Standard for Information technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 2: Logical link control.²⁴

²² IEEE publications are available from The Institute of Electrical and Electronic Engineers (<http://standards.ieee.org>).

²³ IETF RFCs are available from the Internet Engineering Task Force (<https://www.rfc-archive.org/>).

²⁴ ISO/IEC publications are available from the International Organization for Standardization (<https://www.iso.org/>), the International Electrotechnical Commission (<https://www.iec.ch/>), and the American National Standards Institute (<https://www.ansi.org/>).